# 复旦管院智识库

## 项目技术手册

原子级技术文档 · 覆盖全部架构与实现细节

共 15 章 · 涵盖后端/前端/AI/数据库/API 全栈


01 项目概述

02 技术架构

03 后端架构

04 数据库设计

05 RAG搜索引擎

06 AI服务体系

07 AI对话系统

08 会员与权限体系

09 编辑后台CMS

10 前端架构

11 前端页面与交互

12 数据管道

13 国际化与翻译

14 API接口文档



KDCC 团队 · 2026 年 4 月



版本 v1.0

# 01 — 项目概述

文档版本: v1.0 | 最后更新: 2026-04-07 | 状态: 正式发布

本文档为「复旦管院商业智识库」项目的顶层概述，涵盖产品定位、核心价值、数据资产、品牌规范、内容架构与会员体系等关键维度。阅读本文档后，读者应能完整理解本项目的商业目标、技术边界与产品形态。

## 1. 产品定位与愿景

### 1.1 一句话定义

「复旦管院智识库」— 十年商业智慧，一键触达。

本产品以复旦大学管理学院十余年积累的 8,400+ 篇深度商业文章为知识底座，构建一个集 RAG 智能检索、AI 对话助理、标签体系、栏目频道、主题专题五大核心能力于一体的新一代知识库平台。它不是一个传统的新闻发布网站，而是一个将沉淀内容资产转化为可检索、可对话、可发现的 智识平台。

### 1.2 对标产品矩阵

本产品在产品形态上对标三个标杆产品，从各自最强维度汲取设计灵感：

| 对标产品 | 借鉴维度 | 具体能力 |
|---|---|---|
| 36氪 | 栏目化 + 标签化 + 主题策划 | 频道导航（深度洞察、行业观察、学术前沿、院长说）；每篇文章 3-5 个可点击标签；热门标签云；专题策划页 |
| 财新网 | 深度内容 + 知识付费 + 专业检索 | 三级会员体系（游客/免费/付费）；高价值内容分层解锁；精准全文检索 |
| Perplexity | AI 对话式检索 | RAG 驱动的对话问答；引用来源溯源；多轮上下文理解；流式输出 |

### 1.3 目标用户画像

| 用户群 | 典型场景 | 核心需求 |
|---|---|---|
| 管院师生 | 课程研究、案例教学、课堂素材准备 | 按主题快速找到相关案例和观点，获取结构化知识 |
| MBA/EMBA 学员 | 行业研究、毕业论文写作、案例分析 | 跨领域知识关联、深度内容挖掘、时间线梳理 |
| 校友 / 企业家 | 行业动态追踪、投资决策参考 | 栏目订阅、主题专报推送、AI 助理随时问答 |
| 媒体 / 研究者 | 引用查找、数据溯源、趋势分析 | 精确检索、原文溯源、引用链接、批量导出 |

### 1.4 产品愿景全景

本产品的终极愿景是：让复旦管院十余年的商业知识积累，不再沉睡在静态网页和数据库中，而是通过 AI 技术赋能，成为一个 可对话、可导航、可发现的活态知识体。用户可以像和一位资深的商业知识顾问对话一样，随时获取跨越二十余年、覆盖多个行业领域的深度洞见。

## 2. 核心价值主张

### 2.1 从"新闻稿系统"到"智识库平台"的范式跃迁

本项目的核心价值在于将一个传统的、线性的新闻发布系统升级为一个多维度的、AI 驱动的知识平台。以下表格清晰展示了两种范式的本质差异：

| 维度 | 传统新闻稿系统 | 智识库平台 |
|---|---|---|
| 信息获取方式 | 搜索找文章 — 用户必须知道关键词才能找到内容 | 对话问知识 — 用自然语言提问，AI 理解意图并综合多篇文章作答 |
| 内容组织方式 | 线性时间流 — 文章按发布日期单一排列 | 标签 x 栏目 x 主题 三维导航 — 同一篇文章可从多个维度被发现 |
| 阅读体验 | 单篇阅读 — 每次只能看一篇文章，缺乏上下文 | 知识网络关联推荐 — 阅读一篇文章时自动推荐相关内容 |
| 交互模式 | 被动检索 — 用户必须主动搜索 | AI 主动洞察 + 每日推送 — 基于用户兴趣智能推荐 |
| 内容增值 | 静态内容 — 原始文章原样呈现 | AI 增强内容 — 自动生成摘要、观点提取、知识图谱 |

### 2.2 解决的三个核心问题

本项目致力于解决复旦管院在内容资产管理和知识传播中面临的三个根本性问题：

#### 问题一：内容资产沉睡

- 现状：8,431 篇高质量商业文章分散在不同来源（Business Knowledge、学院新闻、微信公众号），大量优质内容发布后即沉入信息海洋，无人问津。
- 解法：通过 AI 自动标签系统为每篇文章建立多维度索引（行业 + 主题 + 类型 + 实体），通过栏目体系和主题专题系统将散落文章编织为结构化的知识网络。每篇文章至少有 3-5 个可点击标签，每个标签聚合相关文章形成专题页。
- 效果：文章不再是"发了就完了"的一次性内容，而是知识网络中的永久节点，可以被反复发现和关联。

### 问题二：检索效率低下

- 现状：传统关键词搜索准确率低（Top-3 命中率约 60%），用户需要反复调整关键词尝试。检索耗时约 3 秒。缺乏语义理解能力，同义词、近义词、跨语言查询无法有效处理。
- 解法：构建 Hybrid Search（向量检索 + BM25 关键词检索混合），引入 Re-Ranker 二阶段排序（粗检召回 Top-50 后用 Gemini 2.5 Pro 精排），支持查询意图识别和多查询扩展。
- 效果：Top-3 命中率提升至 85%+，平均检索耗时降低至 1.5 秒以内。

### 问题三：知识消费门槛高

- 现状：用户必须逐篇阅读全文才能获取信息，缺乏快速消化大量内容的能力。对于"近三年管院在 ESG 领域有什么研究"这类综合性问题，用户需要手动搜索、逐篇阅读、自行整理。
- 解法：引入 AI 对话助理（对标 Perplexity），支持自然语言问答，自动综合多篇文章生成结构化回答，附带引用来源和置信度。支持多轮对话、追问建议、快捷指令（/summarize、/compare、/timeline）。
- 效果：将"读 20 篇文章才能回答的问题"压缩为一次对话交互，知识获取效率提升数个数量级。

# 3. 数据资产概览

## 3.1 文章数据规模

复旦管院智识库的数据底座来源于三个核心渠道，经过系统化采集与清洗，形成了覆盖广泛的知识语料库：

| 数据来源 | 英文标识 | 文章数量 | 内容特征 |
|---|---|---|---|
| 商业知识文章 (Business Knowledge) | business | 2,110 篇 | 深度分析、案例研究、行业报告、专家访谈。内容质量最高，平均篇幅较长，是智识库的核心 |
| 学院新闻 (News) | news | 4,725 篇 | 学院动态、教授观点、活动纪要、科研成果发布。数量最多，覆盖管院二十余年的发展历程。 |
| 微信公众号 (WeChat) | wechat | 1,596 篇 | 公众号推文，内容相对轻量，贴近大众传播语态，涵盖行业洞察和校友故事。 |
| 合计 | — | 8,431 篇 | — |

- 时间跨度：2001 年 - 2025 年，横跨 24 年
- 原始格式：每篇文章以 content.txt + 配图形式存储

## 3.2 结构化存储

| 存储形 | 技术方案 | 容量 | 数据结构 |
|---|---|---|---|
| 关系型数据库 | SQLite | 约 50 MB | articles 表，包含字段：id（主键）、source（来源标识）、title（标题）、publish_date（发布日期）、link（原始链接）、 |
| 向量索引 | FAISS (Facebook AI Similarity Search) | 约 351 MB | 基于 Gemini Embedding 模型（gemini-embedding-001）生成的高维向量表示，用于语义检索 |

## 3.3 分块策略

向量索引基于分块（Chunking）策略构建，将长文切割为适合嵌入模型处理的文本片段：

| 参数 | 值 | 说明 |
|---|---|---|
| 分块大 | 800 字 | 每个文本块的目标长度，确保语义完整性的同时控制在嵌入模型的有效输入范围内 |
| 重叠窗口 | 120 字 | 相邻分块之间的重叠字数（注：PROJECT_BLUEPRINT 中记录为 100 字重叠，config.py 实际参数以代码为准），确保跨分块边界的语义不丢失 |
| 嵌入模型 | gemini-embedding-001 | Google Gemini Embedding 模型，生成高质量中英文语义向量 |

## 3.4 数据处理链路

```
  原始数据采集 (爬虫)
  |
  v
ETL 清洗入库 (build_knowledge_base.py)
  |
  v
SQLite 关系存储 (fudan_knowledge_base.db)
  |
  v
文本分块 + 向量化 (create_vector_db_faiss.py)
  |
  v
FAISS 向量索引 (faiss_index_business/)
  |
  v
AI 自动标签 (Gemini API 批量生成)
  |
  v
栏目 / 专题自动归类
```

### 3.5 AI 模型配置

系统集成了多个 Google Gemini 模型，各司其职：

| 模型标识 | 环境变量 | 默认值 | 用途 |
|---|---|---|---|
| 主对话模型 | GEMINI_CHAT_MODEL | gemini-2.5-pro | AI 助理对话、摘要生成、Re-Ranker 排序 |
| 轻量模型 | GEMINI_FLASH_MODEL | gemini-2.5-flash | 标签生成、意图识别等高吞吐任务 |
| 嵌入模型 | GEMINI_EMBEDDING_MODEL | models/gemini-embedding-001 | 文本向量化，构建 FAISS 索引 |
| 编辑排版模型 | 硬编码 | gemini-3-flash-preview | 文章编辑格式化输出 |

## 4. 品牌规范

### 4.1 品牌色彩体系

复旦管院智识库的视觉体系继承复旦大学的品牌基因，以「复旦蓝」为主色调，「复旦橙」为强调色，构建学术质感与现代交互平衡的视觉语言。

#### 核心品牌三色

| 色彩名称 | 色值 | 色块 | 应用场景 |
|---|---|---|---|
| 复旦蓝 | #0d0783 | ![#0d0783](https://via.placeholder.com/16/0d0783/0d0783.png) | 主色调。用于导航栏、按钮主色、链接色、「深度洞察」栏目强调色、品牌标识 |
| 复旦橙 | #ea6b00 | ![#ea6b00](https://via.placeholder.com/16/ea6b00/ea6b00.png) | 强调色。用于行动召唤按钮、重要提示、「行业观察」栏目强调色、标签高亮 |
| 暗色 / 深蓝 | #0a0560 | ![#0a0560](https://via.placeholder.com/16/0a0560/0a0560.png) | 深色背景。用于 Hero 区域、Footer、深色模式基底、渐变叠加 |

#### 扩展色板 — 标签色系

| 色彩名称 | 色值 | 适用标签类别 |
|---|---|---|
| 科技紫 | #7C3AED | 科技 / AI 相关标签 |
| 金融青 | #0891B2 | 金融 / 投资相关标签 |
| 消费琥珀 | #D97706 | 消费 / 零售相关标签 |
| 医疗翡翠 | #059669 | 医疗 / 健康相关标签 |
| 能源青绿 | #0D9488 | 能源 / 环保相关标签 |
| 教育蓝 | #2563EB | 教育相关标签 |

#### 栏目色系

| 栏目 | 色值 | 说明 |
|---|---|---|
| 深度洞察 | #0d0783 | 复旦蓝，核心栏目 |
| 行业观察 | #ea6b00 | 复旦橙，活力栏目 |
| 学术前沿 | #4f46e5 | 靛蓝色（Indigo），学术气质 |
| 院长说 | #b45309 | 深金色（Amber），权威感 |

### 4.2 字体规范

| 字体名称 | 字体族 | 应用场景 | 设计意图 |
|---|---|---|---|

| | | | |
|---|---|---|---|
| Noto Serif SC | 衬线体 (Serif) | 文章标题、专题标题、Hero 大标题 | 传递学术质感与历史厚重感，呼应管理学院的学术底蕴 |
| Inter | 无衬线体 (Sans-serif) | 英文正文、界面元素、按钮文字、元数据 | 现代、清晰、高可读性的 UI 字体，适合屏幕显示 |
| Noto Sans SC | 无衬线体 (Sans-serif) | 中文正文、标签文字、导航菜单、辅助文本 | 与 Inter 风格统一的中文无衬线体，保证中英文混排的视觉和谐 |

### 4.3 动效规范

| 动效库 | 版本要求 | 应用场景 |
|---|---|---|
| Framer Motion | 与 React 19 兼容版本 | 页面切换过渡、卡片悬浮反馈、搜索面板展开/收起、AI 对话面板出入场、标签云动态效果、无限滚动加载动画 |

动效设计原则：

- 克制：动效服务于功能，不喧宾夺主。每个动效应有明确的信息传达目的。
- 流畅：过渡时间控制在 200ms-400ms 之间，使用 ease-out 缓动曲线。
- 一致：同类交互使用相同的动效模式，建立用户的操作预期。

### 4.4 视觉资产

| 元素 | 说明 |
|---|---|
| 主 Logo | 复旦管院 40 周年院徽 — 蓝底金字圆形章 + 复旦大学管理学院文字 |
| 辅助 Logo | 1985-2025 纪念版 — 金色渐变 + 彩色斜线装饰 |
| 图标库 | Lucide React — 与 Tailwind CSS v4 深度整合的开源图标系统 |

## 5. 四大内容栏目

栏目体系是智识库的内容骨架，将 8,431 篇文章组织为结构化的内容频道。栏目设计对标 36氪 的"快讯/深氪/创投"频道体系和财新网的栏目导航。以下栏目定义来源于 backend/config.py 中的 COLUMN_DEFINITIONS 配置。

### 5.1 栏目总览

| 排序 | 栏目名称 | Slug | 强调色 | 图标 (Lucide) | 内容定义 |
|---|---|---|---|---|---|
| 1 | 深度洞察 | insights | #0d0783 (复旦蓝) | Sparkles | 面向深度分析、专访、案例与长文的核心栏目 |
| 2 | 行业观察 | industry | #ea6b00 (复旦橙) | Building2 | 围绕产业、公司、市场与商业模式的持续观察 |
| 3 | 学术前沿 | research | #4f46e5 (靛蓝) | GraduationCap | 复旦管院及相关学者的研究洞见、论文解读与管理新知 |
| 4 | 院长说 | deans-view | #b45309 (深金) | Mic2 | 聚焦复旦管院教授、院长与核心师资的观点表达 |

### 5.2 各栏目详细说明

#### 5.2.1 深度洞察 (Insights)

- Slug: insights
- 图标: Sparkles (Lucide)
- 强调色: #0d0783 — 复旦蓝，作为核心栏目使用品牌主色
- 内容定义: 面向深度分析、专访、案例与长文的核心栏目
- 内容来源: 主要来自 Business Knowledge 数据源
- 筛选策略: 以内容深度为核心标准，优先收录篇幅较长（>3000 字）的深度分析文章、案例研究和专家访谈
- 定位: 智识库中最核心、最高质量的内容频道，代表管院在商业知识领域的深度思考

#### 5.2.2 行业观察 (Industry)

- Slug: industry
- 图标: Building2 (Lucide)
- 强调色: #ea6b00 — 复旦橙，传达行业动态的活力与时效性
- 内容定义: 围绕产业、公司、市场与商业模式的持续观察
- 内容来源: Business Knowledge + WeChat 数据源
- 筛选策略: 按行业标签分类的动态追踪，覆盖科技互联网、金融投资、消费零售、制造业、医疗健康等行业
- 定位: 面向关注行业趋势的 MBA/EMBA 学员和企业家校友，提供持续的行业情报

#### 5.2.3 学术前沿 (Research)

- Slug: research
- 图标: GraduationCap (Lucide)
- 强调色: #4f46e5 — 靛蓝色，传达学术的严谨与深度
- 内容定义: 复旦管院及相关学者的研究洞见、论文解读与管理新知
- 内容来源: News + WeChat 数据源

- 筛选策略: 聚焦教授研究成果、学术论文解读、管理学前沿理论
- 定位: 面向学术研究者和教师，展示管院的科研实力和学术贡献

#### 5.2.4 院长说（Dean's View）

- Slug: deans-view
- 图标: Mic2 (Lucide)
- 强调色: #b45309 — 深金色，传达权威性与领导力
- 内容定义: 聚焦复旦管院教授、院长与核心师资的观点表达
- 内容来源: 主要来自 News 数据源
- 筛选策略: AI 辅助筛选院长、教授的核心演讲、公开发言和观点文章
- 定位: 精品栏目，展示管院核心领导层的思想和洞见，更新频率低但品质极高

### 5.3 栏目导航设计

顶部导航栏按如下方式排列栏目入口：

```
┌──────────────────────────────────────────────────────────────────────────────┐
│ [Logo] 首页 | 深度洞察 | 行业观察 | 学术前沿 | 院长说 | 时光机 |
└──────────────────────────────────────────────────────────────────────────────┘
```

每个栏目页面的 URL 路由为 /column/:slug，例如：

- 深度洞察: /column/insights
- 行业观察: /column/industry
- 学术前沿: /column/research
- 院长说: /column/deans-view

## 6. 八大主题集群

主题专题系统是智识库的知识发现引擎，围绕特定议题将多篇散落文章编织成结构化的知识专题。主题分为两层：8 个人工策划的核心主题（TOPIC_SEEDS）和 8 个 AI 自动聚类的扩展主题（TOPIC_AUTO_CLUSTERS），共 16 个主题集群。以下定义来源于 backend/config.py。

### 6.1 八大核心主题（TOPIC_SEEDS）

核心主题由人工策划，每个主题有明确的标签匹配规则，用于自动收录相关文章。

#### 6.1.1 AI 重构管理

| 字段 | 值 |
|---|---|
| Slug | ai-management |
| 标题 | AI 重构管理 |
| 内容定位 | 聚焦人工智能、大模型与管理实践交汇地带的持续专题 |
| 核心标签 | AI/人工智能、人工智能、ChatGPT、DeepSeek、生成式AI、智能体 |
| 辅助标签 | 人机协同、人机协作、数字化转型 |

#### 6.1.2 ESG 与可持续转型

| 字段 | 值 |
|---|---|
| Slug | esg-sustainability |
| 标题 | ESG 与可持续转型 |
| 内容定位 | 围绕 ESG、双碳与长期价值创造的商业专题 |
| 核心标签 | ESG/可持续、ESG、可持续发展、双碳、碳中和 |
| 辅助标签 | 绿色转型、企业社会责任 |

#### 6.1.3 领导力与组织变革

| 字段 | 值 |
|---|---|
| Slug | leadership-change |
| 标题 | 领导力与组织变革 |
| 内容定位 | 从团队、组织到文化升级的管理专题 |
| 核心标签 | 领导力、组织变革、企业文化、人才管理 |
| 辅助标签 | 组织管理、组织设计、团队管理 |

#### 6.1.4 数字化转型

| 字段 | 值 |
|---|---|

| | |
|---|---|
| Slug | digital-transformation |
| 标题 | 数字化转型 |
| 内容定位 | 聚合数字化、平台化与商业模式重构相关内容 |
| 核心标签 | 数字化转型、数字经济、数智化、大数据、云计算 |
| 辅助标签 | 平台经济、数据资产、智能制造、商业模式 |

### 6.1.5 创业创新

| 字段 | 值 |
|---|---|
| Slug | entrepreneurship-innovation |
| 标题 | 创业创新 |
| 内容定位 | 跟踪创新企业、创业者与新增长方法论的专题 |
| 核心标签 | 创业创新、创业、创业者、企业家精神 |
| 辅助标签 | 创新、技术创新、商业创新、风险投资 |

### 6.1.6 全球化与出海

| 字段 | 值 |
|---|---|
| Slug | globalization-outbound |
| 标题 | 全球化与出海 |
| 内容定位 | 聚焦跨境经营、全球竞争与产业协同的专题 |
| 核心标签 | 全球化、出海、国际化、跨境 |
| 辅助标签 | 供应链、全球供应链、海外运营 |

### 6.1.7 家族企业与传承

| 字段 | 值 |
|---|---|
| Slug | family-business |
| 标题 | 家族企业与传承 |
| 内容定位 | 关于传承治理、代际更替与长期经营的专题 |
| 核心标签 | 家族企业、家族传承、代际传承、家族财富传承、家族企业传承 |
| 辅助标签 | 接班人、传承治理 |

### 6.1.8 品牌与新消费

| 字段 | 值 |
|---|---|
| Slug | brand-consumer |
| 标题 | 品牌与新消费 |
| 内容定位 | 聚焦品牌塑造、消费者洞察与零售创新的专题 |
| 核心标签 | 品牌营销、新零售、消费零售、消费者洞察、消费心理 |
| 辅助标签 | 品牌资产、营销 |

## 6.2 标签匹配机制

每个核心主题的 match_tags 字段包含完整的匹配标签列表（核心标签 + 辅助标签的并集）。系统在文章入库后，会将文章的 AI 生成标签与各主题的 match_tags 进行匹配，命中一定数量标签的文章自动收录进对应主题。

## 6.3 八大自动专题（TOPIC_AUTO_CLUSTERS）

自动专题由 AI 标签聚类引擎驱动，当满足最小文章数阈值时自动生成。每个自动专题定义了 required_tags（必须同时命中的标签）和 support_tags（辅助加权标签），以及文章数量的上下限。

| 序号 | 专题标题 | Slug | 必需标签 | 辅助标签 | 最小文章数 | 文章上限 | 内容定位 |
|---|---|---|---|---|---|---|---|
| 1 | 资本运作与价值管理 | capital-markets-governance | 金融投资、资本市场 | 公司治理、股权激励、上市公司、并购 | 10 | 18 | 围绕资本运作、价值管理与公司治理的自动专题 |
| 2 | 案例教学与管理教育 | case-teaching-management-education | 案例教学、教育 | 管理教育、商学院 | 10 | 18 | 聚焦案例教学、管理教育与课堂实践转化的自动专题 |
| 3 | 智能制造与产业升级 | smart-manufacturing-upgrade | 制造业、数字化转型 | 智能制造、工业4.0、供应链 | 10 | 18 | 聚焦制造业数智化、产业升级与供应链重构的自动专题 |
| 4 | AI 与智慧医疗 | ai-healthcare | AI/人工智能、医疗健康 | 人工智能、智慧医院 | 8 | 16 | 围绕 AI 技术进入医疗健康与智慧医院场景的自 |
| 5 | 风险投资与创业生态 | venture-capital-innovation | 创业创新、金融投资 | 风险投资、企业家精神、创业 | 10 | 18 | 聚焦风险投资、创业方法论与创新生态的自动专题 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 6 | 新能源与绿色产业 | new-energy-green-industry | 能源环保、新能源汽车 | 制造业、碳中和、可持续发展 | 8 | 16 | 围绕新能源产业、绿色转型与制造升级的自动专题 |
| 7 | 中国品牌出海 | consumer-global-expansion | 消费零售、全球化 | 品牌营销、出海、新零售 | 8 | 16 | 聚焦中国消费品牌出海与全球化增长路径的自动专题 |
| 8 | 科技创业与创新生态 | tech-startup-ecosystem | 科技互联网、创业创新 | AI/人工智能、风险投资、数字化转型 | 10 | 18 | 围绕科技创业、创新生态与平台演化的自动专题 |

### 6.4 核心主题 vs 自动专题的区别

| 维度 | 核心主题（TOPIC_SEEDS） | 自动专题（TOPIC_AUTO_CLUSTERS） |
|---|---|---|
| 策划方 | 人工策划，编辑主导 | AI 标签聚类自动生成 |
| 标签逻 | primary_tags + support_tags 合并为 match_tags，命中任一即可收录 | required_tags 必须全部命中，support_tags 提供加权 |
| 文章数 | 无硬性上下限 | 有 min_articles（最低门槛）和 article_limit（上限） |
| 更新方 | 持续自动更新 | 满足阈值后自动激活，持续追加 |
| 内容定位 | 宽泛的大主题 | 交叉领域的细分专题 |

## 7. 三级会员体系

智识库实现了 四种用户角色（guest / free_member / paid_member / admin）的分级访问控制体系，对标财新网的内容分层解锁模型。

### 7.1 角色总览

| 角色标识 | 角色名称 | 获取方式 | 定位 |
|---|---|---|---|
| guest | 游客 | 无需注册，直接访问 | 内容体验层，引导注册转化 |
| free_member | 免费会员 | 注册账号即可（Supabase Auth） | 读者留存与资产沉淀入口 |
| paid_member | 付费会员 | 订阅月度或年度付费方案 | 高价值内容消费的核心用户 |
| admin | 管理员 | 通过 ADMIN_EMAILS 环境变量配置 | 系统运维与内容管理 |

### 7.2 各角色权限矩阵

| 功能 | 游客（guest） | 免费会员（free_member） | 付费会员（paid_member） | 管理员（admin） |
|---|---|---|---|---|
| 浏览首页 / 栏目 / 标签 | 可以 | 可以 | 可以 | 可以 |
| 基础文章搜索 | 可以 | 可以 | 可以 | 可以 |
| 阅读免费文章全文 | 可以 | 可以 | 可以 | 可以 |
| 保留阅读历史 | 不可以 | 可以 | 可以 | 可以 |
| 点赞与收藏文章 | 不可以 | 可以 | 可以 | 可以 |
| 解锁会员音频/视频基础内容 | 不可以 | 可以 | 可以 | 可以 |
| 解锁付费文章 | 不可以 | 不可以 | 可以 | 可以 |
| 解锁付费音频/视频 | 不可以 | 不可以 | 可以 | 可以 |
| 专题专报 / 课程 / 闭门简报 | 不可以 | 不可以 | 可以 | 可以 |
| AI 对话助理 | 受限次数 | 基础额度 | 完整额度 | 无限制 |
| 管理后台（文章管理、标签管理、专题管理） | 不可以 | 不可以 | 不可以 | 可以 |
| 批量标签生成 / 专题生成 | 不可以 | 不可以 | 不可以 | 可以 |
| 用户管理 | 不可以 | 不可以 | 不可以 | 可以 |

### 7.3 会员升级路径

```
游客 (guest)
 |
 | 注册账号 (Supabase Auth)
 v
免费会员 (free_member)
 |
 | 订阅付费方案 (月度 / 年度)
 v
付费会员 (paid_member)
```

### 7.4 认证技术方案

- 认证服务: Supabase Auth（托管认证，支持邮箱/密码、OAuth 等方式）
- 配置项:
- SUPABASE_URL — Supabase 项目地址
- SUPABASE_ANON_KEY — Supabase 匿名密钥

- SUPABASE_AUTH_TIMEOUT_SECONDS — 认证超时时间（默认 8 秒）
- ADMIN_EMAILS — 管理员邮箱列表（逗号分隔）
- DEV_AUTH_ENABLED — 开发环境认证旁路开关（默认关闭）

# 8. 计费方案

计费体系定义于 backend/config.py 的 BILLING_PLAN_DEFINITIONS 中，当前包含三个订阅方案。支付功能通过 PAYMENTS_ENABLED 环境变量控制开关，支付提供商通过 PAYMENT_PROVIDER 配置（默认为 mock 模拟模式）。

## 8.1 方案总览

| 方案代码 | 方案名称 | 对应角色 | 价格 | 计费周期 | 排序 |
|---|---|---|---|---|---|
| free_member_monthly | 免费会员 | free_member | 0 元 | 月 | 1 |
| paid_member_monthly | 付费会员月订阅 | paid_member | 30 元/月 | 月 | 2 |
| paid_member_yearly | 付费会员年订阅 | paid_member | 300 元/年 | 年 | 3 |

> 注意: 价格在代码中以「分」为单位存储（price_cents），即 30 元 = 3000 分，300 元 = 30000 分。货币单位为人民币（CNY）。

## 8.2 各方案详细说明

### 8.2.1 免费会员（free_member_monthly）

| 字段 | 值 |
|---|---|
| 方案代码 | free_member_monthly |
| 方案名称 | 免费会员 |
| 对应角色 | free_member |
| 价格 | 0 元 (price_cents: 0) |
| 计费周期 | 月 |
| 一句话定位 | 读者留存与资产沉淀入口 |
| 方案描述 | 适合建立阅读账户、保留收藏点赞、解锁基础会员内容 |
| 包含功能 | |
| | 1. 保留阅读历史、点赞与收藏 |
| | 2. 解锁会员音频和视频基础内容 |
| | 3. 作为付费升级的承接层 |

运营定位: 免费会员是用户注册后的默认状态，核心目标是留存用户并沉淀用户行为数据（阅读历史、收藏偏好），为后续的付费转化和个性化推荐提供基础。

### 8.2.2 付费会员月订阅（paid_member_monthly）

| 字段 | 值 |
|---|---|
| 方案代码 | paid_member_monthly |
| 方案名称 | 付费会员月订阅 |
| 对应角色 | paid_member |
| 价格 | 30 元/月 (price_cents: 3000) |
| 计费周期 | 月 |
| 一句话定位 | 高价值会员内容的主力套餐 |
| 方案描述 | 适合深度商业知识服务、闭门内容、专题专报和付费音视频 |
| 包含功能 | |
| | 1. 解锁付费文章、付费音频和付费视频 |
| | 2. 适合专题专报、课程和闭门简报 |
| | 3. 可继续接入续费、试用和优惠券体系 |

运营定位: 月订阅是付费转化的核心方案，定价 30 元/月兼顾了用户付费意愿和内容价值。支持未来接入试用期（如首月 1 元体验）和优惠券体系。

### 8.2.3 付费会员年订阅（paid_member_yearly）

| 字段 | 值 |
|---|---|
| 方案代码 | paid_member_yearly |
| 方案名称 | 付费会员年订阅 |
| 对应角色 | paid_member |
| 价格 | 300 元/年 (price_cents: 30000) |
| 计费周期 | 年 |
| 一句话定位 | 面向机构高价值用户的年度方案 |

| | |
|---|---|
| 方案描述 | 适合长期跟踪行业、专题化学习和稳定续费运营 |
| 包含功能 | |
| | 1. 包含全年付费内容访问权限 |
| | 2. 适合绑定年度专报、课程和线下活动 |
| | 3. 为后续机构版与席位版打基础 |

运营定位：年订阅方案等效 25 元/月（年付 300 元 vs 月付 360 元），提供约 17% 的折扣激励用户长期锁定。该方案面向机构级用户，为后续推出的企业席位版和批量采购模式奠定基础。

### 8.3 定价策略对比

| 维度 | 月订阅 | 年订阅 | 折扣力度 |
|---|---|---|---|
| 月均成本 | 30 元 | 25 元 (300/12) | 节省约 17% |
| 年度总费用 | 360 元 | 300 元 | 节省 60 元 |
| 适用场景 | 短期试用、个人轻度使用 | 长期跟踪、机构采购、深度学习 | |

### 8.4 支付系统配置

| 配置项 | 环境变量 | 默认值 | 说明 |
|---|---|---|---|
| 支付功能开关 | PAYMENTS_ENABLED | 0 (关闭) | 设为 1 启用支付功能 |
| 支付提供商 | PAYMENT_PROVIDER | mock | 当前为模拟模式，后续可接入微信支付、支付宝等 |

## 附录 A：关键配置文件索引

| 文件路径 | 内容说明 |
|---|---|
| backend/config.py | 核心配置：数据库路径、API 密钥、模型选择、栏目定义、主题定义、计费方案 |
| backend/main.py | FastAPI 应用入口，路由注册，CORS 配置 |
| backend/database.py | 数据库连接池与迁移逻辑 |
| build_knowledge_base.py | ETL 数据清洗入库脚本 |
| create_vector_db_faiss.py | FAISS 向量索引构建脚本 |
| business_knowledge_crawler.py | Business Knowledge 数据源爬虫 |

## 附录 B：技术栈速查

| 层级 | 技术选型 | 版本/说明 |
|---|---|---|
| 前端框架 | React | 19 |
| 构建工具 | Vite | — |
| UI 框架 | Tailwind CSS | v4 |
| 前端路由 | react-router-dom | v7 |
| 动画库 | Framer Motion | — |
| 图标库 | Lucide React | — |
| 数据请求 | @tanstack/react-query | — |
| 后端框架 | FastAPI | Python 3.11 |
| 关系数据库 | SQLite | 约 50 MB |
| 向量数据库 | FAISS | 约 351 MB |
| AI 引擎 | Google Gemini | 2.5 Pro / 2.5 Flash / Embedding-001 |
| 图像生成 | Gemini 3 Pro | Image Preview |
| 认证服务 | Supabase Auth | — |
| 后端部署 | Render | — |
| 前端部署 | Vercel | — |

下一篇: [02 — 系统架构](./02-系统架构.md)

# 02 — 技术架构

复旦管院商业智识库 技术架构全景文档 版本：1.0 | 最后更新：2026-04-07

## 目录


1. [整体架构概览](#1-整体架构概览)
2. [架构关键原则](#2-架构关键原则)
3. [后端技术栈详情](#3-后端技术栈详情)
4. [前端技术栈详情](#4-前端技术栈详情)
5. [部署方案](#5-部署方案)
6. [完整项目目录结构](#6-完整项目目录结构)
7. [环境变量说明](#7-环境变量说明)
8. [依赖清单](#8-依赖清单)


## 1. 整体架构概览

本系统采用 前后端完全分离 的架构设计。前端为 React 单页应用（SPA），部署于 Vercel CDN；后端为 Python FastAPI 服务，部署于 Render 云平台。两者通过 HTTP REST API 通信，认证由 Supabase Auth 统一管理。

### 1.1 架构拓扑图

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                 用户浏览器                 │
│            React 19 SPA (Vite 7)           │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
                  │
                  │ HTTPS
                  │
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│            Vercel CDN / Edge Network            │
│        静态资源托管 · SPA 路由 Fallback · GZIP        │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
                  │
                  │ HTTPS REST API
                  │ (Authorization: Bearer <JWT>)
                  │
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                    FastAPI 后端 (Render)                    │
│                                                                                                                                │
│  ┌──────────────────────────┐  ┌──────────────────────────┐  ┌──────────────────────────┐  ┌──────────────────────────┐  ┌──────────────────────────┐  │
│  │ Auth Router │ │ Articles │ │ Chat Router │ │ Search Router │ │ Admin Router │ │
│  │ /api/auth/ │ │ /api/articles │ │ /api/chat/ │ │ /api/search │ │ /api/admin/* │ │
│  └──────────────────────────┘  └──────────────────────────┘  └──────────────────────────┘  └──────────────────────────┘  └──────────────────────────┘  │
│     │          │          │          │          │                                                                                │
│  ┌──────────┴──────────┴──────────┴──────────┴──────────┐                                                                    │
│  │                 Services 业务逻辑层                 │ │
... (共 39 行，已截断显示)
```

### 1.2 数据流简述

1. 用户请求：浏览器向 Vercel 发起静态资源请求，获取 React SPA 资源包。
2. API 调用：SPA 内部通过 fetch / API 模块向 FastAPI 后端发起 REST 请求，请求头携带 Supabase JWT。
3. 认证校验：后端 supabase_auth_service 使用 Supabase Admin API 校验 JWT，提取用户身份。
4. 业务处理：Router 层委托 Service 层执行业务逻辑（文章检索、AI 对话、编辑工坊等）。
5. 混合检索：RAG 引擎同时调用 FAISS 向量检索和 BM25 关键词检索，融合排序后交由 Gemini 大模型生成回答。
6. 响应返回：JSON 格式响应通过 HTTPS 返回前端，SPA 渲染结果。

## 2. 架构关键原则

### 2.1 前后端解耦

前端和后端是两个完全独立的代码模块，各自拥有独立的包管理（package.json / requirements.txt）、独立的构建流程（Vite build / Uvicorn run）、独立的部署目标（Vercel / Render）。它们仅通过 HTTP REST API 契约进行通信，任何一端的版本升级不会强制要求另一端同步变更。

### 2.2 无状态认证

系统采用 Supabase Auth 提供的 JWT Bearer Token 认证模式。后端不维护任何 session 或 cookie 状态，每一个请求都是自包含的——JWT 中携带了用户 ID、邮箱和过期时间。后端仅负责 校验 token 的有效性（通过 Supabase Admin API），不负责签发 token。这使得后端天然无状态，适合水平扩展。

### 2.3 混合检索（Hybrid Retrieval）

知识问答（Chat）功能采用 FAISS 向量检索 + BM25 关键词检索 的混合检索策略：

| 检索方式 | 技术实现 | 优势 |
|---|---|---|
| 语义检索 | FAISS（faiss-cpu）+ Gemini Embedding | 捕获语义相似性，对同义改写鲁棒 |
| 关键词检索 | rank_bm25（BM25 算法） | 精确匹配专有名词、人名、机构名 |

两路检索结果经过 倒数排名融合（RRF） 或分数归一化后合并排序，最终上下文送入 Gemini 大模型进行 RAG 生成。

### 2.4 渐进增强

系统设计遵循渐进增强理念：

- 认证可选：DEV_AUTH_ENABLED 开关允许开发环境跳过认证，方便本地调试。
- 支付可选：PAYMENTS_ENABLED 开关控制是否启用真实支付流程，PAYMENT_PROVIDER 支持 mock 模式。
- AI 可降级：当 Gemini API 配额耗尽时，系统可回退到纯检索模式，不影响基础文章浏览功能。
- 多 API Key 轮转：GEMINI_API_KEYS 支持逗号分隔的多 Key 配置，实现 Key 轮转和限流规避。

### 2.5 双重数据存储

| 存储层 | 技术 | 存储内容 | 特点 |
|---|---|---|---|
| 关系型 | SQLite | 文章元数据、标签、栏目、专题、用户互动、会员、账单、聊天记录 | ACID 事务、SQL 查询、单文件便携 |
| 向量索引 | FAISS | 文章文本的 Gemini Embedding 向量 | 高维近邻搜索、毫秒级响应 |

SQLite 作为唯一的结构化数据库，通过 Render Persistent Disk 持久化。FAISS 索引文件同样存储在磁盘上，服务启动时加载进内存。

## 3. 后端技术栈详情

### 3.1 运行时与框架

| 组件 | 版本/说明 |
|---|---|
| Python | 3.11+ |
| FastAPI | 最新稳定版，异步高性能 Web 框架，自动生成 OpenAPI 文档 |
| Uvicorn | ASGI 服务器，生产环境运行入口 |
| Pydantic v2 | 数据校验与序列化，FastAPI 的核心依赖 |

### 3.2 数据库

| 组件 | 说明 |
|---|---|
| SQLite | 嵌入式关系型数据库，单文件 fudan_knowledge_base.db（约 270 MB），通过 PRAGMA busy_timeout = 60000 处理并发，timeout=60 秒连接超时 |
| FAISS (faiss-cpu) | Facebook 开源的向量相似度搜索库，CPU 版本，存储 Gemini Embedding 向量索引 |

### 3.3 AI 与检索

| 组件 | 说明 |
|---|---|
| LangChain | LLM 应用编排框架，串联检索链和对话链 |
| langchain-google-genai | LangChain 的 Google Gemini 集成 |
| langchain-community | LangChain 社区集成包（FAISS、BM25 等 Retriever） |
| google-genai | Google 官方 Gemini SDK，用于直接调用 Gemini API |
| rank_bm25 | BM25 关键词检索算法实现（langchain-community 依赖） |

### 3.4 Gemini 模型配置

系统配置了 4 个不同用途的 Gemini 模型：

| 配置变量 | 默认值 | 用途 |
|---|---|---|
| GEMINI_CHAT_MODEL | gemini-2.5-pro | RAG 对话生成，知识问答主模型 |

| | | |
|---|---|---|
| GEMINI_FLASH_MODEL | gemini-2.5-flash | 批量文章处理、摘要生成等高吞吐场景 |
| GEMINI_EMBEDDING_MODEL | models/gemini-embedding-001 | 文本向量化，用于 FAISS 索引构建和查询 |
| GEMINI_EDITORIAL_FORMAT_MODEL | gemini-3-flash-preview（硬编码） | 编辑工坊排版格式化，使用最新预览模型 |

## 3.5 认证与权限

| 组件 | 说明 |
|---|---|
| Supabase | 通过 SUPABASE_URL + SUPABASE_ANON_KEY 连接 Supabase Auth 服务 |
| supabase_auth_service | 自研认证服务层，校验 JWT、提取用户身份、判断管理员权限 |
| ADMIN_EMAILS | 管理员邮箱白名单，逗号分隔 |
| DEV_AUTH_ENABLED | 开发模式认证旁路开关 |

## 3.6 工具库

| 组件 | 说明 |
|---|---|
| python-dotenv | .env 文件加载，环境变量管理 |
| python-multipart | 表单文件上传支持（FastAPI 依赖） |
| requests | HTTP 客户端，用于 Supabase API 调用等外部请求 |

## 3.7 路由模块一览

后端通过 20 个独立的 Router 模块组织 API 端点：

| Router 模块 | 文件 | 职责 |
|---|---|---|
| home | home.py | 根路由、健康检查 |
| auth | auth.py | 用户认证、token 校验 |
| articles | articles.py | 文章 CRUD、列表、详情 |
| search | search.py | 全文搜索、混合检索 |
| chat | chat.py | AI 知识问答对话（RAG） |
| columns | columns.py | 栏目管理（深度洞察/行业观察/学术前沿/院长说） |
| topics | topics.py | 专题管理（8 个种子专题 + 8 个自动聚类专题） |
| tags | tags.py | 标签体系 |
| media | media.py | 媒体文件管理（音频/图片） |
| follows | follows.py | 用户关注/收藏 |
| me | me.py | 当前用户信息 |
| membership | membership.py | 会员等级与权益 |
| billing | billing.py | 账单与支付 |
| commerce | commerce.py | 商业化功能 |
| admin | admin.py | 管理后台接口 |
| analytics | analytics.py | 数据分析仪表盘 |
| editorial | editorial.py | 编辑工坊（AI 排版/格式化） |
| publishing | publishing.py | 发布流程 |
| time_machine | time_machine.py | 时光机（历史版本浏览） |
| organizations | organizations.py | 机构管理 |

## 3.8 服务层一览

后端通过 30+ 个 Service 模块封装核心业务逻辑：

| Service 模块 | 职责 |
|---|---|
| ai_service | Gemini API 调用封装、多 Key 轮转、错误重试 |
| rag_engine | RAG 检索增强生成引擎，融合 FAISS + BM25 |
| catalog_service | 文章目录管理、分页查询、筛选排序（最大单文件，约 56 KB） |
| editorial_service | 编辑工坊核心逻辑，AI 排版与格式化（约 48 KB） |
| media_service | 媒体文件上传/下载/音频库同步 |
| billing_service | 订阅计费、支付处理 |
| membership_service | 会员等级判定、权益校验 |
| user_profile_service | 用户画像管理 |
| supabase_auth_service | Supabase JWT 校验 |
| tag_engine | 标签自动生成与管理 |
| taxonomy_service | 分类体系维护 |

| | |
|---|---|
| topic_engine | 专题自动聚类 |
| content_localization | 内容本地化与国际化 |
| translation_service | 翻译服务 |
| article_ai_output_service | 文章 AI 摘要/标签/结构化输出 |
| article_relayout_service | 文章重排版 |
| article_asset_service | 文章关联资源管理 |
| article_visibility_service | 文章可见性规则 |
| display_markdown_service | Markdown 渲染与展示 |
| markdown_structure_quality_service | Markdown 结构质量评估 |
| summary_preview_service | 摘要预览生成 |
| html_renderer | HTML 渲染引擎 |
| fudan_wechat_renderer | 微信公众号排版渲染 |
| local_media_library | 本地音频库管理 |
| commerce_service | 商业化服务 |
| engagement_service | 用户互动服务 |
| analytics_service | 数据分析服务 |
| follow_service | 关注/收藏服务 |
| user_activity_service | 用户行为追踪 |
| clustering_rules | 文章聚类规则 |
| publishing_service | 发布流程服务 |

# 4. 前端技术栈详情

## 4.1 核心框架

| 组件 | 版本 | 说明 |
|---|---|---|
| React | ^19.2.0 | UI 渲染库，使用最新 React 19 |
| React DOM | ^19.2.0 | DOM 渲染层 |
| Vite | ^7.2.4 | 下一代前端构建工具，开发服务器与生产构建 |
| React Router DOM | ^7.0.0 | SPA 路由管理，支持嵌套路由与动态路由 |

## 4.2 样式与设计系统

| 组件 | 版本 | 说明 |
|---|---|---|
| Tailwind CSS | ^4.1.18 | 原子化 CSS 框架（v4 最新版） |
| @tailwindcss/postcss | ^4.1.18 | Tailwind CSS 的 PostCSS 插件 |
| PostCSS | ^8.5.6 | CSS 后处理工具链 |
| Autoprefixer | ^10.4.23 | CSS 浏览器前缀自动添加 |
| Framer Motion | ^12.23.26 | 声明式动画库，页面过渡与交互动效 |
| Lucide React | ^0.562.0 | 现代图标库，提供 SVG 图标组件 |
| clsx | ^2.1.1 | 条件 className 拼接工具 |
| tailwind-merge | ^3.4.0 | Tailwind 类名智能合并，解决冲突 |

## 4.3 品牌设计体系（CSS 自定义变量）

品牌色与字体定义在 frontend/src/index.css 中，通过 Tailwind CSS v4 的 @theme 指令注册：

```
@theme {
 --color-fudan-blue:   #0d0783;  / 复旦蓝 — 主品牌色 /
 --color-fudan-orange: #ea6b00;  / 复旦橙 — 强调色 /
 --color-fudan-dark:   #0a0560;  / 深蓝 — 标题/深色背景 /
 --color-fudan-mist:   #f5f4f0;  / 雾灰 — 次级背景 /
 --color-fudan-paper:  #fcfbf8;  / 纸白 — 主背景色 /

 --font-sans:  "Inter", "Noto Sans SC", "PingFang SC", "Microsoft YaHei", sans-serif;
 --font-serif: "Noto Serif SC", "Songti SC", serif;
}
```

在 Tailwind 中可通过 text-fudan-blue、bg-fudan-paper 等类名直接使用。

body 背景使用双径向渐变叠加线性渐变，营造纸质学术感：

```
  body {
 background:
  radial-gradient(circle at top left, rgba(13, 7, 131, 0.08), transparent 28%),
  radial-gradient(circle at bottom right, rgba(234, 107, 0, 0.08), transparent 24%),
  linear-gradient(180deg, #fcfbf8 0%, #f7f5ef 100%);
}
```

### 4.4 功能库

| 组件 | 版本 | 说明 |
|---|---|---|
| @supabase/supabase-js | ^2.57.4 | Supabase 客户端 SDK，处理用户认证与 session 管理 |
| react-markdown | ^10.1.0 | Markdown 内容渲染为 React 组件 |
| date-fns | ^4.1.0 | 日期格式化与计算库（Tree-shakable） |

### 4.5 开发与质量工具

| 组件 | 版本 | 说明 |
|---|---|---|
| ESLint | ^9.39.1 | JavaScript/JSX 代码规范检查 |
| @eslint/js | ^9.39.1 | ESLint 官方 JS 规则集 |
| eslint-plugin-react-hooks | ^7.0.1 | React Hooks 规则校验 |
| eslint-plugin-react-refresh | ^0.4.24 | React Fast Refresh 兼容性校验 |
| globals | ^16.5.0 | 全局变量定义（ESLint 辅助） |
| @vitejs/plugin-react | ^5.1.1 | Vite 的 React 支持插件（JSX 转换/HMR） |
| @types/react | ^19.2.5 | React 类型定义（IDE 智能提示） |
| @types/react-dom | ^19.2.3 | React DOM 类型定义 |
| Playwright | ^1.58.2 | 端到端测试框架，验收测试与视觉回归测试 |

### 4.6 页面模块一览

前端包含 24 个页面组件：

| 页面组件 | 功能 |
|---|---|
| HomePage | 首页，精选文章与栏目推荐 |
| ArticlePage | 文章详情阅读页 |
| SearchPage | 全站搜索 |
| ChatPage | AI 知识问答对话 |
| LoginPage | 登录注册 |
| ColumnPage | 栏目文章列表 |
| TopicsPage | 专题列表 |
| TopicPage | 专题详情 |
| TagPage | 标签下文章列表 |
| MembershipPage | 会员中心与订阅 |
| MyLibraryPage | 我的收藏/阅读记录 |
| MyFollowingPage | 我的关注 |
| MediaHubPage | 媒体中心 |
| MediaStudioPage | 媒体创作工坊 |
| EditorialWorkbenchPage | 编辑工坊 |
| CommercialPage | 商业化管理 |
| CommercialLeadsPage | 商业线索 |
| AdminConsolePage | 管理控制台 |
| AdminMembershipsPage | 会员管理 |
| AnalyticsPage | 数据分析仪表盘 |
| TimeMachinePage | 时光机 |
| OrganizationsPage | 机构列表 |
| OrganizationPage | 机构详情 |

## 5. 部署方案

### 5.1 后端部署 — Render

| 配置项 | 值 |
|---|---|

| | |
|---|---|
| 平台 | Render Web Service |
| 运行时 | Python 3.11 |
| 启动命令 | uvicorn backend.main:app --host 0.0.0.0 --port $PORT |
| 健康检查 | GET / 返回 {"status": "ok", "service": "复旦管院商业智识库 API", "scope": "business-only"} |
| 持久化存 | Render Persistent Disk，挂载数据目录（SQLite DB、FAISS 索引、音频文件、上传文件） |
| 环境变量 | 通过 Render Dashboard 配置（见第 7 节） |

**启动流程：**

```
  1. ensure_database_ready()    — 检查 SQLite 数据库就绪，包含所有必要表
2. ensure_runtime_tables()    — 创建运行时表（如缺失）
3. MEDIA_UPLOADS_DIR.mkdir()   — 确保上传目录存在
4. sync_local_audio_library()  — 同步本地音频库元数据
5. FastAPI app 启动          — 注册中间件、挂载静态文件、注册路由
6. Uvicorn 开始监听          — 默认端口 8000（Render 通过 $PORT 覆盖）
```

## 5.2 前端部署 — Vercel

| 配置项 | 值 |
|---|---|
| 平台 | Vercel |
| 框架 | Vite（自动检测） |
| 构建命令 | vite build |
| 输出目录 | dist/ |
| SPA 路由 | Vercel 自动配置 Fallback 到 index.html |
| CDN | Vercel Edge Network 全球分发 |
| 环境变量 | 通过 Vercel Dashboard 配置 VITE_* 前缀变量 |

## 5.3 网络拓扑图

```
  ┌──────────────────┐    ┌──────────────────────────┐    ┌──────────────────────────┐
│  终端用户  │──────────▶│  Vercel CDN  │      │  Supabase Cloud  │
│  (浏览器)  │◀──────────│  (前端静态资源)  │      │  (Auth 认证服务)  │
└──────────────────┘    └──────────────────────────┘    └──────────────────────────┘
    │                          │
    │  API 请求                  JWT 校验  │
    │  (HTTPS)                   (HTTPS)  │
    │                          │
  ▼                          ▼
┌──────────────────────────────────────────────────────────────────────────────────────┐
│                Render Web Service                  │
│                                          │
│  FastAPI (Uvicorn)                          │
│  ├── CORS 中间件 (allow_origins 配置)                  │
│  ├── 静态文件挂载 (/audio-files, /media-uploads)          │
│  ├── 20 个 API Router                        │
│  └── 30+ 个 Service 模块                       │
│                                          │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────────────┐  │
│  │ SQLite DB  │  │ FAISS Index  │  │ Render Persistent Disk  │  │
│  │ (~270 MB)  │  │ (向量索引)  │  │ audio/ · uploads/media/  │  │
│  └──────────────────┘  └──────────────────┘  └──────────────────────────┘  │
│                                          │
│              ┌──────────────────────┐              │
│              │ Google Gemini API  │              │
... (共 28 行，已截断显示)
```

## 5.4 CORS 配置

后端通过 FastAPI CORSMiddleware 配置跨域策略：

```
app.add_middleware(
  CORSMiddleware,
  allow_origins=ALLOWED_ORIGINS or ["*"],
  allow_credentials=True,
  allow_methods=["*"],
  allow_headers=["*"],
)
```

ALLOWED_ORIGINS 通过环境变量指定，多个域名用逗号分隔。如未配置则默认允许所有来源。

# 6. 完整项目目录结构

```
fdsmarticles/
│
├── .env                    # 环境变量配置文件（不提交至 Git）
├── .gitignore              # Git 忽略规则
├── requirements.txt        # Python 依赖清单
├── prepare_push.ps1        # 推送前准备脚本（PowerShell）
├── build_knowledge_base.py # 知识库构建入口脚本
├── business_knowledge_crawler.py # 商业知识数据爬虫
├── create_vector_db_faiss.py # FAISS 向量数据库创建脚本
├── PROJECT_BLUEPRINT.md    # 项目蓝图文档
├── 快速启动.md             # 快速启动指南
│
├── fudan_knowledge_base.db # SQLite 主数据库（~270 MB）
├── fudan_knowledge_base.*.db # 数据库备份快照
│
├── backend/                # ===== 后端 Python 包 =====
│   ├── __init__.py         # 包初始化
│   ├── main.py             # FastAPI 应用入口，路由注册与中间件配置
│   ├── config.py           # 全局配置：环境变量、路径、模型、计费方案、栏目/专题定义
│   ├── database.py         # SQLite 连接管理、数据库就绪检查、运行时表创建
│   │
│   ├── models/             # 数据模型层
│   │   ├── __init__.py
│   │   └── schemas.py      # Pydantic 模型定义（请求/响应 Schema）
│   │
... (共 212 行，已截断显示)
```

# 7. 环境变量说明

## 7.1 后端环境变量

所有后端环境变量通过 python-dotenv 从项目根目录 .env 文件加载，或通过部署平台（Render Dashboard）注入。

### AI 模型配置

| 变量名 | 类型 | 默认值 | 必须 | 说明 |
|---|---|---|---|---|
| GOOGLE_API_KEY | string | "" | 推荐 | Google AI 主 API Key，优先使用 |
| GEMINI_API_KEYS | string | "" | 推荐 | 逗号分隔的多 Gemini API Key 列表，用于轮转 |
| GEMINI_CHAT_MODEL | string | gemini-2.5-pro | 否 | RAG 对话使用的模型 |
| GEMINI_FLASH_MODEL | string | gemini-2.5-flash | 否 | 批量处理使用的快速模型 |
| GEMINI_EMBEDDING_MODEL | string | models/gemini-embedding-001 | 否 | 文本向量化模型 |

> 注意：GEMINI_EDITORIAL_FORMAT_MODEL 硬编码为 gemini-3-flash-preview，不可通过环境变量覆盖。

Key 选择逻辑：PRIMARY_GEMINI_KEY = GOOGLE_API_KEY or GEMINI_API_KEYS[0]。如果设置了 GOOGLE_API_KEY，它将作为主 Key；否则使用 GEMINI_API_KEYS 列表中的第一个。

### 认证与权限

| 变量名 | 类型 | 默认值 | 必须 | 说明 |
|---|---|---|---|---|
| SUPABASE_URL | string | "" | 生产必须 | Supabase 项目 URL（如 https://xxx.supabase.co） |
| SUPABASE_ANON_KEY | string | "" | 生产必须 | Supabase 匿名公钥（anon key） |
| SUPABASE_AUTH_TIMEOUT_SECONDS | float | 8 | 否 | Supabase Auth API 调用超时时间（秒） |
| ADMIN_EMAILS | string | "" | 否 | 管理员邮箱白名单，逗号分隔（如 admin@fudan.edu.cn,ops@example.com） |
| DEV_AUTH_ENABLED | string | "0" | 否 | 设为 "1" 启用开发模式认证旁路，跳过 JWT 校验 |

### 支付与计费

| 变量名 | 类型 | 默认值 | 必须 | 说明 |
|---|---|---|---|---|
| PAYMENTS_ENABLED | string | "0" | 否 | 设为 "1" 启用真实支付流程 |
| PAYMENT_PROVIDER | string | "mock" | 否 | 支付提供商（mock 为模拟模式） |

### 网络与跨域

| 变量名 | 类型 | 默认值 | 必须 | 说明 |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| ALLOWED_ORIGINS | string | "*" | 推荐 | 允许跨域访问的前端域名，逗号分隔（如 https://app.example.com,http://localhost:5173） |
| SITE_BASE_URL | string | http://127.0.0.1:4173 | 否 | 网站基础 URL，用于生成绝对链接 |

### 数据目录

| 变量名 | 类型 | 默认值 | 必须 | 说明 |
|---|---|---|---|---|
| FDSM_DATA_DIR | string | 项目根目录 | 否 | 数据文件根目录（SQLite DB、FAISS 索引、上传文件等的父目录） |

## 7.2 前端环境变量

Vite 项目中，环境变量必须以 VITE_ 为前缀才会被注入到客户端代码中。

| 变量名 | 类型 | 说明 |
|---|---|---|
| VITE_API_BASE_URL | string | 后端 API 基础地址（如 https://api.example.com 或 http://localhost:8000） |
| VITE_SUPABASE_URL | string | Supabase 项目 URL（与后端 SUPABASE_URL 相同） |
| VITE_SUPABASE_ANON_KEY | string | Supabase 匿名公钥（与后端 SUPABASE_ANON_KEY 相同） |

> 安全提示：.env 文件已被 .gitignore 排除，不会提交至版本控制。前端 VITE_ 变量会被编译进客户端 JS 包中，因此只能放入公开安全的值（如 anon key），绝不能放入 service_role key 等私密凭证。

# 8. 依赖清单

## 8.1 Python 依赖（requirements.txt）

| 包名 | 用途说明 |
|---|---|
| fastapi | 高性能异步 Web 框架，基于 Starlette + Pydantic，自动生成 OpenAPI/Swagger 文档 |
| uvicorn | ASGI 服务器，FastAPI 的生产运行时，支持异步并发 |
| pydantic | 数据校验与序列化库（v2），定义 API 请求/响应模型，FastAPI 核心依赖 |
| langchain | LLM 应用编排框架，提供 Chain/Agent/Memory 抽象，串联检索与对话流程 |
| langchain-google-genai | LangChain 官方 Google Gemini 集成，提供 ChatGoogleGenerativeAI 和 GoogleGenerativeAIEmbeddings |
| langchain-community | LangChain 社区集成包，提供 FAISS VectorStore、BM25 Retriever 等第三方集成 |
| faiss-cpu | Facebook AI Similarity Search 的 CPU 版本，高维向量近邻搜索库，用于语义检索 |
| google-genai | Google 官方 Generative AI SDK，直接调用 Gemini API（不经过 LangChain 的场景） |
| python-multipart | 表单数据与文件上传解析库，FastAPI 处理 multipart/form-data 请求时必须 |
| requests | 同步 HTTP 客户端库，用于调用 Supabase Auth API 校验 JWT 等外部请求 |
| python-dotenv | 从 .env 文件加载环境变量到 os.environ，本地开发环境管理配置 |

> 隐式依赖说明：langchain-community 会自动引入 rank_bm25（BM25 关键词检索算法），fastapi 会自动引入 starlette（底层 ASGI 框架）和 pydantic（数据校验）。

## 8.2 Node.js 运行时依赖（dependencies）

| 包名 | 版本 | 用途说明 |
|---|---|---|
| react | ^19.2.0 | Facebook 开源 UI 渲染库，组件化开发范式，使用最新 React 19（支持 Server Components 基础设施） |
| react-dom | ^19.2.0 | React 的 DOM 渲染层，负责将虚拟 DOM 挂载到浏览器 DOM |
| react-router-dom | ^7.0.0 | React 官方路由库 v7，支持嵌套路由、动态路由、路由守卫 |
| @supabase/supabase-js | ^2.57.4 | Supabase JavaScript 客户端 SDK，封装 Auth（登录/注册/JWT 管理）、实时订阅等功能 |
| @tailwindcss/postcss | ^4.1.18 | Tailwind CSS v4 的 PostCSS 插件，将 Tailwind 指令编译为原子 CSS |
| framer-motion | ^12.23.26 | React 声明式动画库，提供 motion 组件、AnimatePresence、布局动画、手势交互 |
| lucide-react | ^0.562.0 | 现代 SVG 图标库的 React 封装，提供 1000+ 个精心设计的图标组件 |
| react-markdown | ^10.1.0 | 将 Markdown 字符串渲染为 React 组件，支持自定义渲染器，用于文章内容展示与 AI 回答渲染 |
| date-fns | ^4.1.0 | 轻量级日期处理库，支持 Tree Shaking（按需导入），用于文章发布日期格式化 |
| clsx | ^2.1.1 | 条件 className 拼接工具，替代手动字符串拼接，常与 tailwind-merge 搭配使用 |
| tailwind-merge | ^3.4.0 | Tailwind CSS 类名智能合并工具，自动解决冲突类名（如 p-2 p-4 → p-4） |

## 8.3 Node.js 开发依赖（devDependencies）

| 包名 | 版本 | 用途说明 |
|---|---|---|
| vite | ^7.2.4 | 下一代前端构建工具，基于 ESBuild（开发）和 Rollup（生产），极速 HMR |
| @vitejs/plugin-react | ^5.1.1 | Vite 的 React 支持插件，提供 JSX 转换、React Fast Refresh（HMR）支持 |
| tailwindcss | ^4.1.18 | 原子化 CSS 框架 v4，通过 PostCSS 集成编译，提供 @theme、@layer 等新指令 |
| postcss | ^8.5.6 | CSS 后处理工具链框架，Tailwind CSS 和 Autoprefixer 的运行基础 |
| autoprefixer | ^10.4.23 | 根据 browserslist 自动为 CSS 属性添加浏览器厂商前缀（-webkit-、-moz- 等） |
| eslint | ^9.39.1 | JavaScript/JSX 静态代码分析与规范检查工具（v9 Flat Config 格式） |
| @eslint/js | ^9.39.1 | ESLint 官方 JavaScript 推荐规则集 |
| eslint-plugin-react-hooks | ^7.0.1 | 检查 React Hooks 使用规范（依赖数组、调用顺序等） |
| eslint-plugin-react-refresh | ^0.4.24 | 确保组件导出符合 React Fast Refresh 要求，避免 HMR 失效 |
| globals | ^16.5.0 | 提供各运行环境（browser、node、es2021 等）的全局变量列表，供 ESLint 使用 |
| @types/react | ^19.2.5 | React 的 TypeScript 类型定义，即使在 JS 项目中也为 IDE 提供智能提示 |
| @types/react-dom | ^19.2.3 | React DOM 的 TypeScript 类型定义 |
| playwright | ^1.58.2 | Microsoft 开源端到端测试框架，支持 Chromium/Firefox/WebKit，用于验收测试与视觉回归 |

## 8.4 NPM Scripts

| 脚本命令 | 作用 |
|---|---|
| npm run dev | 启动 Vite 开发服务器（HMR，默认端口 5173） |
| npm run build | 生产构建，输出到 dist/ 目录 |
| npm run preview | 预览生产构建结果（默认端口 4173） |
| npm run lint | ESLint 代码检查 |
| npm run prelaunch:acceptance | 上线前全量验收测试（Playwright） |
| npm run visual:acceptance | 视觉回归测试 |
| npm run userflow:acceptance | 用户流程测试 |

> 文档结束 | 如需了解更多模块细节，请参阅各模块源代码中的内联注释与 docstring。

# 03 — 后端架构

版本: 2026-04-07 适用代码路径: backend/ 运行时: Python 3.11+ / FastAPI / Uvicorn 数据库: SQLite (单文件，WAL 模式兼容) AI 引擎: Google Gemini (2.5-pro / 2.5-flash / 3-flash-preview / embedding-001)

## 目录


1. [FastAPI 应用初始化](#1-fastapi-应用初始化)
2. [启动三步骤](#2-启动三步骤)
3. [CORS 配置](#3-cors-配置)
4. [静态文件挂载](#4-静态文件挂载)
5. [路由注册](#5-路由注册)
6. [请求认证流程](#6-请求认证流程)
7. [错误处理](#7-错误处理)
8. [Service 模块完整概览](#8-service-模块完整概览)


## 1. FastAPI 应用初始化

应用入口位于 backend/main.py。初始化过程分为 模块级预处理 和 app 对象创建 两个阶段。

### 1.1 模块级预处理

```
from backend.config import ALLOWED_ORIGINS, APP_TITLE, MEDIA_UPLOADS_DIR
from backend.database import ensure_database_ready, ensure_runtime_tables
from backend.services.media_service import sync_local_audio_library

ensure_database_ready()        # 步骤 1：数据库就绪检查
ensure_runtime_tables()        # 步骤 2：运行时表创建与迁移
MEDIA_UPLOADS_DIR.mkdir(parents=True, exist_ok=True)  # 确保上传目录存在
sync_local_audio_library()     # 步骤 3：本地音频同步
```

以上代码在 模块导入阶段 执行，即 Uvicorn 加载 backend.main 时立即运行，早于 app 对象创建。

### 1.2 app 对象创建

```
app = FastAPI(title=APP_TITLE)
```

| 属性 | 值 | 来源 |
|---|---|---|
| title | "复旦管院商业智识库 API" | backend/config.py 中的 APP_TITLE 常量 |

创建后，依次完成以下配置：

1. 添加 CORS 中间件
2. 挂载静态文件目录
3. 注册全部 21 个路由器
4. 定义根路径健康检查端点 GET /

### 1.3 健康检查端点

```
@app.get("/")
def health_check():
    return {
        "status": "ok",
        "service": APP_TITLE,
        "scope": "business-only",
    }
```

### 1.4 开发启动入口

```
if __name__ == "__main__":
uvicorn.run(app, host="0.0.0.0", port=8000)
```

## 2. 启动三步骤

应用启动时按严格顺序执行三个初始化步骤。任何步骤失败将阻止 app 对象创建，服务不会接受请求。

## 2.1 步骤 1 — ensure_database_ready()

文件: backend/database.py

职责: 检查 SQLite 数据库文件是否存在，且包含 11 张核心表。如果缺失或不完整，自动触发全量重建。

### 11 张核心表

| # | 表名 | 职责 |
|---|---|---|
| 1 | articles | 文章主表 |
| 2 | tags | 标签表 |
| 3 | article_tags | 文章-标签关联表 |
| 4 | columns | 栏目表 |
| 5 | article_columns | 文章-栏目关联表 |
| 6 | topics | 专题表 |
| 7 | topic_articles | 专题-文章关联表 |
| 8 | topic_tags | 专题-标签关联表 |
| 9 | featured_articles | 首页推荐文章 |
| 10 | chat_sessions | AI 对话会话 |
| 11 | chat_messages | AI 对话消息 |

### 检查逻辑

```
  REQUIRED_TABLES = { 上述 11 张表 }

def database_is_ready() -> bool:
  1. 检查 SQLITE_DB_PATH 文件是否存在
  2. 查询 sqlite_master 获取所有 table 和 view
  3. 返回 REQUIRED_TABLES.issubset(existing)

def ensure_database_ready(force_rebuild=False):
  如果 force_rebuild 或 not database_is_ready():
    调用 backend.scripts.build_business_db.rebuild_database()
```

### 数据库连接配置

```
def get_connection() -> sqlite3.Connection:
connection = sqlite3.connect(SQLITE_DB_PATH, timeout=60)
connection.row_factory = sqlite3.Row     # 行以字典方式访问
connection.execute("PRAGMA busy_timeout = 60000")  # 等待锁 60 秒
return connection
```

- 连接超时: 60 秒
- busy_timeout: 60000 毫秒
- row_factory: sqlite3.Row（字典访问模式）
- 连接管理: 提供 connection_scope() 上下文管理器，自动关闭连接

## 2.2 步骤 2 — ensure_runtime_tables()

文件: backend/database.py

职责: 创建 14+ 张运行时数据表，执行 ALTER TABLE 迁移以补齐新增字段，播种初始数据。

### 2.2.1 创建的运行时表（14 张）

| # | 表名 | 主要字段 | 用途 |
|---|---|---|---|
| 1 | demo_requests | name, organization, role, email, phone, use_case, message, status | 商务演示申请 |
| 2 | editorial_articles | article_id, slug, title, subtitle, author, source_markdown, content_markdown, html_web, html_wechat, status, workflow_status | 编辑部文章（含排版与工作流） |
| 3 | visitor_profiles | visitor_id, user_id, first_seen_at, last_seen_at | 访客画像 |
| 4 | article_view_events | article_id, visitor_id, user_id, view_date, source | 文章阅读事件（按天去重） |
| 5 | article_reactions | article_id, user_id, reaction_type, is_active | 文章点赞/收藏 |
| 6 | user_follows | user_id, entity_type, entity_slug, entity_label | 用户关注（栏目/专题/标签/作 |

者）

| | | | |
|---|---|---|---|
| 7 | user_memberships | user_id, email, tier, status, expires_at | 会员等级 |
| 8 | business_users | user_id, email, display_name, title, organization, tier, role_home_path, locale | 业务用户档案 |
| 9 | admin_role_audit_logs | target_user_id, actor_user_id, previous_tier, next_tier, note | 管理员角色变更审计日志 |
| 10 | media_items | slug, kind, title, summary, speaker, series_name, episode_number, duration_seconds, visibility, media_url, preview_url | 音频/视频媒体项 |
| 11 | billing_plans | plan_code, name, tier, price_cents, currency, billing_period, headline, features_json | 计费套餐定义 |
| 12 | billing_checkout_intents | user_id, plan_code, status, payment_provider, redirect_url | 支付意向记录 |
| 13 | billing_orders | user_id, plan_code, amount_cents, status, payment_provider | 订单记录 |
| 14 | billing_subscriptions | user_id, plan_code, tier, status, started_at, expires_at, auto_renew | 订阅记录 |
| 15 | article_translations | article_id, target_lang, source_hash, title, summary, content, model | 文章翻译缓存 |
| 16 | article_ai_outputs | article_id, source_hash, summary_zh, formatted_markdown_zh/en, html_web_zh/en, summary_status, format_status, translation_status | AI 生成内容中心表 |

### 2.2.2 ALTER TABLE 迁移

使用 _table_has_column() 辅助函数判断列是否存在后，条件性执行 ALTER TABLE ADD COLUMN。这保证了对已有数据库的 向前兼容 迁移，无需手动 schema 管理工具。

迁移覆盖的表和字段：

| 目标表 | 新增字段 |
|---|---|
| articles | access_level |
| editorial_articles | access_level, source_article_id, workflow_status, review_note, scheduled_publish_at, submitted_at, approved_at, ai_synced_at, source_markdown, layout_mode, formatting_notes, formatter_model, last_formatted_at |
| media_items | series_name, episode_number, media_url, preview_url, transcript_markdown, chapter_payload_json |
| business_users | description |
| article_ai_outputs | translation_title_en, translation_excerpt_en, translation_summary_en, summary_html_zh, summary_html_en, translation_content_en, summary_status, format_status, translation_status, summary_error, format_error, translation_error |
| chat_messages | follow_ups_json |

### 2.2.3 数据修复与补齐

迁移之后，还执行一系列 UPDATE 语句进行数据修复：

- articles.access_level 空值回填为 'public'
- editorial_articles.access_level 空值回填为 'public'
- editorial_articles.source_markdown 空值回填为 content_markdown
- editorial_articles.layout_mode 空值回填为 'auto'
- editorial_articles.workflow_status 与 status 同步（已发布文章同步为 'published'）
- media_items 回填 series_name、episode_number、preview_url、transcript_markdown、chapter_payload_json

### 2.2.4 种子数据

- billing_plans: 从 config.py 的 BILLING_PLAN_DEFINITIONS 插入 3 个计费套餐（free_member_monthly / paid_member_monthly / paid_member_yearly），使用 ON CONFLICT DO UPDATE 实现幂等
- media_items: 当表为空时，播种 6 个媒体样本（3 个音频 + 3 个视频），分别覆盖 public / member / paid 三种可见性
- business_users: 调用 ensure_business_user_seed_state() 播种种子用户

## 2.3 步骤 3 — sync_local_audio_library()

文件: backend/services/media_service.py

职责: 扫描项目根目录下的 audio/ 文件夹，将本地音频文件同步到 media_items 表中。

执行逻辑：

1. 检查 audio/ 目录是否存在，不存在则直接返回
2. 删除旧的种子音频记录（slug 匹配 LEGACY_AUDIO_SEED_SLUGS 且 media_url 为空）
3. 遍历本地音频文件，插入或更新 media_items 记录

## 3. CORS 配置

文件: backend/main.py

```
app.add_middleware(
CORSMiddleware,
allow_origins=ALLOWED_ORIGINS or ["*"],
allow_credentials=True,
allow_methods=["*"],
allow_headers=["*"],
)
```

### 配置项详解

| 参数 | 值 | 说明 |
|---|---|---|
| allow_origins | ALLOWED_ORIGINS 或 ["*"] | 从环境变量 ALLOWED_ORIGINS 读取，逗号分隔；未设置时允许所有来源 |
| allow_credentials | True | 允许发送 Cookie 和 Authorization 头 |
| allow_methods | ["*"] | 允许所有 HTTP 方法 |
| allow_headers | ["*"] | 允许所有请求头 |

### 环境变量

```
# .env 文件
ALLOWED_ORIGINS=http://localhost:5173,http://localhost:4173,https://fdsm.example.com
```

ALLOWED_ORIGINS 在 backend/config.py 中解析：

```
ALLOWED_ORIGINS = [
origin.strip()
for origin in os.getenv("ALLOWED_ORIGINS", "*").split(",")
if origin.strip()
]
```

注意：当 allow_credentials=True 时，浏览器不接受 allow_origins=["*"]，此时应配置具体的域名列表。代码中使用 ALLOWED_ORIGINS or ["*"] 确保环境变量为空时不会产生空列表。

## 4. 静态文件挂载

文件: backend/main.py

```
audio_directory = Path(__file__).resolve().parent.parent / "audio"

app.mount("/audio-files", StaticFiles(directory=audio_directory, check_dir=False), name="audio-files")
app.mount("/media-uploads", StaticFiles(directory=MEDIA_UPLOADS_DIR, check_dir=False), name="media-uploads")
```

| URL 前缀 | 文件系统路径 | 用途 | check_dir |
|---|---|---|---|
| /audio-files | <项目根>/audio/ | 本地音频文件直接访问 | False |
| /media-uploads | <DATA_DIR>/uploads/media/ | 用户/管理员上传的媒体文件 | False |

check_dir=False 表示即使目录不存在也不抛出启动异常，这对于首次部署时目录尚未创建的情况是必要的。

MEDIA_UPLOADS_DIR 的定义路径：

```
# backend/config.py
DATA_DIR = Path(os.getenv("FDSM_DATA_DIR", BASE_DIR))
UPLOADS_DIR = DATA_DIR / "uploads"
MEDIA_UPLOADS_DIR = UPLOADS_DIR / "media"
```

## 5. 路由注册

backend/main.py 中通过 app.include_router() 注册了 21 个路由器（其中 publishing_router 无前缀）。路由器按以下顺序注册——注册顺序决定了路径匹配优先级。

### 5.1 路由器完整列表

| # | 变量名 | 前缀 | Tag | 主要职责 |
|---|---|---|---|---|
| 1 | home_router | /api | home | 首页 Feed 聚合（/api/home/feed），返回栏目、推荐文章、专题等聚合数据 |
| 2 | auth_router | /api/auth | auth | 认证状态查询（/status）、密码登录（/login）；返回用户信息、会员等级、角色首页路径 |
| 3 | billing_router | /api/billing | billing | 计费套餐列表（/plans）、用户计费概况（/me）、创建支付意向（/checkout-intent） |
| 4 | membership_router | /api/membership | membership | 当前用户会员等级查询（/me） |
| 5 | follows_router | /api/follows | follows | 用户关注功能：查看关注列表、切换关注状态、获取 watchlist |
| 6 | admin_router | /api/admin | admin | 管理后台：系统概览（/overview）、会员管理（/memberships）、会员角色变更、计费订单查询 |
| 7 | analytics_router | /api/analytics | analytics | 数据分析仪表盘：站点分析概览（/overview），仅管理员可访问 |
| 8 | me_router | /api/me | me | 当前用户仪表盘（/dashboard）、个人阅读库（/library） |
| 9 | media_router | /api/media | media | 媒体管理：管理列表、公开列表、创建/更新/上传媒体项、音频视频 Hub |
| 10 | organizations_router | /api | organizations | 机构列表（/organizations）与详情页（/organizations/{slug}） |
| 11 | search_router | /api | search | 全文搜索（/search，分精确/智能模式）与自动补全（/suggest） |
| 12 | articles_router | /api | articles | 文章列表（最新/热门）、文章详情、封面图、翻译、AI 摘要、互动数据、点赞/收藏 |
| 13 | tags_router | /api | tags | 标签列表（/tags）、标签云（/tags/cloud）、按标签筛选文章、批量标签生成 |
| 14 | columns_router | /api | columns | 栏目列表（/columns）与栏目文章列表 |
| 15 | topics_router | /api | topics | 专题列表与详情（付费会员专属）、专题时间线、专题洞察、自动专题生成 |
| 16 | chat_router | /api | chat | AI 对话：多轮对话、会话管理、每日阅读、文章推荐；支持 /brief、/compare、/timeline 等指令 |
| 17 | time_machine_router | /api | time-machine | 时光机功能（/time_machine）：按指定日期查看历史内容 |
| 18 | commerce_router | /api/commerce | commerce | 商务功能：商务概览、提交/查看/导出演示申请 |
| 19 | editorial_router | /api/editorial | editorial | 编辑部全流程：创建/编辑/排版/审核/发布文章、AI 标签生成、HTML 导出、文件上传、AI 输出管理 |
| 20 | publishing_router | (无前缀) | publishing | SEO 与分发：/sitemap.xml、/rss.xml、/robots.txt |
| 21 | (内联) | / | (无) | 根路径健康检查 GET / |

## 5.2 路由分组示意

```
/
├── GET /                          # 健康检查
├── /sitemap.xml                   # publishing
├── /rss.xml                       # publishing
├── /robots.txt                    # publishing
├── /audio-files/...               # 静态文件
├── /media-uploads/...             # 静态文件
└── /api/
  ├── home/feed                    # home
  ├── auth/
  │  ├── status                    # auth
  │  └── login                     # auth
  ├── billing/
  │  ├── plans                     # billing
  │  ├── me                        # billing
  │  └── checkout-intent           # billing
  ├── membership/me                # membership
  ├── follows/...                  # follows
  ├── admin/
  │  ├── overview                  # admin
  │  ├── memberships               # admin
  │  ├── memberships/{user_id}     # admin
  │  └── billing/orders            # admin
  ├── analytics/overview           # analytics
  ├── me/
... (共 75 行，已截断显示)
```

# 6. 请求认证流程

## 6.1 认证核心函数

认证逻辑集中在 backend/services/supabase_auth_service.py 的 get_authenticated_user() 函数中。

## 6.2 完整流程图

```
客户端请求
 │
 ▼
┌──────────────────────────────────────────────────────────┐
│ 提取 Authorization Header      │
│ 及 X-Debug-User-Id/Email Header │
└──────────────────────────────────────────────────────────┘
      │
      ▼
┌──────────────────────────────────────────────────────────┐
│ is_supabase_auth_enabled()?    │
│ 检查 SUPABASE_URL 和          │
│ SUPABASE_ANON_KEY 是否配置     │
└──────────────────────────────────────────────────────────┘
    │ 未配置      │ 已配置
    ▼           ▼
┌──────────────────────┐  ┌──────────────────────────────────┐
│ DEV_AUTH  │  │ _extract_bearer_token │
│ 走开发模式 │  │ 从 Authorization 提取 │
│ (见 6.3)  │  │ Bearer token        │
└──────────────────────┘  └──────────────────────────────────┘
            │
        ┌──────────────────┐
        │ token 为空?  │
        ├─── 是 ──────────── DEV_AUTH 降级 (见 6.3)
... (共 61 行，已截断显示)
```

### 6.3 DEV_AUTH 开发模式

当 DEV_AUTH_ENABLED=1 且请求携带 X-Debug-User-Id 头时，跳过 Supabase 验证，直接构造调试用户：

```
def _build_debug_user(debug_user_id, debug_user_email):
if not DEV_AUTH_ENABLED or not debug_user_id:
    return None
return {
    "id": debug_user_id.strip(),
    "email": debug_user_email.strip() if debug_user_email else None,
    "raw_user": {"debug": True},
}
```

触发条件：

1. Supabase 未配置 → 直接走 DEV_AUTH
2. Supabase 已配置但无 Bearer token → 降级走 DEV_AUTH
3. DEV_AUTH_ENABLED=0 → 返回 None（匿名用户）

### 6.4 认证状态聚合

get_auth_status_payload() 是认证的高阶封装，在 /api/auth/status 中调用，返回完整的认证态：

```
{
"enabled": bool,        # 认证系统是否启用
"authenticated": bool,   # 当前用户是否已认证
"user": {"id", "email"}, # 用户基本信息
"auth_mode": "password" | "email_otp", # 认证模式
"membership": {...},     # 会员等级信息
"business_profile": {...},# 业务角色信息
"role_home_path": str,   # 角色首页路径
}
```

### 6.5 权限守卫函数

各路由器中通过以下模式进行权限检查：

| 守卫函数 | 所在模块 | 行为 |
|---|---|---|
| require_admin_profile(user) | membership_service | 验证管理员权限，失败抛 403 |
| require_paid_profile(user) | membership_service | 验证付费会员，失败抛 403 |
| 内联 if not user: | 各路由器 | 未登录用户抛 401 |

## 7. 错误处理

### 7.1 HTTPException 使用规范

项目全局使用 FastAPI 的 HTTPException 进行错误返回，按状态码分类：

| 状态码 | 用途 | 典型场景 |
|---|---|---|
| 400 Bad Request | 参数校验失 | 邮箱格式错误（commerce）、翻译参数异常（articles）、无效的媒体类型 |
| 401 Unauthorized | 未认证 | 需要登录的操作（点赞、收藏、个人库）、密码登录失败 |
| 403 Forbidden | 权限不足 | 非管理员访问管理接口（require_admin_profile）、非付费会员访问智能搜索（require_paid_profile） |
| 404 Not Found | 资源不存在 | 文章/媒体/机构/专题不存在 |
| 503 Service Unavailable | 外部服务不可用 | AI 翻译服务调用失败 |

## 7.2 AI 服务容错机制

文件: backend/services/ai_service.py

AI 服务基于 Google Gemini API，实现了三层容错：

### 7.2.1 多 Key 轮转

```
GEMINI_API_KEYS = [item.strip() for item in os.getenv("GEMINI_API_KEYS", "").split(",")]
PRIMARY_GEMINI_KEY = GOOGLE_API_KEY or (GEMINI_API_KEYS[0] if GEMINI_API_KEYS else "")
```

- 支持配置多个 API Key，通过 GEMINI_API_KEYS 环境变量传入（逗号分隔）
- get_gemini_api_keys() 合并去重所有 Key，返回不可变元组
- 请求时使用全局计数器 _gemini_request_counter 轮转起始 Key
- 每次重试自动切换到下一个 Key：api_key = api_keys[(start_offset + attempt) % len(api_keys)]
- 最大重试次数：max(3, len(api_keys) * 2)

### 7.2.2 指数退避重试

```
_GEMINI_RETRYABLE_STATUS = {429, 500, 502, 503, 504}
_GEMINI_TIMEOUT = (10, 90)  # (connect_timeout, read_timeout)

for attempt in range(max_attempts):
    try:
        response = session.post(url, json=payload, timeout=_GEMINI_TIMEOUT)
        if response.status_code == 200:
            return result
        if response.status_code in _GEMINI_RETRYABLE_STATUS:
            time.sleep(min(4.0, 0.6 * (attempt + 1)))  # 线性退避，上限 4 秒
            continue
        raise RuntimeError(...)  # 不可重试的错误直接抛出
    except requests.RequestException:
        time.sleep(min(4.0, 0.6 * (attempt + 1)))
        continue
```

可重试状态码: 429 (Rate Limit)、500、502、503、504

退避策略: min(4.0, 0.6 * (attempt + 1))，即 0.6s → 1.2s → 1.8s → 2.4s → ... → 4.0s（上限）

### 7.2.3 优雅降级

```
# 1. AI 未配置时的检查
def is_ai_enabled() -> bool:
    return bool(get_gemini_api_keys())

# 2. 搜索查询扩展降级
def expand_query(query):
    if not is_ai_enabled():
        return []        # AI 不可用时返回空列表，不影响基础搜索
    try:
        ...
    except Exception:
        return []        # 任何异常都不中断搜索

# 3. 摘要降级
# 如果 AI 预生成摘要可用且 hash 匹配，直接返回缓存
# 否则实时调用 AI 生成（可能失败时向上层抛 503）
```

## 7.3 数据库错误处理

- 连接超时: sqlite3.connect(timeout=60) + PRAGMA busy_timeout = 60000，等待写锁最长 60 秒
- 连接管理: 所有数据库操作通过 connection_scope() 上下文管理器，确保连接关闭
- 幂等迁移: CREATE TABLE IF NOT EXISTS + _table_has_column() 检查，重复执行不报错
- 事务: ensure_runtime_tables() 在最后统一调用 connection.commit()

### 7.4 JSON 解析容错

AI 返回内容解析使用多级降级策略：

```
def _parse_json_payload(raw: str):
text = _strip_code_fence(raw)    # 1. 去除 ``` 代码围栏
try:
    return json.loads(text)      # 2. 直接解析
except json.JSONDecodeError:
    # 3. 尝试提取第一个 JSON 数组
    # 4. 尝试提取第一个 JSON 对象
    raise
```

## 8. Service 模块完整概览

backend/services/ 目录包含 33 个 Python 文件（含 __init__.py），按职责分为 9 大类。

### 8.1 按职责分组

#### 8.1.1 认证与用户管理

| 文件 | 用途 |
|---|---|
| supabase_auth_service.py | Supabase 认证集成：Bearer token 验证、DEV_AUTH 模式、认证状态聚合 |
| user_profile_service.py | 业务用户档案管理：种子用户创建、仪表盘数据、管理员概览、角色首页路径计算、密码登录（preview 账户） |
| membership_service.py | 会员体系：会员等级查询/更新、权限守卫（require_admin_profile / require_paid_profile）、审计日志、会员列表 |

#### 8.1.2 内容检索与 AI

| 文件 | 用途 |
|---|---|
| ai_service.py | Gemini AI 核心封装：多 Key 轮转、重试退避、提示词调用、摘要生成、查询扩展、标签推荐、格式化指令 |
| rag_engine.py | RAG 检索增强生成引擎：FAISS 向量搜索、精确搜索、混合排序、搜索建议 |
| translation_service.py | AI 翻译服务：文章多语言翻译（中→英）、翻译缓存管理、hash 校验 |
| content_localization.py | 内容本地化：CJK 检测、语言标记、多语言文本处理工具集 |

#### 8.1.3 文章内容管理

| 文件 | 用途 |
|---|---|
| catalog_service.py | 最大的 Service 文件 (56KB)：文章列表/详情/封面、首页 Feed、栏目/专题/标签/机构数据聚合、对话会话管理、时光机、推荐文章 |
| article_asset_service.py | 文章资源管理：封面图路径解析与资源定位 |
| article_visibility_service.py | 文章可见性服务：根据 access_level 和会员等级决定内容可见性 |
| summary_preview_service.py | 摘要预览服务：判断 HTML 摘要是否有效、将 Markdown 摘要渲染为 HTML 预览 |
| display_markdown_service.py | 展示 Markdown 服务：标准化摘要的显示 Markdown 格式 |
| article_ai_output_service.py | 文章 AI 输出管理：查询/管理 article_ai_outputs 表中的 AI 生成内容（摘要、格式化、翻译状态） |

#### 8.1.4 编辑与发布

| 文件 | 用途 |
|---|---|
| editorial_service.py | 编辑部全流程 (48KB)：编辑文章 CRUD、自动排版（AI 驱动）、HTML 生成（Web/微信）、工作流（draft→submitted→approved→published）、AI 导入、源文章管理 |
| article_relayout_service.py | 文章重排版：AI 驱动的 Markdown 结构化重排版，支持多种 layout_mode |
| publishing_service.py | SEO 发布：生成 sitemap.xml、rss.xml、robots.txt |
| html_renderer.py | HTML 渲染器：将 Markdown 渲染为适配 Web 和微信公众号的 HTML |
| fudan_wechat_renderer.py | 复旦微信渲染器：针对微信公众号的专用 HTML 渲染模板 |
| markdown_structure_quality_service.py | Markdown 结构质量评估：分析文档结构、标题层级、段落质量 |

#### 8.1.5 标签与分类

| 文件 | 用途 |
|---|---|
| tag_engine.py | 标签引擎：AI 驱动的自动标签生成、批量标签生成、标签标准化 |

| | |
|---|---|
| taxonomy_service.py | 分类体系服务：标签分类法、层级关系管理、标签规范化映射 |
| clustering_rules.py | 聚类规则：自动专题的标签匹配与聚类逻辑配置 |

### 8.1.6 主题与专题

| 文件 | 用途 |
|---|---|
| topic_engine.py | 专题引擎：自动专题生成、基于标签匹配的文章聚合、专题时间线构建 |

### 8.1.7 媒体管理

| 文件 | 用途 |
|---|---|
| media_service.py | 媒体全流程：音频/视频 CRUD、文件上传（支持 mp3/wav/m4a/aac/ogg/mp4/mov/m4v/webm）、本地音频同步、媒体 Hub 列表 |
| local_media_library.py | 本地媒体库：扫描本地 audio/ 目录，构建内存中的音频项列表 |

### 8.1.8 用户互动与行为

| 文件 | 用途 |
|---|---|
| engagement_service.py | 互动服务：文章浏览记录、点赞/收藏（toggle 语义）、互动数据统计 |
| follow_service.py | 关注服务：关注/取关（栏目/专题/标签/作者）、关注列表、watchlist 聚合 |
| user_activity_service.py | 用户活跃度：阅读历史、个人图书馆数据 |
| analytics_service.py | 分析服务：站点数据分析概览（仅管理员） |

### 8.1.9 会员与计费

| 文件 | 用途 |
|---|---|
| billing_service.py | 计费服务 (15KB)：套餐管理、支付意向创建、订单管理、订阅生命周期、支持 mock 支付提供商 |
| commerce_service.py | 商务服务：商务概览统计、演示申请 CRUD、CSV 导出 |

## 8.2 Service 依赖关系图

下图展示 Service 模块之间的主要调用关系。箭头方向表示"依赖/调用"。

```
          ┌──────────────────────────────────────────────────────────┐
          │   supabase_auth_service   │
          │  (认证入口)              │
          └──────────────────────────┬──────────────────────────────┘
                                     │        │
          ┌──────────────────────────▼────┐  ┌──────────▼──────────────────┐
          │membership │  │ user_profile    │
          │_service   │──│ _service        │
          │(会员等级) │  │(业务档案/仪表盘) │
          └──────────┬────────────────────┘  └──────────────────────────────┘
                     │
    ┌────────────────┼──────────────────────────────────────────────────────┐
    │        │                │
    ▼        ▼                ▼
┌──────────────────────────┐  ┌──────────────────────────┐    ┌──────────────────────────┐
│ billing    │ │ admin/     │      │ commerce    │
│ _service   │ │ analytics  │      │ _service    │
│ (计费/订阅) │ │ _service   │      │ (商务/Demo) │
└──────────────────────────┘  └──────────────────────────┘    └──────────────────────────┘

┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                catalog_service                 │
│  (文章/栏目/专题/标签/机构/Chat/Feed/时光机 — 数据中枢)  │
└──────────────────────┬──────────────────────┬──────────────────────────┬──────────────────────┘
     │       │        │        │
... (共 116 行，已截断显示)
```

## 8.3 核心 Service 文件规模

| 文件 | 大小 | 行数级别 | 备注 |
|---|---|---|---|
| catalog_service.py | 56 KB | ~1500 行 | 数据中枢，覆盖几乎所有读取操作 |
| editorial_service.py | 48 KB | ~1200 行 | 编辑部完整工作流 |
| user_profile_service.py | 32 KB | ~800 行 | 用户档案与仪表盘 |
| display_markdown_service.py | 25 KB | ~650 行 | Markdown 展示处理 |
| media_service.py | 22 KB | ~550 行 | 媒体管理全流程 |
| article_relayout_service.py | 17 KB | ~430 行 | AI 排版 |
| html_renderer.py | 16 KB | ~400 行 | HTML 渲染 |
| rag_engine.py | 16 KB | ~400 行 | RAG 搜索引擎 |
| membership_service.py | 15 KB | ~400 行 | 会员体系 |
| billing_service.py | 15 KB | ~400 行 | 计费系统 |
| ai_service.py | 15 KB | ~400 行 | AI 基础设施 |
| taxonomy_service.py | 13 KB | ~350 行 | 分类体系 |
| content_localization.py | 13 KB | ~330 行 | 本地化工具 |
| topic_engine.py | 11 KB | ~280 行 | 专题引擎 |
| translation_service.py | 10 KB | ~270 行 | 翻译服务 |
| article_ai_output_service.py | 9 KB | ~230 行 | AI 输出管理 |
| tag_engine.py | 8 KB | ~220 行 | 标签引擎 |
| follow_service.py | 8 KB | ~210 行 | 关注系统 |
| summary_preview_service.py | 7 KB | ~180 行 | 摘要预览 |
| markdown_structure_quality_service.py | 6 KB | ~170 行 | Markdown 质量 |
| engagement_service.py | 6 KB | ~160 行 | 互动服务 |
| commerce_service.py | 4 KB | ~120 行 | 商务功能 |
| fudan_wechat_renderer.py | 4 KB | ~110 行 | 微信渲染 |
| analytics_service.py | 4 KB | ~110 行 | 分析服务 |
| publishing_service.py | 4 KB | ~110 行 | SEO 发布 |
| local_media_library.py | 3 KB | ~90 行 | 本地媒体库 |
| supabase_auth_service.py | 3 KB | ~107 行 | 认证核心 |
| clustering_rules.py | 2 KB | ~65 行 | 聚类规则 |
| article_visibility_service.py | 2 KB | ~60 行 | 可见性 |
| article_asset_service.py | 2 KB | ~50 行 | 文章资源 |
| user_activity_service.py | 2 KB | ~50 行 | 用户活跃度 |

## 8.4 环境变量速查

| 变量名 | 默认值 | 用途 |
|---|---|---|
| FDSM_DATA_DIR | 项目根目录 | 数据目录（数据库、FAISS 索引、上传文件） |

| | | |
|---|---|---|
| GOOGLE_API_KEY | "" | 主 Gemini API Key |
| GEMINI_API_KEYS | "" | 多 Key 列表（逗号分隔） |
| GEMINI_CHAT_MODEL | gemini-2.5-pro | 对话/摘要模型 |
| GEMINI_FLASH_MODEL | gemini-2.5-flash | 轻量级快速模型 |
| GEMINI_EMBEDDING_MODEL | models/gemini-embedding-001 | 向量嵌入模型 |
| ALLOWED_ORIGINS | * | CORS 允许来源 |
| SITE_BASE_URL | http://127.0.0.1:4173 | 站点基础 URL |
| SUPABASE_URL | "" | Supabase 项目 URL |
| SUPABASE_ANON_KEY | "" | Supabase 匿名 Key |
| SUPABASE_AUTH_TIMEOUT_SECONDS | 8 | Supabase 认证超时（秒） |
| ADMIN_EMAILS | "" | 管理员邮箱列表（逗号分隔） |
| DEV_AUTH_ENABLED | 0 | 开发模式认证开关 |
| PAYMENTS_ENABLED | 0 | 支付功能开关 |
| PAYMENT_PROVIDER | mock | 支付提供商（mock/实际） |

## 8.5 AI 模型配置

| 用途 | 模型 | 配置来源 |
|---|---|---|
| 对话与摘要 | gemini-2.5-pro | GEMINI_CHAT_MODEL |
| 标签生成与轻量任务 | gemini-2.5-flash | GEMINI_FLASH_MODEL |
| 编辑排版 | gemini-3-flash-preview | 硬编码 GEMINI_EDITORIAL_FORMAT_MODEL |
| 向量嵌入 | models/gemini-embedding-001 | GEMINI_EMBEDDING_MODEL |

文档结束

# 04 — 数据库设计

本文档详细说明复旦商业知识库平台的数据库架构，包括技术选型、所有表结构定义、表间关系、迁移机制及种子数据初始化策略。

对应代码文件：backend/database.py、backend/config.py、backend/scripts/build_business_db.py

## 4.1 技术选型

### 4.1.1 为什么选择 SQLite

| 考量维度 | SQLite 的优势 |
|---|---|
| 部署简单 | 无须独立数据库进程，数据库即单个 .db 文件，随应用一起部署即可 |
| 读多写少 | 商业知识库以文章检索、阅读为主，写入操作（浏览计数、点赞、收藏等）频次远低于读取，SQLite 的单写者模型完全胜任 |
| 文件级备份 | 可直接拷贝 .db 文件实现冷备份，也可通过 WAL 日志实现热备，运维成本极低 |
| 零配置 | 不需要管理用户权限、网络端口、连接池等基础设施，适合中小型内容平台 |

### 4.1.2 连接参数

连接逻辑集中在 backend/database.py 的 get_connection() 函数中：

```
def get_connection() -> sqlite3.Connection:
connection = sqlite3.connect(SQLITE_DB_PATH, timeout=60)
connection.row_factory = sqlite3.Row
connection.execute("PRAGMA busy_timeout = 60000")
return connection
```

| 参数 | 值 | 说明 |
|---|---|---|
| timeout | 60（秒） | sqlite3.connect() 的 Python 层超时，当数据库被锁时最多等待 60 秒再抛出 OperationalError |
| row_factory | sqlite3.Row | 查询结果以类字典对象返回，支持 row["column_name"] 方式访问字段，避免依赖列索引 |
| PRAGMA busy_timeout | 60000（毫秒） | SQLite 引擎层的忙等待超时，当 WAL 写锁被占用时，最多等待 60 秒后再报 SQLITE_BUSY。与 Python timeout 参数配合，提供双层超时保护 |

此外，数据库构建阶段通过 PRAGMA journal_mode=WAL 启用 WAL（Write-Ahead Logging）模式，允许读写并发，进一步减少锁竞争。

### 4.1.3 连接管理

使用 connection_scope() 上下文管理器统一管理连接的创建与关闭：

```
@contextmanager
def connection_scope():
connection = get_connection()
try:
yield connection
finally:
connection.close()
```

所有业务代码均通过 with connection_scope() as connection: 获取连接，确保连接在异常情况下也能正确关闭，避免连接泄漏。

## 4.2 核心表（Core Tables）

核心表在数据库构建阶段（backend/scripts/build_business_db.py 的 create_schema() 函数）创建，构建时采用 DROP TABLE IF EXISTS + CREATE TABLE 的方式全量重建。这些表构成平台数据的骨架。

代码中使用 REQUIRED_TABLES 集合来校验数据库是否就绪：

```
REQUIRED_TABLES = {
"articles", "tags", "article_tags", "columns", "article_columns",
"topics", "topic_articles", "topic_tags", "featured_articles",
"chat_sessions", "chat_messages",
}
```

### 4.2.1 meta — 元数据表

存储构建时的全局键值对信息。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|

| | | | |
|---|---|---|---|
| key | TEXT | PRIMARY KEY | 元数据键名 |
| value | TEXT | NOT NULL | 元数据值 |

当前存储的键值包括 business_article_count（文章总数）和 updated_at（最近构建时间）。

### 4.2.2 articles — 文章主表

整个平台最核心的实体表，存储从商业知识库爬取并经 Gemini 分析后的全部文章。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| id | INTEGER | PRIMARY KEY | 文章 ID（构建时按遍历顺序分配） |
| doc_id | TEXT | UNIQUE NOT NULL | 文档指纹，由 relative_path 的 SHA-1 前 20 位生成 |
| slug | TEXT | UNIQUE NOT NULL | URL 友好的文章标识，格式为 {publish_date}-{title}，截断至 96 字符 |
| relative_path | TEXT | UNIQUE NOT NULL | 文章在 business_data/ 目录下的相对路径 |
| source | TEXT | NOT NULL | 数据来源，固定为 "business" |
| source_mode | TEXT | | 来源模式（如 business、wechat 等） |
| title | TEXT | NOT NULL | 文章标题 |
| publish_date | TEXT | NOT NULL | 发布日期（ISO 8601 格式，如 2024-03-15） |
| link | TEXT | | 原始来源链接 |
| content | TEXT | NOT NULL | 文章正文全文 |
| excerpt | TEXT | | 摘要，从正文前 180 字符截取 |
| main_topic | TEXT | | 主题分类（由 Gemini 模型分析得出） |
| article_type | TEXT | | 文章类型：专访、对谈、案例报道、研究/论文解读、活动/预告、书摘/书评、评论/ |
| series_or_column | TEXT | | 系列/专栏归属 |
| primary_org_name | TEXT | | 主要关联机构名称 |
| tag_text | TEXT | DEFAULT '' | 标签文本（以 ` \ ` 分隔），用于 |
| people_text | TEXT | DEFAULT '' | 关联人物文本（以 ` \ ` 分隔），用于 |
| org_text | TEXT | DEFAULT '' | 关联机构文本（以 ` \ ` 分隔），用于 |
| search_text | TEXT | DEFAULT '' | 聚合搜索文本（标题+主题+标签+人物+机构+摘要） |
| word_count | INTEGER | DEFAULT 0 | 文章字数（去除空白字符后统计） |
| cover_image_path | TEXT | | 封面图片的相对路径 |
| view_count | INTEGER | DEFAULT 0 | 阅读量（构建时基于确定性算法生成的模拟数据） |
| is_featured | INTEGER | DEFAULT 0 | 是否为推荐文章（前 24 篇） |
| access_level | TEXT | NOT NULL DEFAULT 'public' | 访问级别：public（公开）、member（会员）、paid（付费）— 通过迁移添加 |
| created_at | TEXT | | 记录创建时间（ISO 8601） |
| updated_at | TEXT | | 记录更新时间（ISO 8601） |

索引：

- idx_articles_date — publish_date DESC
- idx_articles_view_count — view_count DESC

### 4.2.3 tags — 标签表

存储从文章中抽取、合并后的分类标签。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT | 标签 ID |
| name | TEXT | NOT NULL | 标签名称（如"AI/人工智能"、"ESG/可持续"） |
| slug | TEXT | UNIQUE NOT NULL | URL 友好标识（如 topic-ai-人工智能） |
| category | TEXT | NOT NULL | 标签分类：topic（主题）、industry（行业）、type（类型）、entity（实体）、series（系列） |
| description | TEXT | | 描述（自动生成为 "{name} 相关文章聚合标签"） |

| | | | |
|---|---|---|---|
| color | TEXT | | 展示颜色（按 category 映射：industry=#ea6b00, topic=#0d0783, type=#4f46e5, entity=#64748b, series=#7c3aed） |
| article_count | INTEGER | DEFAULT 0 | 关联文章数量 |

约束：UNIQUE(name, category) — 同一分类下标签名不重复。

索引：idx_tags_category — category, article_count DESC

### 4.2.4 article_tags — 文章-标签关联表

多对多关联表，记录文章与标签的关系及置信度。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| article_id | INTEGER | NOT NULL, FK -> articles(id) | 文章 ID |
| tag_id | INTEGER | NOT NULL, FK -> tags(id) | 标签 ID |
| confidence | REAL | DEFAULT 1.0 | 标签关联置信度（0.0-1.0） |

主键：(article_id, tag_id)

索引：idx_article_tags_tag — tag_id, article_id

### 4.2.5 columns — 栏目表

编辑栏目定义，从 COLUMN_DEFINITIONS 配置初始化。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT | 栏目 ID |
| name | TEXT | NOT NULL | 栏目名称 |
| slug | TEXT | UNIQUE NOT NULL | URL 友好标识 |
| description | TEXT | | 栏目描述 |
| icon | TEXT | | 图标标识（如 Sparkles、Building2） |
| sort_order | INTEGER | DEFAULT 0 | 排序序号 |
| accent_color | TEXT | | 主题色（十六进制颜色值） |

### 4.2.6 article_columns — 文章-栏目关联表

多对多关联表，将文章分配到编辑栏目中。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| article_id | INTEGER | NOT NULL, FK -> articles(id) | 文章 ID |
| column_id | INTEGER | NOT NULL, FK -> columns(id) | 栏目 ID |
| is_featured | INTEGER | DEFAULT 0 | 是否为栏目推荐 |
| sort_order | INTEGER | DEFAULT 0 | 在栏目内的排序 |

主键：(article_id, column_id)

索引：idx_article_columns_column — column_id, article_id

### 4.2.7 topics — 专题表

编辑策划的内容专题，支持手动策划（seed）和自动聚类（auto）两种类型。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT | 专题 ID |
| title | TEXT | NOT NULL | 专题标题 |
| slug | TEXT | UNIQUE NOT NULL | URL 友好标识 |
| description | TEXT | | 专题描述 |
| cover_image | TEXT | | 封面图片路径 |
| cover_article_id | INTEGER | | 封面文章 ID |
| type | TEXT | NOT NULL | 专题类型：seed（策划）或 auto（自动聚类） |
| auto_rules | TEXT | | 自动聚类规则（JSON 格式） |
| status | TEXT | DEFAULT 'draft' | 状态：draft、published |
| created_at | TEXT | | 创建时间 |
| updated_at | TEXT | | 更新时间 |
| view_count | INTEGER | DEFAULT 0 | 浏览量 |

### 4.2.8 topic_articles — 专题-文章关联表

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| topic_id | INTEGER | NOT NULL, FK -> topics(id) | 专题 ID |
| article_id | INTEGER | NOT NULL, FK -> articles(id) | 文章 ID |
| sort_order | INTEGER | DEFAULT 0 | 在专题内的排序 |
| editor_note | TEXT | | 编辑批注 |

主键：(topic_id, article_id)

索引：idx_topic_articles_topic — topic_id, sort_order

### 4.2.9 topic_tags — 专题-标签关联表

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| topic_id | INTEGER | NOT NULL, FK -> topics(id) | 专题 ID |
| tag_id | INTEGER | NOT NULL, FK -> tags(id) | 标签 ID |

主键：(topic_id, tag_id)

### 4.2.10 featured_articles — 推荐文章表

控制首页和频道页的文章推荐位。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT | 记录 ID |
| article_id | INTEGER | NOT NULL, FK -> articles(id) | 文章 ID |
| position | TEXT | NOT NULL | 推荐位标识：hero（头条）或 editor-{n}（编辑推荐第 n 位） |
| start_date | TEXT | | 推荐开始日期 |
| end_date | TEXT | | 推荐结束日期 |
| is_active | INTEGER | DEFAULT 1 | 是否激活 |

构建时自动选出评分最高的文章作为 hero，其后 6 篇作为 editor-1 到 editor-6。

### 4.2.11 chat_sessions — 聊天会话表

AI 对话功能的会话记录。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| id | TEXT | PRIMARY KEY | 会话 ID（UUID 格式） |
| title | TEXT | NOT NULL | 会话标题 |
| created_at | TEXT | NOT NULL | 创建时间 |
| updated_at | TEXT | NOT NULL | 最近更新时间 |
| last_question | TEXT | NOT NULL | 最近一条用户提问 |

### 4.2.12 chat_messages — 聊天消息表

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT | 消息 ID |
| session_id | TEXT | NOT NULL, FK -> chat_sessions(id) | 所属会话 ID |
| role | TEXT | NOT NULL | 角色：user 或 assistant |
| content | TEXT | NOT NULL | 消息内容 |
| sources_json | TEXT | | 引用来源（JSON 数组） |
| follow_ups_json | TEXT | | 后续推荐问题（JSON 数组）— 通过迁移添加 |
| created_at | TEXT | NOT NULL | 创建时间 |

## 4.3 运行时表（Runtime Tables）

运行时表在 ensure_runtime_tables() 函数中使用 CREATE TABLE IF NOT EXISTS 创建，不依赖全量重建，随应用启动按需创建。这些表支撑用户交互、会员体系、计费系统、媒体内容、编辑后台和 AI 产出等功能模块。

### 4.3.1 demo_requests — 演示申请表

收集潜在客户的产品演示申请。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT | 记录 ID |
| name | TEXT | NOT NULL | 申请人姓名 |

| | | | |
|---|---|---|---|
| organization | TEXT | NOT NULL | 所在机构 |
| role | TEXT | NOT NULL | 职位角色 |
| email | TEXT | NOT NULL | 邮箱 |
| phone | TEXT | | 电话（可选） |
| use_case | TEXT | NOT NULL | 使用场景描述 |
| message | TEXT | | 附加留言 |
| status | TEXT | NOT NULL DEFAULT 'new' | 处理状态 |
| created_at | TEXT | NOT NULL | 提交时间 |

索引：idx_demo_requests_created_at — created_at DESC

### 4.3.2 editorial_articles — 编辑文章表

编辑后台管理的文章实体，支持从爬取文章（article_id）关联，或独立创建。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT | 记录 ID |
| article_id | INTEGER | FK -> articles(id) | 关联的原始文章 ID |
| source_article_id | INTEGER | FK -> articles(id) | 来源文章 ID（迁移添加） |
| slug | TEXT | UNIQUE NOT NULL | URL 友好标识 |
| title | TEXT | NOT NULL | 文章标题 |
| subtitle | TEXT | | 副标题 |
| author | TEXT | | 作者 |
| organization | TEXT | | 所属机构 |
| publish_date | TEXT | NOT NULL | 发布日期 |
| source_url | TEXT | | 原始来源 URL |
| cover_image_url | TEXT | | 封面图片 URL |
| primary_column_slug | TEXT | | 主栏目标识 |
| article_type | TEXT | | 文章类型 |
| main_topic | TEXT | | 主题分类 |
| source_markdown | TEXT | NOT NULL DEFAULT '' | 原始 Markdown 源文（迁移添加） |
| layout_mode | TEXT | NOT NULL DEFAULT 'auto' | 排版模式：auto（自动）、manual（手动）— 迁移添加 |
| formatting_notes | TEXT | | 排版备注（迁移添加） |
| formatter_model | TEXT | | 排版使用的模型（迁移添加） |
| last_formatted_at | TEXT | | 最近排版时间（迁移添加） |
| content_markdown | TEXT | NOT NULL | 正文 Markdown |
| plain_text_content | TEXT | NOT NULL | 纯文本内容（用于搜索和摘要） |
| excerpt | TEXT | | 摘要 |
| tag_payload_json | TEXT | NOT NULL DEFAULT '[]' | 标签数据（JSON 数组） |
| html_web | TEXT | | Web 渲染 HTML |
| html_wechat | TEXT | | 微信公众号 HTML |
| status | TEXT | NOT NULL DEFAULT 'draft' | 发布状态：draft、published |
| access_level | TEXT | NOT NULL DEFAULT 'public' | 访问级别（迁移添加） |
| workflow_status | TEXT | NOT NULL DEFAULT 'draft' | 工作流状态：draft、submitted、approved、published（迁移添加） |
| review_note | TEXT | | 审核备注（迁移添加） |
| scheduled_publish_at | TEXT | | 定时发布时间（迁移添加） |
| submitted_at | TEXT | | 提交审核时间（迁移添加） |
| approved_at | TEXT | | 审核通过时间（迁移添加） |
| ai_synced_at | TEXT | | AI 同步时间 |
| created_at | TEXT | NOT NULL | 创建时间 |
| updated_at | TEXT | NOT NULL | 更新时间 |
| published_at | TEXT | | 正式发布时间 |

索引：

- idx_editorial_articles_status — status, updated_at DESC
- idx_editorial_articles_article_id — article_id

- idx_editorial_articles_source_article_id — source_article_id

### 4.3.3 visitor_profiles — 访客档案表

匿名和已登录访客的基本画像。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| visitor_id | TEXT | PRIMARY KEY | 访客唯一标识（浏览器指纹或 cookie ID） |
| user_id | TEXT | | 关联的用户 ID（登录后关联） |
| first_seen_at | TEXT | NOT NULL | 首次访问时间 |
| last_seen_at | TEXT | NOT NULL | 最近访问时间 |

### 4.3.4 article_view_events — 文章浏览事件表

按天去重的文章浏览事件，用于统计阅读量和用户行为分析。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT | 记录 ID |
| article_id | INTEGER | NOT NULL, FK -> articles(id) | 文章 ID |
| visitor_id | TEXT | NOT NULL | 访客 ID |
| user_id | TEXT | | 用户 ID |
| view_date | TEXT | NOT NULL | 浏览日期（用于按天去重） |
| source | TEXT | | 来源渠道标识 |
| created_at | TEXT | NOT NULL | 记录创建时间 |

唯一约束：UNIQUE(article_id, visitor_id, view_date) — 同一访客同一天对同一文章只计一次。

索引：

- idx_article_view_events_article — article_id, view_date DESC
- idx_article_view_events_user — user_id, created_at DESC

### 4.3.5 article_reactions — 文章互动表

记录用户对文章的点赞和收藏等互动行为。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT | 记录 ID |
| article_id | INTEGER | NOT NULL, FK -> articles(id) | 文章 ID |
| user_id | TEXT | NOT NULL | 用户 ID |
| reaction_type | TEXT | NOT NULL | 互动类型（如 like、bookmark） |
| is_active | INTEGER | NOT NULL DEFAULT 1 | 是否生效（0=已取消，1=生效） |
| created_at | TEXT | NOT NULL | 首次互动时间 |
| updated_at | TEXT | NOT NULL | 最近更新时间 |

唯一约束：UNIQUE(article_id, user_id, reaction_type) — 每种互动类型每用户每篇文章唯一。

索引：

- idx_article_reactions_article — article_id, reaction_type, is_active
- idx_article_reactions_user — user_id, updated_at DESC

### 4.3.6 user_follows — 用户关注表

记录用户关注的标签、栏目、专题等实体。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT | 记录 ID |
| user_id | TEXT | NOT NULL | 用户 ID |
| entity_type | TEXT | NOT NULL | 实体类型（如 tag、column、topic） |
| entity_slug | TEXT | NOT NULL | 实体标识 |
| entity_label | TEXT | NOT NULL | 实体显示名称 |
| created_at | TEXT | NOT NULL | 关注时间 |

唯一约束：UNIQUE(user_id, entity_type, entity_slug)

索引：idx_user_follows_user — user_id, created_at DESC

### 4.3.7 user_memberships — 用户会员表

管理用户的会员资格和等级。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| user_id | TEXT | PRIMARY KEY | 用户 ID |
| email | TEXT | | 邮箱 |
| tier | TEXT | NOT NULL DEFAULT 'free_member' | 会员等级：free_member、paid_member、admin 等 |
| status | TEXT | NOT NULL DEFAULT 'active' | 状态：active、expired、suspended |
| note | TEXT | | 备注 |
| started_at | TEXT | NOT NULL | 会员开始时间 |
| expires_at | TEXT | | 到期时间 |
| created_at | TEXT | NOT NULL | 创建时间 |
| updated_at | TEXT | NOT NULL | 更新时间 |

索引：

- idx_user_memberships_tier — tier, status, updated_at DESC
- idx_user_memberships_email — email

### 4.3.8 business_users — 业务用户表

平台的核心用户实体，存储来自 Supabase 认证的用户信息。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| user_id | TEXT | PRIMARY KEY | 用户 ID（对应 Supabase 的 UUID） |
| email | TEXT | | 邮箱 |
| display_name | TEXT | NOT NULL | 显示名称 |
| title | TEXT | | 职称 |
| organization | TEXT | | 所属机构 |
| bio | TEXT | | 个人简介 |
| description | TEXT | | 详细描述（迁移添加） |
| tier | TEXT | NOT NULL DEFAULT 'free_member' | 用户等级 |
| status | TEXT | NOT NULL DEFAULT 'active' | 账户状态 |
| role_home_path | TEXT | NOT NULL DEFAULT '/' | 登录后默认跳转路径 |
| auth_source | TEXT | NOT NULL DEFAULT 'supabase' | 认证来源 |
| locale | TEXT | NOT NULL DEFAULT 'zh-CN' | 语言偏好 |
| is_seed | INTEGER | NOT NULL DEFAULT 0 | 是否为种子用户 |
| created_at | TEXT | NOT NULL | 创建时间 |
| updated_at | TEXT | NOT NULL | 更新时间 |
| last_seen_at | TEXT | NOT NULL | 最近活跃时间 |

索引：

- idx_business_users_tier — tier, status, updated_at DESC
- idx_business_users_email — email

### 4.3.9 admin_role_audit_logs — 角色变更审计日志

记录管理员对用户角色/等级的所有变更操作，用于审计追踪。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT | 记录 ID |
| target_user_id | TEXT | NOT NULL | 被变更的用户 ID |
| actor_user_id | TEXT | | 执行变更的管理员 ID |
| actor_email | TEXT | | 管理员邮箱 |
| previous_tier | TEXT | | 变更前等级 |
| next_tier | TEXT | NOT NULL | 变更后等级 |
| previous_status | TEXT | | 变更前状态 |
| next_status | TEXT | NOT NULL | 变更后状态 |
| note | TEXT | | 操作备注 |
| created_at | TEXT | NOT NULL | 操作时间 |

索引：idx_admin_role_audit_target — target_user_id, created_at DESC

### 4.3.10 media_items — 媒体内容表

存储音频和视频内容（播客、视频简报等）。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|

| | | | |
|---|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT | 记录 ID |
| slug | TEXT | UNIQUE NOT NULL | URL 友好标识 |
| kind | TEXT | NOT NULL | 类型：audio（音频）或 video（视频） |
| title | TEXT | NOT NULL | 标题 |
| summary | TEXT | NOT NULL | 摘要 |
| speaker | TEXT | | 主讲人/发言人 |
| series_name | TEXT | | 系列名称（迁移添加，默认按 kind 映射：音频->"管理者音频"，视频->"管理者视频"） |
| episode_number | INTEGER | NOT NULL DEFAULT 1 | 集数编号（迁移添加） |
| publish_date | TEXT | NOT NULL | 发布日期 |
| duration_seconds | INTEGER | NOT NULL DEFAULT 0 | 时长（秒） |
| visibility | TEXT | NOT NULL DEFAULT 'public' | 可见性：public、member、paid |
| status | TEXT | NOT NULL DEFAULT 'published' | 状态：published、draft |
| cover_image_url | TEXT | | 封面图片 URL |
| media_url | TEXT | | 媒体文件 URL（迁移添加） |
| preview_url | TEXT | | 预览/试听 URL（迁移添加） |
| source_url | TEXT | | 来源 URL |
| body_markdown | TEXT | NOT NULL DEFAULT '' | 正文 Markdown |
| transcript_markdown | TEXT | NOT NULL DEFAULT '' | 文字稿 Markdown（迁移添加） |
| chapter_payload_json | TEXT | NOT NULL DEFAULT '[]' | 章节数据 JSON（迁移添加） |
| sort_order | INTEGER | NOT NULL DEFAULT 0 | 排序序号 |
| created_at | TEXT | NOT NULL | 创建时间 |
| updated_at | TEXT | NOT NULL | 更新时间 |

索引：idx_media_items_kind — kind, status, publish_date DESC, sort_order ASC

### 4.3.11 billing_plans — 计费方案表

定义平台的会员订阅方案。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| plan_code | TEXT | PRIMARY KEY | 方案编码（如 free_member_monthly） |
| name | TEXT | NOT NULL | 方案名称 |
| tier | TEXT | NOT NULL | 对应的用户等级 |
| price_cents | INTEGER | NOT NULL DEFAULT 0 | 价格（分） |
| currency | TEXT | NOT NULL DEFAULT 'CNY' | 货币 |
| billing_period | TEXT | NOT NULL DEFAULT 'month' | 计费周期：month、year |
| headline | TEXT | | 方案标语 |
| description | TEXT | | 方案描述 |
| features_json | TEXT | NOT NULL DEFAULT '[]' | 功能列表（JSON 数组） |
| is_public | INTEGER | NOT NULL DEFAULT 1 | 是否公开展示 |
| is_enabled | INTEGER | NOT NULL DEFAULT 0 | 是否启用（取决于 PAYMENTS_ENABLED 配置） |
| sort_order | INTEGER | NOT NULL DEFAULT 0 | 排序序号 |
| created_at | TEXT | NOT NULL | 创建时间 |
| updated_at | TEXT | NOT NULL | 更新时间 |

### 4.3.12 billing_checkout_intents — 结账意向表

记录用户发起的结账/支付意向。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT | 记录 ID |
| user_id | TEXT | | 用户 ID |
| email | TEXT | | 邮箱 |
| plan_code | TEXT | NOT NULL | 订阅方案编码 |
| status | TEXT | NOT NULL | 状态 |

| | | | |
|---|---|---|---|
| payment_provider | TEXT | NOT NULL | 支付提供商 |
| redirect_url | TEXT | | 支付回跳 URL |
| note | TEXT | | 备注 |
| created_at | TEXT | NOT NULL | 创建时间 |
| updated_at | TEXT | NOT NULL | 更新时间 |

索引：idx_billing_checkout_intents_user — user_id, created_at DESC

### 4.3.13 billing_orders — 订单表

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT | 订单 ID |
| user_id | TEXT | | 用户 ID |
| email | TEXT | | 邮箱 |
| plan_code | TEXT | NOT NULL | 订阅方案编码 |
| amount_cents | INTEGER | NOT NULL DEFAULT 0 | 金额（分） |
| currency | TEXT | NOT NULL DEFAULT 'CNY' | 货币 |
| status | TEXT | NOT NULL | 订单状态 |
| payment_provider | TEXT | NOT NULL | 支付提供商 |
| checkout_intent_id | INTEGER | | 关联的结账意向 ID |
| provider_order_id | TEXT | | 第三方支付系统的订单 ID |
| created_at | TEXT | NOT NULL | 创建时间 |
| updated_at | TEXT | NOT NULL | 更新时间 |

索引：idx_billing_orders_user — user_id, created_at DESC

### 4.3.14 billing_subscriptions — 订阅表

记录用户的有效订阅关系。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT | 记录 ID |
| user_id | TEXT | NOT NULL | 用户 ID |
| email | TEXT | | 邮箱 |
| plan_code | TEXT | NOT NULL | 方案编码 |
| tier | TEXT | NOT NULL | 对应等级 |
| status | TEXT | NOT NULL | 订阅状态 |
| started_at | TEXT | NOT NULL | 开始时间 |
| expires_at | TEXT | | 到期时间 |
| auto_renew | INTEGER | NOT NULL DEFAULT 0 | 是否自动续费 |
| payment_provider | TEXT | NOT NULL | 支付提供商 |
| provider_subscription_id | TEXT | | 第三方系统的订阅 ID |
| created_at | TEXT | NOT NULL | 创建时间 |
| updated_at | TEXT | NOT NULL | 更新时间 |

索引：idx_billing_subscriptions_user — user_id, status, updated_at DESC

### 4.3.15 article_translations — 文章翻译表

存储文章的多语言翻译结果，通过 source_hash 实现内容变更感知。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| article_id | INTEGER | NOT NULL, FK -> articles(id) | 文章 ID |
| target_lang | TEXT | NOT NULL | 目标语言（如 en） |
| source_hash | TEXT | NOT NULL | 原文内容的哈希值，用于判断翻译是否过期 |
| title | TEXT | NOT NULL | 翻译后标题 |
| excerpt | TEXT | | 翻译后摘要 |
| summary | TEXT | NOT NULL | 翻译后总结 |
| content | TEXT | NOT NULL | 翻译后正文 |
| model | TEXT | NOT NULL | 使用的翻译模型标识 |
| created_at | TEXT | NOT NULL | 创建时间 |
| updated_at | TEXT | NOT NULL | 更新时间 |

主键：(article_id, target_lang, source_hash)

索引：idx_article_translations_lookup — article_id, target_lang, updated_at DESC

### 4.3.16 article_ai_outputs — AI 产出表

存储文章的所有 AI 处理结果：摘要、排版、翻译、HTML 渲染等。该表是编辑后台 AI 管线的核心存储。

| 字段 | 类型 | 约束 | 说明 |
|---|---|---|---|
| article_id | INTEGER | NOT NULL, FK -> articles(id) | 文章 ID |
| doc_id | TEXT | | 文档指纹 |
| slug | TEXT | | 文章标识 |
| relative_path | TEXT | | 文件相对路径 |
| source_hash | TEXT | NOT NULL | 原文哈希 |
| source_lang | TEXT | NOT NULL DEFAULT 'zh-CN' | 源语言 |
| target_lang | TEXT | NOT NULL DEFAULT 'en' | 目标语言 |
| source_title | TEXT | NOT NULL | 原始标题 |
| source_excerpt | TEXT | | 原始摘要 |
| summary_zh | TEXT | | 中文摘要 |
| summary_html_zh | TEXT | | 中文摘要 HTML（迁移添加） |
| summary_model | TEXT | | 摘要模型标识 |
| formatted_markdown_zh | TEXT | | 排版后的中文 Markdown |
| formatted_markdown_en | TEXT | | 排版后的英文 Markdown |
| translation_title_en | TEXT | | 英文标题翻译（迁移添加） |
| translation_excerpt_en | TEXT | | 英文摘要翻译（迁移添加） |
| translation_summary_en | TEXT | | 英文摘要翻译（迁移添加） |
| summary_html_en | TEXT | | 英文摘要 HTML（迁移添加） |
| translation_content_en | TEXT | | 英文正文翻译（迁移添加） |
| html_web_zh | TEXT | | 中文 Web HTML |
| html_wechat_zh | TEXT | | 中文微信 HTML |
| html_web_en | TEXT | | 英文 Web HTML |
| html_wechat_en | TEXT | | 英文微信 HTML |
| summary_status | TEXT | NOT NULL DEFAULT 'pending' | 摘要任务状态（迁移添加） |
| format_status | TEXT | NOT NULL DEFAULT 'pending' | 排版任务状态（迁移添加） |
| translation_status | TEXT | NOT NULL DEFAULT 'pending' | 翻译任务状态（迁移添加） |
| summary_error | TEXT | | 摘要错误信息（迁移添加） |
| format_error | TEXT | | 排版错误信息（迁移添加） |
| translation_error | TEXT | | 翻译错误信息（迁移添加） |
| translation_model | TEXT | | 翻译模型标识 |
| format_model | TEXT | | 排版模型标识 |
| format_template | TEXT | NOT NULL DEFAULT 'fudan-business-knowledge-v1' | 排版模板标识 |
| status | TEXT | NOT NULL DEFAULT 'pending' | 总体状态 |
| error_message | TEXT | | 错误信息 |
| worker_name | TEXT | | 处理该任务的 worker 标识 |
| started_at | TEXT | | 任务开始时间 |
| completed_at | TEXT | | 任务完成时间 |
| created_at | TEXT | NOT NULL | 创建时间 |
| updated_at | TEXT | NOT NULL | 更新时间 |

主键：(article_id, source_hash)

索引：

- idx_article_ai_outputs_article — article_id, updated_at DESC
- idx_article_ai_outputs_status — status, updated_at DESC

## 4.4 表关系描述

### 4.4.1 核心内容关系

```
articles (1) ──┬── () article_tags () ──── (1) tags
        ├── () article_columns () ── (1) columns
        ├── () topic_articles () ── (1) topics
        ├── (*) featured_articles
        ├── (*) article_view_events
        ├── (*) article_reactions
        ├── (*) article_translations
        └── (*) article_ai_outputs

topics (1) ──┬── (*) topic_articles
        └── () topic_tags () ──── (1) tags
```

- articles 是数据中心：所有内容表（标签、栏目、专题、推荐、浏览、互动、翻译、AI 产出）都通过外键指向 articles.id。
- tags 通过 article_tags 与文章实现多对多关联，同时通过 topic_tags 关联到专题。
- columns 通过 article_columns 与文章实现多对多关联。
- topics 通过 topic_articles 关联文章，通过 topic_tags 关联标签。

### 4.4.2 用户与会员关系

```
business_users (1) ──┬── user_memberships (通过 user_id 逻辑关联)
          ├── user_follows
          ├── article_reactions
          ├── article_view_events
          ├── billing_subscriptions
          ├── billing_orders
          └── billing_checkout_intents

admin_role_audit_logs ── 引用 target_user_id / actor_user_id
```

- business_users 是用户中心表，与 user_memberships 通过 user_id 逻辑关联（非显式外键）。
- 用户的互动行为（关注、点赞/收藏、浏览）和计费数据均通过 user_id 关联。

### 4.4.3 编辑工作流关系

```
articles (1) ──── (*) editorial_articles (通过 article_id / source_article_id)
editorial_articles ──── AI 管线 ──── article_ai_outputs
```

- editorial_articles 可以关联原始 articles 记录，也可以独立存在。
- AI 管线产出存储在 article_ai_outputs，通过 article_id 与 articles 关联。

### 4.4.4 计费链路

```
billing_plans ←── plan_code ──→ billing_checkout_intents
                |
                ↓
            billing_orders
                |
                ↓
          billing_subscriptions ──→ user_memberships (升级 tier)
```

用户从选择方案、发起结账、完成支付到激活订阅，形成完整的计费生命周期。

## 4.5 迁移机制

### 4.5.1 设计原则

本项目不使用独立的迁移框架（如 Alembic），而是在 ensure_runtime_tables() 函数中实现幂等的增量迁移。核心思路：

1. 表级别：使用 CREATE TABLE IF NOT EXISTS 确保表存在，不会重复创建。
2. 列级别：使用 _table_has_column() 检查列是否已存在，仅在缺失时执行 ALTER TABLE ADD COLUMN。
3. 数据修补：使用 UPDATE ... WHERE ... IS NULL OR ... = '' 对空值字段进行默认值回填。

### 4.5.2 幂等实现

_table_has_column() 是迁移的核心辅助函数：

```
def _table_has_column(connection, table_name, column_name) -> bool:
rows = connection.execute(f"PRAGMA table_info({table_name})").fetchall()
return any(row["name"] == column_name for row in rows)
```

每次迁移都遵循以下模式：

```
if not _table_has_column(connection, "表名", "新列名"):
connection.execute("ALTER TABLE 表名 ADD COLUMN 新列名 类型 约束")
```

这保证了：

- 首次运行：列不存在，执行 ALTER TABLE，添加新列。
- 重复运行：列已存在，跳过 ALTER TABLE，无副作用。
- 安全性：SQLite 的 ALTER TABLE ADD COLUMN 只支持在表末尾追加列，不支持删除或修改已有列，这一限制在此机制下反而成为优势——不会因为迁移冲突破坏已有数据。

### 4.5.3 当前迁移清单

以下列是通过迁移机制添加的（按表分组）：

articles 表：

- access_level — TEXT NOT NULL DEFAULT 'public'

editorial_articles 表：

- access_level, source_article_id, workflow_status, review_note, scheduled_publish_at, submitted_at, approved_at, ai_synced_at, source_markdown, layout_mode, formatting_notes, formatter_model, last_formatted_at

media_items 表：

- series_name, episode_number, media_url, preview_url, transcript_markdown, chapter_payload_json

business_users 表：

- description

article_ai_outputs 表：

- translation_title_en, translation_excerpt_en, translation_summary_en, summary_html_zh, summary_html_en, translation_content_en, summary_status, format_status, translation_status, summary_error, format_error, translation_error

chat_messages 表：

- follow_ups_json

### 4.5.4 数据修补逻辑

迁移完成后，ensure_runtime_tables() 还会执行一系列数据修补 UPDATE 语句：

1. 将 articles.access_level 和 editorial_articles.access_level 的空值回填为 'public'。
2. 将 editorial_articles.source_markdown 的空值回填为 content_markdown 的内容（COALESCE(NULLIF(source_markdown, ''), content_markdown)）。
3. 将 editorial_articles.layout_mode 的空值回填为 'auto'。
4. 将 editorial_articles.workflow_status 与 status 同步：已发布的文章设为 'published'，其余默认 'draft'。
5. 将 media_items 的 series_name（按 kind 映射为"管理者音频"或"管理者视频"）、episode_number（至少为 1）、preview_url（回退为 source_url）、transcript_markdown（回填空串）、chapter_payload_json（回填 '[]'）回填默认值。

## 4.6 种子数据

### 4.6.1 栏目种子（COLUMN_DEFINITIONS）

数据库构建时（insert_documents()），栏目从 backend/config.py 的 COLUMN_DEFINITIONS 初始化：

| slug | name | description | icon | sort_order | accent_color |
|---|---|---|---|---|---|
| insights | 深度洞 | 面向深度分析、专访、案例与长文的核心栏目。 | Sparkles | 1 | #0d0783 |
| industry | 行业观 | 围绕产业、公司、市场与商业模式的持续观察。 | Building2 | 2 | #ea6b00 |
| research | 学术前 | 复旦管院及相关学者的研究洞见、论文解读与管理新知。 | GraduationCap | 3 | #4f46e5 |
| deans-view | 院长说 | 聚焦复旦管院教授、院长与核心师资的观点表达。 | Mic2 | 4 | #b45309 |

### 4.6.2 专题种子（TOPIC_SEEDS + TOPIC_AUTO_CLUSTERS）

专题分为两类：

手动策划专题（TOPIC_SEEDS）— 8 个：

| slug | title | 核心标签 |
|---|---|---|
| ai-management | AI 重构管理 | AI/人工智能, ChatGPT, DeepSeek, 生成式AI, 智能体 |
| esg-sustainability | ESG 与可持续转型 | ESG/可持续, 双碳, 碳中和 |
| leadership-change | 领导力与组织变革 | 领导力, 组织变革, 企业文化, 人才管理 |
| digital-transformation | 数字化转型 | 数字化转型, 数字经济, 大数据, 云计算 |
| entrepreneurship-innovation | 创业创新 | 创业创新, 创业者, 企业家精神 |
| globalization-outbound | 全球化与出海 | 全球化, 出海, 国际化, 跨境 |

| | | |
|---|---|---|
| family-business | 家族企业与传承 | 家族企业, 家族传承, 代际传承 |
| brand-consumer | 品牌与新消费 | 品牌营销, 新零售, 消费者洞察 |

自动聚类专题（TOPIC_AUTO_CLUSTERS） — 5+ 个：

| slug | title | 必需标签 | 最少文章数 |
|---|---|---|---|
| capital-markets-governance | 资本运作与价值管理 | 金融投资, 资本市场 | 10 |
| case-teaching-management-education | 案例教学与管理教育 | 案例教学, 教育 | 10 |
| smart-manufacturing-upgrade | 智能制造与产业升级 | 制造业, 数字化转型 | 10 |
| ai-healthcare | AI 与智慧医疗 | AI/人工智能, 医疗健康 | 8 |
| venture-capital-innovation | 风险投资与创业生态 | 创业创新, 金融投资 | 10 |

每个专题通过 required_tags 和 support_tags 规则自动匹配文章，由 topic_engine.rebuild_topics() 在数据库构建时执行。

### 4.6.3 计费方案种子（BILLING_PLAN_DEFINITIONS）

在 ensure_runtime_tables() 中通过 INSERT … ON CONFLICT(plan_code) DO UPDATE 幂等写入：

| plan_code | name | tier | price_cents | currency | billing_period | headline |
|---|---|---|---|---|---|---|
| free_member_monthly | 免费会员 | free_member | 0 | CNY | month | 读者留存与资产沉淀入口 |
| paid_member_monthly | 付费会员月订 | paid_member | 3000 | CNY | month | 高价值会员内容的主力套餐 |
| paid_member_yearly | 付费会员年订阅 | paid_member | 30000 | CNY | year | 面向机构高价值用户的年度方案 |

其中 is_enabled 的值取决于环境配置 PAYMENTS_ENABLED：启用支付时为 1，否则为 0。使用 ON CONFLICT DO UPDATE 确保每次启动时方案定义都能与代码中的配置保持同步。

### 4.6.4 媒体内容种子

当 media_items 表为空时（SELECT COUNT(*) = 0），自动插入 6 条种子记录（3 条音频 + 3 条视频），覆盖三种可见性级别：

| slug | kind | title | visibility | duration_seconds | speaker |
|---|---|---|---|---|---|
| audio-ai-decision-lab | audio | AI 决策实验室 | public | 1220 | 复旦商业知识库编辑 |
| audio-case-briefing-weekly | audio | 案例 Briefing 周报 | member | 1560 | 案例研究团队 |
| audio-chairman-private-brief | audio | 董事长私享研判 | paid | 2280 | 商业研究院 |
| video-industry-observer | video | 行业观察视频简报 | public | 520 | 行业观察编辑部 |
| video-classroom-clip | video | 课堂精华剪辑 | member | 930 | 高管教育中心 |
| video-ceo-deep-briefing | video | CEO 深度闭门简报 | paid | 1980 | 复旦商业知识库研究团队 |

插入后还会执行一次 UPDATE 为所有种子媒体补充 series_name、episode_number、media_url、preview_url、transcript_markdown、chapter_payload_json 字段。

### 4.6.5 种子用户

ensure_runtime_tables() 末尾调用 ensure_business_user_seed_state(connection) 初始化种子用户数据（如管理员账户），确保系统启动后即可登录管理后台。

## 4.7 就绪检查

database_is_ready() 函数在应用启动时被调用，验证数据库文件存在且包含所有核心表：

```
def database_is_ready() -> bool:
if not SQLITE_DB_PATH.exists():
    return False
with connection_scope() as connection:
    rows = connection.execute(
        "SELECT name FROM sqlite_master WHERE type IN ('table', 'view')"
    ).fetchall()
existing = {row["name"] for row in rows}
return REQUIRED_TABLES.issubset(existing)
```

若检查不通过，ensure_database_ready() 会触发全量重建（rebuild_database()），该过程包括：

1. 解析 business_data/ 目录下的原始文本文件
2. 调用 Gemini AI 分析结果构建文档对象（标题、主题、标签、关键人物等）
3. 创建表结构（create_schema()）
4. 插入所有数据（insert_documents()）
5. 重建专题（rebuild_topics()）
6. 刷新搜索缓存（refresh_search_cache()）

# 05 — RAG 搜索引擎

本文档详细说明复旦商业知识库平台的搜索功能技术实现，涵盖整体架构、三路混合搜索算法、AI 重排、查询扩展、搜索模式、缓存策略以及 FAISS 向量索引构建流程。

对应代码文件：backend/services/rag_engine.py、backend/services/ai_service.py、create_vector_db_faiss.py

## 5.1 搜索架构概览

本系统采用三路混合搜索 + AI 重排的架构，融合传统信息检索与现代语义搜索的优势，在中文商业知识库场景下实现高质量的检索结果。

```
用户查询
│
├──── 1. 查询扩展（Gemini 生成 2-4 个替代表述）
│       │
│       ▼
│   terms = [原始查询, 分词片段, AI 扩展词]（最多 6 个）
│
├──── 2a. 词法搜索 (_relevance_score)
│       加权字段匹配 + 时效性 + 热度
│       输出: lexical_score
│
├──── 2b. BM25 搜索 (_bm25_scores)
│       BM25Okapi + CJK bigram/trigram
│       输出: bm25_score（归一化 0-18）
│
├──── 2c. 向量搜索 (_vector_scores)
│       FAISS + GoogleGenerativeAIEmbeddings
│       输出: vector_score（归一化 0-14）
│
├──── 3. 综合评分
│       score = lexical + (bm25 × 1.35) + vector
│
├──── 4. 排序
│
└──── 5. AI 重排 (_rerank_rows)
... (共 27 行，已截断显示)
```

核心设计理念：

- 词法搜索提供精确匹配能力，确保标题、标签中包含查询词的文章得到高分。
- BM25 搜索提供基于词频-逆文档频率的统计相关性，弥补简单词匹配在长文档中的不足。
- 向量搜索提供语义理解能力，能够找到与查询语义相关但不包含相同关键词的文章。
- AI 重排利用大模型的深层理解，对候选结果做最终的质量筛选。

## 5.2 查询预处理

### 5.2.1 查询词构建（_build_terms）

用户输入的查询经过以下处理步骤生成搜索词列表：

```
def _build_terms(query: str) -> list[str]:
base = query.strip()
parts = TOKEN_SPLIT_PATTERN.split(base)  # 按空格、逗号、句号等分词
terms = [base]                  # 第一个 term 始终是完整查询
for item in parts:
    if item not in terms:
        terms.append(item)
for expanded in ai_service.expand_query(base):  # AI 查询扩展
    if expanded not in terms:
        terms.append(expanded)
return terms[:6]                # 最多保留 6 个 term
```

分词模式：TOKEN_SPLIT_PATTERN = re.compile(r"[\s,，。；;、/|]+")，涵盖中英文常用分隔符。

输出示例：

| 用户输入 | 生成的 terms |
|---|---|
| "AI 数字化转型" | ["AI 数字化转型", "AI", "数字化转型", "人工智能驱动的企业数字化", "AI赋能转型升级"] |
| "ESG" | ["ESG", "ESG评级", "可持续发展", "环境社会治理"] |

### 5.2.2 查询扩展（ai_service.expand_query）

当 Gemini API 可用时，系统会调用大模型生成 2-4 个替代表述：

```
def expand_query(query: str) -> list[str]:
prompt = (
    "You rewrite search queries for a Chinese business knowledge base.\n"
    "Return 2 to 4 short alternative queries in the same language...\n"
    "User query:\n{query}\n\nJSON:"
)
raw = _invoke_prompt(prompt, {"query": query})
# 解析 JSON，去重，最多返回 4 个
```

关键特性：

- 使用 Gemini Pro 模型，温度 0.2（低随机性）。
- 要求返回 JSON 格式，便于解析。
- 如果 AI 不可用或调用失败，优雅降级为空列表，不影响搜索。
- 扩展词与原始查询去重后附加到 terms 列表。

### 5.2.3 文本归一化

所有文本比较前经过统一归一化处理：

```
def _normalize_text(text: str | None) -> str:
return (text or "").strip().lower()
```

确保大小写不敏感、前后空白不影响匹配。

## 5.3 词法搜索（_relevance_score）

词法搜索是最基础也是最可靠的搜索通道，通过在文章各字段中统计查询词出现次数，并根据字段重要性加权计分。

### 5.3.1 字段权重

| 字段 | 权重 | 说明 |
|---|---|---|
| title（标题） | 12 | 标题命中是最强的相关性信号 |
| main_topic（主题） | 8 | 主题分类的命中表明高度相关 |
| tag_text（标签） | 7 | 标签是编辑精选的关键词 |
| people_text（人物） | 6 | 关联人物匹配表示主题相关 |
| org_text（机构） | 5 | 机构名称匹配 |
| excerpt（摘要） | 4 | 摘要中的命中有中等信号强度 |
| content（正文） | 1.2 | 正文命中信号最弱，且限制最多 8 次计数（min(content_hits, 8)） |

### 5.3.2 完整评分公式

```
  # 对每个 term 计算得分
for index, term in enumerate(terms):
  weight = 1.0 if index == 0 else 0.65      # 首个 term（完整查询）权重 1.0，其余 0.65
  term_score = (
    title_hits * 12
    + topic_hits * 8
    + tag_hits * 7
    + people_hits * 6
    + org_hits * 5
    + excerpt_hits * 4
    + min(content_hits, 8) * 1.2
  )
  score += term_score * weight

# 如果没有任何 term 命中，直接返回 0
if not matched:
  return 0.0

# 加上时效性和热度加权
score = score + recency_boost + popularity_boost
```

### 5.3.3 时效性加权（Recency Boost）

```
  age_days = max((date.today() - date.fromisoformat(published)).days, 1)
recency_boost = max(0.0, 1.5 - (age_days / 3650))
```

| 文章发布距今 | recency_boost |
|---|---|
| 当天 | 1.5（最大值） |
| 1 年 (365 天) | 约 1.4 |
| 5 年 (1825 天) | 约 1.0 |
| 10 年 (3650 天) | 0.0 |
| 10 年以上 | 0.0（下限） |

设计意图：在同等相关性下，新发布的文章略微优先。最大加权 1.5 分，10 年以上的文章不再获得时效性加成。

### 5.3.4 热度加权（Popularity Boost）

```
popularity_boost = min((row.get("view_count") or 0) / 3000, 2.2)
```

| view_count | popularity_boost |
|---|---|
| 0 | 0.0 |
| 1500 | 0.5 |
| 3000 | 1.0 |
| 6600+ | 2.2（最大值，上限封顶） |

设计意图：高阅读量的文章获得额外加分，但封顶 2.2 分，防止热门文章过度压制相关性更高的冷门文章。

### 5.3.5 Term 权重衰减

- 第一个 term（即完整的用户查询）权重为 1.0。
- 后续 term（分词片段、AI 扩展词）权重为 0.65。

这确保了完整查询的匹配始终优先于部分匹配或扩展匹配。

## 5.4 BM25 搜索（_bm25_scores）

### 5.4.1 算法选择

使用 rank_bm25 库的 BM25Okapi 算法，这是 BM25 家族中最经典的变体，在信息检索领域广泛使用。

### 5.4.2 CJK 分词策略（_tokenize_for_bm25）

由于中文没有天然的词边界，系统采用 bigram + trigram 的 n-gram 分词策略：

```
def _tokenize_for_bm25(text: str | None) -> list[str]:
tokens: list[str] = []
for chunk in TEXT_TOKEN_PATTERN.findall(text.lower()):
    tokens.append(chunk)                    # 原始 token
    if all("\u4e00" <= c <= "\u9fff" for c in chunk):  # 如果是纯中文
        for size in (2, 3):                 # bigram 和 trigram
            if len(chunk) >= size:
                tokens.extend(
                    chunk[i : i + size]
                    for i in range(len(chunk) - size + 1)
                )
return tokens
```

TEXT_TOKEN_PATTERN：re.compile(r"[a-z0-9]+|[\u4e00-\u9fff]+", re.IGNORECASE)

分词示例：

| 输入文本 | 产生的 tokens |
|---|---|
| "数字化转型" | ["数字化转型", "数字", "字化", "化转", "转型", "数字化", "字化转", "化转型"] |
| "AI strategy" | ["ai", "strategy"] |

这种策略的优势：

- 无须依赖中文分词词典，避免 OOV（Out-of-Vocabulary）问题。
- bigram 能捕获大部分双字词组合。
- trigram 能捕获三字词，提高召回率。
- 英文和数字保持原始 token。

### 5.4.3 文档构建（_build_bm25_document）

每篇文章构建为一个 BM25 文档时，各字段通过重复来实现隐式加权：

| 字段 | 重复次数 | 说明 |
|---|---|---|

| | | |
|---|---|---|
| title | 3 | 标题最重要 |
| main_topic | 2 | 主题次之 |
| tag_text | 2 | 标签同样重要 |
| people_text | 1 | 人物名 |
| org_text | 1 | 机构名 |
| excerpt | 1 | 摘要 |
| content（前 4000 字符） | 1 | 正文截断避免过长文档稀释词频 |

### 5.4.4 归一化

BM25 原始分数通过以下方式归一化到 0-18 区间：

```
max_score = max(positive_scores)
normalized = (float(score) / max_score) * 18
```

归一化后四舍五入保留 4 位小数。仅保留分数大于 0 的文章。

### 5.4.5 查询 Token 构建

搜索时取 terms 前 4 个，对每个 term 分别分词，合并去重后作为 BM25 查询向量：

```
for term in terms[:4]:
for token in _tokenize_for_bm25(term):
  if token not in query_tokens:
    query_tokens.append(token)
```

## 5.5 向量搜索（_vector_scores）

### 5.5.1 技术栈

| 组件 | 选择 | 说明 |
|---|---|---|
| 向量数据库 | FAISS（Facebook AI Similarity Search） | 本地部署，无须外部服务 |
| 嵌入模型 | GoogleGenerativeAIEmbeddings | 使用 Gemini 的 embedding 模型 |
| 距离策略 | 余弦距离（COSINE） | 语义相似度的标准度量 |

### 5.5.2 搜索流程

```
def _vector_scores(terms: list[str]) -> dict[int, float]:
store = _load_vectorstore()
scores: dict[int, float] = {}
for index, term in enumerate(terms[:3]):      # 取前 3 个 term
  results = store.similarity_search_with_score(term, k=10)  # 每个 term 取 top-10
  weight = 1.0 if index == 0 else 0.7       # 首个 term 权重 1.0，其余 0.7
  for document, distance in results:
    article_id = document.metadata.get("article_id")
    similarity = max(0.0, 1 - float(distance))  # 余弦距离转相似度
    current = scores.get(article_id, 0.0)
    scores[article_id] = max(current, similarity  14  weight)
return scores
```

关键参数：

| 参数 | 值 | 说明 |
|---|---|---|
| terms 数量 | 前 3 个 | 避免过多查询降低性能 |
| top-k | 10 | 每个 term 取最相似的 10 个文档块 |
| 首 term 权重 | 1.0 | 完整查询的向量搜索结果最重要 |
| 后续 term 权重 | 0.7 | 扩展词的权重稍低 |
| 归一化系数 | 14 | 最大可能得分为 14 分 |
| 分数合并策略 | 取最大值（max） | 同一文章被多个 term 命中时取最高分 |

### 5.5.3 余弦距离到相似度的转换

FAISS 返回的是余弦距离（0 表示完全相同，2 表示完全相反），转换为相似度：

```
similarity = max(0.0, 1 - distance)
```

- 距离 0 -> 相似度 1.0（完全匹配）
- 距离 0.5 -> 相似度 0.5

- 距离 1.0 -> 相似度 0.0

最终得分 = similarity 14 weight，因此理论最大向量得分为 14 分。

### 5.5.4 向量存储加载

```
@lru_cache(maxsize=1)
def _load_vectorstore():
    if not PRIMARY_GEMINI_KEY:
        return None
    index_file = FAISS_DB_DIR / "index.faiss"
    if not index_file.exists():
        return None
    embeddings = GoogleGenerativeAIEmbeddings(
        model=GEMINI_EMBEDDING_MODEL,
        google_api_key=PRIMARY_GEMINI_KEY,
        task_type="retrieval_query",
    )
    return FAISS.load_local(
        str(FAISS_DB_DIR), embeddings,
        allow_dangerous_deserialization=True,
        distance_strategy=DistanceStrategy.COSINE,
    )
```

注意 `task_type="retrieval_query"` 表示这是查询端的嵌入，与构建索引时使用的 `task_type="retrieval_document"` 配对，这是 Google embedding 模型的推荐做法。

## 5.6 综合评分

三路搜索的分数通过以下公式合并：

```
score = lexical_score + (bm25_score * 1.35) + vector_score
```

| 通道 | 理论最大分 | 权重系数 | 加权后最大分 | 占比 |
|---|---|---|---|---|
| 词法搜索（lexical） | 无固定上限（取决于命中次数），典型 20-80 | 1.0 | 20-80 | 主导 |
| BM25 搜索 | 18 | 1.35 | 24.3 | 补充 |
| 向量搜索 | 14 | 1.0 | 14 | 语义补充 |

BM25 乘以 1.35 的系数是为了将其贡献提升到与向量搜索相当的水平。词法搜索因为有字段加权机制，在精确匹配场景中天然占据主导地位。

仅当综合分数 > 0 时，文章才进入候选集。

## 5.7 AI 重排（_rerank_rows）

### 5.7.1 触发条件

- AI 服务已启用（ai_service.is_ai_enabled() 返回 True，即 Gemini API key 已配置）。
- 搜索模式为 smart（不在 exact 模式下触发）。
- 排序方式为 relevance（按日期或热度排序时不触发重排）。
- 候选结果非空。

### 5.7.2 重排流程

```
全部候选结果（按综合分排序）
    |
    ▼
取 top 18 篇作为候选
    |
    ▼
构建候选摘要（id, title, publish_date, main_topic, excerpt, tags, columns）
    |
    ▼
发送给 Gemini Pro 模型
    |
Prompt: "Score each candidate from 0 to 10 based only on relevance"
    |
    ▼
返回 JSON: [{"id": 1, "score": 8.6}, {"id": 2, "score": 3.2}, ...]
    |
    ▼
将 AI 分数乘以 4 加到原始分数上：
final_score = original_score + (ai_score × 4)
    |
    ▼
重新排序 top 18，拼接剩余结果
```

### 5.7.3 关键参数

| 参数 | 值 | 说明 |
|---|---|---|
| 候选数量 | 18 | 取综合评分前 18 名进行重排 |
| AI 评分范围 | 0-10 | Gemini 返回的相关性评分 |
| AI 分数权重 | 4 | AI 评分乘以 4 后加到原始分数 |
| AI 最大加分 | 40（10 × 4） | 极高相关性的文章可获得最多 40 分加成 |
| 排序策略 | 先 AI 分数，再原始分数 | sorted(key=lambda: (rerank_score, original_score)) |

### 5.7.4 候选摘要结构

发送给 Gemini 的每篇候选文章包含以下字段：

```
  {
  "id": 42,
  "title": "AI 如何重塑企业决策",
  "publish_date": "2024-03-15",
  "main_topic": "人工智能",
  "excerpt": "...",
  "tags": ["AI/人工智能", "数字化转型"],    # 最多 4 个
  "columns": ["深度洞察"],                  # 最多 3 个
}
```

标签限制为前 4 个，栏目限制为前 3 个，以控制 prompt 长度和 token 消耗。

### 5.7.5 容错机制

- 如果 Gemini 调用失败（网络错误、JSON 解析失败等），返回空字典，不影响原始排序。
- AI 分数被强制限制在 [0.0, 10.0] 区间内（max(0.0, min(float(score), 10.0))）。
- 重排仅影响 top 18 的内部顺序，第 19 名及以后的文章保持原始排序不变。

## 5.8 搜索模式

### 5.8.1 smart 模式（默认）

完整的三路混合搜索 + AI 重排流程。需要 Gemini API key 才能发挥全部能力（查询扩展 + 向量搜索 + AI 重排），但在 API key 不可用时会自动降级为词法搜索 + BM25。

评分公式：lexical + (bm25 × 1.35) + vector

### 5.8.2 exact 模式

纯精确匹配模式，所有用户均可使用，不依赖 AI 服务。

评分公式：

```
exact_score = _term_occurrences(title, query) * 10 + _term_occurrences(content, query)
```

只在标题和正文中搜索完整的用户查询字符串（不分词、不扩展）。标题命中权重 10，正文命中权重 1。仅当 exact_score > 0 时返回结果。

### 5.8.3 模式对比

| 特性 | smart | exact |
|---|---|---|
| 查询扩展 | 有（Gemini） | 无 |
| 词法搜索 | 有 | 无 |
| BM25 搜索 | 有 | 无 |
| 向量搜索 | 有（FAISS） | 无 |
| AI 重排 | 有（Gemini） | 无 |
| 精确匹配 | 通过词法搜索隐含 | 唯一搜索方式 |
| AI 依赖 | 可选（降级运行） | 无 |
| 适用场景 | 探索性搜索、模糊查询 | 已知关键词的精确查找 |

## 5.9 排序与过滤

### 5.9.1 排序方式

搜索结果支持三种排序方式（sort 参数）：

| sort | 排序逻辑 | AI 重排 |
|---|---|---|
| relevance（默认） | 按综合分降序，分数相同按日期降序 | 触发 |
| date | 按发布日期降序，日期相同按分数降序 | 不触发 |
| popularity | 按阅读量降序，阅读量相同按分数降序 | 不触发 |

### 5.9.2 过滤器

_matches_filters() 支持以下过滤条件：

| 过滤字段 | 类型 | 匹配逻辑 |
|---|---|---|
| tags | 字符串数组 | 文章的 tag_text 中至少包含一个匹配的标签（交集非空） |
| columns | 字符串数组 | 文章所属栏目中至少包含一个匹配的栏目 |
| start_date | 字符串 | 文章发布日期 >= start_date |
| end_date | 字符串 | 文章发布日期 <= end_date |

过滤在评分之前执行，不符合过滤条件的文章直接跳过，不参与后续计算。

### 5.9.3 分页

```
safe_page = max(page, 1)
safe_page_size = max(1, min(page_size, MAX_PAGE_SIZE))
start = (safe_page - 1) * safe_page_size
page_rows = matched_rows[start : start + safe_page_size]
```

页码从 1 开始，页大小受 MAX_PAGE_SIZE 上限控制。

## 5.10 缓存策略

系统使用 Python 标准库的 functools.lru_cache 实现内存级缓存，避免重复加载大量数据。

### 5.10.1 三层缓存

| 缓存函数 | maxsize | 缓存内容 | 说明 |
|---|---|---|---|
| _load_search_rows() | 1 | 全部文章的搜索字段（dict 列表） | 从数据库加载所有文章，按 publish_date DESC 排序 |
| _load_bm25_index() | 1 | BM25Okapi 索引 + article_ids 映射 | 基于搜索行数据构建 BM25 语料库 |
| _load_vectorstore() | 1 | FAISS 向量存储对象 | 从磁盘加载 FAISS 索引 + embedding 模型 |

maxsize=1 意味着每个缓存只保存最新一次计算的结果。由于搜索数据在应用运行期间很少变化，单条缓存足够。

### 5.10.2 缓存刷新

```
def refresh_search_cache() -> None:
_load_search_rows.cache_clear()
_load_bm25_index.cache_clear()
_load_vectorstore.cache_clear()
```

在以下场景调用：

- 数据库全量重建后（rebuild_database()）。
- 手动触发缓存刷新时。

刷新后，下一次搜索请求会自动触发重新加载。

### 5.10.3 搜索行数据结构

_load_search_rows() 返回的每个 dict 包含以下字段：

```
  {
  "id", "title", "slug", "publish_date", "source", "excerpt",
  "article_type", "main_topic", "view_count", "cover_image_path",
  "link", "content", "tag_text", "people_text", "org_text",
  "column_text"  # 通过 JOIN 聚合的栏目标识
}
```

column_text 通过子查询实时聚合：GROUP_CONCAT(c.slug, '|')，将文章所属的所有栏目用 | 分隔。

## 5.11 FAISS 索引构建

索引构建由独立脚本 create_vector_db_faiss.py 完成，可单独运行。

### 5.11.1 构建流程

```
  1. 从 SQLite 加载文章 (load_documents)
    |
    ▼
2. 构建 Document 对象（含元数据）
    |
    ▼
3. 文本分块 (split_documents)
    |  chunk_size=800, chunk_overlap=120
    |
    ▼
4. 批量嵌入 + 构建索引 (create_index)
    |  batch_size=50, embedding model = Gemini
    |
    ▼
5. 保存到 faiss_index_business/ 目录
```

### 5.11.2 文档加载（load_documents）

从 articles 表加载所有文章，每篇文章构建为一个 LangChain Document 对象：

page_content 格式：

```
  Title: {title}
Date: {publish_date}
Type: {article_type}
Main Topic: {main_topic}
Tags: {tag_text}

{content}
```

metadata 结构：

```
  {
  "article_id": int,     # 文章 ID，用于搜索结果回溯
  "title": str,
  "publish_date": str,
  "link": str,
  "source": "business",
}
```

### 5.11.3 文本分块（split_documents）

使用 LangChain 的 RecursiveCharacterTextSplitter：

| 参数 | 值 | 说明 |
|---|---|---|
| chunk_size | 800 | 每个文本块最大 800 字符 |
| chunk_overlap | 120 | 相邻块之间重叠 120 字符，确保跨块语义连贯 |
| length_function | len | 按字符数计算长度（对中文友好） |

设计考量：

- 800 字符对中文来说约 400-500 个汉字，接近一个完整段落的长度。

- 120 字符的重叠确保段落边界处的语义不会丢失。
- 每个 chunk 继承原始 Document 的 metadata（包括 article_id），搜索时可回溯到源文章。

### 5.11.4 嵌入与索引构建（create_index）

| 参数 | 值 | 说明 |
|---|---|---|
| 嵌入模型 | GEMINI_EMBEDDING_MODEL（来自 config） | Google Generative AI Embeddings |
| task_type | "retrieval_document" | 文档端嵌入（与查询端的 retrieval_query 配对） |
| 批量大小 | 50 | 每次向 API 发送 50 个文本块进行嵌入 |
| 距离策略 | DistanceStrategy.COSINE | 余弦距离 |
| 重试策略 | 最多 3 次，每次失败后等待 2 秒 | 应对 API 限流和瞬时错误 |

批量处理流程：

```
for index in range(0, len(chunks), BATCH_SIZE):
batch = chunks[index : index + BATCH_SIZE]
for attempt in range(3):        # 最多重试 3 次
  try:
    if vectorstore is None:
      vectorstore = FAISS.from_documents(batch, embeddings, ...)
    else:
      vectorstore.add_documents(batch)
    break
  except Exception:
    if attempt == 2:
      raise
    time.sleep(2)        # 失败后等待 2 秒
```

第一个批次使用 FAISS.from_documents() 创建新索引，后续批次使用 add_documents() 追加。

### 5.11.5 索引存储

构建完成后保存到 faiss_index_business/ 目录：

```
os.makedirs(FAISS_DB_DIR, exist_ok=True)
vectorstore.save_local(FAISS_DB_DIR)
```

该目录包含两个文件：

- index.faiss — FAISS 二进制索引文件
- index.pkl — 文档元数据的 pickle 文件

### 5.11.6 API Key 获取

脚本按以下顺序获取 Google API Key：

1. 环境变量 GOOGLE_API_KEY
2. 环境变量 GEMINI_API_KEYS（逗号分隔，取第一个）
3. 均不存在时抛出 ValueError

## 5.12 搜索建议（suggest）

除了主搜索功能外，系统还提供搜索建议接口 suggest()，用于搜索框的自动补全。

### 5.12.1 空查询建议

当用户未输入任何内容时，返回热门标签：

- 中文模式：从 tags 表中按 article_count DESC 取前 8 个 topic 或 industry 类标签的名称。
- 英文模式：同样取热门标签，但通过 localize_tag_name() 本地化为英文，过滤掉仍含中文的结果，最多返回 8 个。

### 5.12.2 有查询建议

当用户已输入部分文本时，使用 LIKE %query% 在以下来源中搜索：

- 中文模式：标题（前 8 条）+ 标签名（前 6 条），合并去重后返回前 10 条。
- 英文模式：英文翻译标题（article_translations.title，前 8 条）+ 标签名英文本地化（前 160 个标签中匹配的），合并去重后返回前 10 条。

## 5.13 搜索结果响应结构

search_articles() 返回的字典结构：

```
  {
  "query": "AI 数字化转型",          # 原始查询
  "mode": "smart",                  # 搜索模式
  "total": 42,                      # 匹配总数
  "page": 1,                        # 当前页码
  "page_size": 12,                  # 每页数量
  "query_terms": ["AI 数字化转型", "AI", "数字化转型", ...],  # 展开后的搜索词
  "items": [                        # 当前页的文章列表
    {
      "id": 1,
      "title": "...",
      "slug": "...",
      "score": 45.6789,          # 综合得分（含 AI 重排加分）
      ...                        # 其他 ArticleCard 字段
    }
  ]
}
```

每个 item 是经过 _serialize_articles() 处理的文章卡片数据，额外附带 score 字段用于调试和排序透明化。

# 06 --- AI 服务体系

本章完整拆解后端 backend/services/ai_service.py 的设计与实现，覆盖模型选型、密钥管理、两条调用路径、七大核心功能的 Prompt 工程，以及 AI 不可用时的降级策略。

## 6.1 模型配置总览

系统同时使用四个 Google Gemini 模型，各司其职：

| 用途 | 模型标识符 | 配置来源 | 说明 |
|---|---|---|---|
| 对话 / 重排 | gemini-2.5-pro | 环境变量 GEMINI_CHAT_MODEL，缺省值 gemini-2.5-pro | 默认的主力 LLM，用于所有 LangChain 路径调用（查询扩展、文章摘要、知识库问答、搜索重排、编辑元数据建议 |
| 翻译 | gemini-2.5-flash | 环境变量 GEMINI_FLASH_MODEL，缺省值 gemini-2.5-flash | 专用于 translate_article_to_english，通过 LangChain 路径调用 |
| 编辑排版 | gemini-3-flash-preview | 硬编码常量 GEMINI_EDITORIAL_FORMAT_MODEL | 专用于 auto_format_editorial_markdown，通过 REST 直连路径调用 |
| 向量嵌入 | models/gemini-embedding-001 | 环境变量 GEMINI_EMBEDDING_MODEL，缺省值 models/gemini-embedding-001 | 用于 FAISS 向量检索的文本嵌入生成（在 create_vector_db_faiss.py 及 rag_engine 中使用） |

模型实例通过 _build_llm(model_name) 工厂函数构建，参数固定为：

```
  ChatGoogleGenerativeAI(
  model=model_name,
  google_api_key=keys[0],        # 取去重后的第一个 key
  temperature=0.2,
  convert_system_message_to_human=True,
)
```

两个 LangChain 模型实例（get_llm 和 get_flash_llm）各自被 @lru_cache(maxsize=1) 缓存，进程生命周期内只创建一次。

## 6.2 API Key 轮转机制

### 6.2.1 密钥来源与去重

```
  @lru_cache(maxsize=1)
def get_gemini_api_keys() -> tuple[str, ...]:
  keys: list[str] = []
  for key in (PRIMARY_GEMINI_KEY, *GEMINI_API_KEYS):
    cleaned = str(key or "").strip()
    if cleaned and cleaned not in keys:
      keys.append(cleaned)
  return tuple(keys)
```

- PRIMARY_GEMINI_KEY：优先取 GOOGLE_API_KEY 环境变量，若为空则取 GEMINI_API_KEYS 列表的第一个元素。
- GEMINI_API_KEYS：从环境变量 GEMINI_API_KEYS 按逗号分割，最多可配 6 个 key。
- 去重逻辑：遍历时用 list 保序，遇到已出现的 key 跳过，最终返回不可变 tuple，并被 @lru_cache 缓存。
- 返回结果为空元组时，is_ai_enabled() 返回 False，全部 AI 功能降级。

### 6.2.2 轮转计数器

```
  _gemini_request_counter = itertools.count()
```

这是一个进程级的无限自增计数器（itertools.count），每次调用 _request_gemini_text 时执行 next(_gemini_request_counter) 取得 start_offset。

### 6.2.3 轮转逻辑

在 _request_gemini_text 的重试循环中：

```
  max_attempts = max(3, len(api_keys) * 2)

for attempt in range(max_attempts):
  api_key = api_keys[(start_offset + attempt) % len(api_keys)]
```

- 最大重试次数：max(3, key数量 * 2)。若有 6 个 key，则最多 12 次尝试。
- Key 选择：(start_offset + attempt) % len(api_keys)。start_offset 逐次递增，保证不同请求从不同 key 开始，均匀分摊配额。

### 6.2.4 指数退避

```
time.sleep(min(4.0, 0.6 * (attempt + 1)))
```

| attempt | 等待时间 |
|---|---|
| 0 | 0.6s |
| 1 | 1.2s |
| 2 | 1.8s |
| 3 | 2.4s |
| 4 | 3.0s |
| 5 | 3.6s |
| 6+ | 4.0s（上限） |

退避仅在以下两种情况触发：

1. HTTP 状态码属于可重试集合 {429, 500, 502, 503, 504}。
2. 发生 requests.RequestException（网络超时、连接中断等）。

对于其他 HTTP 错误（如 400、401、403），直接抛出 RuntimeError，不进入重试。

## 6.3 两条调用路径

### 6.3.1 LangChain 调用路径 --- _invoke_prompt

```
  PromptTemplate.from_template(template)
    |
    v
ChatGoogleGenerativeAI (gemini-2.5-pro 或 gemini-2.5-flash)
    |
    v
StrOutputParser()
    |
    v
chain.invoke(payload).strip()
```

**核心签名：**

```
  def _invoke_prompt(
  template: str,
  payload: dict[str, str],
  *,
  llm: ChatGoogleGenerativeAI | None = None,
) -> str
```

- template：包含 {variable} 占位符的 Prompt 字符串。
- payload：键值对，由 LangChain 的 PromptTemplate 填充。
- llm：可选，默认使用 get_llm()（gemini-2.5-pro）。翻译功能传入 get_flash_llm()。
- 内部通过 | 管道运算符将三个组件串成 chain，调用 chain.invoke(payload) 完成端到端推理。

使用此路径的功能：
expand_query、summarize_article、answer_with_sources、rerank_search_results、suggest_editorial_metadata、translate_article_to_english。

### 6.3.2 REST 直连路径 --- _request_gemini_text

```
POST https://generativelanguage.googleapis.com/v1beta/models/{model}:generateContent?key={api_key}
```

**核心签名：**

```
  def _request_gemini_text(
  *,
  prompt: str,
  model_name: str,
  response_mime_type: str = "text/plain",
) -> str
```

**请求体结构：**

```
  {
 "contents": [{"parts": [{"text": "<prompt>"}]}],
 "generationConfig": {
  "temperature": 0.25,
  "responseMimeType": "text/plain"
 }
}
```

超时配置： (10, 90) --- 连接超时 10 秒，读取超时 90 秒。使用 requests.Session 保持连接复用。

响应解析： _extract_gemini_text 从 response.json()["candidates"][0]["content"]["parts"] 中提取并拼接所有 text 字段。

使用此路径的功能： auto_format_editorial_markdown。选择 REST 直连而非 LangChain 的原因是该功能需要指定 responseMimeType 并使用尚未被 LangChain 适配器完全支持的 gemini-3-flash-preview 模型。

## 6.4 JSON 解析工具链

_parse_json_payload(raw) 负责将 LLM 返回的非结构化文本解析为 Python 对象：

1. 代码围栏剥离：_strip_code_fence 移除 ``json ... `` 包裹。
2. 直接解析：先尝试 json.loads(text)。
3. 数组兜底：查找最外层 [...] 并解析。
4. 对象兜底：查找最外层 {...} 并解析。
5. 若全部失败，抛出 json.JSONDecodeError。

该工具链被 expand_query、rerank_search_results、suggest_editorial_metadata、translate_article_to_english 共同使用。

## 6.5 七大核心功能详解

### 6.5.1 expand_query --- 查询扩展

目的： 将用户的单条搜索查询改写为 2--4 个语义近似的替代查询，提升检索召回率。

调用路径： LangChain（_invoke_prompt -> gemini-2.5-pro）

**完整 Prompt 模板：**

```
  You rewrite search queries for a Chinese business knowledge base.
Return 2 to 4 short alternative queries in the same language as the original query.
Do not broaden the meaning, do not explain, and return JSON only.

User query:
{query}

JSON:
```

**参数：**

| 参数 | 类型 | 说明 |
|---|---|---|
| query | str | 用户原始搜索词 |

返回值： list[str]，最多 4 条去重后的替代查询。

**后处理逻辑：**

1. 调用 _parse_json_payload 解析 JSON 数组。
2. 过滤非字符串元素。
3. 去除与原始 query 完全相同的条目。
4. 用 set 去重，保留插入顺序。
5. 截断到最多 4 条。

降级： AI 不可用或解析异常时返回空列表 []。

### 6.5.2 summarize_article --- 文章摘要

目的： 将一篇文章压缩为 6--10 个高密度要点的 Markdown 简报。

调用路径： LangChain（_invoke_prompt -> gemini-2.5-pro）

**完整 Prompt 模板：**

```
You are an editor for a Chinese business knowledge product.
Summarize the article into a high-density Markdown brief.
Keep the original logic, do not invent facts, avoid filler, and answer in the same language as the article.
Target 6 to 10 short sections or bullets.

Title:
{title}

Article:
{content}

Summary:
```

**参数：**

| 参数 | 类型 | 说明 |
|---|---|---|
| title | str | 文章标题 |
| content | str | 文章正文，截断到前 18,000 字符 |

返回值： str，Markdown 格式摘要。

降级： AI 不可用或调用异常时，退回 build_extractive_summary，该函数简单提取正文前 6 个非空段落拼接。

### 6.5.3 answer_with_sources --- 带引用的知识库问答

目的： 根据检索到的知识库材料，用 [1]、[2] 等编号引用回答用户问题，是 RAG 问答的核心生成环节。

调用路径： LangChain（_invoke_prompt -> gemini-2.5-pro）

**完整 Prompt 模板：**

```
You are a business knowledge assistant.
Answer only from the supplied materials and never invent facts.
If the evidence is insufficient, say that clearly.
Respond in Markdown and cite sources with [1], [2], etc.
{language_instruction}

Conversation history:
{history}

User question:
{question}

Materials:
{context_blocks}

Answer:
```

其中 {language_instruction} 根据 response_language 参数动态切换：

| response_language | 指令文本 |
|---|---|
| "zh" | Respond in concise, professional Simplified Chinese. |
| "en" | Respond in concise, polished English. |
| "auto"（默认） | Use the same language as the user question. |

**参数：**

| 参数 | 类型 | 说明 |
|---|---|---|
| question | str | 用户当前问题 |
| history | str | 对话历史文本，为空时填 "None" |
| context_blocks | str | 由 _build_context_blocks 拼装的检索材料 |
| response_language | str | 响应语言，"auto" / "zh" / "en" |

返回值： str，带有编号引用的 Markdown 回答。

降级： 该函数本身不做降级；由调用方 _answer_from_sources 在异常时退回 _fallback_answer（纯阅读路径建议列表）。

### 6.5.4 rerank_search_results --- 搜索结果重排

目的： 对检索引擎返回的候选文章列表，让 LLM 按与查询的相关性打 0--10 分，用于 smart 模式的结果重排。

调用路径： LangChain（_invoke_prompt -> gemini-2.5-pro）

**完整 Prompt 模板：**

```
You rerank search results for a business knowledge base.
Score each candidate from 0 to 10 based only on relevance to the user query.
Return JSON only in this format: [{"id": 1, "score": 8.6}].

User query:
{query}

Candidates:
{candidates}

JSON:
```

**参数：**

| 参数 | 类型 | 说明 |
|---|---|---|
| query | str | 用户查询 |
| candidates | list[dict] | 候选文章列表，序列化为 JSON 字符串传入 Prompt |

返回值： dict[int, float]，键为候选 ID，值为 0.0--10.0 的相关性分数（已做 clamp）。

**后处理逻辑：**

1. _parse_json_payload 解析 JSON 数组。
2. 遍历每个 {"id": int, "score": float} 对象。
3. 用 max(0.0, min(float(score), 10.0)) 钳制到合法范围。

降级： AI 不可用、候选列表为空或解析异常时返回空字典 {}。

### 6.5.5 suggest_editorial_metadata --- 编辑元数据建议

目的： 自动为文章生成摘要（excerpt）、文章类型、主题标签、栏目归属等编辑元数据，减轻人工录入负担。

调用路径： LangChain（_invoke_prompt -> gemini-2.5-pro）

**完整 Prompt 模板：**

```
You are helping an editorial CMS for a Chinese business knowledge site.
Read the article title and content, then return strict JSON with this shape:
{
 "excerpt": "80-140 Chinese characters summary",
 "article_type": "short type label",
 "main_topic": "primary topic label",
 "column_slug": "insights|industry|research|deans-view",
 "tags": [{"name": "tag name", "category": "topic|industry|type|entity", "confidence": 0.0}]
}
Use concise Chinese labels when possible, do not invent facts, return 4 to 8 tags, and return JSON only.

Title:
{title}

Content:
{content}

JSON:
```

**参数：**

| 参数 | 类型 | 说明 |
|---|---|---|
| title | str | 文章标题 |
| content | str | 文章正文，截断到前 12,000 字符 |

返回值： dict，包含 excerpt、article_type、main_topic、column_slug、tags 字段。

**返回示例：**

```
 {
 "excerpt": "复旦大学管理学院发布最新研究报告，探讨生成式AI在金融行业的落地路径...",
 "article_type": "深度分析",
 "main_topic": "AI与金融科技",
 "column_slug": "research",
 "tags": [
  {"name": "生成式AI", "category": "topic", "confidence": 0.95},
  {"name": "金融科技", "category": "industry", "confidence": 0.88}
 ]
}
```

降级： AI 不可用或解析异常时返回空字典 {}。

### 6.5.6 auto_format_editorial_markdown --- 自动编辑排版

目的： 将原始 Markdown 稿件重排为微信公众号级别的出版就绪格式，支持 4 种布局模式。

调用路径： REST 直连（_request_gemini_text -> gemini-3-flash-preview）

**4 种布局模式（layout_mode）：**

| 模式 | 布局引导词 |
|---|---|
| auto（默认） | Default to a polished WeChat-style long-form layout with a short lead, clear H2/H3 hierarchy, and dense but readable information blocks. |
| insight | Favor deeper analysis, explicit argument structure, section transitions, and slightly longer analytical paragraphs. |
| briefing | Favor rapid scanning with short sections, bullet lists, key takeaways, and compact summaries. |
| interview | Preserve speaker turns and Q&A logic while adding a stronger title, lead, and readable section breaks. |

**完整 Prompt 模板：**

```
You are the formatting engine for a top-tier Chinese WeChat official-account editorial CMS.
Rewrite the source into publication-ready Markdown only.
Hard requirements:
1. Use model-side judgment to create a readable, professional Chinese long-form layout similar to the automatic layout mode used by high-end WeChat publishing tools.
2. Keep every important fact, number, company name, quote, and conclusion. Do not invent facts.
3. The first heading must be a single H1 title.
4. Add a short lead paragraph after the title when useful.
5. Use clean H2/H3 hierarchy, short paragraphs, lists, and quotes where appropriate.
6. Preserve interview, dialogue, and speech structure when present.
7. Do not output HTML. Do not output code fences. Output Markdown only.
8. Do not mention the model, prompt, or formatting rules in the answer.

Layout mode guidance: {layout_guidance}
Additional editorial notes: {formatting_notes}

Metadata:
- Title: {normalized_title}
- Excerpt hint: {excerpt}
- Main topic: {main_topic}
- Article type: {article_type}
- Organization: {organization}
- Tags: {tags_text}

Source content:
{normalized_source}
```

**参数：**

| 参数 | 类型 | 必填 | 说明 |
|---|---|---|---|
| title | str | 是 | 文章标题，为空时默认 "未命名文稿" |
| source_markdown | str | 是 | 原始 Markdown 内容，不可为空 |
| excerpt | str | 是 | 文章摘要提示 |
| main_topic | str | 是 | 主题标签 |
| article_type | str | 是 | 文章类型 |
| organization | str | 是 | 来源机构 |
| tags | list[str] | 否 | 标签列表 |
| layout_mode | str | 否 | 布局模式，默认 "auto" |
| formatting_notes | str | 否 | 额外编辑备注 |

返回值： dict[str, str]，包含 markdown（排版后的 Markdown）和 model（使用的模型名称）。

**后处理：**

1. _strip_code_fence 去除可能的代码围栏包裹。
2. 若输出不以 # 开头，自动补上 # {title} 作为一级标题。

降级： 无静默降级；源内容为空或 API 返回为空时直接抛出 RuntimeError。

## 6.5.7 translate_article_to_english --- 全文英译

目的： 将中文商业文章整篇翻译为专业英文，同时保留原文结构。

调用路径： LangChain（_invoke_prompt -> gemini-2.5-flash），通过 llm=flash_llm 参数显式指定 Flash 模型。

**完整 Prompt 模板：**

```
You are translating a Chinese business article for an English-language knowledge product.
Translate the title, short deck, summary, and full visible content into polished professional English.
Preserve structure, headings, lists, quotations, and paragraph breaks.
Do not invent facts and do not omit visible source content.
Return strict JSON only with this shape:
{
 "title": "English title",
 "excerpt": "English deck or short intro",
 "summary": "Markdown summary in English",
 "content": "Full translated content in Markdown"
}

Source title:
{title}

Source excerpt:
{excerpt}

Source content:
{content}

JSON:
```

**参数：**

| 参数 | 类型 | 说明 |
|---|---|---|
| title | str | 中文标题 |
| excerpt | str | 中文摘要 |
| content | str | 中文正文 |

返回值： dict[str, str]，包含 title、excerpt、summary、content、model 五个字段。

**后处理逻辑：**

1. _parse_json_payload 解析 JSON 对象。
2. 各字段做空值兜底：标题、摘要、内容若为空则回退到原文。
3. summary 若为空，退回到 excerpt 或正文第一行。
4. 附加 model 字段记录使用的模型名。

降级： Flash 模型未配置时抛出 RuntimeError。

## 6.6 is_ai_enabled() 降级策略

```
def is_ai_enabled() -> bool:
return bool(get_gemini_api_keys())
```

这是整个 AI 服务体系的总开关。当没有配置任何有效的 API Key 时，返回 False。各功能对降级的处理方式汇总如下：

| 功能 | 降级行为 | 用户感知 |
|---|---|---|
| expand_query | 返回 [] | 搜索不做查询扩展，仅用原始关键词检索 |
| summarize_article | 返回 build_extractive_summary 的结果（前 6 段原文） | 摘要质量下降但仍可阅读 |
| answer_with_sources | 不做内部降级，由 _answer_from_sources 捕获异常后退回 _fallback_answer | 返回阅读路径建议列表替代生成式回答 |
| rerank_search_results | 返回 {} | 搜索结果不做 LLM 重排，仅依赖向量/BM25 分数 |
| suggest_editorial_metadata | 返回 {} | 编辑界面不显示 AI 建议，需人工填写 |
| auto_format_editorial_markdown | 抛出 RuntimeError | 前端提示排版服务不可用 |
| translate_article_to_english | 抛出 RuntimeError | 前端提示翻译服务不可用 |

设计原则： 查询侧功能（扩展、摘要、重排）做静默降级以保持搜索可用；生成侧功能（排版、翻译）做显式报错以避免返回低质量内容。

## 6.7 辅助函数清单

| 函数名 | 职责 |
|---|---|
| _strip_code_fence(text) | 移除 LLM 返回内容外层的 Markdown 代码围栏 |
| _parse_json_payload(raw) | 多策略 JSON 提取（直接解析 -> 最外层数组 -> 最外层对象） |
| _extract_gemini_text(response_payload) | 从 Gemini REST 响应体中提取拼接文本 |
| build_extractive_summary(content) | 无 AI 时的文章摘要降级方案，提取前 6 个非空段落 |

## 6.8 模块依赖关系

```
backend/config.py
├── GEMINI_API_KEYS, PRIMARY_GEMINI_KEY
├── GEMINI_CHAT_MODEL        ("gemini-2.5-pro")
├── GEMINI_FLASH_MODEL       ("gemini-2.5-flash")
└── GEMINI_EDITORIAL_FORMAT_MODEL ("gemini-3-flash-preview")

backend/services/ai_service.py
├── langchain_core.output_parsers.StrOutputParser
├── langchain_core.prompts.PromptTemplate
├── langchain_google_genai.ChatGoogleGenerativeAI
└── requests (REST 直连)
```

被下游消费的路径：

- backend/routers/chat.py --- 调用 answer_with_sources、is_ai_enabled
- backend/services/rag_engine.py --- 调用 expand_query、rerank_search_results
- backend/routers/articles.py --- 调用 summarize_article、suggest_editorial_metadata、auto_format_editorial_markdown、translate_article_to_english

# 07 --- AI 对话系统

本章完整拆解 backend/routers/chat.py 中的对话路由、六大斜杠命令、自由问答 RAG 流程、会话持久化机制，以及置信度计算规则。

## 7.1 端点规格

### 7.1.1 POST /api/chat --- 主对话入口

**请求体 ChatRequest：**

```
class ChatMessageIn(BaseModel):
  role: str        # "user" 或 "assistant"
  content: str      # 消息文本

class ChatRequest(BaseModel):
  messages: list[ChatMessageIn]  # 对话消息列表
  session_id: str | None = None  # 会话 ID，首次为 None 时自动生成
  mode: str = "precise"        # 搜索模式（预留，当前未使用）
  language: str = "zh"         # 响应语言，"zh" 或 "en"
```

**响应体 ChatResponse：**

```
class ChatResponse(BaseModel):
  session_id: str                          # 会话 ID
  answer: str                              # Markdown 格式的回答
  sources: list[ArticleCard] = Field(default_factory=list)  # 引用的文章卡片
  follow_up_questions: list[str] = Field(default_factory=list)  # 后续建议问题（最多 3 条）
  confidence: float = 0.0                  # 置信度 0.0-1.0
```

### 7.1.2 辅助端点

| 方法 | 路径 | 响应模型 | 说明 |
|---|---|---|---|
| GET | /api/chat/sessions | list[ChatSessionSummary] | 列出最近 30 个会话 |
| GET | /api/chat/session/{session_id} | ChatSessionDetail | 获取单个会话的完整消息记录 |
| DELETE | /api/chat/session/{session_id} | ChatSessionDeleteResponse | 删除会话及其所有消息 |

## 7.2 请求处理主流程

POST /api/chat 的 chat() 函数按以下步骤执行：

```
  1. 提取最近一条 user 消息作为 latest_question
2. 语言规范化 _normalize_language（CJK 检测强制 zh）
3. 会话 ID：取 request.session_id，为 None 则 uuid4().hex 新建
4. 截取最近 6 条消息拼为 history_text
5. 尝试 _dispatch_command(latest_question, ...)
  ├── 命中命令 → 走命令流程
  └── 未命中 → 走自由问答 RAG 流程
6. 持久化：store_chat_session + append_chat_message（user + assistant）
7. 返回 ChatResponse，follow_up_questions 截断到 3 条
```

语言规范化细节： _normalize_language(requested_language, text) 的判断优先级为：

1. 若 text 包含 CJK 字符 → 强制 "zh"。
2. 否则，若 requested_language 显式为 "en" → "en"。
3. 其余情况 → "zh"。

## 7.3 命令解析与别名系统

### 7.3.1 解析流程

```
def _parse_command(command_text: str) -> tuple[str, str] | None
```

1. 检查文本是否以 / 开头，不是则返回 None（交给自由问答）。
2. 以第一个空格分割为 command_token 和 raw_argument。
3. command_token 转小写后在 COMMAND_ALIASES 中查找规范名。

### 7.3.2 完整别名映射表

| 输入别名 | 规范命令 |
|---|---|
| /brief | /brief |
| /简报 | /brief |
| /summarize | /summarize |
| /总结 | /summarize |
| /摘要 | /summarize |
| /compare | /compare |
| /比较 | /compare |
| /对比 | /compare |
| /timeline | /timeline |
| /时间线 | /timeline |
| /脉络 | /timeline |
| /today | /today |
| /今日一读 | /today |
| /一读 | /today |
| /今日 | /today |
| /今天 | /today |
| /今日简报 | /today |
| /recommend | /recommend |
| /继续阅读 | /recommend |
| /延伸阅读 | /recommend |
| /推荐 | /recommend |

## 7.4 六大命令详解

### 7.4.1 /brief [topic] --- 战略简报

别名： /简报

参数： topic（必填），缺失时返回用法提示。

**执行逻辑：**

1. 调用 rag_engine.search_articles(topic, mode="smart", page=1, page_size=6)，检索 6 篇文章。
2. 构建专用 Prompt 问题："Create an executive brief on '{topic}'. Cover the core signals, business implications, what deserves immediate attention, and the next questions worth tracking."
3. 调用 _answer_from_sources -> ai_service.answer_with_sources 生成回答。
4. 置信度：有源文章时 0.88，无源时 0.3。

后续建议： /timeline {topic}、/summarize {topic}、/recommend {topic}

### 7.4.2 /summarize [topic] --- 主题总结

别名： /总结、/摘要

参数： topic（必填），缺失时返回用法提示。

**执行逻辑：**

1. 调用 rag_engine.search_articles(topic, mode="smart", page=1, page_size=6)，检索 6 篇文章。
2. 构建专用 Prompt 问题："Summarize the core viewpoints, representative themes, and open questions around '{topic}'. Keep the answer dense and source-grounded."
3. 调用 _answer_from_sources -> ai_service.answer_with_sources 生成回答。
4. 置信度：有源文章时 0.85，无源时 0.3。

后续建议： /brief {topic}、/timeline {topic}、/recommend {topic}

### 7.4.3 /timeline [topic] --- 时间线梳理

别名： /时间线、/脉络

参数： topic（必填），缺失时返回用法提示。

**执行逻辑：**

1. 调用 rag_engine.search_articles(topic, mode="smart", sort="date", page=1, page_size=8)，检索 8 篇文章，按 日期排序。

2. 构建专用 Prompt 问题："Explain the evolution of '{topic}' as a timeline. Highlight key dates, turning points, and how the framing changed over time."
3. 调用 _answer_from_sources -> ai_service.answer_with_sources 生成回答。
4. 置信度：有源文章时 0.83，无源时 0.3。

后续建议： /brief {topic}、/summarize {topic}、/recommend {topic}

### 7.4.4 /compare [A] vs [B] --- 差异对比

别名： /比较、/对比

参数： 两个主题，用 vs、versus、与、和 分隔（正则 \s+(?:vs\.?|versus|与|和)\s+，不区分大小写）。格式不正确时返回用法提示。

**执行逻辑：**

1. 分别检索两个主题：
- rag_engine.search_articles(left, mode="smart", page=1, page_size=4) -> 4 篇
- rag_engine.search_articles(right, mode="smart", page=1, page_size=4) -> 4 篇
2. 调用 _dedupe_sources(left_sources, right_sources) 按文章 ID 去重合并（最多 8 篇，实际可能更少）。
3. 构建专用 Prompt 问题："Compare '{left}' and '{right}'. Show the overlap, the major differences, and what a business reader should read next."
4. 调用 _answer_from_sources -> ai_service.answer_with_sources 生成回答。
5. 置信度：有源文章时 0.84，无源时 0.3。

后续建议： /timeline {left}、/timeline {right}、/brief {left}

### 7.4.5 /today [YYYY-MM-DD] --- 今日一读

别名： /今日一读、/一读、/今日、/今天、/今日简报

参数： 可选的 ISO 日期字符串 YYYY-MM-DD。不传则使用当天日期。格式错误（非空且不匹配 \d{4}-\d{2}-\d{2}）时返回用法提示。

**执行逻辑：**

1. 调用 catalog_service.get_daily_read(target_date, language) 获取每日推荐文章。
2. 选文算法细节：
- 从数据库取不超过 anchor_date 的最近 240 篇文章。
- 过滤非商业安全文章（排除 editorial、test、demo 等标记）。
- 筛选近 180 天内的候选文章，无合格则取前 96 篇。
- 按 _daily_read_priority 综合评分排序取 TOP 12：
- freshness = max(0, 180 - age_days) / 18 --- 越新越高，最高 10 分
- popularity = min(view_count / 1800, 3.4) --- 阅读量加成，上限 3.4
- feature_boost = 2.2 if is_featured else 0.0 --- 编辑推荐加成
- depth_boost = min(word_count / 2600, 1.4) --- 长文加成
- SHA1 轮转选择：hashlib.sha1(anchor_date.isoformat()).hexdigest()[:8] 转为整数对候选列表长度取模，同一日期永远选同一篇。
3. 调用 _build_daily_read_answer 构建结构化回答，包含标题、发布日期、引语、主线提示、选文说明。
4. 置信度：有源文章时 0.88，无源时 0.45。

后续建议： 基于选中文章主题动态生成 /brief、/timeline、/recommend。

### 7.4.6 /recommend [subject] --- 继续阅读 / 延伸推荐

别名： /继续阅读、/延伸阅读、/推荐

参数： 可选的主题关键词 subject。

**执行逻辑（两条路径）：**

**路径 A --- 有 subject 参数（标签关联推荐）：**

1. 调用 rag_engine.search_articles(subject, mode="smart", page=1, page_size=5) 检索 5 篇。
2. recommendation_mode = "topic"。

**路径 B --- 无 subject 参数（高信号文章推荐）：**

1. 调用 catalog_service.get_recommended_articles(session_id, limit=5) 获取推荐。
2. 内部逻辑：
- 先通过 _load_context_source_ids 从当前会话的最近 8 条 assistant 消息中提取已引用文章 ID。
- 若有上下文文章 ID，基于标签关联查找相关文章（contextual 模式）。
- 若无上下文，退回全站高信号文章列表。
3. recommendation_mode 由 get_recommended_articles 返回（"contextual" 或 "high_signal"）。

**通用后处理：**

1. 调用 _build_continue_reading_answer 生成带有"为什么读"标注的阅读列表。
2. 置信度：有源文章时 0.8，无源时 0.35。

后续建议： /today、/brief {topic}、/timeline {topic}

## 7.5 自由问答流程（RAG）

当用户输入不以 / 开头时，进入自由问答流程：

```
  用户问题 latest_question
    |
    v
rag_engine.search_articles(
  latest_question,
  mode="smart",
  page=1,
  page_size=5,     # 检索 5 篇
  language=response_language,
)
    |
    v
sources = search_payload["items"]
    |
    v
_answer_from_sources(
  latest_question,
  history_text,      # 最近 6 条消息拼接
  sources,
  latest_question,    # fallback_question
  response_language,
)
    |
    ├── AI 可用且有源 → ai_service.answer_with_sources(...)
    |     构建 context_blocks：每篇格式为
 ... (共 35 行，已截断显示)
```

**关键参数差异（与命令对比）：**

| 维度 | 自由问答 | 命令 |
|---|---|---|
| 检索数量 | 5 篇 | 6/8/4+4 篇不等 |
| 搜索模式 | smart | smart |
| 排序 | 默认（相关性） | /timeline 用 date |
| Prompt 问题 | 用户原文 | 预设的结构化指令 |
| 置信度 | 动态计算 | 固定值 |

## 7.6 会话持久化

### 7.6.1 数据表结构

**chat_sessions 表：**

| 列名 | 类型 | 说明 |
|---|---|---|
| id | TEXT PK | 会话 ID（uuid4().hex 或客户端传入） |
| title | TEXT | 会话标题（取用户首条问题前 40 字符） |
| created_at | TEXT (ISO 8601) | 创建时间 |
| updated_at | TEXT (ISO 8601) | 最后更新时间 |
| last_question | TEXT | 最近一次用户问题 |

**chat_messages 表：**

| 列名 | 类型 | 说明 |
|---|---|---|
| id | INTEGER PK AUTOINCREMENT | 自增主键（隐含） |
| session_id | TEXT FK | 关联的会话 ID |
| role | TEXT | "user" 或 "assistant" |
| content | TEXT | 消息内容 |
| sources_json | TEXT | JSON 序列化的 list[dict]，记录引用文章 |
| follow_ups_json | TEXT | JSON 序列化的 list[str]，记录后续建议 |
| created_at | TEXT (ISO 8601) | 消息时间戳 |

### 7.6.2 写入时序

每次 /api/chat 请求产生以下写操作：

```
  1. store_chat_session(session_id, title, latest_question)
  └─── INSERT ... ON CONFLICT(id) DO UPDATE SET updated_at, last_question
    # 首次创建会话，后续只更新时间和最后问题

2. append_chat_message(session_id, "user", latest_question)
  └─── INSERT INTO chat_messages (session_id, role, content, sources_json, follow_ups_json, created_at)
    # sources_json = "[]", follow_ups_json = "[]"

3. append_chat_message(session_id, "assistant", answer, sources, follow_up_questions)
  └─── INSERT INTO chat_messages (...)
    # sources_json = JSON 序列化的引用文章列表
    # follow_ups_json = JSON 序列化的建议问题列表
  └─── UPDATE chat_sessions SET updated_at = ? WHERE id = ?
    # 每条消息写入后刷新会话时间戳
```

### 7.6.3 读取接口

**列表查询 `list_chat_sessions()`：**

- 按 updated_at DESC 排序。
- 限制最多返回 30 条。
- 返回字段：session_id、title、updated_at、last_question。

**详情查询 `get_chat_session_detail(session_id)`：**

- 查询会话元信息 + 所有消息（按 id ASC 排序）。
- 每条消息的 sources_json 和 follow_ups_json 实时反序列化。
- 会话不存在时返回 HTTP 404。

**删除 `delete_chat_session(session_id)`：**

- 先删 chat_messages 中该 session 的所有记录，再删 chat_sessions 本行。
- 会话不存在时返回 HTTP 404。
- 返回 {"session_id": "...", "deleted": true}。

## 7.7 后续问题建议生成

### 7.7.1 自由问答的 `_build_follow_ups`

```
def _build_follow_ups(question, sources, language):
topic = _compact_follow_up_subject(
    _first_topic(sources, question),  # 优先取源文章的 main_topic
    question,                         # 回退到用户问题
    language,
)
if not topic:
    return ["/brief AI战略", "/今日一读", "/继续阅读"]  # 默认建议（中文示例）
return _build_command_follow_ups(topic, language)
```

### 7.7.2 `_compact_follow_up_subject` 主题压缩

将主题文本压缩到适合命令参数的长度：

- 中文：在第一个句号/逗号/分号等标点处截断，上限 16 字符。
- 英文：在第一个 .!?;: 处截断，上限 44 字符，并在最近的空格处断词。

### 7.7.3 `_build_command_follow_ups` 命令建议生成

根据压缩后的主题，固定生成三条建议：

```
  [
  "/brief {subject}",    # 或 "/简报 {subject}"
  "/timeline {subject}",  # 或 "/时间线 {subject}"
  "/recommend {subject}", # 或 "/继续阅读 {subject}"
]
```

命令标签根据 language 自动在中英文之间切换。

### 7.7.4 命令流程的 `follow_up_questions`

各命令在返回时硬编码自己的后续建议，通常为三条互补命令。具体见 7.4 节各命令的"后续建议"说明。

## 7.8 置信度计算规则总表

| 场景 | 置信度 | 计算方式 |
|---|---|---|
| /brief --- 有源文章 | 0.88 | 固定值 |
| /brief --- 无源文章 | 0.30 | 固定值 |
| /summarize --- 有源文章 | 0.85 | 固定值 |
| /summarize --- 无源文章 | 0.30 | 固定值 |
| /timeline --- 有源文章 | 0.83 | 固定值 |
| /timeline --- 无源文章 | 0.30 | 固定值 |
| /compare --- 有源文章 | 0.84 | 固定值 |
| /compare --- 无源文章 | 0.30 | 固定值 |
| /today --- 有源文章 | 0.88 | 固定值 |
| /today --- 无源文章 | 0.45 | 固定值 |
| /recommend --- 有源文章 | 0.80 | 固定值 |
| /recommend --- 无源文章 | 0.35 | 固定值 |
| 命令格式不完整 | 0.20 | 固定值 |
| 不支持的命令 | 0.20 | 固定值 |
| 自由问答 --- 有源文章 | 动态 | min(1.0, sources[0]["score"] / 12) |
| 自由问答 --- 无源文章 | 0.45 | 固定值 |

自由问答动态置信度说明： 取最相关文章的原始检索分数除以 12，再钳制到 [0, 1.0]。例如检索分数 9.6 对应置信度 0.8，分数 12+ 对应 1.0。

## 7.9 降级与容错

### 7.9.1 _fallback_answer 降级回答

当 ai_service.is_ai_enabled() 为 False 或 answer_with_sources 调用抛出异常时，_answer_from_sources 退回到 _fallback_answer：

- 无源文章：返回"当前知识库里暂时没有找到足够相关的业务材料。建议换一个更具体的主题继续问..."
- 有源文章：返回"基于当前检索到的材料，可以先从下列阅读路径入手"，列出最多 4 篇文章的标题、日期和摘要，末尾提示可继续做简报/时间线/对比。

### 7.9.2 命令参数缺失的 _command_usage

返回格式化的用法提示，附带三条默认建议命令，置信度固定 0.2。

## 7.10 上下文构建细节

### 7.10.1 _build_context_blocks --- 检索材料格式化

将检索结果列表转换为 LLM 可读的文本块，传入 answer_with_sources 的 context_blocks 参数：

```
[1] Title: 生成式AI在金融行业的落地路径
Date: 2025-03-15
Topic: AI与金融科技
Excerpt: 复旦管院最新报告指出...

[2] Title: OpenAI与谷歌的大模型竞争格局
Date: 2025-03-10
Topic: 大模型竞争
Excerpt: 2025年Q1以来...
```

### 7.10.2 history_text --- 对话历史

从 request.messages 的最后 6 条消息中按 "{role}: {content}" 格式拼接，用换行分隔。超出 6 条的早期消息被截断。

## 7.11 模块依赖关系

```
backend/routers/chat.py
├── backend/models/schemas.py
│   └── ChatRequest, ChatResponse, ChatSessionSummary,
│       ChatSessionDetail, ChatSessionDeleteResponse
├── backend/services/ai_service.py
│   └── answer_with_sources, is_ai_enabled
├── backend/services/rag_engine.py
│   └── search_articles
├── backend/services/catalog_service.py
│   └── store_chat_session, append_chat_message,
│       list_chat_sessions, get_chat_session_detail,
│       delete_chat_session, get_daily_read,
│       get_recommended_articles
└── backend/services/content_localization.py
    └── contains_cjk
```

# 08 — 会员与权限体系

本章全面阐述 FDSMArticles 平台的用户身份认证、会员等级管理、内容访问控制、计费订阅以及审计日志等核心机制。系统围绕 四级会员体系 和 三级内容访问控制 构建，支持 Supabase 云认证与本地 DEV_AUTH 双模式，并配备完整的计费方案管理与角色变更审计链路。

## 8.1 四级会员等级（Membership Tiers）

系统定义了四个互斥的会员等级，由 membership_service.py 中的 TIER_LABELS 和 BENEFIT_MAP 两个字典统一管理：

| 等级键值 | 中文标签 | 说明 |
|---|---|---|
| guest | 游客 | 未登录状态，仅可浏览公开内容 |
| free_member | 免费会员 | 注册用户的默认等级 |
| paid_member | 付费会员 | 已订阅付费方案的用户 |
| admin | 管理员 | 拥有完整后台权限 |

### 8.1.1 各等级权益范围

**guest（游客）**

- 可浏览公开文章与公开音视频样刊
- 可查看商业化方案和知识库结构

**free_member（免费会员）**

- 可保留点赞、收藏与阅读历史
- 可访问会员专属音频和视频栏目中的基础内容
- 可逐步沉淀个人知识资产与推荐偏好

**paid_member（付费会员）**

- 可访问付费音频、付费视频和深度会员内容
- 适合扩展为专题专报、课程、闭门简报与陪跑服务
- 可承接更高价值的商业知识服务与内容订阅

**admin（管理员）**

- 拥有会员配置权限
- 可查看完整会员列表并调整等级
- 可继续扩展到专题运营、内容发布与商业后台管理

### 8.1.2 等级合法值校验

VALID_TIERS = {"free_member", "paid_member", "admin"} 定义了可写入数据库的合法等级集合（不含 guest，因为 guest 仅为序列化默认值，不存储在 user_memberships 表中）。调用 _normalize_tier() 时，若传入值不在此集合内，将直接抛出 HTTP 400 错误。

### 8.1.3 会员状态（Membership Status）

| 状态键值 | 中文标签 | 说明 |
|---|---|---|
| anonymous | 未登录 | 仅用于 guest 序列化，不存储 |
| active | 有效 | 会员资格正常 |
| trial | 试用中 | 试用期内 |
| paused | 已暂停 | 暂停会员权益 |
| expired | 已过期 | 会员已到期 |

合法可写入状态集合：VALID_STATUSES = {"active", "trial", "paused", "expired"}。

### 8.1.4 权限能力矩阵

_serialize_profile() 函数根据等级自动计算能力标记：

| 能力字段 | guest | free_member | paid_member | admin |
|---|---|---|---|---|
| is_authenticated | False | True | True | True |
| is_admin | False | False | False | True |
| can_access_member | False | True | True | True |
| can_access_paid | False | False | True | True |

计算逻辑：

```
can_access_member = tier in {"free_member", "paid_member", "admin"}
can_access_paid   = tier in {"paid_member", "admin"}
is_admin          = tier == "admin"
```

### 8.1.5 权限守卫函数

系统提供两个权限守卫函数，供路由层直接调用：

- require_admin_profile(user) -- 若 user 为 None 则抛出 401，若不是 admin 则抛出 403。
- require_paid_profile(user) -- 若 user 为 None 则抛出 401，若不是 paid_member 或 admin 则抛出 403。

## 8.2 内容访问控制 — build_content_access()

每篇文章均有一个 access_level 字段，取值为三级之一。build_content_access() 函数根据文章的访问级别和当前用户的会员状态，生成完整的访问控制描述对象。

### 8.2.1 三级访问级别

| access_level | 中文标签 | 含义 |
|---|---|---|
| public | 公开 | 所有访客均可阅读全文 |
| member | 会员 | 需要 free_member 及以上等级 |
| paid | 付费 | 需要 paid_member 及以上等级 |

### 8.2.2 build_content_access() 返回结构

```
def build_content_access(access_level: str, membership: dict | None) -> dict:
```

该函数首先调用 normalize_access_level() 对 access_level 进行标准化（空值默认为 "public"），然后通过 membership_can_access() 判断当前用户是否有权访问。

返回字典结构：

| 字段 | 类型 | 说明 |
|---|---|---|
| access_level | str | 标准化后的访问级别 |
| access_label | str | 中文标签（"公开"/"会员"/"付费"） |
| locked | bool | True 表示用户无权查看完整内容 |
| required_membership | `str \ | None` 解锁所需的最低会员等级 |
| required_membership_label | `str \ | None` 所需等级的中文标签 |
| message | str | 面向用户的提示文案 |

### 8.2.3 权限判定逻辑 — membership_can_access()

```
def membership_can_access(membership: dict | None, access_level: str) -> bool:
```

判定规则：

1. public -- 直接返回 True，任何人都可访问。
2. member -- 检查 membership["can_access_member"]，即 free_member / paid_member / admin 均可。
3. paid -- 检查 membership["can_access_paid"]，仅 paid_member / admin 可。

若 membership 为 None（未登录游客），则默认使用 _serialize_profile(None) 生成 guest 模板，此时 can_access_member 和 can_access_paid 均为 False。

### 8.2.4 各级别提示文案

| 级别 | locked 时的 message |
|---|---|
| public | "公开内容，所有访客均可阅读全文。" |
| member | "登录成为免费会员后可阅读全文与完整 AI 摘要。" |
| paid | "升级为付费会员后可阅读全文、完整 AI 摘要与高价值会员内容。" |

## 8.3 锁定内容预览 — preview_markdown()

当 locked == True 时，前端不应展示完整正文。系统通过 preview_markdown() 函数（位于 article_ai_output_service.py）生成预览片段。

### 8.3.1 函数签名

```
def preview_markdown(
  content: str,
  paragraph_limit: int = 4,
  char_limit: int = 900,
) -> str
```

### 8.3.2 截断逻辑

1. 将内容按行分割，过滤掉空行，得到段落列表。
2. 按顺序逐段收集，受两个限制条件约束：
- 段落数上限 -- 默认为 4 段。
- 字符数上限 -- 默认为 900 个字符。
3. 当某段落长度超过剩余字符配额时，截断该段并追加省略号 "..."。
4. 收集完毕后用双换行 "\n\n" 拼接输出。

### 8.3.3 使用场景

catalog_service.py
中调用此函数对锁定内容进行预览裁切，确保付费墙前只暴露开头有限内容，既给予用户对文章质量的初步感知，又保护了完整内容的商业价值。

## 8.4 Supabase 认证

系统通过 supabase_auth_service.py 实现与 Supabase Auth 的集成，作为生产环境的主要认证方式。

### 8.4.1 环境变量

| 变量 | 默认值 | 说明 |
|---|---|---|
| SUPABASE_URL | "" | Supabase 项目 URL |
| SUPABASE_ANON_KEY | "" | Supabase anon/public key |
| SUPABASE_AUTH_TIMEOUT_SECONDS | 8 | HTTP 请求超时秒数 |

### 8.4.2 认证是否启用

```
def is_supabase_auth_enabled() -> bool:
return bool(SUPABASE_URL and SUPABASE_ANON_KEY)
```

当 SUPABASE_URL 和 SUPABASE_ANON_KEY 均非空时，Supabase 认证启用。

### 8.4.3 Bearer Token 验证流程

get_authenticated_user() 是认证入口函数，完整流程如下：

```
  请求进入
 |
 v
is_supabase_auth_enabled()?
 |-- No --> 尝试 DEV_AUTH 模式 --> 返回 debug user 或 None
 |-- Yes
 v
从 Authorization 头提取 Bearer token
 |
 |-- token 为空 --> 尝试 DEV_AUTH 模式
 |-- token 存在
 v
GET {SUPABASE_URL}/auth/v1/user
 Headers:
  apikey: {SUPABASE_ANON_KEY}
  Authorization: Bearer {access_token}
 Timeout: SUPABASE_AUTH_TIMEOUT_SECONDS
 |
 |-- 网络异常 (RequestException) --> 返回 None
 |-- HTTP status != 200 --> 返回 None
 |-- 200 OK
 v
解析 JSON，提取 id 和 email
 |-- id 为空 --> 返回 None
 |-- id 存在
 ... (共 27 行，已截断显示)
```

### 8.4.4 Bearer Token 提取

_extract_bearer_token() 函数从 Authorization 头提取 token：

- 头部值为空 → 返回 None
- 不以 "bearer " 开头（不区分大小写） → 返回 None
- 提取 "bearer " 之后的内容，去除首尾空白

### 8.4.5 auth/status 端点

路由 GET /api/auth/status 调用 get_auth_status_payload()，返回完整认证状态，包含：

| 字段 | 说明 |
|---|---|
| enabled | 认证是否已配置（Supabase 或 DEV_AUTH 任一启用） |
| authenticated | 当前请求是否已认证 |
| user | {id, email} 或 null |
| auth_mode | "password"（DEV_AUTH 模式）或 "email_otp"（Supabase 模式） |
| membership | 完整会员状态对象 |
| business_profile | 业务用户档案对象 |
| role_home_path | 该角色的默认首页路径 |

### 8.4.6 角色首页路径

ROLE_HOME_PATHS 定义了各等级登录后的默认跳转路径：

| 等级 | 路径 |
|---|---|
| guest | / |
| free_member | /me |
| paid_member | /membership |
| admin | /admin |

## 8.5 DEV_AUTH 模式

当 DEV_AUTH_ENABLED = True（环境变量 DEV_AUTH_ENABLED=1）时，系统启用本地调试认证模式，无需连接 Supabase。

### 8.5.1 启用条件

DEV_AUTH 在以下两种场景中生效：

1. Supabase 未配置 -- SUPABASE_URL 或 SUPABASE_ANON_KEY 为空。
2. Supabase 已配置但请求未携带 Bearer token -- 此时回退到 DEV_AUTH。

### 8.5.2 调试请求头

| Header | 说明 |
|---|---|
| X-Debug-User-Id | 模拟用户 ID，如 mock-admin |
| X-Debug-User-Email | 模拟用户邮箱 |

_build_debug_user() 函数的处理逻辑：

1. 若 DEV_AUTH_ENABLED 为 False，直接返回 None。
2. 若 debug_user_id 为空或仅含空白字符，返回 None。
3. 构造并返回 {"id": user_id, "email": email, "raw_user": {"debug": True}}。

### 8.5.3 密码登录端点

路由 POST /api/auth/login 仅在 DEV_AUTH 模式下有效，接受 email + password 登录：

```
  @router.post("/login", response_model=AuthLoginResponse)
def auth_login(payload: AuthPasswordLoginRequest):
  account = authenticate_preview_account(payload.email, payload.password)
```

authenticate_preview_account() 函数检查：

1. DEV_AUTH_ENABLED 是否为 True。
2. 邮箱是否在 PREVIEW_ACCOUNT_PASSWORDS 字典中。
3. 密码是否严格匹配（区分大小写）。
4. 邮箱对应的种子用户定义是否存在。

全部通过后返回种子用户定义字典，否则返回 None，路由层抛出 401 错误。

## 8.6 管理员自动提升

系统通过 ADMIN_EMAILS 环境变量实现管理员邮箱白名单自动提升机制。

### 8.6.1 配置方式

环境变量 ADMIN_EMAILS 接受逗号分隔的邮箱列表：

```
ADMIN_EMAILS="alice@example.com,bob@example.com"
```

加载后转为小写去重的集合：

```
  ADMIN_EMAILS = {
  item.strip().lower()
  for item in os.getenv("ADMIN_EMAILS", "").split(",")
  if item.strip()
}
```

### 8.6.2 自动提升时机

邮箱匹配检查通过 _is_admin_email() 函数实现（大小写不敏感），在以下两个环节触发自动提升：

1. 首次创建会员记录 -- 在 _ensure_membership_row() 中，若新用户的邮箱匹配 ADMIN_EMAILS，直接以 admin 等级创建记录，而非默认的 free_member。
2. 已有用户登录 -- 在 _ensure_membership_row() 中，若已有用户的邮箱匹配 ADMIN_EMAILS 但当前等级不是 admin，自动将 tier 更新为 admin。
3. 管理员手动更新会员 -- 在 update_membership() 中，若被操作用户的邮箱匹配 ADMIN_EMAILS，即使管理员设定了其他等级，最终也会被强制覆盖为 admin。

### 8.6.3 判定函数

```
def _is_admin_email(email: str | None) -> bool:
return bool(email and email.strip().lower() in ADMIN_EMAILS)
```

## 8.7 会员记录自动创建 — _ensure_membership_row()

每次用户通过认证后，系统调用 get_membership_profile() → _ensure_membership_row() 来保证 user_memberships 表中存在该用户的记录。

### 8.7.1 完整流程

```
  用户通过认证，得到 user = {id, email}
 |
 v
get_membership_profile(user)
 |
 v
_ensure_membership_row(connection, user_id, email)
 |
 v
SELECT user_id FROM user_memberships WHERE user_id = ?
 |
 |-- 不存在（新用户）
 |   |
 |   v
 |  判定初始等级：
 |   邮箱匹配 ADMIN_EMAILS → tier = "admin"
 |   否则 → tier = "free_member"
 |   |
 |   v
 |  INSERT INTO user_memberships
 |  (status = 'active', started_at = now)
 |
 |-- 已存在（老用户）
 |   |
 |   v
 ... (共 34 行，已截断显示)
```

### 8.7.2 与业务用户档案联动

创建或更新会员记录后，函数会调用 ensure_business_user_profile() 将等级和状态同步到 business_users 表。判定 auth_source 的规则：

- user_id 以 "mock-" 开头 → auth_source = "seed"
- 其他情况 → auth_source = "supabase"

## 8.8 种子账户

系统内置 3 个测试账户，定义在 user_profile_service.py 的 SEED_USER_DEFINITIONS 列表中，用于开发和演示环境。

### 8.8.1 账户列表

| user_id | 邮箱 | 密码 | 等级 | 中文昵称 | 角色描述 |
|---|---|---|---|---|---|
| mock-free-member | reader@example.com | Reader2026! | free_member | 陈知远 | 知识库读者 |
| mock-paid-member | paid@example.com | Paid2026! | paid_member | 林策言 | 深度会员 |
| mock-admin | admin@example.com | Admin2026! | admin | 管理员 | 运营控制台 |

### 8.8.2 账户详细信息

每个种子账户包含完整的业务用户档案字段：

```
{
  "user_id": "mock-free-member",
  "email": "reader@example.com",
  "display_name": "陈知远",
  "title": "知识库读者",
  "organization": "FDSM Reader Lab",
  "bio": "负责持续整理个人阅读资产...",
  "description": "免费会员账号，可查看收藏、点赞、历史与基础会员内容。",
  "tier": "free_member",
  "status": "active",
  "auth_source": "seed",
  "locale": "zh-CN",
}
```

### 8.8.3 密码存储

密码以明文形式存储在 PREVIEW_ACCOUNT_PASSWORDS 字典中，仅在 DEV_AUTH_ENABLED=1 时启用密码登录逻辑，不应用于生产环境。

```
PREVIEW_ACCOUNT_PASSWORDS = {
  "reader@example.com": "Reader2026!",
  "paid@example.com": "Paid2026!",
  "admin@example.com": "Admin2026!",
}
```

### 8.8.4 登录后默认跳转

| 账户 | 等级 | 默认首页 |
|---|---|---|
| reader@example.com | free_member | /me |
| paid@example.com | paid_member | /membership |
| admin@example.com | admin | /admin |

## 8.9 计费系统

计费系统由 billing_service.py 实现，管理订阅方案、结账意向（checkout intent）、订单和订阅记录。

### 8.9.1 环境变量

| 变量 | 默认值 | 说明 |
|---|---|---|
| PAYMENTS_ENABLED | "0" | 设为 "1" 启用支付功能 |
| PAYMENT_PROVIDER | "mock" | 支付提供商标识 |
| SITE_BASE_URL | "http://127.0.0.1:4173" | 站点基础 URL，用于生成回调地址 |

### 8.9.2 三个计费方案

系统预置 3 个计费方案，在 billing_plans 表中管理，并在 PLAN_LOCALIZATIONS 中维护中英文描述：

| plan_code | 中文名 | 等级 | 计费周期 | 英文名 |
|---|---|---|---|---|
| free_member_monthly | 免费会员 | free_member | 月付 | Free Member |
| paid_member_monthly | 付费会员（月付） | paid_member | 月付 | Paid Member Monthly |
| paid_member_yearly | 付费会员（年付） | paid_member | 年付 | Paid Member Yearly |

#### 方案特性详情

`free_member_monthly`

- Keep reading history, likes, and bookmarks
- Unlock foundational member audio and video content
- Serve as the bridge into paid upgrades

`paid_member_monthly`

- Unlock paid articles, premium audio, and premium video
- Fit topic briefings, courses, and closed-door notes
- Stay compatible with renewals, trials, and promotional pricing

`paid_member_yearly`

- Includes a full year of paid-content access
- Fits annual briefings, course bundles, and offline programs
- Prepares the path for institution and seat-based editions

### 8.9.3 计费周期标签

BILLING_PERIOD_LABELS 提供中英文计费周期标签：

| 键值 | 中文 | 英文 |
|---|---|---|
| month | 月付 | month |
| quarter | 季付 | quarter |
| year | 年付 | year |
| oneoff | 一次性 | one-off |

### 8.9.4 方案列表 API

```
def list_billing_plans(language: str = "zh") -> dict
```

从 billing_plans 表查询所有 is_public = 1 的方案，按 sort_order 和 price_cents 排序。返回结构：

```
{
 "payments_enabled": true,
 "payment_provider": "mock",
 "items": [ / 序列化后的方案列表 / ]
}
```

每个方案的序列化字段包括：plan_code、name、tier、tier_label、price_cents、currency、billing_period、billing_period_label、headline、description、features（JSON 数组）、is_public、is_enabled、checkout_available、sort_order。

其中 checkout_available = PAYMENTS_ENABLED and is_enabled -- 仅当全局支付启用且该方案已启用时，才允许结账。

### 8.9.5 Checkout Intent 流程

create_checkout_intent() 函数实现结账意向的创建流程：

```
  用户请求结账
 |
 v
校验用户是否已登录（否则 401）
校验 plan_code 是否存在且 is_public（否则 404）
 |
 v
判定 payments_ready = PAYMENTS_ENABLED and plan_row["is_enabled"]
 |
 |-- payments_ready = False
 |    status = "disabled"
 |    redirect_url = None
 |    note = "Payment infrastructure is configured, but live payment
 |           capture is not enabled in this environment."
 |
 |-- payments_ready = True
 |    status = "pending"
 |    redirect_url = success_url 或
 |     "{SITE_BASE_URL}/membership?checkout=mock&plan={plan_code}"
 |    note = "A checkout intent has been created. Replace the mock
 |           redirect with your live payment gateway when ready."
 |
 v
INSERT INTO billing_checkout_intents
 (user_id, email, plan_code, status, payment_provider,
 ... (共 43 行，已截断显示)
```

### 8.9.6 Mock 支付说明

当前系统使用 PAYMENT_PROVIDER = "mock" 作为默认支付提供商。这意味着：

1. 不会真正触发支付网关。
2. checkout_url 在 payments_ready = True 时指向站内 mock 回调地址。
3. 订单状态始终为 "pending" 或 "disabled"，需要接入真实支付网关后替换逻辑。
4. 系统已预留 provider_order_id 字段用于存储第三方支付平台的订单号。

### 8.9.7 用户计费概览

```
def get_billing_profile(user: dict | None, language: str = "zh") -> dict
```

返回结构：

| 字段 | 说明 |
|---|---|
| payments_enabled | 支付是否启用 |
| payment_provider | 支付提供商 |
| membership | 当前会员状态 |
| active_subscription | 当前有效订阅（status 为 active 或 trial 的最近一条） |
| recent_orders | 最近 12 条订单记录 |

### 8.9.8 订单管理

```
def list_billing_orders(limit: int = 100, query: str = "") -> dict
```

支持按 user_id、email、plan_code、status 进行模糊搜索。最大返回 200 条。返回 {items: [...], total: N}。

### 8.9.9 国际化支持

计费系统完整支持中英双语：

- _normalize_language() 将语言标识规范化为 "zh" 或 "en"。
- _localized_plan_copy() 根据语言从 PLAN_LOCALIZATIONS 获取英文文案，或回退到数据库中的中文文案。
- 英文模式下，若回退文案包含 CJK 字符（通过 contains_cjk() 检测），则返回空字符串而非乱码。

## 8.10 审计日志

系统通过 admin_role_audit_logs 表记录所有会员等级变更操作，确保权限变更的可追溯性。

### 8.10.1 记录函数

```
    def record_admin_role_audit(
    connection,
    *,
    target_user_id: str,
    actor_user_id: str | None,
    actor_email: str | None,
    previous_tier: str | None,
    next_tier: str,
    previous_status: str | None,
    next_status: str,
    note: str | None = None,
) -> None
```

### 8.10.2 记录时机

审计日志在 update_membership() 函数中、会员信息写入数据库之后立即记录。无论是新建还是更新会员记录，均会产生审计条目。

### 8.10.3 记录字段

| 字段 | 说明 |
|---|---|
| target_user_id | 被操作的用户 ID |
| actor_user_id | 执行操作的管理员 ID（可为 None） |
| actor_email | 执行操作的管理员邮箱（可为 None） |
| previous_tier | 变更前等级（新建时为 None） |
| next_tier | 变更后等级 |
| previous_status | 变更前状态（新建时为 None） |
| next_status | 变更后状态 |
| note | 操作备注 |
| created_at | 记录创建时间（ISO 格式） |

### 8.10.4 管理后台概览

get_admin_overview() 函数提供管理后台的统计概览，包括：

- 总用户数（business_users 表）
- 付费会员数
- 管理员数
- 审计日志总数
- 各等级会员分布统计

### 8.10.5 数据完整性

- 审计日志仅追加，不可修改或删除。
- 日志记录与会员变更在同一个事务内（connection.commit() 在两者之后），确保原子性。
- actor_user_id 和 actor_email 来自 actor_user 参数，通常为当前登录的管理员信息。

## 8.11 会员管理 API 汇总

### 8.11.1 认证相关

| 方法 | 路径 | 说明 | 权限要求 |
|---|---|---|---|
| GET | /api/auth/status | 获取当前认证状态与会员信息 | 无（匿名可用） |
| POST | /api/auth/login | DEV_AUTH 密码登录 | DEV_AUTH 模式下有效 |

### 8.11.2 会员管理（管理员操作）

| 函数 | 说明 |
|---|---|
| list_memberships(limit, query) | 列出所有会员，支持搜索，按等级权重排序 |
| update_membership(user_id, ...) | 更新会员等级/状态/备注/到期时间 |
| get_membership_profile(user) | 获取当前用户的会员档案 |

### 8.11.3 会员列表排序规则

查询结果按以下优先级排序：

1. 等级权重：admin > paid_member > free_member > 其他

2. 更新时间倒序

## 8.12 数据表结构参考

### user_memberships

| 列名 | 说明 |
|---|---|
| user_id | 用户唯一标识（主键） |
| email | 用户邮箱 |
| tier | 会员等级 |
| status | 会员状态 |
| note | 管理员备注 |
| started_at | 会员起始时间 |
| expires_at | 会员到期时间（可为空） |
| created_at | 记录创建时间 |
| updated_at | 记录更新时间 |

### admin_role_audit_logs

| 列名 | 说明 |
|---|---|
| target_user_id | 被操作用户 ID |
| actor_user_id | 操作者 ID |
| actor_email | 操作者邮箱 |
| previous_tier | 变更前等级 |
| next_tier | 变更后等级 |
| previous_status | 变更前状态 |
| next_status | 变更后状态 |
| note | 备注 |
| created_at | 记录时间 |

### billing_plans / billing_orders / billing_subscriptions / billing_checkout_intents

分别存储计费方案定义、订单记录、订阅状态和结账意向记录，详见 8.9 节。

## 8.13 关键源文件索引

| 文件路径 | 职责 |
|---|---|
| backend/services/membership_service.py | 会员等级、权限判定、访问控制核心逻辑 |
| backend/services/supabase_auth_service.py | Supabase 认证与 DEV_AUTH 回退 |
| backend/services/billing_service.py | 计费方案、checkout intent、订单管理 |
| backend/services/user_profile_service.py | 种子账户、业务用户档案、审计日志 |
| backend/services/article_ai_output_service.py | preview_markdown() 锁定内容预览 |
| backend/routers/auth.py | 认证 API 路由 |
| backend/config.py | 环境变量加载与全局常量 |

# 09 — 编辑后台 CMS

本章全面阐述 FDSMArticles 编辑后台的内容管理系统（CMS），涵盖文稿全生命周期管理：从草稿创建、AI 批处理导入、AI 自动排版与标签生成，到工作流状态机驱动的审核发布流程，以及 HTML 渲染、导出与前端 UI 交互。

## 9.1 工作流状态机

编辑后台的核心是一个五级工作流状态机，定义在 editorial_service.py 的 WORKFLOW_LABELS 字典中：

```
draft ──→ in_review ──→ approved ──→ scheduled ──→ published
↑ ____________|_______________|        |
|      |        |       |
└──────────────────────┴──────────────────────────┘        |
  (可回退到 draft)            |
                        v
                    published（终态，不可逆）
```

### 9.1.1 状态定义

| 状态键值 | 中文标签 | 说明 |
|---|---|---|
| draft | 草稿 | 初始状态，可自由编辑 |
| in_review | 待审核 | 已提交审核，等待审批 |
| approved | 已通过 | 审核通过，可进入排期或直接发布 |
| scheduled | 定时发布 | 已设定发布时间，等待定时触发 |
| published | 已发布 | 终态，内容已写入公开 articles 表 |

### 9.1.2 状态标准化

```
def _normalize_workflow_status(value: str | None) -> str:
workflow_status = (value or "").strip() or "draft"
if workflow_status not in WORKFLOW_LABELS:
    raise HTTPException(status_code=400, detail="Unsupported workflow status")
return workflow_status
```

空值默认为 "draft"，非法值直接抛出 HTTP 400。

## 9.2 草稿创建

系统支持三种草稿创建方式：手动创建、文件上传和 AI 批处理导入。

### 9.2.1 手动创建

路由： POST /api/editorial/articles

函数： create_editorial_article(payload: dict)

接受字段：

| 字段 | 必填 | 说明 |
|---|---|---|
| title | 是 | 标题，不能为空 |
| content_markdown | 是 | Markdown 正文 |
| source_markdown | 否 | 原始来源内容（为空时复制 content_markdown） |
| subtitle | 否 | 副标题 |
| author | 否 | 作者 |
| organization | 否 | 机构名称 |
| publish_date | 否 | 发布日期（默认当天） |
| source_url | 否 | 原文链接 |
| cover_image_url | 否 | 封面图 URL |
| primary_column_slug | 否 | 主栏目 slug |
| article_type | 否 | 文章类型 |
| main_topic | 否 | 主题 |
| access_level | 否 | 访问级别（默认 "public"） |
| layout_mode | 否 | 排版模式（默认 "auto"） |

| | | |
|---|---|---|
| formatting_notes | 否 | 排版备注 |
| slug | 否 | 自定义 URL slug |

创建流程：

1. 校验 title 和 content_markdown 非空。
2. source_markdown 和 content_markdown 互相兜底：任一为空时取另一方的值。
3. 调用 strip_markdown() 从 Markdown 生成纯文本。
4. 调用 _extract_excerpt() 从纯文本提取前 140 字符摘要。
5. 生成唯一 slug：_unique_slug() 基于标题或自定义 slug 生成候选值，若数据库中已存在同名 slug，则追加 SHA1 哈希后缀。
6. 初始 status = "draft"，workflow_status = "draft"。
7. tag_payload_json = "[]"，html_web = NULL，html_wechat = NULL。

### 9.2.2 文件上传

路由： POST /api/editorial/upload

函数： create_editorial_from_upload(filename, raw_bytes)

支持的文件格式：

| 后缀 | 解析方式 |
|---|---|
| .md | 直接按文本读取，尝试多种编码 |
| .txt | 同上 |
| .html / .htm | 通过 _html_to_markdown_like() 转换为 Markdown 风格文本 |
| .docx | 通过 _docx_to_markdown_like() 解析 Word XML |

#### 文本编码检测

_decode_text_bytes() 按以下顺序尝试解码：

1. utf-8-sig（带 BOM 的 UTF-8）
2. utf-8
3. gb18030（中文 Windows 常用编码）
4. utf-16
5. 最后回退到 utf-8 的 errors="replace" 模式

#### DOCX 解析

_docx_to_markdown_like() 处理流程：

1. 将 bytes 作为 ZIP 文件打开。
2. 读取 word/document.xml（不存在则抛出 400）。
3. 遍历所有 <w:p> 段落元素。
4. 检查段落样式：Heading1 → # ，Heading2 → ## ，Heading3 → ### 。
5. 提取所有 <w:t> 文本节点，拼接为段落文本。
6. 用双换行连接所有段落。

#### HTML → Markdown 转换

_html_to_markdown_like() 使用正则表达式进行标签转换：

- <br> → 换行
- </p> → 双换行
- <h1> → # ，<h2> → ## ，<h3> → ###
- <li> → -
-  / <b> → ...
-  / <i> → ...
- 其余标签 → 空格
- 最后 html.unescape() 还原 HTML 实体

#### 标题推断

上传完成后，系统尝试从文件内容推断标题：

1. 若第一个非空行以 # 开头，去掉标记符号作为标题，剩余内容作为正文。
2. 若第一个非空行不超过 48 个字符，作为标题。
3. 否则使用文件名（去掉扩展名）作为标题。

最终调用 create_editorial_article() 创建草稿。

## 9.3 AI 批处理导入

路由： POST /api/editorial/source-articles/{article_id}/import-ai

函数： import_editorial_ai_draft(source_article_id, payload)

此功能将 article_ai_outputs 表中的 AI 格式化结果导入编辑后台，创建或更新编辑稿。

### 9.3.1 前置条件

1. 对应的 article_ai_outputs 记录必须存在，且 format_ready = True。
2. formatted_markdown_zh（中文格式化版本）不能为空。

### 9.3.2 导入流程

```
  请求 {source_article_id, editorial_id?}
 |
 v
获取 AI 输出：get_article_ai_output_detail(source_article_id)
 |-- format_ready = False → 400 错误
 |-- formatted_markdown_zh 为空 → 400 错误
 |
 v
从 articles 表读取源文章信息
从 article_tags 获取源文章标签（最多 12 个）
从 article_columns 获取源文章主栏目
 |
 v
editorial_id 是否已指定?
 |
 |-- 是（更新已有编辑稿）
 |   |-- 已发布 → 400 "Published editorial article cannot be overwritten"
 |   |-- 未发布 → UPDATE editorial_articles SET ...
 |
 |-- 否（创建新编辑稿）
 |   → INSERT INTO editorial_articles
 |
 v
返回完整编辑稿详情
```

### 9.3.3 导入时自动设置的字段

| 字段 | 来源 |
|---|---|
| title | 源文章标题 |
| author | 已有编辑稿保留原值，否则 "Fudan Business Knowledge Editorial Desk" |
| organization | 已有编辑稿保留原值，否则源文章的 primary_org_name |
| source_markdown | 源文章的原始 content |
| content_markdown | AI 的 formatted_markdown_zh |
| excerpt | 优先使用 AI 的 summary_zh，否则源文章 excerpt |
| tag_payload_json | 源文章的标签（序列化为 JSON） |
| primary_column_slug | 源文章的栏目 slug，默认 "insights" |
| access_level | 源文章的 access_level |
| layout_mode | "auto" |
| formatter_model | AI 输出的 format_model |
| ai_synced_at | 当前时间戳 |
| html_web / html_wechat | 重置为 NULL（需重新渲染） |

### 9.3.4 相关查询端点

| 路由 | 说明 |
|---|---|
| GET /api/editorial/source-articles | 列出可导入的源文章（通过 article_ai_output_service） |
| GET /api/editorial/source-articles/{id}/ai-output | 查看某篇源文章的 AI 处理详情 |

## 9.4 AI 自动排版

路由： POST /api/editorial/articles/{editorial_id}/auto-format

函数： auto_format_editorial_article(editorial_id, payload)

### 9.4.1 排版模型

使用 Google Gemini gemini-3-flash-preview 模型（配置项 GEMINI_EDITORIAL_FORMAT_MODEL）。

### 9.4.2 四种排版模式

在 ai_service.py 的 LAYOUT_MODE_GUIDANCE 字典中定义：

| layout_mode | 中文标签 | 排版指引 |
|---|---|---|
| auto | 自动排版 | 默认使用高端微信公众号长文风格，短导言、清晰 H2/H3 层级、信息密集但可读 |
| insight | 深度长文 | 偏重深度分析，显式论证结构，章节过渡，稍长的分析段落 |
| briefing | 快报简版 | 快速扫描友好，短章节、列表、关键要点、紧凑摘要 |
| interview | 访谈实录 | 保留说话人轮次和问答逻辑，同时加强标题、导言和可读性段落分隔 |

### 9.4.3 排版流程

```
  请求 {editorial_id, source_markdown?, layout_mode?, formatting_notes?}
 |
 v
读取当前编辑稿（已发布则拒绝 → 400）
 |
 v
确定 source_markdown：
 请求参数 > current.source_markdown > current.content_markdown
 |-- 全部为空 → 400 "Source content is empty"
 |
 v
构建 Gemini 提示词，包含：
 - 系统角色："top-tier Chinese WeChat official-account editorial CMS"
 - 8 条硬性排版规则
 - 排版模式指引（layout_mode_guidance）
 - 编辑备注（formatting_notes）
 - 文章元数据（title, excerpt, main_topic, article_type, organization, tags）
 - 完整 source_markdown
 |
 v
调用 Gemini API → 获取格式化后的 Markdown
 |-- 返回为空 → 502 "Auto-formatting returned empty content"
 |-- 返回不以 # 开头 → 自动补上 H1 标题
 |
 v
 ... (共 34 行，已截断显示)
```

### 9.4.4 Gemini 提示词中的 8 条硬性规则

1. 使用模型判断力创建可读、专业的中文长文排版，类似高端微信发布工具的自动排版模式。
2. 保留每个重要事实、数字、公司名称、引语和结论，不得捏造事实。
3. 第一个标题必须是单个 H1 标题。
4. 在标题后适当添加一段简短导言。
5. 使用清晰的 H2/H3 层级结构，短段落、列表和引用。
6. 保留访谈、对话和演讲的原始结构。
7. 不输出 HTML，不输出代码块围栏，仅输出 Markdown。
8. 不在答案中提及模型、提示词或排版规则。

## 9.5 AI 自动标签

路由： POST /api/editorial/articles/{editorial_id}/autotag

函数： generate_editorial_tags(editorial_id)

### 9.5.1 双源标签生成策略

系统采用 启发式 + Gemini 合并去重 的双源策略：

#### 启发式标签（Heuristic）

调用 tag_engine._derive_tag_entries()，基于文章的标题、主题、摘要、正文、文章类型和机构名等字段，通过规则匹配生成标签。输入数据结构：

```
  {
  "title": "...",
  "main_topic": "...",
  "excerpt": "...",
  "content": plain_text_content,
  "search_text": plain_text_content,
  "tag_text": "",
  "article_type": "...",
  "series_or_column": "",
  "people_text": "",
  "org_text": organization,
}
```

同时传入数据库中所有已知标签的 {name: category} 映射，用于分类校准。

#### Gemini AI 标签

调用 ai_service.suggest_editorial_metadata(title, content)，请求 Gemini 返回 JSON 结构，包含建议的标签列表（每个标签有 name、category、confidence 字段）。

### 9.5.2 合并去重逻辑 — _dedupe_tags()

```
def _dedupe_tags(entries: list[tuple[str, str, float]]) -> list[dict]
```

1. 将所有标签条目按 (name, category) 对进行合并，取最高 confidence。
2. 标签名去前后空白，空名跳过。
3. category 若不在 TAG_PRIORITY 中，回退为 "topic"。
4. 排序规则：
- 一级：按 TAG_PRIORITY 排序（industry=0 > topic=1 > type=2 > entity=3 > series=4）
- 二级：按 confidence 倒序
- 三级：按名称字典序
5. 截取前 12 个标签。
6. 为每个标签生成 slug（slugify("{category}-{name}")，截断 80 字符）和颜色（从 COLOR_BY_CATEGORY 映射）。

### 9.5.3 标签分类优先级

| category | 优先级 | 说明 |
|---|---|---|
| industry | 0 | 行业标签，最高优先 |
| topic | 1 | 主题标签 |
| type | 2 | 类型标签 |
| entity | 3 | 实体标签 |
| series | 4 | 系列标签 |

### 9.5.4 Gemini 附带的元数据更新

除标签外，Gemini 还可能返回以下建议，系统会在有值时覆盖现有字段：

| AI 返回字段 | 更新目标 | 截断限制 |
|---|---|---|
| excerpt | editorial_articles.excerpt | 180 字符 |
| article_type | editorial_articles.article_type | 32 字符 |
| main_topic | editorial_articles.main_topic | 48 字符 |
| column_slug | editorial_articles.primary_column_slug | 需在 VALID_COLUMN_SLUGS 内 |

### 9.5.5 栏目 slug 回退推断

当 AI 未返回有效 column_slug 且当前编辑稿也没有栏目时，调用 _infer_column_slug() 基于关键词匹配推断：

| 匹配关键词 | 推断栏目 |
|---|---|
| 研究、论文、学术、案例教学 | research |
| 院长、教授、复旦、管理学院 | deans-view |
| 标签中含 industry 类别 | industry |
| 默认 | insights |

## 9.6 工作流状态转换规则

路由： POST /api/editorial/articles/{editorial_id}/workflow

函数： update_editorial_workflow(editorial_id, payload)

### 9.6.1 可用操作（action）

| action | 转换目标状态 | 说明 |
|---|---|---|
| save_draft | draft | 回退/保存为草稿，清除定时发布设置 |
| submit_review | in_review | 提交审核，记录 submitted_at |
| approve | approved | 审核通过，记录 approved_at |
| schedule | scheduled | 设定定时发布，必须提供 scheduled_publish_at |

### 9.6.2 前置校验

- 已发布（status == "published"）的文章 不可 进行任何工作流操作 → 400 "Published article workflow is locked"。
- schedule 操作必须提供合法的 scheduled_publish_at datetime 字符串 → 400 "Scheduled publish time is required"。

### 9.6.3 状态转换时更新的字段

```
  UPDATE editorial_articles
SET workflow_status = ?,
  review_note = ?,
  scheduled_publish_at = ?,
  submitted_at = ?,
  approved_at = ?,
  updated_at = ?
WHERE id = ?
```

各操作对字段的影响：

| 字段 | save_draft | submit_review | approve | schedule |
|---|---|---|---|---|
| workflow_status | draft | in_review | approved | scheduled |
| submitted_at | 保持不变 | 更新为当前时间 | 保持不变 | 保持不变 |
| approved_at | 保持不变 | 保持不变 | 更新为当前时间 | 保持不变 |
| scheduled_publish_at | 清空为 NULL | 清空为 NULL | 保持不变 | 更新为指定时间 |
| review_note | 若请求中有值则更新 | 同左 | 同左 | 同左 |

### 9.6.4 定时发布时间格式

_normalize_schedule_datetime() 函数将输入解析为 ISO 格式并去除微秒。非法格式抛出 400 "Invalid scheduled publish time"。

## 9.7 HTML 渲染

路由： POST /api/editorial/articles/{editorial_id}/render-html

函数： render_editorial_html(editorial_id)

系统将 Markdown 内容渲染为两种 HTML 版本：标准 Web 版和微信公众号版。渲染引擎位于 html_renderer.py。

### 9.7.1 渲染流程

```
  请求渲染
 |
 v
获取编辑稿详情
 |
 v
标签是否为空?
 |-- 是 → 自动调用 generate_editorial_tags() 先生成标签
 |
 v
调用 render_editorial_package(article, tags)
 → 返回 {html_web, html_wechat, summary}
 |
 v
更新 editorial_articles 表:
 SET excerpt, html_web, html_wechat, updated_at
 |
 v
返回 {article_id, html_web, html_wechat, summary}
```

### 9.7.2 Markdown → HTML 转换引擎

markdown_to_html() 函数是一个纯 Python 实现的 Markdown 解析器，支持：

| 语法元素 | HTML 输出 |
|---|---|
| # / ## / ### / #### | <h1> ~ <h4> |
| 段落文本 | <p> |
| ` ` ` 代码块 | <pre><code> |
| - / * / + 无序列表 | <ul><li> |
| 1. 2. 有序列表 | <ol><li> |
| > 引用 | <blockquote><p> |
| --- / *** 分隔线 | <hr /> |
| 加粗 |  |
| 斜体 |  |
| ` 行内代码 ` | <code> |
| [文本](URL) | <a href="..." target="_blank"> |

所有文本输出均经过 html.escape() 处理以防止 XSS。

### 9.7.3 Web 版 HTML（标准版）

Web 版使用完整的 CSS 类名和 CSS 变量系统，特点包括：

- 配色方案：采用复旦蓝（#0d0783）和复旦橙（#ea6b00）为品牌色
- 布局：max-width: 940px 居中容器
- Hero 区域：渐变背景卡片，包含标题、副标题、作者、机构、发布日期、摘要
- 标签芯片：圆角胶囊样式，颜色来自标签的 color 字段
- 正文区域：白色圆角卡片，衬线标题字体 + 无衬线正文字体
- 页脚：来源链接和生成模板标记

中英文字体差异：

| 场景 | 中文 | 英文 |
|---|---|---|
| 正文字 | Inter, PingFang SC, Microsoft YaHei | Aptos, Segoe UI, Helvetica Neue, Arial |
| 标题字 | Noto Serif SC, Songti SC, serif | Iowan Old Style, Palatino Linotype, Georgia |
| 正文行 | 2 | 1.82 |
| 摘要行 | 1.9 | 1.78 |
| 正文字 | 16px | 17px |
| 标题字号 | 52px | 48px |

### 9.7.4 微信公众号版 HTML（内联样式版）

微信版专为微信公众号排版设计，所有样式均使用 内联 CSS（style="..."），因为微信编辑器不支持外部样式表和大部分 CSS 类名。

特点：

- 所有样式直接写在元素的 style 属性中
- 使用 wechat-shell（max-width: 760px）容器
- wechat-card 白色圆角卡片背景
- 标签使用 wechat-tag span 样式
- 正文内容同样从 Markdown 转换而来

### 9.7.5 渲染包输入数据

render_editorial_package() 从 article 字典中提取：

| 字段 | 用途 | 英文回退值 |
|---|---|---|
| content_markdown | 正文渲染 | - |
| excerpt | 摘要区域 | "Summary unavailable" |
| title | 标题 | - |
| subtitle | 副标题（英文模式下如含 CJK 则清空） | - |
| author | 作者 | "Editorial Desk" |
| organization | 机构 | "Fudan Business Knowledge" |
| publish_date | 发布日期 | - |
| source_url | 来源链接 | - |

英文模式下，标签列表会过滤掉含 CJK 字符的标签。

### 9.7.6 纯文本提取 — strip_markdown()

```
def strip_markdown(content: str) -> str
```

从 Markdown 内容中移除所有格式标记，保留纯文本。处理顺序：

1. 移除代码块（```...```）
2. 移除行内代码标记
3. 移除图片语法 ![]()
4. 提取链接文本 [text](url) → text
5. 移除标题标记 #
6. 移除引用标记 >
7. 移除列表标记 - * +
8. 移除有序列表标记 1.
9. 移除 Markdown 特殊字符 * _ ~ # > -
10. 压缩多余换行和空白

## 9.8 发布流程

路由： POST /api/editorial/articles/{editorial_id}/publish

函数： publish_editorial_article(editorial_id)

发布是编辑稿生命周期的终态操作，将编辑稿内容正式写入公开的 articles 表。

### 9.8.1 发布前自动补全

1. 若 tags 为空 → 自动调用 generate_editorial_tags() 生成标签。
2. 若 html_web 或 html_wechat 为空 → 自动调用 render_editorial_html() 渲染 HTML。

### 9.8.2 发布详细流程

```
  编辑稿就绪
 |
 v
自动补全标签和 HTML（如缺失）
 |
 v
生成公开 slug：_unique_slug(connection, "articles", ...)
 |
 v
editorial_articles.article_id 是否已关联且目标文章存在？
 |
 |-- 是（更新已有文章）
 |   → UPDATE articles SET doc_id, slug, title, publish_date,
 |     content, excerpt, main_topic, article_type, tag_text,
 |     org_text, search_text, word_count, access_level, ...
 |
 |-- 否（创建新文章）
 |   → INSERT INTO articles (...)
 |   → 获取 article_id = last_insert_rowid()
 |
 v
写入标签关联：
 DELETE FROM article_tags WHERE article_id = ?
 对每个标签：
  _ensure_tag(name, category) → 获取或创建 tag_id
 ... (共 61 行，已截断显示)
```

### 9.8.3 写入 articles 表的字段映射

| articles 字段 | 来源 |
|---|---|
| doc_id | SHA1 哈希 "editorial:{id}:{slug}" 取前 20 位 |
| slug | 基于编辑稿 slug 生成的唯一 slug |
| relative_path | "editorial/{slug}.md" |
| source | "editorial" |
| source_mode | "cms" |
| title | 编辑稿标题 |
| publish_date | 编辑稿发布日期 |
| link | 编辑稿的 source_url |
| content | 纯文本内容（plain_text_content） |
| excerpt | 编辑稿摘要 |
| main_topic | 编辑稿主题 |
| article_type | 编辑稿类型 |
| series_or_column | 固定为 "内容后台" |
| tag_text | 标签名称以 `\` 连接 |
| org_text | 机构名称 |
| search_text | "{title} {excerpt} {content[:4000]}" 用于全文搜索 |
| word_count | 纯文本字数（去除空白后的字符数） |
| access_level | 编辑稿的访问级别 |

### 9.8.4 搜索缓存刷新

发布完成后调用 refresh_search_cache()（来自 rag_engine 模块），确保新发布的文章可被即时搜索。

## 9.9 导出功能

路由： GET /api/editorial/articles/{editorial_id}/export?variant=web|wechat

函数： export_editorial_html(editorial_id, variant)

### 9.9.1 导出流程

1. 获取编辑稿详情。
2. 若 HTML 未渲染（html_web 或 html_wechat 为空），自动触发 render_editorial_html() 渲染。
3. 校验 variant 参数：仅接受 "web" 或 "wechat"，否则 400。
4. 根据 variant 选择对应的 HTML 内容。
5. 生成下载文件名：{slug}-{variant}.html。
6. 返回 Response，MIME 类型为 text/html; charset=utf-8，设置 Content-Disposition: attachment 头触发浏览器下载。

### 9.9.2 导出格式对比

| 属性 | Web 版 | 微信版 |
|---|---|---|
| CSS 方式 | <style> 标签内的类名/变量 | 内联 style 属性 |
| 目标平台 | 浏览器/网站嵌入 | 微信公众号编辑器粘贴 |
| 容器宽度 | 940px | 760px |
| 设计风格 | 渐变 Hero + 卡片布局 | 简洁白卡 + 紧凑排版 |
| 标签展示 | 圆角芯片（CSS 变量色） | 简单 span 标签 |
| 适配场景 | 桌面端阅读 | 移动端微信阅读 |

## 9.10 EditorialWorkbenchPage 前端 UI 交互

编辑后台的前端以 EditorialWorkbenchPage 组件为核心，提供完整的 CMS 工作台界面。以下结合后端 API 路由描述前端的主要 UI 交互流程。

### 9.10.1 权限控制

所有编辑后台路由（/api/editorial/*）均通过 _require_editorial_admin() 函数进行权限校验：

```
def _require_editorial_admin(authorization, debug_user_id, debug_user_email):
user = get_authenticated_user(authorization, ...)
require_admin_profile(user)
```

前端需确保当前用户已以 admin 身份登录，否则所有 API 调用将返回 401 或 403。

### 9.10.2 仪表盘视图

API： GET /api/editorial/dashboard?limit=6

前端展示的仪表盘数据结构：

| 指标 | 字段 | 说明 |
|---|---|---|
| 草稿数 | draft_count | workflow_status = "draft" 的文章数 |
| 已发布数 | published_count | workflow_status = "published" 的文章数 |
| 待审核数 | pending_review_count | workflow_status = "in_review" 的文章数 |
| 已通过数 | approved_count | workflow_status = "approved" 的文章数 |
| 定时发布数 | scheduled_count | workflow_status = "scheduled" 的文章数 |
| 最近发布时间 | latest_published_at | 最后一篇已发布文章的时间 |
| 导出就绪数 | export_ready_count | html_web 和 html_wechat 均非空的文章数 |
| 工作流分布 | workflow_counts | 各状态的数量列表 |
| 最近编辑 | recent_items | 最近更新的文章列表 |

### 9.10.3 文章列表与筛选

API： GET /api/editorial/articles?limit=40&status=...&workflow_status=...

前端可按以下维度筛选：

- status -- 文章发布状态（如 "draft", "published"）
- workflow_status -- 工作流状态

列表按 updated_at DESC 排序，最大返回 100 条。

### 9.10.4 文章详情与编辑

查看详情： GET /api/editorial/articles/{editorial_id}

返回完整的 EditorialArticleDetail，包含所有元数据、Markdown 正文、标签、HTML 渲染结果和工作流状态。若文章有关联的源文章（source_article_id），还会附带 source_article_ai 字段。

更新文章： PUT /api/editorial/articles/{editorial_id}

可更新的字段列表：

```
allowed_fields = {
"title", "subtitle", "author", "organization", "publish_date",
"source_url", "cover_image_url", "primary_column_slug",
"article_type", "main_topic", "access_level", "source_markdown",
"layout_mode", "formatting_notes", "content_markdown",
}
```

更新时自动重新生成：

- plain_text_content（从 content_markdown 提取纯文本）

- excerpt（从纯文本提取摘要）
- html_web 和 html_wechat 被清空为 NULL（需重新渲染）

### 9.10.5 前端操作流程汇总

以下是编辑人员典型的工作流程：

```
  1. 创建文稿
 |-- 手动创建：填写标题和正文 → POST /api/editorial/articles
 |-- 上传文件：拖拽 .md/.docx/.txt → POST /api/editorial/upload
 |-- AI 导入：选择源文章 → POST /api/editorial/source-articles/{id}/import-ai
 |
 v
2. 编辑与完善
 |-- 修改元数据/正文 → PUT /api/editorial/articles/{id}
 |-- AI 自动排版 → POST /api/editorial/articles/{id}/auto-format
 |-- AI 自动标签 → POST /api/editorial/articles/{id}/autotag
 |
 v
3. 预览渲染
 |-- 渲染 HTML → POST /api/editorial/articles/{id}/render-html
 |-- 导出 HTML → GET /api/editorial/articles/{id}/export?variant=web|wechat
 |
 v
4. 审核流程
 |-- 提交审核 → POST .../workflow {action: "submit_review"}
 |-- 审核通过 → POST .../workflow {action: "approve"}
 |-- 定时发布 → POST .../workflow {action: "schedule", scheduled_publish_at: "..."}
 |-- 回退草稿 → POST .../workflow {action: "save_draft"}
 |
 v
5. 正式发布
 ... (共 30 行，已截断显示)
```

### 9.10.6 编辑稿序列化字段全表

_serialize_editorial_row() 返回的完整字段：

| 字段 | 类型 | 说明 | |
|---|---|---|---|
| id | int | 编辑稿 ID | |
| article_id | `int \ | None` | 关联的已发布文章 ID |
| source_article_id | `int \ | None` | 关联的源文章 ID |
| slug | str | URL slug | |
| title | str | 标题 | |
| subtitle | `str \ | None` | 副标题 |
| author | `str \ | None` | 作者 |
| organization | `str \ | None` | 机构 |
| publish_date | str | 发布日期 | |
| source_url | `str \ | None` | 原文链接 |
| cover_image_url | `str \ | None` | 封面图 URL |
| primary_column_slug | `str \ | None` | 主栏目 slug |
| article_type | `str \ | None` | 文章类型 |
| main_topic | `str \ | None` | 主题 |
| access_level | str | 访问级别（public/member/paid） | |
| access_label | str | 访问级别中文标签 | |
| workflow_status | str | 工作流状态 | |
| workflow_label | str | 工作流状态中文标签 | |
| review_note | `str \ | None` | 审核备注 |
| scheduled_publish_at | `str \ | None` | 定时发布时间 |
| submitted_at | `str \ | None` | 提交审核时间 |
| approved_at | `str \ | None` | 审核通过时间 |
| ai_synced_at | `str \ | None` | AI 同步时间 |
| layout_mode | str | 排版模式 | |
| formatting_notes | `str \ | None` | 排版备注 |
| formatter_model | `str \ | None` | 使用的 AI 排版模型 |
| last_formatted_at | `str \ | None` | 最后排版时间 |
| source_markdown | str | 原始 Markdown 内容 | |
| content_markdown | str | 当前 Markdown 正文 | |

| | | | |
|---|---|---|---|
| plain_text_content | str | 纯文本内容 | |
| excerpt | str | 摘要 | |
| tags | list[dict] | 标签列表 | |
| html_web | `str \ | None` | Web 版 HTML |
| html_wechat | `str \ | None` | 微信版 HTML |
| status | str | 文章状态 | |
| created_at | str | 创建时间 | |
| updated_at | str | 更新时间 | |
| published_at | `str \ | None` | 发布时间 |

## 9.11 API 路由汇总

| 方法 | 路径 | 说明 | 权限 |
|---|---|---|---|
| GET | /api/editorial/dashboard | 仪表盘概览 | admin |
| GET | /api/editorial/articles | 编辑稿列表 | admin |
| POST | /api/editorial/articles | 创建编辑稿 | admin |
| POST | /api/editorial/upload | 上传文件创建编辑稿 | admin |
| GET | /api/editorial/articles/{id} | 编辑稿详情 | admin |
| PUT | /api/editorial/articles/{id} | 更新编辑稿 | admin |
| POST | /api/editorial/articles/{id}/auto-format | AI 自动排版 | admin |
| POST | /api/editorial/articles/{id}/autotag | AI 自动标签 | admin |
| POST | /api/editorial/articles/{id}/workflow | 工作流状态转换 | admin |
| POST | /api/editorial/articles/{id}/render-html | 渲染 HTML | admin |
| GET | /api/editorial/articles/{id}/export | 导出 HTML 文件 | admin |
| POST | /api/editorial/articles/{id}/publish | 正式发布 | admin |
| GET | /api/editorial/source-articles | 可导入的源文章列表 | admin |
| GET | /api/editorial/source-articles/{id}/ai-output | 源文章 AI 输出详情 | admin |
| POST | /api/editorial/source-articles/{id}/import-ai | AI 批处理导入 | admin |

## 9.12 关键源文件索引

| 文件路径 | 职责 |
|---|---|
| backend/services/editorial_service.py | 编辑后台核心业务逻辑（草稿、导入、排版、标签、发布） |
| backend/services/html_renderer.py | Markdown → HTML 渲染引擎（Web 版 + 微信版） |
| backend/services/ai_service.py | Gemini API 调用（排版和标签建议） |
| backend/services/article_ai_output_service.py | AI 批处理输出管理 |
| backend/services/tag_engine.py | 启发式标签推导引擎 |
| backend/services/rag_engine.py | 搜索缓存刷新 |
| backend/services/membership_service.py | 访问级别标准化与管理员权限校验 |
| backend/routers/editorial.py | 编辑后台 API 路由定义 |
| backend/models/schemas.py | Pydantic 请求/响应模型定义 |
| backend/scripts/build_business_db.py | slugify 工具函数和颜色映射 |

# 10 — 前端架构

本文档覆盖"复旦管院智识库"前端（React 18 + Vite）的完整架构实现，包括应用启动链路、Provider 层级、路由设计、认证体系、API 客户端、访客追踪、角色体验系统、国际化、布局组件和共享组件。 源码入口：frontend/src/main.jsx

## 10.1 Provider 栈

应用采用严格的 Provider 嵌套顺序，每一层向下层提供特定的 Context：

```
StrictMode
└── BrowserRouter        ← react-router-dom，提供路由能力
  └── LanguageProvider  ← 国际化上下文（语言、翻译函数 t()）
    └── AuthProvider ← 认证上下文（用户、会话、角色、权限标志）
      └── App  ← 路由表 + Suspense 懒加载
```

### 10.1.1 各层职责

| 层级 | 组件 | 来源文件 | 职责 |
|---|---|---|---|
| 1 | StrictMode | React 内置 | 开发模式下双渲染检测副作用 |
| 2 | BrowserRouter | react-router-dom | 提供 useNavigate、useLocation、Routes 等路由原语 |
| 3 | LanguageProvider | src/i18n/LanguageProvider.jsx | 管理当前语言状态（zh / en），暴露 t(key) 翻译函数、isEnglish 标志、setLanguage / toggleLanguage 切换方法 |
| 4 | AuthProvider | src/auth/AuthProvider.jsx | 管理认证状态（登录/登出）、会员信息（membership）、商务档案（businessProfile）、角色首页路径（roleHomePath）、权限标志集（isAdmin / isPaidMember / canUseAiAssistant 等） |
| 5 | App | src/App.jsx | 使用 Suspense + lazy() 实现路由级代码分割，定义全部路由及守卫规则 |

### 10.1.2 嵌套顺序的设计约束

- LanguageProvider 在 AuthProvider 之上：AuthProvider 内部需要调用 useLanguage() 获取 isEnglish 以生成本地化错误提示（如"邮箱或密码不正确"vs."Incorrect email or password."），因此语言层必须先于认证层。
- BrowserRouter 在最外层：LanguageProvider 在初始化时需要读取 window.location.search 中的 ?lang=en 参数，因此 BrowserRouter 必须先于 LanguageProvider 挂载。
- AuthProvider 在 App 之上：App 中的 ProtectedRoute 需要消费认证上下文中的权限标志来做路由守卫。

## 10.2 路由设计

所有路由定义在 App.jsx 中，使用 react-router-dom 的 <Routes> + <Route> 声明式路由。

### 10.2.1 懒加载策略

所有页面组件均通过 React.lazy() + 动态 import() 加载，在路由匹配时才会触发对应 chunk 的下载。
全局 <Suspense> 的 fallback 使用 <PageFallback />，显示一行翻译后的"加载中"提示文字。

```
const HomePage = lazy(() => import('./pages/HomePage.jsx'))
// ... 其他 22 个页面同理
```

### 10.2.2 完整路由表

所有路由均嵌套在 <Route element={<SiteLayout />}> 下，SiteLayout 提供统一的 Navbar + Footer + 浮动 ChatPanel。

| 路径 | 组件 | 懒加载 | 守卫类型 | 守卫参数 | 说明 |
|---|---|---|---|---|---|
| / | HomePage | 是 | 无 | — | 公开首页，所有角色可访问 |
| /login | LoginPage | 是 | 无 | — | 登录页面，已登录用户会自动跳转到角色首 |
| /search | SearchPage | 是 | 无 | — | 搜索页面，智能搜索需付费会员权限 |
| /analytics | AnalyticsPage | 是 | requireAdmin | fallbackPath="/" | 内容分析仪表板 |
| /admin | AdminConsolePage | 是 | requireAdmin | fallbackPath="/" | 管理员控制台 |
| /admin/memberships | AdminMembershipsPage | 是 | requireAdmin | fallbackPath="/admin" | 会员管理 |
| /article/:id | ArticlePage | 是 | 无 | — | 文章详情页，按 access_level 在后端控制可见性 |

| | | | | | |
|---|---|---|---|---|---|
| /audio | MediaHubPage | 是 | 无 | — | 音频媒体页，kind="audio" |
| /column/:slug | ColumnPage | 是 | 无 | — | 栏目详情页 |
| /tag/:slug | TagPage | 是 | 无 | — | 标签文章列表 |
| /membership | MembershipPage | 是 | 无 | — | 会员方案与权益页 |
| /following | MyFollowingPage | 是 | 无 | — | 我的关注（需登录才有数据） |
| /organizations | OrganizationsPage | 是 | 无 | — | 机构索引列表 |
| /organization/:slug | OrganizationPage | 是 | 无 | — | 单个机构的文章聚合 |
| /topic/:slug | TopicPage | 是 | 无 | — | 专题详情页（后端按权限判断是否返回数据 |
| /topics | TopicsPage | 是 | 无 | — | 专题广场（后端按权限返回 401/403 或数据） |
| /time-machine | TimeMachinePage | 是 | 无 | — | 时光机 |
| /chat | ChatPage | 是 | requireAiAssistant | fallbackPath="/membership" | AI 助理全页模式 |
| /commercial | CommercialPage | 是 | 无 | — | 商业方案介绍 |
| /commercial/leads | CommercialLeadsPage | 是 | requireAdmin | fallbackPath="/admin" | 商务线索管理 |
| /editorial | EditorialWorkbenchPage | 是 | requireAdmin | fallbackPath="/admin" | 编辑工作台 |
| /media-studio | MediaStudioPage | 是 | requireAdmin | fallbackPath="/admin" | 媒体管理后台 |
| /me | MyLibraryPage | 是 | 无 | — | 我的资产库 |
| /video | MediaHubPage | 是 | 无 | — | 视频媒体页，kind="video" |
| * | Navigate to="/" | — | — | — | 未匹配路由重定向到首页 |

### 10.2.3 路由参数化设计

- 动态段（:id、:slug）：通过 useParams() 在页面组件内获取。
- 查询参数：SearchPage 使用 useSearchParams() 解析 q、mode、sort、start_date、end_date、tags、columns 等参数。LoginPage 读取 redirect 参数以在登录后跳转回原页面。
- Props 传参：MediaHubPage 通过 JSX prop kind="audio" / kind="video" 区分音频和视频两种模式，复用同一个页面组件。

## 10.3 认证系统

认证系统由四个文件构成：

| 文件 | 职责 |
|---|---|
| auth/AuthContext.js | 创建 AuthContext，暴露 useAuth() hook |
| auth/AuthProvider.jsx | 认证状态管理的核心 Provider |
| auth/AuthDialog.jsx | 弹窗式登录对话框 |
| auth/debugAuth.js | localStorage 调试认证持久化 |
| auth/ProtectedRoute.jsx | 路由守卫组件 |

### 10.3.1 AuthProvider 状态管理

AuthProvider 维护以下核心状态：

| 状态字段 | 类型 | 初始值 | 说明 | |
|---|---|---|---|---|
| session | `object \ | null` | null | 会话对象，包含 access_token 和 user |
| user | `object \ | null` | null | 用户对象，包含 id、email、user_metadata.display_name |
| loading | boolean | true | 初始认证状态加载中 | |
| submitting | boolean | false | 登录请求提交 | |
| authMessage | string | '' | 认证成功提示 | |
| authError | string | '' | 认证错误提示 | |
| dialogOpen | boolean | false | 登录对话框是否打开 | |
| membership | object | GUEST_MEMBERSHIP | 会员信息对象 | |
| businessProfile | object | GUEST_BUSINESS_PROFILE | 商务档案 | |

| | | | |
|---|---|---|---|
| backendAuthEnabled | boolean | false | 后端认证是否启用 |
| roleHomePath | string | '/' | 角色默认首页 |
| authMode | string | 'password' | 认证模式 |

### 10.3.2 GUEST_MEMBERSHIP 默认对象

当用户未登录时，使用如下访客会员对象：

```
 {
 tier: 'guest',
 tier_label: '访客',
 status: 'anonymous',
 status_label: '未登录',
 is_authenticated: false,
 is_admin: false,
 can_access_member: false,
 can_access_paid: false,
 user_id: null,
 email: null,
 note: null,
 started_at: null,
 expires_at: null,
 benefits: [],
}
```

### 10.3.3 GUEST_BUSINESS_PROFILE 默认对象

```
 {
 user_id: null,
 email: null,
 display_name: '访客',
 title: '公开访客',
 organization: null,
 bio: null,
 tier: 'guest',
 tier_label: '访客',
 status: 'anonymous',
 status_label: '未登录',
 role_home_path: '/',
 auth_source: 'guest',
 locale: 'zh-CN',
 is_seed: false,
 is_authenticated: false,
 is_admin: false,
}
```

### 10.3.4 派生权限标志

AuthProvider 向下游暴露以下派生标志（通过 useAuth() 获取）：

| 标志 | 计算逻辑 | 用途 |
|---|---|---|
| isAuthenticated | Boolean(user) | 是否已登录 |
| isGuest | !user | 是否为访客 |
| isMember | Boolean(membership?.can_access_member) | 是否具有会员访问权限 |
| isPaidMember | Boolean(membership?.can_access_paid) | 是否为付费会员 |
| isAdmin | Boolean(membership?.is_admin) | 是否为管理员 |
| canUseAiAssistant | `Boolean(user) && (can_access_paid \| \| is_admin)` | 是否可使用 AI 助理 |
| authEnabled | `backendAuthEnabled \| \| loading` | 认证系统是否可用（loading 期间假定可用） |

### 10.3.5 signInWithPassword 登录流程

```
  用户提交 email + password
 ↓
调用 loginWithPassword({ email, password }) → POST /api/auth/login
 ↓
成功响应 payload 包含:
 - payload.user (id, email)
 - payload.membership (tier, benefits, etc.)
 - payload.business_profile (display_name, role_home_path, etc.)
 - payload.role_home_path
 ↓
saveDebugAuth(account)       ← 将账号写入 localStorage
setUser(buildLocalUser(account))
setSession(buildLocalSession(account))
setMembership(payload.membership)
setBusinessProfile(payload.business_profile)
setRoleHomePath(payload.role_home_path)
 ↓
syncStatus()              ← 再次调用 GET /api/auth/status 同步最新状态
setDialogOpen(false)         ← 关闭登录对话框
```

错误处理：

- HTTP 401 → 显示"邮箱或密码不正确"
- 其他错误 → 显示"登录暂不可用"

### 10.3.6 signOut 登出流程

```
  clearDebugAuth()     ← 清除 localStorage 中的调试认证数据
setSession(null)
setUser(null)
setMembership(GUEST_MEMBERSHIP)
setBusinessProfile(GUEST_BUSINESS_PROFILE)
setRoleHomePath('/')
setAuthMessage('')
setAuthError('')
```

### 10.3.7 初始化流程

应用启动时（useEffect([], ...)）：

1. 从 localStorage 加载 loadDebugAuth()，若存在则恢复用户和会话状态
2. 调用 syncStatus()（GET /api/auth/status）获取后端认证和会员状态
3. 无论成功或失败，将 loading 设为 false

### 10.3.8 AuthDialog 弹窗登录

AuthProvider 内渲染了 <AuthDialog> 组件，任何页面组件可通过 useAuth().openAuthDialog() 打开弹窗式登录，无需跳转到 /login 页面。

## 10.4 ProtectedRoute 路由守卫

ProtectedRoute 支持三种独立的守卫模式，可组合使用：

### 10.4.1 守卫判定流程

```
  loading?
 → 显示 "Loading access..."
 ↓
isAuthenticated?
 → false: 重定向到 /login?redirect=<当前路径>
 ↓
requireAdmin && !isAdmin?
 → 重定向到 fallbackPath（默认 "/"）
 ↓
requirePaid && !isPaidMember?
 → 重定向到 fallbackPath（默认 "/"）
 ↓
requireAiAssistant && !canUseAiAssistant?
 → 重定向到 fallbackPath（默认 "/"）
 ↓
渲染 children
```

### 10.4.2 三种守卫 Props

| Prop | 类型 | 默认值 | 说明 |
|---|---|---|---|

| | | | |
|---|---|---|---|
| requireAdmin | boolean | false | 要求管理员权限 |
| requirePaid | boolean | false | 要求付费会员权限 |
| requireAiAssistant | boolean | false | 要求 AI 助理权限（付费会员或管理员） |
| fallbackPath | string | '/' | 权限不足时的回退路径 |

### 10.4.3 实际使用统计

| 守卫类型 | 使用路由 | fallbackPath |
|---|---|---|
| requireAdmin | /analytics、/admin、/admin/memberships、/commercial/leads、/editorial、/media-studio | /、/admin |
| requireAiAssistant | /chat | /membership |
| requirePaid | （当前未使用，作为扩展预留） | — |

### 10.4.4 重定向机制

未登录用户被守卫拦截时，会被重定向到 /login?redirect=<原路径>。
buildLoginPath() 函数负责构建含 redirect 参数的登录路径，保留完整的 pathname + search。

## 10.5 API 客户端

API 客户端定义在 src/api/index.js，为整个前端提供统一的后端通信层。

### 10.5.1 基础配置

```
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL || 'http://127.0.0.1:8000/api'
const API_ORIGIN = API_BASE_URL.replace(/\/api\/?$/, '')
```

- API_BASE_URL：所有 API 请求的基础路径，通过 Vite 环境变量 VITE_API_BASE_URL 配置，默认指向本地开发服务器。
- API_ORIGIN：去掉 /api 后缀的服务器源地址，用于 resolveApiUrl() 拼接相对路径的资源 URL（如图片、音频文件）。

### 10.5.2 核心 request() 函数

```
async function request(path, options = {}) { ... }
```

自动注入的 Headers：

| Header | 条件 | 来源 | 说明 |
|---|---|---|---|
| Content-Type: application/json | 请求体不是 FormData 时 | 自动 | JSON 请求内容类型 |
| X-Visitor-Id | 浏览器环境下 | getOrCreateVisitorId() | 访客追踪 UUID |
| Authorization: Bearer <token> | options.authToken 有值时 | 调用方传入 | JWT 认证令牌 |
| X-Debug-User-Id | 无 authToken 且有调试认证数据时 | getDebugAuthHeaders() | 调试模式用户 ID |
| X-Debug-User-Email | 同上且有 email 时 | getDebugAuthHeaders() | 调试模式用户邮箱 |

FormData 特殊处理：当 options.body 为 FormData 实例时，不设置 Content-Type header，让浏览器自动添加带 boundary 的 multipart/form-data。

错误处理：非 2xx 响应时抛出 Error，并在 error 对象上附加 status 属性，便于上层根据 HTTP 状态码做差异化处理。

### 10.5.3 resolveApiUrl() 辅助函数

```
export function resolveApiUrl(url) {
  if (!url) return null
  if (url.startsWith('http://') || url.startsWith('https://')) return url
  return ${API_ORIGIN}${url}
}
```

用于将后端返回的相对路径（如 /uploads/cover/123.jpg）解析为完整的可访问 URL。如果已经是绝对 URL，直接返回。

### 10.5.4 API 函数分类索引

#### 首页与内容

| 函数 | 方法 | 路径 | 说明 |
|---|---|---|---|
| fetchHomeFeed(language) | GET | /home/feed?language= | 首页信息流 |
| fetchLatestArticles(limit, offset, language) | GET | /articles/latest | 最新文章分 |
| fetchTrendingArticles(limit, offset) | GET | /articles/trending | 热门文章 |

## 文章

| 函数 | 方法 | 路径 | 说明 |
|---|---|---|---|
| fetchArticle(id, authToken) | GET | /article/:id | 文章详情 |
| fetchArticleSummary(id, authToken) | GET | /summarize_article/:id | AI 摘要 |
| fetchArticleTranslation(id, lang, authToken) | GET | /article/:id/translation | 文章翻译 |
| fetchArticleEngagement(id, authToken) | GET | /article/:id/engagement | 互动数据（点赞/收藏状态 |
| submitArticleReaction(id, payload, authToken) | POST | /article/:id/reaction | 提交反应（点赞/收藏/浏览） |

## 搜索

| 函数 | 方法 | 路径 | 说明 |
|---|---|---|---|
| searchArticles(payload, authToken) | POST | /search | 统一搜索（exact/smart 模式） |
| fetchSuggestions(query, language) | GET | /suggest | 搜索建议词 |

## 栏目 / 标签 / 专题

| 函数 | 方法 | 路径 | 说明 |
|---|---|---|---|
| fetchColumns() | GET | /columns | 栏目列表 |
| fetchColumnArticles(slug, page, pageSize) | GET | /columns/:slug/articles | 栏目文章列表 |
| fetchTags() | GET | /tags | 标签列表（含分组和热门标 |
| fetchTagArticles(slug, page, pageSize) | GET | /tags/:slug/articles | 标签文章列表 |
| fetchTopics(authToken) | GET | /topics | 专题列表 |
| fetchTopic(slug, page, pageSize, authToken) | GET | /topics/:slug | 专题详情及文章 |
| fetchTopicTimeline(topicId, authToken) | GET | /topics/:id/timeline | 专题时间线 |
| fetchTopicInsights(topicId, authToken) | GET | /topics/:id/insights | 专题洞察 |

## 机构

| 函数 | 方法 | 路径 | 说明 |
|---|---|---|---|
| fetchOrganizations(limit) | GET | /organizations | 机构列表 |
| fetchOrganization(slug, page, pageSize) | GET | /organizations/:slug | 机构详情及文章 |

## 时光机

| 函数 | 方法 | 路径 | 说明 |
|---|---|---|---|
| fetchTimeMachine(targetDate, language) | GET | /time_machine | 按日期检索历史文章卡片 |

## AI 聊天

| 函数 | 方法 | 路径 | 说明 |
|---|---|---|---|
| sendChatMessage(payload) | POST | /chat | 发送聊天消息 |
| fetchChatSessions() | GET | /chat/sessions | 获取会话列表 |
| fetchChatSession(sessionId) | GET | /chat/session/:id | 获取单个会话历史 |
| deleteChatSession(sessionId) | DELETE | /chat/session/:id | 删除会话 |

## 认证

| 函数 | 方法 | 路径 | 说明 |
|---|---|---|---|
| fetchAuthStatus(authToken) | GET | /auth/status | 获取认证状态和会员信息 |
| loginWithPassword(payload) | POST | /auth/login | 邮箱密码登录 |

## 会员与计费

| 函数 | 方法 | 路径 | 说明 |
|---|---|---|---|
| fetchMembershipProfile(authToken) | GET | /membership/me | 当前会员档案 |
| fetchBillingPlans(language) | GET | /billing/plans | 计费方案列表 |
| fetchBillingProfile(authToken, language) | GET | /billing/me | 当前计费档案（含订阅和订 |
| createBillingCheckoutIntent(payload, authToken) | POST | /billing/checkout-intent | 创建结账意向单 |

## 媒体

| 函数 | 方法 | 路径 | 说明 |
|---|---|---|---|
| fetchMediaHub(kind, authToken, limit) | GET | /media/:kind | 前台媒体列表 |

| | | | |
|---|---|---|---|
| fetchMediaAdminItems(kind, status, limit) | GET | /media/admin/items | 管理端媒体列表 |
| fetchMediaAdminItem(id) | GET | /media/admin/items/:id | 管理端媒体详情 |
| createMediaAdminItem(payload) | POST | /media/admin/items | 创建媒体项 |
| updateMediaAdminItem(id, payload) | PUT | /media/admin/items/:id | 更新媒体项 |
| uploadMediaAdminFile(file, kind, usage) | POST | /media/admin/upload | 上传媒体文件（FormData） |

### 编辑后台

| 函数 | 方法 | 路径 | 说明 |
|---|---|---|---|
| fetchEditorialArticles(limit, status) | GET | /editorial/articles | 编辑文章列表 |
| fetchEditorialSourceArticles(query, limit) | GET | /editorial/source-articles | 源文稿搜索 |
| fetchEditorialSourceAiOutput(articleId) | GET | /editorial/source-articles/:id/ai-output | 源文稿 AI 输出 |
| importEditorialSourceAi(articleId, payload) | POST | /editorial/source-articles/:id/import-ai | 导入 AI 输出到编辑文章 |
| fetchEditorialDashboard(limit) | GET | /editorial/dashboard | 编辑仪表板统计 |
| fetchEditorialArticle(id) | GET | /editorial/articles/:id | 编辑文章详情 |
| createEditorialArticle(payload) | POST | /editorial/articles | 创建编辑文章 |
| updateEditorialArticle(id, payload) | PUT | /editorial/articles/:id | 更新编辑文章 |
| autoFormatEditorialArticle(id, payload) | POST | /editorial/articles/:id/auto-format | AI 一键排版 |
| uploadEditorialFile(file) | POST | /editorial/upload | 上传文稿文件（FormData） |
| autotagEditorialArticle(id) | POST | /editorial/articles/:id/autotag | 自动标签 |
| updateEditorialWorkflow(id, payload) | POST | /editorial/articles/:id/workflow | 更新工作流状态 |
| renderEditorialHtml(id) | POST | /editorial/articles/:id/render-html | 渲染 HTML 预览 |
| publishEditorialArticle(id) | POST | /editorial/articles/:id/publish | 发布文章 |
| editorialHtmlExportUrl(id, variant) | — | /editorial/articles/:id/export | 导出 HTML（返回 URL 字符串） |

### 管理后台

| 函数 | 方法 | 路径 | 说明 |
|---|---|---|---|
| fetchAdminOverview(authToken) | GET | /admin/overview | 管理总览（指标+角色分布+审计） |
| fetchAdminMemberships(authToken, limit, query) | GET | /admin/memberships | 会员列表 |
| updateAdminMembership(userId, payload, authToken) | PUT | /admin/memberships/:userId | 更新会员等级/状态 |
| fetchAdminBillingOrders(authToken, limit, query) | GET | /admin/billing/orders | 账单订单列表 |

### 商务

| 函数 | 方法 | 路径 | 说明 |
|---|---|---|---|
| fetchCommerceOverview() | GET | /commerce/overview | 商务总览 |
| submitDemoRequest(payload) | POST | /commerce/demo-request | 提交 Demo 咨询 |
| fetchDemoRequests(limit) | GET | /commerce/demo-requests | 获取线索列表 |

### 个人资产

| 函数 | 方法 | 路径 | 说明 |
|---|---|---|---|
| fetchMyLibrary(authToken, limit) | GET | /me/library | 收藏、点赞、最近浏览 |
| fetchMyDashboard(authToken) | GET | /me/dashboard | 个人仪表板统计 |

### 关注

| 函数 | 方法 | 路径 | 说明 |
|---|---|---|---|
| fetchFollows(authToken) | GET | /follows | 关注列表 |
| toggleFollow(payload, authToken) | POST | /follows | 切换关注状态 |
| fetchWatchlist(authToken, limit, filters) | GET | /follows/watchlist | 关注项相关文章 |

## 10.6 访客追踪

文件：src/lib/visitor.js

### 10.6.1 UUID 生成策略

```
function generateVisitorId() {
  if (typeof crypto !== 'undefined' && typeof crypto.randomUUID === 'function') {
    return visitor-${crypto.randomUUID()}
  }
  return visitor-${Date.now()}-${Math.random().toString(36).slice(2, 10)}
}
```

- 优先使用 crypto.randomUUID() 生成标准 UUID v4
- 降级方案：时间戳 + 随机字符串

### 10.6.2 存储与获取

| 函数 | 行为 |
|---|---|
| getOrCreateVisitorId() | 从 localStorage 读取 fdsm-visitor-id；若不存在则生成新 ID 并存入 localStorage，返回该 ID |
| getStoredVisitorId() | 仅读取 localStorage，不创建新 ID |

### 10.6.3 使用方式

request() 函数在每次 API 调用时都会通过 getOrCreateVisitorId() 获取访客 ID，并作为 X-Visitor-Id header 发送给后端。这使得后端可以追踪匿名访客的行为（浏览量统计等），而无需用户登录。

## 10.7 角色体验系统

文件：src/lib/roleExperience.js

角色体验系统为每种用户角色定义一套完整的 UI 配置，包括文案、导航链接和快捷操作，使整个应用可以根据用户角色动态调整界面表现。

### 10.7.1 四级角色定义

| 角色 tier | 标签 | 入口路径 | 说明 |
|---|---|---|---|
| guest | 游客 / Guest | /login | 未登录用户 |
| free_member | 免费会员 / Free Member | /me | 已登录但未付费的用户 |
| paid_member | 付费会员 / Paid Member | /membership | 已订阅付费方案的用户 |
| admin | 管理员 / Admin | /admin | 系统管理员 |

### 10.7.2 resolveRoleTier() 角色解析

```
export function resolveRoleTier({ membership, isAuthenticated = false } = {}) {
  if (!isAuthenticated) return 'guest'
  const tier = (membership?.tier || '').trim()
  if (tier === 'admin' || tier === 'paid_member' || tier === 'free_member') return tier
  return 'free_member'
}
```

优先级：未登录 → guest；已登录但 tier 不在已知值中 → 默认为 free_member。

### 10.7.3 每级角色配置（ROLE_DATA）

每个角色包含以下配置项（均支持中英文双语）：

| 配置项 | 说明 |
|---|---|
| label | 角色显示名 |
| heroKicker | Hero 区域角标文字 |
| heroTitle | Hero 区域大标题 |
| heroBody | Hero 区域描述文段 |
| entryLabel | 入口按钮文字 |
| entryPath | 入口按钮跳转路径 |
| statCards | 3 张统计卡片（label, value, detail） |
| quickActions | 快捷操作按钮组（label, path, tone, description） |
| navLinks | 导航链接组（label, to） |

### 10.7.4 各角色导航链接

Guest（游客）：首页、会员、专题、机构、时光机

Free Member（免费会员）：首页、我的资产、我的关注、会员、专题

Paid Member（付费会员）：首页、会员、音频、视频、我的资产、我的关注

Admin（管理员）：管理总览、会员管理、内容分析、文章后台、媒体后台、销售线索、前台首页

### 10.7.5 各角色快捷操作

Guest：公开首页（primary）、会员方案（secondary）、统一搜索（plain）

Free Member：我的资产（primary）、我的关注（secondary）、升级会员（plain）

Paid Member：会员权益（primary）、音频页面（secondary）、视频页面（plain）

Admin：管理总览（primary）、会员管理（secondary）、内容分析（plain）、文章后台（plain）、媒体后台（plain）、销售线索（plain）

### 10.7.6 getRoleExperience() 返回值

getRoleExperience(tier, isEnglish) 返回已翻译后的扁平对象，所有文案字段直接为当前语言的字符串，不再需要调用方处理双语。

## 10.8 国际化

文件：src/i18n/LanguageProvider.jsx + src/i18n/LanguageContext.js + src/i18n/messages.js

### 10.8.1 语言检测优先级

1. URL 查询参数 ?lang=en
2. localStorage fdsm-language
3. 浏览器 navigator.language（en* → en，其他 → zh）
4. 默认 zh

### 10.8.2 语言切换副作用

每次语言变更时：

- 写入 localStorage
- 更新 document.documentElement.lang（en 或 zh-CN）
- 更新 URL 查询参数（英文时添加 ?lang=en，中文时移除）

### 10.8.3 翻译函数 t(key)

- 支持点号分隔的嵌套 key，如 t('navbar.searchPlaceholder')
- 先从当前语言的 messages 中查找
- 找不到时回退到中文 messages
- 仍找不到则返回 key 本身

### 10.8.4 暴露的 Context 值

| 字段 | 类型 | 说明 |
|---|---|---|
| language | `'zh' \ 'en'` | 当前语言 |
| isEnglish | boolean | 是否为英文 |
| setLanguage(lang) | function | 设置语言 |
| toggleLanguage() | function | 切换语言 |
| t(key) | function | 翻译函数 |

## 10.9 布局组件

### 10.9.1 SiteLayout

文件：src/components/layout/SiteLayout.jsx

SiteLayout 是所有页面共享的布局壳，通过 React Router 的 <Outlet> 渲染子路由：

```
  <div className="min-h-screen">
  <Navbar />
  {authMessage 提示条}
  {authError 错误条}
  <main className="min-h-[calc(100vh-180px)]">
    <Outlet />  ← 子路由页面在此渲染
  </main>
  <Footer />
  {canUseAiAssistant ? <ChatPanel variant="floating" /> : null}
</div>
```

关键特性：

- 认证消息（authMessage）和认证错误（authError）以全宽横幅形式显示在 Navbar 下方
- 当用户具有 AI 助理权限时（canUseAiAssistant），自动渲染浮动 ChatPanel
- main 区域设有最小高度，确保页脚不会出现在屏幕中间

### 10.9.2 Navbar

文件：src/components/layout/Navbar.jsx

Navbar 是应用最复杂的布局组件，根据用户角色动态渲染不同的导航内容。

结构分层（桌面视图 xl 断点以上）：

```
  <header sticky top-0 z-40>
  第一行：Logo + 语言切换 + 会员徽章 + 登录/登出按钮
  第二行：搜索框 + 桌面端操作按钮组 + 角色入口按钮
  第三行：导航链接（NavLink 带 active 状态高亮）
</header>
```

移动端适配（xl 以下）：

- 操作按钮区域隐藏
- 导航链接合并为水平滚动的药片式按钮（pill buttons）
- 将桌面端操作项并入移动端导航列表

根据角色不同的操作按钮组（desktopActionItems）：

| 角色 | 显示按钮 |
|---|---|
| admin | 管理总览、AI 助理（若有权限）、文章后台、媒体后台、销售线索 |
| paid_member | 会员空间、AI 助理（若有权限）、音频、视频、我的资产 |
| free_member | 我的资产、我的关注、视频、升级会员、商业方案 |
| guest | 登录中心、会员方案、音频、视频 |

搜索行为：

- 默认搜索模式根据角色自动选择：付费会员/管理员使用 smart，其他使用 exact
- 提交后导航到 /search?q=<query>&mode=<mode>

语言切换：内嵌 LanguageSwitch 组件，中英文两个 pill 按钮，当前语言高亮为 fudan-blue 背景。

会员徽章：根据 membership.tier 显示不同样式的圆角徽章：

- guest：白底灰字，灰色边框
- free_member：蓝色半透明背景，fudan-blue 文字
- paid_member：橙色半透明背景，fudan-orange 文字
- admin：绿色半透明背景，emerald 文字

导航链接构建：

1. 从 roleExperience.navLinks 获取角色专属链接
2. 追加发现区链接：专题广场、深度洞察、行业观察、学术前沿、院长说
3. 去重（按 to 字段）

### 10.9.3 Footer

文件：src/components/layout/Footer.jsx

简洁的页脚，包含：

- 左侧：应用名 + 描述文段
- 右侧：商业合作按钮（链接到 /commercial）+ 品牌标识 "Fudan Blue x Fudan Orange"

## 10.10 共享组件

### 10.10.1 ArticleCard

文件：src/components/shared/ArticleCard.jsx

可复用的文章卡片组件，在首页、搜索页、栏目页、标签页、专题页、机构页、我的资产页等多处使用。

Props：
- article：文章数据对象
- compact：紧凑模式（默认 false），影响标题字号、摘要长度和标签数量

渲染内容：
- 封面图（有则显示图片，无则显示渐变占位）
- 访问层级标签（公开/会员/付费，不同颜色）
- 标签组（TagBadge，最多 2-3 个）
- 标题（font-serif, fudan-blue）
- 摘要（截断到 92/150 字符）
- 发布日期 + 所属栏目
- 浏览数 + 点赞数 + 收藏数
- 整个卡片可点击，链接到 /article/:id

### 10.10.2 ChatPanel

文件：src/components/shared/ChatPanel.jsx

AI 对话面板，支持两种模式：

| 模式 | variant | 使用场景 |
|---|---|---|
| 浮动模式 | "floating" | SiteLayout 中的全局悬浮按钮，点击展开面板 |
| 页面模式 | "page" | ChatPage 中内嵌的全宽面板 |

功能特性：
- 快捷指令按钮：/今日一读、/简报 AI战略、/总结 ESG转型、/比较 AI 与 ESG、/时间线 生成式AI、/继续阅读
- 会话管理：列出历史会话、切换会话、删除会话、新建会话
- 消息渲染：使用 ReactMarkdown 渲染 AI 回复内容
- 引用来源：每条 AI 回复可附带 sources（可点击文章链接）和 followUps（后续操作建议）
- 中英文双语 UI 文案

调用的 API：
- sendChatMessage(payload) — POST /api/chat
- fetchChatSessions() — GET /api/chat/sessions
- fetchChatSession(sessionId) — GET /api/chat/session/:id
- deleteChatSession(sessionId) — DELETE /api/chat/session/:id

### 10.10.3 SearchBar

文件：src/components/shared/SearchBar.jsx

搜索栏组件，在 HomePage 和 SearchPage 中使用。

Props：
- initialQuery：初始搜索词
- initialMode：初始搜索模式（'smart' / 'exact'）
- onSearch(query, mode)：搜索回调
- variant：'hero'（大号）或其他（小号）

功能特性：
- 搜索模式切换：智能搜索（仅付费会员/管理员可用）和精确搜索
- 如果用户无 smart 搜索权限，仅显示"精确搜索"一个 tab
- 非登录用户点击 smart 搜索 tab 时触发 openAuthDialog()
- 输入自动建议：输入 220ms 后调用 fetchSuggestions() 获取建议词，以下拉列表呈现
- 建议词可点击，直接触发搜索

### 10.10.4 TagBadge

文件：src/components/shared/TagBadge.jsx

标签徽章组件，根据标签类别（category）显示不同颜色样式。

Props：
- tag：标签对象（{ slug, name, category }）
- clickable：是否可点击（默认 true，点击导航到 /tag/:slug）
- variant：'default' 或 'inverse'（反色，用于深色背景）

类别样式映射：

| category | 样式 |
| --- | --- |
| industry | 橙色系（fudan-orange） |
| topic | 蓝色系（fudan-blue） |
| type | 靛蓝系（indigo） |
| entity | 灰色系（slate） |
| series | 紫色系（purple） |

### 10.10.5 AutoHeightPreviewFrame

文件：src/components/shared/AutoHeightPreviewFrame.jsx

自适应高度的 iframe 预览组件，在 EditorialWorkbenchPage 和 ArticlePage 中使用。

Props：

- title：iframe 的 title 属性
- srcDoc：注入 iframe 的 HTML 文档字符串
- minHeight：最小高度（默认 200px）
- className：额外 CSS 类

自适应原理：

1. 监听 iframe 的 load 事件
2. 读取 iframe 内部文档的 body.scrollHeight 和 documentElement.scrollHeight
3. 取较大值作为 iframe 的实际高度
4. 使用 ResizeObserver 持续监听 iframe 内容尺寸变化
5. 通过 requestAnimationFrame 避免布局抖动

安全设置：

- sandbox="allow-same-origin"：允许同源访问但禁止脚本执行
- scrolling="no"：禁用 iframe 自带滚动条

## 10.11 调试认证（debugAuth）

文件：src/auth/debugAuth.js

这是一个简化的本地持久化方案，用于在开发和调试阶段维持登录状态。

### 10.11.1 存储 Key

```
localStorage key: "fdsm-debug-auth"
```

### 10.11.2 存储格式

```
{
 "user_id": "uuid-string",
 "email": "user@example.com",
 "display_name": "管理员",
 "tier": "admin"
}
```

### 10.11.3 函数列表

| 函数 | 说明 |
| --- | --- |
| loadDebugAuth() | 从 localStorage 读取并解析调试认证数据，校验 user_id 存在性 |
| saveDebugAuth(account) | 将账号信息写入 localStorage |
| clearDebugAuth() | 删除 localStorage 中的调试认证数据 |
| getDebugAuthHeaders() | 返回调试认证 HTTP headers：X-Debug-User-Id 和 X-Debug-User-Email |

### 10.11.4 与 AuthProvider 的关系

- 登录时：signInWithPassword 成功后调用 saveDebugAuth() 持久化
- 登出时：signOut 调用 clearDebugAuth() 清除
- 应用启动时：useEffect 中调用 loadDebugAuth() 恢复上次登录状态
- API 请求时：如果没有 authToken，request() 函数会调用 getDebugAuthHeaders() 注入调试 headers，后端通过这些 headers 识别用户

## 10.12 设计语言与样式体系

### 10.12.1 CSS 框架

- 使用 Tailwind CSS 进行原子化样式编写
- 自定义颜色 token：fudan-blue（#0d0783）、fudan-orange（#ea6b00）、fudan-dark

### 10.12.2 布局 Token

- page-shell：页面最大宽度容器，水平居中
- fudan-panel：圆角卡片容器（border + shadow）
- fudan-card：可交互卡片容器
- section-kicker：区块角标（小号大写字母间距文字）
- section-title：区块大标题（font-serif + font-black）
- paper-divider：分割线

### 10.12.3 渐变背景模式

多处 Hero 区域使用统一的品牌渐变背景：

```
  background: linear-gradient(135deg,
 rgba(13,7,131,0.98),     / 深蓝 /
 rgba(10,5,96,0.86) 58%,  / 中蓝 /
 rgba(234,107,0,0.xx)     / 橙色点缀，透明度因页面而异 /
);
```

### 10.12.4 图标库

使用 lucide-react 图标库，常用图标包括：

Search、LogIn、LogOut、Crown、Shield、UserRound、MessageSquare、PenTool、Film、Headphones、Sparkles、ArrowRight、Bookmark、Heart、Eye、Lock 等。

# 11 — 前端页面与交互

本文档详细描述"复旦管院智识库"前端所有 23 个页面的路由路径、 访问权限、UI 布局、核心交互逻辑、调用的 API 端点及状态管理。 页面均通过 React.lazy() 懒加载，在 App.jsx 中注册路由。

## 页面总览

| 序号 | 页面 | 路由 | 守卫 | 说明 |
|---|---|---|---|---|
| 1 | HomePage | / | 无 | 公开首页 |
| 2 | LoginPage | /login | 无 | 登录中心 |
| 3 | SearchPage | /search | 无 | 统一搜索 |
| 4 | ArticlePage | /article/:id | 无 | 文章详情 |
| 5 | ChatPage | /chat | requireAiAssistant | AI 助理 |
| 6 | EditorialWorkbenchPage | /editorial | requireAdmin | 编辑工作台 |
| 7 | AdminConsolePage | /admin | requireAdmin | 管理控制台 |
| 8 | AdminMembershipsPage | /admin/memberships | requireAdmin | 会员管理 |
| 9 | AnalyticsPage | /analytics | requireAdmin | 内容分析 |
| 10 | MembershipPage | /membership | 无 | 会员方案与权益 |
| 11 | MediaHubPage (audio) | /audio | 无 | 音频媒体 |
| 12 | MediaHubPage (video) | /video | 无 | 视频媒体 |
| 13 | MediaStudioPage | /media-studio | requireAdmin | 媒体管理后台 |
| 14 | TopicsPage | /topics | 无 | 专题广场 |
| 15 | TopicPage | /topic/:slug | 无 | 专题详情 |
| 16 | TimeMachinePage | /time-machine | 无 | 时光机 |
| 17 | CommercialPage | /commercial | 无 | 商业方案 |
| 18 | CommercialLeadsPage | /commercial/leads | requireAdmin | 商务线索 |
| 19 | MyLibraryPage | /me | 无 | 我的资产 |
| 20 | MyFollowingPage | /following | 无 | 我的关注 |
| 21 | ColumnPage | /column/:slug | 无 | 栏目详情 |
| 22 | TagPage | /tag/:slug | 无 | 标签文章列表 |
| 23 | OrganizationsPage | /organizations | 无 | 机构索引 |
| 24 | OrganizationPage | /organization/:slug | 无 | 机构详情 |

## 11.1 HomePage — 公开首页

### 基本信息

| 属性 | 值 |
|---|---|
| 路由 | / |
| 文件 | src/pages/HomePage.jsx |
| 守卫 | 无，所有角色可访问 |
| 懒加载 | 是 |

### 页面布局

```
  Hero 区域
  ├── 搜索框（SearchBar 组件）
  ├── 热门标签快捷入口（最多 6 个 TagBadge）
  └── 快速专题入口（最多 2 个）

编辑推荐区（editors_picks，最多 4 篇）
  └── 每篇带封面图、标题、摘要、栏目标签

体验能力卡片区（experienceCards，5 张卡片）
  ├── 统一搜索
  ├── AI 助理（或升级提示）
  ├── 专题阅读（或升级提示）
  ├── 时光机
  └── 会员访问

角色引导面板（rolePanel）
  └── 根据当前角色（admin/paid_member/free_member/guest）显示不同引导

产品能力卡片（productCards，3 张）
  ├── 教学与研究
  ├── 管理者洞察
  └── 会员阅读

AI 助理 CTA
  └── 已有权限 → "进入 AI 助理" 链接到 /chat
 ... (共 30 行，已截断显示)
```

### 核心交互

1. 搜索提交：调用 navigate(/search?q=...&mode=...)，搜索模式根据角色自动选择（付费/管理员 → smart，其他 → exact）
2. 加载更多文章：调用 fetchLatestArticles(12, offset, language)，将返回结果追加到 latestArticles 数组；不足 12 条时隐藏"加载更多"按钮
3. 体验能力卡片动态化：AI 助理和专题卡片根据当前权限显示不同文案——有权限时显示功能说明和入口链接；无权限时显示升级引导
4. 角色面板差异化：guest → 介绍登录价值；free_member → 引导升级；paid_member → 突出权益；admin → 引导进入管理台

### 调用的 API

| API 函数 | 端点 | 触发时机 |
|---|---|---|
| fetchHomeFeed(language) | GET /api/home/feed | 组件挂载时、语言切换时 |
| fetchLatestArticles(12, offset, language) | GET /api/articles/latest | 点击"加载更多"时 |

### 数据来源（fetchHomeFeed 返回值）

| 字段 | 说明 |
|---|---|
| hot_tags | 热门标签数组 |
| topics | 推荐专题数组 |
| editors_picks | 编辑推荐文章 |
| latest | 最新文章（初始批次） |

## 11.2 ArticlePage — 文章详情

### 基本信息

| 属性 | 值 |
|---|---|
| 路由 | /article/:id |
| 文件 | src/pages/ArticlePage.jsx |
| 守卫 | 无（后端按 access_level 控制正文可见范围） |
| 懒加载 | 是 |

### 页面布局

```
 文章元信息区
 ├── 访问层级标签（公开/会员/付费）
 ├── 会员状态标签
 ├── 标题（font-serif，超大字号）
 ├── 副标题
 ├── 作者 + 机构（链接到 /organization/:slug）
 ├── 发布日期
 └── 标签组（TagBadge 组件）

封面图（如果存在）

付费墙提示（如果 access.locked === true）
 └── 锁定提示文字
 └── 升级链接（→ /membership 或 → /login）

正文区域
 ├── 如果有 html_content → AutoHeightPreviewFrame 渲染 HTML
 └── 如果只有 markdown → 内置 Markdown 渲染器将 markdown 转 HTML

操作工具条
 ├── 点赞按钮（Heart 图标，带计数）
 ├── 收藏按钮（Bookmark 图标，带计数）
 ├── 浏览量（Eye 图标）
 └── 翻译按钮（Languages 图标）

... (共 33 行，已截断显示)
```

## 核心交互

1. 文章加载：根据 URL 参数 :id 调用 fetchArticle(id, authToken) 获取文章数据
2. 互动数据加载：调用 fetchArticleEngagement(id, authToken) 获取当前用户的点赞/收藏状态和计数
3. 点赞/收藏：调用 submitArticleReaction(id, { type: 'like'|'bookmark', active: true|false }, authToken) 切换状态
4. AI 摘要：点击"AI 摘要"按钮调用 fetchArticleSummary(id, authToken)，首次加载后缓存
5. 文章翻译：点击"翻译"按钮调用 fetchArticleTranslation(id, 'en', authToken)，首次加载后缓存
6. Markdown 渲染器：内置简易 Markdown → HTML 转换器，支持标题（h1-h6）、段落、无序列表、有序列表、水平线、加粗、斜体、行内代码
7. HTML 正文渲染：使用 AutoHeightPreviewFrame 组件在沙盒 iframe 中渲染 HTML，自动适配高度
8. 付费墙逻辑：后端在 access 字段中返回 locked: true 和 access_level（'member'/'paid'）；前端根据此信息显示锁定提示，并引导用户登录或升级

## 调用的 API

| API 函数 | 端点 | 触发时机 |
|---|---|---|
| fetchArticle(id, authToken) | GET /api/article/:id | 页面加载、id 变化 |
| fetchArticleEngagement(id, authToken) | GET /api/article/:id/engagement | 页面加载 |
| submitArticleReaction(id, payload, authToken) | POST /api/article/:id/reaction | 点赞/收藏操作 |
| fetchArticleSummary(id, authToken) | GET /api/summarize_article/:id | 点击 AI 摘要 |
| fetchArticleTranslation(id, lang, authToken) | GET /api/article/:id/translation | 点击翻译 |

## 访问控制映射

| 文章 access_level | guest | free_member | paid_member | admin |
|---|---|---|---|---|
| public | 全文 | 全文 | 全文 | 全文 |
| member | 预览 | 全文 | 全文 | 全文 |
| paid | 预览 | 预览 | 全文 | 全文 |

# 11.3 SearchPage — 统一搜索

## 基本信息

| 属性 | 值 |
|---|---|
| 路由 | /search |
| 文件 | src/pages/SearchPage.jsx |
| 守卫 | 无 |
| 懒加载 | 是 |

## 查询参数

| 参数 | 说明 | 默认值 |
|---|---|---|

| | | |
|---|---|---|
| q | 搜索关键词 | '' |
| mode | 搜索模式（smart / exact） | 'smart' |
| sort | 排序（relevance / date_desc / date_asc） | 'relevance' |
| start_date | 起始日期过滤 | '' |
| end_date | 结束日期过滤 | '' |
| tags | 标签 slug 列表（逗号分隔） | '' |
| columns | 栏目 slug 列表（逗号分隔） | '' |

## 页面布局

```
SearchBar 组件（带模式切换）

搜索历史（无关键词时显示，最多 8 条，localStorage 持久化）

智能搜索权限提示（仅无权限时显示）

筛选器面板
  ├── 排序切换（相关性 / 最新 / 最早）
  ├── 日期范围筛选器（起止日期输入）
  ├── 标签多选筛选器（从 fetchTags 获取全部标签组）
  └── 栏目多选筛选器（从 fetchColumns 获取全部栏目）

搜索结果区
  ├── 结果计数 + 搜索耗时
  ├── 当前激活的过滤条件标签（可单独移除）
  ├── AI 智能摘要（仅 smart 模式返回）
  └── ArticleCard 结果列表
```

## 核心交互

1. 搜索执行：SearchBar 提交后更新 URL 查询参数 → useEffect 监听参数变化 → 调用 searchArticles(payload, authToken)
2. 搜索模式自动降级：如果当前用户无 smart 搜索权限且 URL 中 mode=smart，自动替换为 mode=exact 并更新 URL
3. 搜索历史：每次搜索将关键词写入 localStorage（key: fdsm-search-history），最多保留 8 条，去重，最新在前
4. 筛选器联动：修改任何筛选条件时重新构建 URL 并导航，触发新搜索
5. 标签/栏目多选：点击标签/栏目 toggle 其选中状态，已选标签以 TagBadge 形式显示在结果区上方，可点击 X 移除

## 调用的 API

| API 函数 | 端点 | 触发时机 |
|---|---|---|
| searchArticles(payload, authToken) | POST /api/search | 查询参数变化时 |
| fetchTags() | GET /api/tags | 组件挂载时 |
| fetchColumns() | GET /api/columns | 组件挂载时 |

## 搜索请求体

```
{
 "query": "AI 战略",
 "mode": "smart",
 "language": "zh",
 "sort": "relevance",
 "page": 1,
 "page_size": 18,
 "filters": {
  "start_date": "2024-01-01",
  "end_date": null,
  "tags": ["人工智能"],
  "columns": ["insights"]
 }
}
```

## 搜索模式对比

| 特性 | exact（精确搜索） | smart（智能搜索） |
|---|---|---|
| 权限 | 所有用户 | 付费会员 + 管理员 |
| 底层 | BM25 关键词匹配 | 向量检索 + 语义理解 |
| 返回 | 文章列表 | 文章列表 + AI 摘要 |
| 建议词 | 支持 | 支持 |

# 11.4 ChatPage — AI 助理

## 基本信息

| 属性 | 值 |
|---|---|
| 路由 | /chat |
| 文件 | src/pages/ChatPage.jsx |
| 守卫 | requireAiAssistant，fallbackPath /membership |
| 懒加载 | 是 |

## 访问权限

- 付费会员（isPaidMember）和管理员（isAdmin）可访问
- 未登录用户 → 重定向到 /login?redirect=/chat
- 已登录但非付费/管理员 → 重定向到 /membership

## 页面布局

```
 页面标题区
├── Section kicker: "AI 助理"
└── 标题: "围绕知识库继续提问、比较与整合"

ChatPanel（variant="page" 模式）
├── 会话侧边栏（仅 page 模式显示）
│   ├── "新建会话" 按钮
│   ├── 历史会话列表（可点击加载、可删除）
│   └── 空状态提示
│
├── 对话主区域
│   ├── 问候消息（含快捷指令介绍）
│   ├── 快捷指令按钮组（6 个）
│   ├── 对话消息流
│   │   ├── 用户消息
│   │   └── AI 回复（ReactMarkdown 渲染）
│   │       ├── 引用来源（可点击链接到文章）
│   │       └── 后续操作建议（可点击自动填入）
│   └── 输入框 + 发送按钮
│
└── Loading 状态指示
```

## 核心交互

1. 发送消息：用户输入后调用 sendChatMessage({ message, session_id, language })
2. 快捷指令：点击快捷按钮将预设文本填入输入框并自动发送
3. 会话管理：
- 列表：fetchChatSessions() 获取所有会话
- 加载：fetchChatSession(sessionId) 获取会话历史
- 删除：deleteChatSession(sessionId) 删除指定会话
- 新建：清空当前消息和 sessionId
4. 后续操作：AI 回复可附带 followUps 数组，渲染为可点击按钮，点击后自动填入输入框发送

## 快捷指令列表

| 中文指令 | 英文指令 | 功能 |
|---|---|---|
| /今日一读 | /today | 获取今日推荐阅读 |
| /简报 AI战略 | /brief AI strategy | 生成主题简报 |
| /总结 ESG转型 | /summarize ESG transition | 总结主题内容 |
| /比较 AI 与 ESG | /compare AI vs ESG | 对比两个主题 |
| /时间线 生成式AI | /timeline Generative AI | 生成时间线 |
| /继续阅读 | /recommend | 获取阅读推荐 |

## 调用的 API

| API 函数 | 端点 | 触发时机 |
|---|---|---|
| sendChatMessage(payload) | POST /api/chat | 发送消息时 |
| fetchChatSessions() | GET /api/chat/sessions | 组件挂载时、sessionId 变化时 |
| fetchChatSession(id) | GET /api/chat/session/:id | 点击历史会话时 |

| | | |
|---|---|---|
| deleteChatSession(id) | DELETE /api/chat/session/:id | 点击删除会话时 |

# 11.5 EditorialWorkbenchPage — 编辑工作台

## 基本信息

| 属性 | 值 |
|---|---|
| 路由 | /editorial |
| 文件 | src/pages/EditorialWorkbenchPage.jsx |
| 守卫 | requireAdmin，fallbackPath /admin |
| 懒加载 | 是 |

## 页面布局

```
Hero 区域（深蓝渐变背景）
├── 标题："上传、改稿、自动排版、预览、发布"
├── 说明文字
├── "新建草稿" 按钮
├── "上传文稿" 按钮（支持 .md .txt .html .htm .docx）
└── Dashboard 统计卡片（3 张）
  ├── 草稿数
  ├── 待审核数
  └── 已发布数

主内容区（两栏布局）
├── 左侧（0.72fr）
│  ├── 稿件列表面板
│  │  └── 按钮列表，每项显示标题和工作流状态
│  │  └── 选中项高亮
│  └── 流程控制面板
│     ├── 审核备注 textarea
│     ├── 定时发布 datetime-local 输入
│     ├── "提交审核" 按钮
│     └── "发布" 按钮
│
└── 右侧（1.28fr）
  ├── 操作按钮栏
  │  ├── 保存
  │  ├── 一键自由排版（AI，使用 gemini-3-flash-preview）
... (共 42 行，已截断显示)
```

## 核心交互

1. 文稿上传：选择文件 → uploadEditorialFile(file) → 后端解析并创建文章 → 刷新列表
2. 新建草稿：清空所有表单字段和选中 ID
3. 保存：调用 persist()（create 或 update） → 刷新列表
4. 一键自由排版：先保存 → 调用 autoFormatEditorialArticle(id, { source_markdown, layout_mode, formatting_notes }) → 后端使用 Gemini 模型排版 → 刷新
5. 自动标签：先保存 → 调用 autotagEditorialArticle(id) → 后端 AI 生成标签 → 刷新
6. 生成预览：先保存 → 调用 renderEditorialHtml(id) → 后端生成 HTML → 刷新预览区
7. 提交审核：先保存 → 调用 updateEditorialWorkflow(id, { action: 'submit_review', review_note, ... })
8. 发布：先保存 → 调用 publishEditorialArticle(id) → 文章状态变为已发布
9. 稿件切换：点击列表中的稿件 → syncDetail(id) 加载详情并填充表单

## 调用的 API

| API 函数 | 端点 | 触发时机 |
|---|---|---|
| fetchEditorialArticles(80) | GET /api/editorial/articles | 初始化、每次操作 |
| fetchEditorialDashboard(8) | GET /api/editorial/dashboard | 初始化、每次操作 |
| fetchEditorialArticle(id) | GET /api/editorial/articles/:id | 选中稿件时 |
| fetchColumns() | GET /api/columns | 初始化 |
| createEditorialArticle(payload) | POST /api/editorial/articles | 新建保存 |
| updateEditorialArticle(id, payload) | PUT /api/editorial/articles/:id | 更新保存 |
| autoFormatEditorialArticle(id, payload) | POST /api/editorial/articles/:id/auto-format | 一键排版 |
| autotagEditorialArticle(id) | POST /api/editorial/articles/:id/autotag | 自动标签 |
| renderEditorialHtml(id) | POST /api/editorial/articles/:id/render-html | 生成预览 |
| updateEditorialWorkflow(id, payload) | POST /api/editorial/articles/:id/workflow | 提交审核 |

| | | |
|---|---|---|
| publishEditorialArticle(id) | POST /api/editorial/articles/:id/publish | 发布 |
| uploadEditorialFile(file) | POST /api/editorial/upload | 上传文稿 |
| editorialHtmlExportUrl(id, variant) | — | 导出链接构建 |

### 表单默认值

```
 {
 title: '',
 subtitle: '',
 author: 'Fudan Business Knowledge Editorial Desk',
 organization: 'Fudan Business Knowledge',
 publish_date: new Date().toISOString().slice(0, 10), // 今日日期
 source_url: '',
 cover_image_url: '',
 primary_column_slug: 'insights',
 article_type: '',
 main_topic: '',
 access_level: 'public',
 layout_mode: 'auto',
 formatting_notes: '',
 source_markdown: '',
 content_markdown: '',
}
```

### 默认栏目列表

| slug | 名称 |
|---|---|
| insights | 深度洞察 |
| industry | 行业观察 |
| research | 学术前沿 |
| deans-view | 院长说 |

## 11.6 AdminConsolePage — 管理控制台

### 基本信息

| 属性 | 值 |
|---|---|
| 路由 | /admin |
| 文件 | src/pages/AdminConsolePage.jsx |
| 守卫 | requireAdmin，fallbackPath / |
| 懒加载 | 是 |

### 页面布局

```
 Hero 区域（深蓝+绿色点缀渐变）
 ├── 标题（来自 roleExperience admin 配置）
 ├── 说明文字
 └── 统计卡片（3 张）
   ├── 当前账号（display_name + email）
   ├── 主要入口（role_home_path）
   └── 主要任务（会员 / 内容 / 审核）

指标卡片区（4 列网格，data.metrics 动态渲染）
 └── 每张包含 label、value（大字号）、detail

两栏布局
 ├── 左侧
 │  ├── 快捷入口面板
 │  │  ├── 进入会员管理（→ /admin/memberships）
 │  │  ├── 进入编辑工作台（→ /editorial）
 │  │  ├── 进入内容分析（→ /analytics）
 │  │  ├── 进入媒体后台（→ /media-studio）
 │  │  └── 查看商务线索（→ /commercial/leads）
 │  └── 角色分布面板
 │     └── data.role_counts 渲染（tier_label + total）
 │
 └── 右侧
   ├── 最近审核面板
   │  └── data.recent_audits 渲染
... (共 28 行，已截断显示)
```

## 核心交互

1. 数据加载：挂载时调用 fetchAdminOverview(accessToken) 获取管理总览数据
2. 快捷导航：5 个快捷入口按钮，链接到各管理子页面
3. 角色分布查看：显示各 tier 的用户总数
4. 审计日志查看：显示最近的角色变更记录

## 调用的 API

| API 函数 | 端点 | 触发时机 |
|---|---|---|
| fetchAdminOverview(accessToken) | GET /api/admin/overview | 组件挂载时 |

# 11.7 AdminMembershipsPage — 会员管理

## 基本信息

| 属性 | 值 |
|---|---|
| 路由 | /admin/memberships |
| 文件 | src/pages/AdminMembershipsPage.jsx |
| 守卫 | requireAdmin，fallbackPath /admin |
| 懒加载 | 是 |

## 页面布局

```
  标题 + 描述

新建会员面板
  ├── user_id 输入
  ├── email 输入
  ├── tier 下拉选择（free_member / paid_member / admin）
  ├── status 下拉选择（active / trial / paused / expired）
  ├── note 输入
  ├── expires_at datetime 输入
  └── "新建" 按钮

搜索/过滤
  └── 关键词搜索输入框 + 刷新按钮

角色统计行（data.counts，每个 tier 的总数）

会员列表
  └── 每行一个会员
    ├── user_id
    ├── email（可编辑）
    ├── tier 下拉（可编辑）
    ├── status 下拉（可编辑）
    ├── note（可编辑）
    ├── expires_at（可编辑）
    └── "保存" 按钮
```

### 核心交互

1. 加载会员列表：fetchAdminMemberships(accessToken, 120, query) → 填充列表并生成本地可编辑副本（drafts）
2. 编辑会员：修改 drafts 中的字段 → 点击保存 → updateAdminMembership(userId, payload, accessToken) → 刷新列表
3. 新建会员：填写新建表单 → updateAdminMembership(newMember.user_id, payload, accessToken) → 刷新列表
4. 搜索：输入关键词 → 触发 loadData → 重新获取列表

### 调用的 API

| API 函数 | 端点 | 触发时机 |
|---|---|---|
| fetchAdminMemberships(authToken, limit, query) | GET /api/admin/memberships | 初始化、搜索、 |
| updateAdminMembership(userId, payload, authToken) | PUT /api/admin/memberships/:userId | 保存修改、新建 |

## 11.8 MembershipPage — 会员方案与权益

### 基本信息

| 属性 | 值 |
|---|---|
| 路由 | /membership |
| 文件 | src/pages/MembershipPage.jsx |
| 守卫 | 无，所有角色可访问 |
| 懒加载 | 是 |

### 页面布局

```
  Hero 区域（使用 roleExperience 配置）
  ├── 角色角标 + 标题 + 描述
  └── 统计卡片（roleExperience.statCards）

快捷操作区（roleExperience.quickActions）

当前会员状态面板（已登录时显示）
  ├── 等级标签
  ├── 状态标签
  ├── 权益列表（benefits 数组）
  └── 到期时间

付费方案卡片区（featuredPlans）
  └── 每张卡片
    ├── 方案名称
    ├── 价格（CNY xxx / 月/季/年）
    ├── 权益列表
    ├── 结账按钮（如果 paymentsEnabled）
    └── 方案确认按钮（如果是 mock 支付）

活跃订阅信息（activeSubscription，如果存在）
  ├── 方案名称
  ├── 状态
  ├── 开始/到期时间
  └── 计费周期
... (共 28 行，已截断显示)
```

### 核心交互

1. 数据加载：并行调用 fetchBillingPlans(language) 和 fetchBillingProfile(authToken, language)
2. 角色体验动态化：根据 resolveRoleTier() 获取当前角色，使用 getRoleExperience() 渲染对应的 Hero 和快捷操作
3. 结账流程：点击方案卡片的结账按钮 → createBillingCheckoutIntent({ plan_code }, authToken) → 显示结果消息（如支付链接或确认信息）
4. 访客 vs 会员视图：
- guest：显示方案市场（showMarketplace），引导注册
- 已登录用户：显示个人工作空间（showMemberWorkspace），包含当前状态和订阅信息

### 调用的 API

| API 函数 | 端点 | 触发时机 |
|---|---|---|
| fetchBillingPlans(language) | GET /api/billing/plans | 组件挂载 |
| fetchBillingProfile(authToken, language) | GET /api/billing/me | 组件挂载 |
| createBillingCheckoutIntent(payload, authToken) | POST /api/billing/checkout-intent | 点击结账按钮 |

## 11.9 AnalyticsPage — 内容分析

### 基本信息

| 属性 | 值 |
|---|---|
| 路由 | /analytics |
| 文件 | src/pages/AnalyticsPage.jsx |
| 守卫 | requireAdmin，fallbackPath / |
| 懒加载 | 是 |

### 页面布局

```
  标题区
  ├── Kicker: "Analytics"
  ├── 标题: "内容表现与读者行为分析"
  └── 说明: 仅管理员可见

指标卡片区（4 列，data.metrics 动态渲染）
  └── 每张: label + value（大字号）+ detail

两栏布局
  ├── 7 日浏览趋势（柱状图，CSS 渐变条）
  |   └── 每天一行: 日期 + 数值 + 进度条（宽度按比例）
  |
  └── 右侧内容
    ├── 热门文章（按浏览数排序，ArticleCard）
    ├── 最多点赞文章
    └── 最多收藏文章
```

### 调用的 API

| API 函数 | 端点 | 触发时机 |
|---|---|---|
| fetchAnalyticsOverview(accessToken) | GET /api/analytics/overview | 组件挂载时 |

## 11.10 MediaHubPage — 音频/视频媒体

### 基本信息

| 属性 | 值 |
|---|---|
| 路由 | /audio（kind=audio）、/video（kind=video） |
| 文件 | src/pages/MediaHubPage.jsx |
| 守卫 | 无 |
| 懒加载 | 是 |

### 页面布局

```
  Hero 区域（音频为橙色点缀，视频同）
  ├── 标题 + 描述（根据 kind 切换文案）
  ├── "进入媒体后台" 按钮（仅管理员显示）
  ├── "查看权限层级" 按钮（→ /membership）
  └── 统计卡片（3 张）
    ├── 当前身份（viewer_tier）
    ├── 媒体总数（items.length）
    └── 总时长（所有项目 duration_seconds 之和）

当前选中项播放区
  ├── 封面图
  ├── 播放器
  |   ├── audio kind → <audio controls>
  |   └── video kind → <video controls>
  ├── 标题 + 权限标签
  ├── 演讲者 + 时长 + 发布日期
  ├── 权限访问提示（如果 locked）
  └── 富文本内容（body_markdown 或 transcript_markdown，ReactMarkdown 渲染）

媒体列表
  └── 网格卡片，每张可点击选中
    ├── 封面图 + 播放图标叠加
    ├── 标题
    ├── 演讲者 + 时长
    └── 权限标签（公开/会员/付费 + 锁/可访问图标）
```

### 核心交互

1. 数据加载：fetchMediaHub(kind, accessToken, 24) 获取媒体列表
2. 选择播放：点击列表中的卡片 → 更新 selectedSlug → 播放区显示对应项
3. 权限控制：后端返回每个 item 的 accessible 标志和 media_url / preview_url
   - 可访问 → 使用 media_url
   - 不可访问 → 使用 preview_url（试听/试看）
4. 管理员入口：管理员可看到 "进入媒体后台" 按钮

### 调用的 API

| API 函数 | 端点 | 触发时机 |
|---|---|---|
| fetchMediaHub(kind, authToken, 24) | GET /api/media/:kind | 组件挂载时、kind 或 token 变化时 |

## 11.11 MediaStudioPage — 媒体管理后台

### 基本信息

| 属性 | 值 |
|---|---|
| 路由 | /media-studio |
| 文件 | src/pages/MediaStudioPage.jsx |
| 守卫 | requireAdmin，fallbackPath /admin |
| 懒加载 | 是 |

### 页面布局

```
Hero + 标题

创建/编辑表单
├── kind 切换（audio / video）
├── 标题、摘要、演讲者
├── 系列名、集数、发布日期
├── 时长（秒）
├── 可见性（public / member / paid）
├── 状态（draft / published）
├── 封面图 URL、媒体文件 URL、试听 URL、来源 URL
├── 正文 Markdown、文字稿 Markdown
├── 章节列表编辑
├── 文件上传按钮
└── 保存 / 新建按钮

媒体列表（左侧面板）
└── 按 kind 和 status 可筛选
└── 每项可点击加载到表单
```

### 核心交互

1. 列表加载：fetchMediaAdminItems(kind, status, 60) 获取管理端媒体列表
2. 项目加载：点击列表项 → fetchMediaAdminItem(id) → 填充表单
3. 保存：createMediaAdminItem(payload) 或 updateMediaAdminItem(id, payload)
4. 文件上传：uploadMediaAdminFile(file, kind, 'media') 上传音频/视频文件，返回 URL

### 调用的 API

| API 函数 | 端点 | 触发时机 |
|---|---|---|
| fetchMediaAdminItems(kind, status, limit) | GET /api/media/admin/items | 初始化、筛选变 |
| fetchMediaAdminItem(id) | GET /api/media/admin/items/:id | 点击列表项 |
| createMediaAdminItem(payload) | POST /api/media/admin/items | 新建保存 |
| updateMediaAdminItem(id, payload) | PUT /api/media/admin/items/:id | 更新保存 |
| uploadMediaAdminFile(file, kind, usage) | POST /api/media/admin/upload | 文件上传 |

## 11.12 TopicsPage — 专题广场

### 基本信息

| 属性 | 值 |
|---|---|
| 路由 | /topics |
| 文件 | src/pages/TopicsPage.jsx |
| 守卫 | 无（但后端按权限返回 401/403） |
| 懒加载 | 是 |

### 访问权限（后端控制）

- 付费会员和管理员：正常返回专题列表

- 未登录用户（401）：显示"登录后可继续查看"提示 + 登录按钮
- 免费会员（403）：显示"仅对付费会员开放"提示 + 升级按钮

### 页面布局

```
权限不足时
└── 提示面板 + 登录/升级按钮

正常访问时
├── 标题: "专题广场" / "按议题进入结构化商业阅读"
└── 专题卡片网格（4 列）
    └── 每张卡片
        ├── 类型标签（formatTopicType）
        ├── 标题
        ├── 描述
        ├── 标签组（最多 3 个 TagBadge）
        ├── 文章数量
        └── 链接到 /topic/:slug
```

### 调用的 API

| API 函数 | 端点 | 触发时机 |
|---|---|---|
| fetchTopics(accessToken) | GET /api/topics | 组件挂载时 |

## 11.13 TopicPage — 专题详情

### 基本信息

| 属性 | 值 |
|---|---|
| 路由 | /topic/:slug |
| 文件 | src/pages/TopicPage.jsx |
| 守卫 | 无（后端按权限控制） |
| 懒加载 | 是 |

### 页面布局

```
专题头部
├── 类型标签
├── 标题
├── 描述
├── 标签组
├── 关注/取关按钮（需登录）
└── 文章总数

文章列表（分页加载，每页 12 篇）
└── ArticleCard 网格 + "加载更多" 按钮
```

### 核心交互

1. 专题加载：fetchTopic(slug, page, pageSize, accessToken)
2. 关注状态：fetchFollows(accessToken) 检查是否已关注
3. 切换关注：toggleFollow({ entity_type: 'topic', entity_slug, active }, accessToken)
4. 分页加载：点击"加载更多" → 加载下一页追加到列表

### 调用的 API

| API 函数 | 端点 | 触发时机 |
|---|---|---|
| fetchTopic(slug, page, pageSize, authToken) | GET /api/topics/:slug | 初始化、加载更多 |
| fetchFollows(authToken) | GET /api/follows | 初始化 |
| toggleFollow(payload, authToken) | POST /api/follows | 点击关注/取关 |

## 11.14 TimeMachinePage — 时光机

### 基本信息

| 属性 | 值 |
| --- | --- |
| 路由 | /time-machine |
| 文件 | src/pages/TimeMachinePage.jsx |
| 守卫 | 无 |
| 懒加载 | 是 |

## 页面布局

```
  标题区: "时光机" / "从某一天切回当时的阅读现场"

日期输入面板
  ├── 日期输入框（文本格式，如 "2023-06-21"）
  └── "启动时光机" 按钮

Loading 提示

结果卡片（唱片封面式大卡片，居中展示）
  ├── 封面图（或渐变占位）
  ├── 发布日期
  ├── 文章标题
  ├── 引用语（data.quote）
  ├── 摘要
  ├── "阅读文章" 按钮（→ /article/:id）
  └── "换一篇" 按钮（重新调用 API）
```

## 核心交互

1. 初始加载：不传日期调用 fetchTimeMachine('', language) 获取随机历史文章
2. 指定日期：输入日期后点击"启动时光机" → fetchTimeMachine(targetDate, language)
3. 换一篇：点击"换一篇" → 重新调用（同日期或空日期随机）

## 调用的 API

| API 函数 | 端点 | 触发时机 |
| --- | --- | --- |
| fetchTimeMachine(date, language) | GET /api/time_machine | 初始化、搜索、换一篇 |

# 11.15 CommercialPage — 商业方案

## 基本信息

| 属性 | 值 |
| --- | --- |
| 路由 | /commercial |
| 文件 | src/pages/CommercialPage.jsx |
| 守卫 | 无 |
| 懒加载 | 是 |

## 页面布局

```
  Hero 区域（深蓝+橙色渐变）
  ├── 标题："让复旦商业知识库进入会员服务与机构项目"
  ├── 说明文字
  ├── 搜索入口按钮
  └── 会员入口按钮

能力卡片区（3 张）
  ├── 机构知识中枢
  ├── 研究与教学支持
  └── 会员媒体服务

能力清单（5 条）
  ├── 统一搜索
  ├── AI 助理
  ├── 四层访问体系
  ├── 订阅基础能力
  └── 编辑后台

版本方案卡片（3 张）
  ├── 知识库版本
  ├── 会员版本
  └── 运营版本

价格信息区
  └── 从 fetchBillingPlans 获取真实价格显示
 ... (共 36 行，已截断显示)
```

### 核心交互

1. 数据加载：并行调用 fetchCommerceOverview() 和 fetchBillingPlans(language)
2. 提交咨询：填写表单 → submitDemoRequest(form) → 显示成功消息 → 清空表单

### 调用的 API

| API 函数 | 端点 | 触发时机 |
|---|---|---|
| fetchCommerceOverview() | GET /api/commerce/overview | 组件挂载时 |
| fetchBillingPlans(language) | GET /api/billing/plans | 组件挂载时 |
| submitDemoRequest(payload) | POST /api/commerce/demo-request | 提交表单 |

## 11.16 CommercialLeadsPage — 商务线索

### 基本信息

| 属性 | 值 |
|---|---|
| 路由 | /commercial/leads |
| 文件 | src/pages/CommercialLeadsPage.jsx |
| 守卫 | requireAdmin，fallbackPath /admin |
| 懒加载 | 是 |

### 页面布局

```
  标题区 + "导出 CSV" 按钮 + "返回商业方案页" 链接

两栏布局
  ├── 线索列表（fetchDemoRequests）
  │   └── 每条线索
  │       ├── 姓名、机构、职位
  │       ├── 邮箱、电话
  │       ├── 使用场景、留言
  │       └── 创建时间
  │
  └── 最近订单列表（fetchAdminBillingOrders）
      └── 每条订单
          ├── 订单 ID
          ├── 状态
          └── 创建时间
```

### 调用的 API

| API 函数 | 端点 | 触发时机 |
|---|---|---|

| | | |
|---|---|---|
| fetchDemoRequests(80) | GET /api/commerce/demo-requests | 组件挂载时 |
| fetchAdminBillingOrders(authToken, 20) | GET /api/admin/billing/orders | 组件挂载时（仅管理员） |

## 11.17 MyLibraryPage — 我的资产

### 基本信息

| 属性 | 值 |
|---|---|
| 路由 | /me |
| 文件 | src/pages/MyLibraryPage.jsx |
| 守卫 | 无（未登录时显示引导页） |
| 懒加载 | 是 |

### 访问行为

- 未登录：显示 guest roleExperience 引导页（Hero + 统计卡片 + 快捷操作 + 登录入口）
- 已登录：显示完整的个人资产库

### 页面布局（已登录）

```
Hero 区域（roleExperience 配置）
├── 角色角标 + 标题 + 描述
├── 用户信息标签（display_name / tier label）
└── 统计卡片（roleExperience.statCards）

资产概览卡片（4 张）
├── 收藏数
├── 点赞数
├── 历史数
└── 关注数

快捷操作区（roleExperience.quickActions）

三个资产区块（LibrarySection 组件）
├── 收藏（Bookmark 图标）→ data.bookmarks
├── 点赞（Heart 图标）→ data.likes
└── 最近浏览（Eye 图标）→ data.recent_views
└── 每个区块内: ArticleCard 网格 或 空状态提示
```

### 调用的 API

| API 函数 | 端点 | 触发时机 |
|---|---|---|
| fetchMyDashboard(authToken) | GET /api/me/dashboard | 组件挂载时 |
| fetchMyLibrary(authToken, 12) | GET /api/me/library | 组件挂载时（仅已登录） |

## 11.18 MyFollowingPage — 我的关注

### 基本信息

| 属性 | 值 |
|---|---|
| 路由 | /following |
| 文件 | src/pages/MyFollowingPage.jsx |
| 守卫 | 无（未登录时数据为空） |
| 懒加载 | 是 |

### 页面布局

```
  标题区

关注列表（左侧/顶部）
  └── 每个关注项
    ├── 实体类型标签（标签/专题/栏目）
    ├── 实体名称
    ├── 可点击选中（高亮）
    └── 取消关注按钮

选中关注项的文章列表（右侧/底部）
  └── fetchWatchlist 返回的文章
  └── ArticleCard 网格
  └── 空状态提示
```

### 核心交互

1. 加载关注列表：fetchFollows(accessToken) → 渲染关注项列表
2. 选择关注项：点击关注项 → 设置 selectedFollowKey → fetchWatchlist(authToken, 24, { entityType, entitySlug }) 加载相关文章
3. 取消关注：toggleFollow({ entity_type, entity_slug, active: false }, accessToken) → 刷新列表

### 调用的 API

| API 函数 | 端点 | 触发时机 |
|---|---|---|
| fetchFollows(authToken) | GET /api/follows | 组件挂载时 |
| fetchWatchlist(authToken, limit, filters) | GET /api/follows/watchlist | 选中关注项 |
| toggleFollow(payload, authToken) | POST /api/follows | 取消关注时 |

## 11.19 LoginPage — 登录中心

### 基本信息

| 属性 | 值 |
|---|---|
| 路由 | /login |
| 文件 | src/pages/LoginPage.jsx |
| 守卫 | 无 |
| 懒加载 | 是 |

### 页面布局

```
 两栏布局
 ├── 左侧（信息面板）
 |   ├── Hero: "登录复旦商业知识库"
 |   └── 3 张说明卡片
 |     ├── 访问: 一个登录入口覆盖所有权限
 |     ├── 延续: 登录后回到原始目标页
 |     └── 访客浏览: 公开内容继续开放
 |
 └── 右侧（登录面板）
   ├── 已登录状态
   |   ├── 当前角色面板（display_name + tier）
   |   ├── 默认首页路径
   |   ├── "进入首页" 按钮
   |   └── "切换账号" 按钮（signOut）
   |
   └── 未登录状态
     ├── 邮箱输入框
     ├── 密码输入框
     ├── 错误提示（authError）
     ├── "登录" 按钮
     └── "查看会员方案" 链接
```

### 核心交互

1. 自动跳转：已登录用户访问 /login 时，自动跳转到 redirect 参数指定的路径，或 roleHomePath，或 roleExperience.entryPath
2. 登录提交：调用 signInWithPassword({ email, password })，错误信息由 AuthProvider 管理
3. 登出/切换：已登录时点击"切换账号" → signOut() → 清空状态回到登录表单

### 自动跳转目标优先级

1. URL 查询参数 ?redirect=<path>
2. roleHomePath（后端返回的角色首页，如 /admin）
3. currentExperience.entryPath（roleExperience 配置的入口路径）
4. /（最终兜底）

## 11.20 ColumnPage — 栏目详情

### 基本信息

| 属性 | 值 |
|---|---|
| 路由 | /column/:slug |
| 文件 | src/pages/ColumnPage.jsx |
| 守卫 | 无 |
| 懒加载 | 是 |

### 核心交互

1. 栏目加载：fetchColumnArticles(slug, page, PAGE_SIZE) 获取栏目文章（每页 18 篇）
2. 关注功能：检查是否已关注 → 显示关注/取关按钮 → toggleFollow({ entity_type: 'column', entity_slug, active }, authToken)
3. 分页加载：点击"加载更多" → 加载下一页追加到列表

### 调用的 API

| API 函数 | 端点 | 触发时机 |
|---|---|---|
| fetchColumnArticles(slug, page, pageSize) | GET /api/columns/:slug/articles | 初始化、加载更多 |
| fetchFollows(authToken) | GET /api/follows | 初始化（仅已登录） |
| toggleFollow(payload, authToken) | POST /api/follows | 点击关注/取关 |

## 11.21 TagPage — 标签文章列表

### 基本信息

| 属性 | 值 |
|---|---|
| 路由 | /tag/:slug |
| 文件 | src/pages/TagPage.jsx |
| 守卫 | 无 |
| 懒加载 | 是 |

### 核心交互

1. 标签文章加载：fetchTagArticles(slug, page, PAGE_SIZE) 获取该标签下的文章（每页 18 篇）
2. 关注功能：同 ColumnPage，entity_type 为 'tag'
3. 分页加载：追加模式

### 调用的 API

| API 函数 | 端点 | 触发时机 |
|---|---|---|
| fetchTagArticles(slug, page, pageSize) | GET /api/tags/:slug/articles | 初始化、加载更多 |
| fetchFollows(authToken) | GET /api/follows | 初始化 |
| toggleFollow(payload, authToken) | POST /api/follows | 关注/取关 |

## 11.22 OrganizationsPage — 机构索引

### 基本信息

| 属性 | 值 |
|---|---|
| 路由 | /organizations |
| 文件 | src/pages/OrganizationsPage.jsx |
| 守卫 | 无 |

| | |
|---|---|
| 懒加载 | 是 |

### 页面布局

```
  标题: "机构索引"
说明: "从文章中的机构字段聚合而来"

机构卡片网格（3 列）
  └── 每张卡片
    ├── 机构名称
    ├── 相关文章数
    ├── 最近更新日期
    └── 链接到 /organization/:slug
```

### 调用的 API

| API 函数 | 端点 | 触发时机 |
|---|---|---|
| fetchOrganizations(80) | GET /api/organizations | 组件挂载时 |

## 11.23 OrganizationPage — 机构详情

### 基本信息

| 属性 | 值 |
|---|---|
| 路由 | /organization/:slug |
| 文件 | src/pages/OrganizationPage.jsx |
| 守卫 | 无 |
| 懒加载 | 是 |

### 页面布局

```
  机构头部
  ├── 机构名称
  ├── 文章总数
  └── 最近更新日期

文章列表（分页加载，每页 18 篇）
  └── ArticleCard 网格 + "加载更多" 按钮
```

### 调用的 API

| API 函数 | 端点 | 触发时机 |
|---|---|---|
| fetchOrganization(slug, page, pageSize) | GET /api/organizations/:slug | 初始化、加载更多 |

## 11.24 页面间导航关系图

```
                ┌──────────────────────┐
                │  HomePage   │
                │    /     │
                └──────────────────────┘
                          │
          ┌──────────────────────────┼──────────────────────────┐
          │          │          │
     ┌────────────▼────────────┐  ┌──────────▼──────────┐  ┌──────────────▼──────────────┐
     │ SearchPage  │  │ LoginPage │  │ MembershipPage │
     │   /search   │  │ /login  │  │ /membership  │
     └────────────────────────┘  └────────────────────┘  └────────────────────────────┘
          │          │          │
     ┌────────────▼────────────┐    │      ┌──────────▼──────────┐
     │ ArticlePage │    │       │ ChatPage  │
     │ /article/:id │    │       │   /chat   │
     └────────────────────────┘    │      └────────────────────┘
                │
          ┌──────────────────────────┼──────────────────────────┐
          │          │          │
     ┌────────────▼────────────┐    │   ┌──────────▼──────────┐
     │ MyLibraryPage │    │    │ AdminConsole  │
     │    /me      │    │    │   /admin    │
     └────────────────────────┘    │   └────────────────────┘
                │    ┌──────────────┼──────────────┐
                │    │    │       │
              ┌──────────▼──┐  ┌▼──────────▼┐  ┌──────────▼──────────┐
... (共 29 行，已截断显示)
```

## 11.25 全局交互模式

### 11.25.1 公共数据加载模式

所有页面采用一致的数据加载模式：

```
  const [data, setData] = useState(null)
const [error, setError] = useState('')

useEffect(() => {
 fetchXxx()
  .then(setData)
  .catch(() => setError('加载失败'))
}, [dependencies])
```

部分页面增加了 cleanup 函数防止 unmount 后的状态更新：

```
  useEffect(() => {
 let mounted = true
 fetchXxx().then((payload) => {
  if (!mounted) return
  setData(payload)
 })
 return () => { mounted = false }
}, [])
```

### 11.25.2 分页加载模式

多个列表页面（Column、Tag、Topic、Organization、HomePage latest）使用相同的"加载更多"模式：

```
  const handleLoadMore = async () => {
 setLoadingMore(true)
 try {
  const nextPage = data.page + 1
  const payload = await fetchXxx(slug, nextPage, PAGE_SIZE)
  setData((current) => ({
   ...payload,
   items: [...(current?.items || []), ...payload.items],
  }))
 } finally {
  setLoadingMore(false)
 }
}
```

### 11.25.3 关注模式

Column、Tag、Topic 三类实体共享相同的关注交互：

1. fetchFollows(accessToken) 检查当前用户的关注列表

2. some() 判断是否已关注当前实体
3. toggleFollow({ entity_type, entity_slug, active }) 切换关注状态
4. 未登录时点击关注 → openAuthDialog() 弹出登录对话框

### 11.25.4 多语言文案模式

所有页面使用两种国际化策略：

1. messages.js 翻译：通过 t('key') 获取预定义翻译（用于固定 UI 文案）
2. 行内 byLanguage：isEnglish ? 'English text' : '中文文字'（用于动态/页面级文案）

### 11.25.5 错误处理模式

```
{error ? <div className="mt-6 text-sm text-red-500">{error}</div> : null}
```

错误信息统一以红色文字显示在页面主内容区上方。部分页面（如 AdminMembershipsPage）根据 HTTP 状态码（401、403）显示差异化提示。

### 11.25.6 Loading 状态模式

- 全局 Suspense fallback：PageFallback 组件显示"加载中"文字
- 页面级 loading：各页面自行管理 loading state
- 操作级 busy：EditorialWorkbenchPage 使用 busy 字符串标识当前正在执行的操作，对应按钮显示 LoaderCircle 旋转图标

# 12 --- 数据管道

本章完整描述复旦商业知识库从原始网页到可检索、可翻译、可对话的全流程数据管道。整条管道分为四个阶段，每个阶段对应一个或多个脚本，前一阶段的输出即后一阶段的输入。

## 12.1 管道总览

```
  Stage 1          Stage 2          Stage 3          Stage 4
网页爬虫          数据库构建        FAISS 索引构建      AI 批处理
(crawler)       (build_business_db)  (create_vector_db)  (article_ai_batch)

微信公众号 ──┐                                  ┌── 摘要 (summary)
复旦管院  ──┤    content.txt ──────── SQLite DB ──────── FAISS   ├── 排版 (formatted_md)
第三方源  ──┘    + 图片          articles   索引     └── 翻译 (translation)
            |      tags              |
            |      columns           ▼
            ▼      topics        article_ai_outputs
        Fudan_Business_        featured_articles   article_translations
        Knowledge_Data/        chat_sessions
        YYYY-MM/日期_标题/
```

**执行顺序**

| 顺序 | 脚本 | 入口命令 | 说明 |
|---|---|---|---|
| 1 | business_knowledge_crawler.py | python business_knowledge_crawler.py | 爬取网页，保存到本地文件系统 |
| 2 | build_knowledge_base.py | python build_knowledge_base.py | 读取本地文件，构建 SQLite 数据库 |
| 3 | create_vector_db_faiss.py | python create_vector_db_faiss.py | 从 SQLite 构建 FAISS 向量索引 |
| 4 | backend/scripts/article_ai_batch.py | python -m backend.scripts.article_ai_batch run-all | 批量生成摘要、排版、翻译 |

> 阶段 1 和阶段 4 均涉及外部网络请求（分别是网页爬取和 Gemini API 调用），运行时间较长。阶段 2 和阶段 3 纯本地计算，通常在数分钟内完成。

## 12.2 网页爬虫 --- business_knowledge_crawler.py

### 12.2.1 职责

从预定义的链接文件中逐条读取文章 URL，发送 HTTP 请求，解析 HTML 页面，提取正文文本和图片，保存到本地文件系统的标准目录结构中。

### 12.2.2 配置区域

| 配置项 | 值/说明 |
|---|---|
| SAVE_ROOT | 项目根目录下的源目录名（如 锦缎），按数据来源命名 |
| LINKS_FILE | 同级目录下的 {源名}_links.txt，每行一条链接记录 |
| HEADERS | 模拟 Chrome 121 的 User-Agent，避免被反爬策略拦截 |
| MAX_WORKERS | 并发线程数，默认 5 |

### 12.2.3 链接文件格式

```
  2025-12-19 | 文章标题文本 | https://mp.weixin.qq.com/s/xxxxxxxx
2025-12-18 | 另一篇标题  | https://www.fdsm.fudan.edu.cn/xxxxx
```

每行由 | 分隔为三段：日期、标题、URL。脚本会忽略空行和不符合三段格式的行。URL 必须以 http 开头。

### 12.2.4 输出目录结构

```
Fudan_Business_Knowledge_Data/
2025-12/
  2025-12-19_文章标题最多50字/
    content.txt      <- 正文文本
    image_1.jpg      <- 页面内图片
    image_2.png
  2025-12-18_另一标题/
    content.txt
    image_1.jpg
2025-11/
  ...
```

**目录层级规则：**

- 第一层：YYYY-MM 月份目录，取自日期字符串前 7 位
- 第二层：YYYY-MM-DD_安全标题 文章目录，标题经 clean_filename() 清洗后截断前 50 字符
- 文件：content.txt 为正文，image_{序号}.{ext} 为图片

**文件名清洗规则（clean_filename）：**

```
re.sub(r'[\\/*?:\"<>|\n\t]', "", text).strip()
```

移除所有 Windows 文件系统不支持的字符。

### 12.2.5 content.txt 文件格式

```
标题: 文章标题
日期: 2025-12-19
链接: https://mp.weixin.qq.com/s/xxxxxxxx
来源模式: wechat
----------------------------------------

正文内容...
```

文件头包含四个元数据字段（标题、日期、链接、来源模式），以 40 个短横线 ---------- 分隔后接正文。正文由 BeautifulSoup.get_text(separator="\n", strip=True) 提取，保留自然换行。

### 12.2.6 页面解析策略

爬虫采用三级回退策略识别页面正文区域：

| 优先级 | 识别方式 | 来源模式（source_mode） | 说明 |
|---|---|---|---|
| 1 | div#js_content 或 div.rich_media_content | wechat | 微信公众号标准正文容器 |
| 2 | div.detail-con | school_native | 复旦管院官网正文容器 |
| 3 | 全局搜索 mmbiz.qpic.cn 图片 | wechat_fallback_global | 纯图片文章的兜底策略 |

**详细流程：**

1. 首先尝试查找 div#js_content 或 div.rich_media_content（微信公众号正文区域）
2. 如果未找到，尝试查找 div.detail-con（学校官网正文区域）
3. 如果仍未找到，进入全局回退模式：
- 搜索页面中所有包含 mmbiz.qpic.cn 域名的图片（同时检查 src 和 data-src 属性）
- 如果找到图片，保存图片并写入 content.txt（标记为 "Fallback Global" 模式）
- 如果连图片都没有，打印警告并跳过该页面

**微信图片处理注意事项：**

- 微信图片的真实 URL 存储在 data-src 属性而非 src（微信模式下优先取 data-src）
- 图片后缀通过 URL 参数判断：wx_fmt=png -> .png，wx_fmt=gif -> .gif，wx_fmt=jpeg 或 wx_fmt=jpg -> .jpg
- 协议相对路径（//example.com/img.png）自动补全 https: 前缀
- 非微信模式下优先取 src 属性

### 12.2.7 断点续爬

```
if os.path.exists(content_file):
safe_print(f"  [跳过] (已存在) {safe_title}...")
return
```

如果目标目录的 content.txt 已存在，直接跳过。这意味着：

- 脚本可以安全地重复执行
- 增量爬取只需在链接文件中追加新行
- 需要重爬时，手动删除对应目录即可

### 12.2.8 并发控制与线程安全

```
    MAX_WORKERS = 5

with ThreadPoolExecutor(max_workers=MAX_WORKERS) as executor:
    futures = [executor.submit(process_item, line) for line in lines]
    for future in futures:
        try: future.result()
        except: pass
```

使用 concurrent.futures.ThreadPoolExecutor，默认 5 个并发线程。所有 print 输出通过 threading.Lock (print_lock) 保护的 safe_print() 函数进行，防止终端输出交错。

每个线程独立处理一行链接记录，异常被静默捕获（except: pass），不会导致整个批次中断。

### 12.2.9 错误处理

| 错误场景 | 处理方式 |
|---|---|
| 链接格式不正确（少于 3 段） | 直接返回，跳过该行 |
| URL 不以 http 开头 | 直接返回，跳过该行 |
| HTTP 请求失败 | 捕获异常，打印 [错误] 日志 |
| 图片下载失败 | 捕获异常，打印 Warning 日志，不影响文本保存 |
| 无法识别页面结构 | 打印 [警告]，跳过该页面 |

## 12.3 数据库构建 --- build_business_db.py

### 12.3.1 职责

遍历 Fudan_Business_Knowledge_Data/ 下所有 content.txt 文件，解析元数据与正文，进行标签分配、栏目分配、特征计算，最终将全部数据写入 SQLite 数据库。

入口脚本 build_knowledge_base.py 是一个薄包装层，直接调用 backend.scripts.build_business_db.main()。

### 12.3.2 完整处理流程

```
  content.txt 文件
  |
  ▼
split_header_and_body()   —— 分离头部元数据与正文
  |
  ▼
parse_header()          —— 提取 标题/日期/链接/来源模式
  |
  ▼
load_ai_output()        —— 合并 Gemini 批处理预分析结果 (如存在)
  |
  ▼
infer_article_type()    —— 推断文章类型 (专访/对谈/访谈/案例报道/...)
  |
  ▼
taxonomy_service        —— 标签分配 (build_tag_entries)
  |                       - industry 行业标签
  |                       - topic 主题标签
  |                       - type 类型标签
  |                       - entity 实体标签
  |                       - series 系列标签
  |
  ▼
clustering_rules        —— 栏目分配 (derive_column_slugs)
  |
 ... (共 42 行，已截断显示)
```

### 12.3.3 数据模型 --- RawDocument

RawDocument 是内部数据传输对象（dataclass），包含一篇文章的所有计算结果：

| 字段 | 类型 | 说明 | |
|---|---|---|---|
| article_id | int | 自增主键，按文件遍历顺序分配 | |
| doc_id | str | 基于 relative_path 的 SHA1 哈希前 20 位，跨重建稳定 | |
| slug | str | URL 友好标识，格式 {日期}-{标题}，最长 96 字符 | |
| relative_path | str | 相对于 BUSINESS_DATA_DIR 的路径 | |
| title | str | 文章标题，优先取 content.txt 头部，其次取 AI 输出，最后取目录名 | |
| publish_date | str | 发布日期 YYYY-MM-DD，缺失时默认 2000-01-01 | |
| link | str | 原始来源 URL | |
| source_mode | str | 来源模式 (wechat / school_native / business) | |
| content | str | 正文文本 | |
| excerpt | str | 正文前 180 字符摘要，由 excerpt_from_content() 生成 | |
| word_count | int | 去空格后的字符数 | |
| article_type | str | 文章类型（专访/对谈/评论观点等） | |
| main_topic | `str \ | None` | 主话题（来自 AI 分析） |
| series_or_column | `str \ | None` | 所属系列/专栏（来自 AI 分析） |
| primary_org_name | `str \ | None` | 主要关联机构 |
| cover_image_path | `str \ | None` | 封面图片相对路径（取目录内按字母排序首个图片） |
| tag_names | list[tuple[str, str, float]] | 标签列表 (名称, 分类, 置信度) | |
| column_slugs | list[str] | 分配到的栏目 slug 列表 | |
| people_text | str | 人物名称，`\ | ` 分隔，最多 4 人 |
| org_text | str | 机构名称，`\ | ` 分隔，最多 4 个 |
| tag_text | str | 标签名称，`\ | ` 分隔 |
| search_text | str | 全文搜索索引文本（拼接标题+主话题+标签+人物+机构+摘要） | |
| view_count | int | 初始浏览量（算法生成） | |
| is_featured | int | 是否精选（0/1） | |

### 12.3.4 元数据解析

```
split_header_and_body(text)
```

按照 ----------（40 个短横线）分隔符将 content.txt 分为头部和正文。如果文件中没有分隔符，则取前 6 行作为头部，其余为正文。

`parse_header(header_lines)`

从头部行中提取四个关键字段：

| 前缀 | 字段名 |
|---|---|
| 标题: | title |
| 日期: | publish_date |
| 链接: | link |
| 来源模式: | source_mode |

`slugify(value)`

将标题转换为 URL 友好的 slug：

1. 转小写
2. 替换 /、|、｜、・、空格为 -
3. 用正则移除非 \w、非中文、非 - 的字符
4. 合并连续 -，去掉首尾 -
5. 如果结果为空，使用 SHA1 哈希前 12 位

### 12.3.5 AI 预分析结果合并

load_ai_output() 从 GEMINI_AGGREGATE_PATH 加载 JSON 文件，按 relative_path 建立索引。每篇文章的字段优先级为：

| 字段 | 优先级：content.txt 头部 > AI 输出 > 默认值 |
|---|---|
| title | 头部 标题: > ai_row.title > 目录名 |
| publish_date | 头部 日期: > ai_row.publish_date > "2000-01-01" |
| link | 头部 链接: > ai_row.source_url > "" |
| source_mode | 头部 来源模式: > ai_row.source_mode > "business" |

AI 预分析提供以下独有增强字段（model_output 子对象）：

| AI 输出字段 | 用途 |
|---|---|
| article_type | 文章类型分类 |
| main_topic | 主话题 |
| series_or_column | 所属系列 |
| topic_keywords | 主题关键词列表（最多取 5 个） |
| primary_org_name | 主要关联机构 |
| people | 文中提及人物列表（含 person_name、org_name、fdsm_relation） |

### 12.3.6 文章类型推断

当 AI 预分析结果的 article_type 不可用时，infer_article_type() 通过标题和正文前 200 字的关键词规则推断：

| 检测条件 | 推断类型 |
|---|---|
| 专访 出现在标题或正文前200字 | 专访 |
| 对谈 出现在标题 | 对谈 |
| 访谈 出现在标题 | 访谈 |
| 案例 出现在标题 | 案例报道 |
| 研究 或 论文 出现在标题 | 研究/论文解读 |
| 预告 或 报名 出现在标题 | 活动/预告 |
| 书 + (阅读 / 书评 / 翻书) 出现在标题 | 书摘/书评 |
| 以上均不匹配 | 评论/观点 |

### 12.3.7 标签分配

标签通过 taxonomy_service.build_tag_entries() 生成，每个标签包含三元组 (name, category, confidence)。标签分为五个类别：

| 分类（category） | 颜色 | 说明 | 示例 |
|---|---|---|---|
| industry | #ea6b00 (橙色) | 行业标签 | 金融科技、新能源 |
| topic | #0d0783 (深蓝) | 主题标签 | AI/人工智能、ESG/可持续 |
| type | #4f46e5 (靛蓝) | 类型标签 | 专访、研究 |
| entity | #64748b (灰色) | 实体标签 | 机构名称、人物名称 |
| series | #7c3aed (紫色) | 系列标签 | 翻书党 |

**关键词筛选规则（全局阈值过滤）：**

| 规则 | 条件 |
|---|---|
| 关键词准入 | 出现次数 >= 6 次 |
| 系列准入 | 出现次数 >= 8 次 |
| 关键词长度上限 | 18 字符 |
| 系列名称长度上限 | 20 字符 |
| 停用词过滤 | KEYWORD_STOPWORDS 集合 |
| 噪声系列过滤 | SERIES_NOISE 集合 |

这种全局阈值设计确保只有在语料库中具有统计显著性的标签才会被保留，避免长尾噪声标签污染标签云。

### 12.3.8 栏目分配

clustering_rules.derive_column_slugs() 根据以下特征将文章分配到一个或多个栏目：

| 输入特征 | 说明 |
|---|---|
| word_count | 字数 |
| article_type | 文章类型 |
| series_or_column | 系列/专栏名称 |
| tag_entries | 标签列表 |
| fdsm_hits | 复旦管院关联关键词命中情况 |

**复旦管院关联关键词（FDSM_RELATION_KEYWORDS）：**

教师、教授、院长、教职、老师、学术院长、特聘教授

normalize_fdsm_relation() 检查 AI 输出中的 fdsm_relation 字段是否包含上述关键词，有则返回匹配的关键词，供栏目分配使用。

栏目定义来自 backend.config.COLUMN_DEFINITIONS，每个栏目包含以下字段：name、slug、description、icon、sort_order、accent_color。

### 12.3.9 浏览量计算

stable_view_count() 基于文档特征生成一个确定性的初始浏览量，不依赖真实用户行为：

```
age_days = max((date.today() - date.fromisoformat(publish_date)).days, 1)
recency_factor = max(0.2, 3.2 - age_days / 1200)  # 时效因子：越新越高
length_factor = min(2.4, word_count / 1800)       # 长度因子：越长越高，上限 2.4
noise = int(sha1(doc_id[:6]).hexdigest(), 16) % 1800  # 确定性随机噪声 0-1799
view_count = int(520 + recency_factor  950 + length_factor  420 + hot_tag_count * 180 + noise)
```

| 因子 | 范围 | 说明 |
|---|---|---|
| 基础值 | 520 | 所有文章的最低浏览量基线 |
| 时效因子 | 0.2 -- 3.2 | 发布日距今越近，值越大 |
| 长度因子 | 0 -- 2.4 | 字数越多，值越大 |
| 热门标签计数 | 0 -- N | topic 和 industry 类标签的数量 |
| 确定性噪声 | 0 -- 1799 | 由 doc_id 的 SHA1 决定，保证跨重建稳定 |

该算法保证同一篇文章在多次数据库重建中获得相同的浏览量值（由 doc_id 决定的确定性计算）。

### 12.3.10 精选文章

feature_score() 综合以下因素为每篇文章评分：

| 因子 | 权重/加分 |
|---|---|
| view_count / 600 | 基础分 |
| word_count / 2000 (上限 3.5) | 长文加分 |
| 文章类型为专访/访谈/对谈/案例报道 | +1.2 |
| 包含 "AI/人工智能" 标签 | +0.8 |
| 包含 "ESG/可持续" 标签 | +0.6 |
| 系列属于 SERIES_NOISE | -1.4 |

排名前 24 的文章标记为 is_featured = 1，并写入 featured_articles 表：

| 排名 | position 值 |
|---|---|
| 第 1 名 | hero |
| 第 2--7 名 | editor-1 至 editor-6 |

### 12.3.11 数据库表结构

create_schema() 创建以下全部数据库表（先 DROP 再 CREATE，每次全量重建）：

### meta --- 元信息表

| 列名 | 类型 | 说明 |
|---|---|---|
| key | TEXT PK | 键 |
| value | TEXT NOT NULL | 值 |

存储 business_article_count 和 updated_at。

### articles --- 文章主表

| 列名 | 类型 | 说明 |
|---|---|---|
| id | INTEGER PK | 主键 |
| doc_id | TEXT UNIQUE | 文档唯一标识（SHA1 前 20 位） |
| slug | TEXT UNIQUE | URL 友好标识 |
| relative_path | TEXT UNIQUE | 相对路径 |
| source | TEXT NOT NULL | 来源，固定 "business" |
| source_mode | TEXT | 来源模式 |
| title | TEXT NOT NULL | 标题 |
| publish_date | TEXT NOT NULL | 发布日期 |
| link | TEXT | 原始 URL |
| content | TEXT NOT NULL | 正文 |
| excerpt | TEXT | 摘要 |
| main_topic | TEXT | 主话题 |
| article_type | TEXT | 文章类型 |
| series_or_column | TEXT | 系列/专栏 |
| primary_org_name | TEXT | 主要关联机构 |
| tag_text | TEXT | 标签文本 |
| people_text | TEXT | 人物文本 |
| org_text | TEXT | 机构文本 |
| search_text | TEXT | 搜索索引文本 |
| word_count | INTEGER | 字数 |
| cover_image_path | TEXT | 封面图路径 |
| view_count | INTEGER | 浏览量 |
| is_featured | INTEGER | 是否精选 |
| created_at | TEXT | 创建时间 |
| updated_at | TEXT | 更新时间 |

### tags --- 标签表

| 列名 | 类型 | 说明 |
|---|---|---|
| id | INTEGER PK AUTO | 主键 |
| name | TEXT NOT NULL | 标签名 |
| slug | TEXT UNIQUE | URL 标识 |
| category | TEXT NOT NULL | 分类 |
| description | TEXT | 描述 |
| color | TEXT | 颜色代码 |
| article_count | INTEGER | 文章计数 |

UNIQUE 约束：(name, category)

### article_tags --- 文章-标签关联表

| 列名 | 类型 | 说明 |
|---|---|---|
| article_id | INTEGER FK | 文章 ID |
| tag_id | INTEGER FK | 标签 ID |
| confidence | REAL | 置信度，默认 1.0 |

### columns --- 栏目表

| 列名 | 类型 | 说明 |
|---|---|---|
| id | INTEGER PK AUTO | 主键 |
| name | TEXT NOT NULL | 栏目名 |
| slug | TEXT UNIQUE | URL 标识 |

| | | |
|---|---|---|
| description | TEXT | 描述 |
| icon | TEXT | 图标 |
| sort_order | INTEGER | 排序 |
| accent_color | TEXT | 强调色 |

### article_columns --- 文章-栏目关联表

| 列名 | 类型 | 说明 |
|---|---|---|
| article_id | INTEGER FK | 文章 ID |
| column_id | INTEGER FK | 栏目 ID |
| is_featured | INTEGER | 是否精选 |
| sort_order | INTEGER | 排序 |

### featured_articles --- 精选文章表

| 列名 | 类型 | 说明 |
|---|---|---|
| id | INTEGER PK AUTO | 主键 |
| article_id | INTEGER FK | 文章 ID |
| position | TEXT NOT NULL | 位置（hero / editor-N） |
| start_date | TEXT | 开始日期 |
| end_date | TEXT | 结束日期 |
| is_active | INTEGER | 是否激活 |

### topics --- 专题表

| 列名 | 类型 | 说明 |
|---|---|---|
| id | INTEGER PK AUTO | 主键 |
| title | TEXT NOT NULL | 专题标题 |
| slug | TEXT UNIQUE | URL 标识 |
| description | TEXT | 描述 |
| cover_image | TEXT | 封面图 |
| cover_article_id | INTEGER | 封面文章 ID |
| type | TEXT NOT NULL | 类型 |
| auto_rules | TEXT | 自动规则 JSON |
| status | TEXT | 状态，默认 draft |
| created_at | TEXT | 创建时间 |
| updated_at | TEXT | 更新时间 |
| view_count | INTEGER | 浏览量 |

### topic_articles --- 专题-文章关联表

| 列名 | 类型 | 说明 |
|---|---|---|
| topic_id | INTEGER FK | 专题 ID |
| article_id | INTEGER FK | 文章 ID |
| sort_order | INTEGER | 排序 |
| editor_note | TEXT | 编辑备注 |

### topic_tags --- 专题-标签关联表

| 列名 | 类型 | 说明 |
|---|---|---|
| topic_id | INTEGER FK | 专题 ID |
| tag_id | INTEGER FK | 标签 ID |

### chat_sessions --- 对话会话表

| 列名 | 类型 | 说明 |
|---|---|---|
| id | TEXT PK | 会话 ID（UUID） |
| title | TEXT NOT NULL | 会话标题 |
| created_at | TEXT NOT NULL | 创建时间 |
| updated_at | TEXT NOT NULL | 更新时间 |
| last_question | TEXT NOT NULL | 最后一个问题 |

#### chat_messages --- 对话消息表

| 列名 | 类型 | 说明 |
|---|---|---|
| id | INTEGER PK AUTO | 主键 |
| session_id | TEXT FK | 会话 ID |
| role | TEXT NOT NULL | 角色（user/assistant） |
| content | TEXT NOT NULL | 消息内容 |
| sources_json | TEXT | 来源文章 JSON |
| follow_ups_json | TEXT | 后续问题 JSON |
| created_at | TEXT NOT NULL | 创建时间 |

**数据库索引：**

| 索引名 | 字段 | 用途 |
|---|---|---|
| idx_articles_date | publish_date DESC | 按日期排序查询 |
| idx_articles_view_count | view_count DESC | 按热度排序查询 |
| idx_tags_category | category, article_count DESC | 标签分类查询 |
| idx_article_columns_column | column_id, article_id | 栏目文章查询 |
| idx_article_tags_tag | tag_id, article_id | 标签文章查询 |
| idx_topic_articles_topic | topic_id, sort_order | 专题文章排序查询 |

数据库使用 WAL 模式 (PRAGMA journal_mode=WAL) 以提高并发读取性能。

### 12.3.12 main() 执行流程

```
def main():
rebuild_database()
print(f"Business knowledge base rebuilt at {SQLITE_DB_PATH}")
```

rebuild_database() 内部流程：

1. build_documents() --- 遍历所有 content.txt，构建 RawDocument 列表
2. 连接 SQLite 数据库，设置 row_factory = sqlite3.Row
3. create_schema() --- 删除旧表、创建新表（全量重建）
4. insert_documents() --- 写入文章、标签、栏目、精选、专题等全部数据
5. 关闭数据库连接
6. refresh_search_cache() --- 刷新搜索缓存

> 注意：数据库构建采用全量重建策略（先 DROP 再 CREATE），每次执行都会清空旧数据。运行时数据（如用户的 chat_sessions、article_reactions）也会被重建。运行时表由 ensure_runtime_tables() 在应用启动时按需创建。

## 12.4 FAISS 索引构建 --- create_vector_db_faiss.py

### 12.4.1 职责

从 SQLite 数据库读取文章数据，进行文本分块和向量化，构建 FAISS 向量索引，用于语义搜索和 RAG 对话。

### 12.4.2 配置

| 配置项 | 值 |
|---|---|
| SQLITE_DB_PATH | 项目根/fudan_knowledge_base.db |
| FAISS_DB_DIR | 项目根/faiss_index_business/ |
| BATCH_SIZE | 50（每批嵌入的文档块数） |
| 嵌入模型 | GEMINI_EMBEDDING_MODEL（来自 backend.config） |
| 距离策略 | DistanceStrategy.COSINE（余弦相似度） |

### 12.4.3 数据加载 --- load_documents()

从 articles 表按 id ASC 排序读取全部记录，提取以下字段：

```
SELECT id, title, publish_date, link, content, article_type, main_topic, tag_text
FROM articles
ORDER BY id ASC
```

将每篇文章格式化为结构化文本：

```
Title: 文章标题
Date: 2025-12-19
Type: 专访
Main Topic: 人工智能
Tags: AI/人工智能 | 金融科技

正文内容全文...
```

每条记录包装为 LangChain Document 对象，metadata 包含：

| 元数据字段 | 来源 | 说明 |
|---|---|---|
| article_id | articles.id | 用于检索结果回溯 |
| title | articles.title | 展示用 |
| publish_date | articles.publish_date | 展示用 |
| link | articles.link | 原始来源链接 |
| source | 固定值 "business" | 来源标识 |

### 12.4.4 文本分块 --- split_documents()

使用 LangChain 的 RecursiveCharacterTextSplitter：

| 参数 | 值 | 说明 |
|---|---|---|
| chunk_size | 800 | 每个文本块最大 800 字符 |
| chunk_overlap | 120 | 相邻块重叠 120 字符，保证上下文连续性 |
| length_function | len | 按 Python 字符数计算长度（中文字符等价于1） |

分块后每个 chunk 继承原文档的 metadata，便于检索结果回溯到原始文章。RecursiveCharacterTextSplitter 会优先在段落、句子、单词边界处分割，尽量保持语义完整性。

### 12.4.5 向量索引构建 --- create_index()

```
embeddings = GoogleGenerativeAIEmbeddings(
  model=GEMINI_EMBEDDING_MODEL,
  google_api_key=google_api_key,
  task_type="retrieval_document",
)
```

**API Key 解析优先级：**

1. GOOGLE_API_KEY 环境变量（优先）
2. GEMINI_API_KEYS 环境变量（逗号分隔列表，取第一个）
3. 以上均无则抛出 ValueError

**构建过程：**

1. 初始化 GoogleGenerativeAIEmbeddings，task_type="retrieval_document" 表示为文档索引生成嵌入向量
2. 按 BATCH_SIZE=50 分批处理全部文本块
3. 第一批使用 FAISS.from_documents() 创建向量存储，指定 COSINE 距离策略
4. 后续批次使用 vectorstore.add_documents() 追加到已有索引
5. 每批最多重试 3 次（覆盖 API 限流等临时错误），重试间隔 2 秒
6. 第 3 次重试仍失败则抛出异常
7. 全部处理完成后调用 vectorstore.save_local(FAISS_DB_DIR) 持久化

使用 tqdm 进度条展示构建进度，单位为 batch。

### 12.4.6 输出文件

```
faiss_index_business/
index.faiss      <- FAISS 二进制索引文件（向量数据）
index.pkl        <- Python pickle 文件（Document 对象和元数据序列化）
```

### 12.4.7 主入口

```
if __name__ == "__main__":
create_index(split_documents(load_documents()))
```

三个函数管道式调用：load_documents() -> split_documents() -> create_index()。

## 12.5 AI 批处理 --- article_ai_batch.py

### 12.5.1 职责

对数据库中的所有文章进行三类 AI 处理：中文摘要生成、Markdown 排版格式化、英文全文翻译。使用多工作进程并行调用 Gemini API，支持断点续跑和状态追踪。

### 12.5.2 目录结构

```
Fudan_Business_Knowledge_Data/article_ai_batch/
manifest.json        <- 待处理文章清单（从数据库生成）
run_meta.json        <- 运行元信息（开始时间、配置等）
logs/                <- 工作进程日志目录
state/               <- 工作进程状态文件目录
  orchestrator.json  <- 编排器状态
  worker-0.json      <- 各分片工作进程状态
  worker-1.json
  ...
```

### 12.5.3 命令行接口

脚本通过 argparse 提供四个子命令：

#### `manifest` --- 构建/重建文章清单

```
python -m backend.scripts.article_ai_batch manifest [--limit N] [--rebuild]
```

| 参数 | 说明 |
|---|---|
| --limit N | 限制清单中的文章数量（0 = 全部） |
| --rebuild | 强制重建清单，忽略已有 manifest.json |

从 SQLite 数据库查询文章列表，生成 manifest.json。

#### `run-worker` --- 运行单个工作分片

```
python -m backend.scripts.article_ai_batch run-worker \
--shard-index 0 --total-shards 10 [--limit N] [--rebuild-manifest] [--worker-name NAME]
```

| 参数 | 说明 |
|---|---|
| --shard-index | 当前分片索引（必需） |
| --total-shards | 总分片数（必需） |
| --limit N | 处理文章数量上限 |
| --rebuild-manifest | 先重建清单再处理 |
| --worker-name | 自定义工作进程名 |

#### `run-all` --- 运行完整多工作进程批处理

```
python -m backend.scripts.article_ai_batch run-all \
[--workers 20] [--limit N] [--poll-seconds 15] [--rebuild-manifest]
```

| 参数 | 默认值 | 说明 |
|---|---|---|
| --workers | 20 | 并发工作进程数 |
| --poll-seconds | 15 | 编排器轮询子进程状态的间隔秒数 |
| --limit | 0（全部） | 处理文章数量上限 |
| --rebuild-manifest | false | 先重建清单 |

编排器通过 subprocess 启动多个子进程，每个子进程运行 run-worker，分片处理清单中的文章。

#### `status` --- 查看批处理状态

```
python -m backend.scripts.article_ai_batch status
```

读取 state/ 目录下的状态文件，汇总展示整体进度和各工作进程状态。

### 12.5.4 模型配置

| 用途 | 模型变量 | 默认值 |
|---|---|---|
| 摘要生成 | FLASH_MODEL | gemini-2.5-flash |
| 排版格式化 | FLASH_MODEL | gemini-2.5-flash |
| 英文翻译 | TRANSLATION_MODEL_CHAIN | [FLASH_MODEL, CHAT_MODEL] 依次尝试 |

翻译模型链：先使用 GEMINI_FLASH_MODEL，如果失败再回退到 GEMINI_CHAT_MODEL。如果两者相同则链中只有一个模型。

## 12.5.5 Prompt 模板

### 摘要 Prompt（SUMMARY_PROMPT）

角色定位为"复旦商业知识编辑部的摘要助手"，要求：

1. 保留原文逻辑，不编造事实
2. 优先用 5--8 个短标题或要点组织内容
3. 语言凝练、专业，避免空话
4. 仅输出 Markdown

输入变量：{title}、{content}

### 排版 Prompt（FORMAT_PROMPT）

角色定位为"复旦商业知识编辑部的排版助手"，要求：

1. 保留全部关键事实、论点、案例与引语
2. 补出标题层级、导语、小标题、列表与引用
3. 一级标题必须是文章标题
4. 采访/对话/演讲/案例文章保留原有说话人和结构
5. 不输出 HTML，只输出 Markdown
6. 参考"复旦商业知识 / AI_writer"公众号长文排版风格

输入变量：{title}、{excerpt}、{main_topic}、{article_type}、{organization}、{tags}、{content}

### 翻译 Prompt（TRANSLATION_PROMPT）

英文翻译，要求：

1. 独立于摘要和排版，直接从中文原文翻译
2. 翻译全文，保留所有有意义的内容
3. 保留段落、列表、引用结构
4. 返回严格 JSON 格式

输出 JSON 结构：

```
 {
 "title": "English title",
 "excerpt": "English deck or short intro",
 "summary": "English Markdown summary",
 "content": "Full translated Markdown body"
}
```

## 12.5.6 翻译字段别名映射

TRANSLATION_FIELD_ALIASES 定义了翻译 JSON 输出中各字段的可接受别名，增强对 AI 输出格式变化的容错性：

| 标准字段 | 接受的别名 |
|---|---|
| title | title, english_title, translated_title |
| excerpt | excerpt, deck, intro, lead, subtitle |
| summary | summary, abstract, overview, brief |
| content | content, body, translation, translated_content, article |

## 12.5.7 错误处理与重试

| 参数 | 值 | 说明 |
|---|---|---|
| MAX_API_ATTEMPTS | 8 | 单个文章单个任务的最大重试次数 |
| CONNECT_TIMEOUT_SECONDS | 30 | HTTP 连接超时 |
| READ_TIMEOUT_SECONDS | 300 | HTTP 读取超时（5 分钟，适应长文翻译） |

**重试策略：**

- 错误消息以 NONRETRYABLE: 前缀开头的为不可重试错误（如输入格式问题），直接跳过
- is_non_retryable_batch_error() 检测该前缀
- 可重试错误（API 限流、网络超时等）在最多 8 次尝试内重试

**状态持久化：**

- 每个工作进程的状态通过 JSON 文件持久化到 state/ 目录
- write_json_atomic() 使用临时文件（.tmp 后缀）+ Path.replace() 保证写入原子性
- 支持断点续跑：重启后读取状态文件，跳过已完成的文章

## 12.5.8 API Key 管理

```
def load_api_keys() -> list[str]:
keys = [item.strip() for item in GEMINI_API_KEYS if item.strip()]
if PRIMARY_GEMINI_KEY and PRIMARY_GEMINI_KEY not in keys:
    keys.insert(0, PRIMARY_GEMINI_KEY)
if not keys:
    raise BatchError("No Gemini API keys are configured.")
return keys
```

- 从 GEMINI_API_KEYS（列表）加载所有可用 Key
- PRIMARY_GEMINI_KEY 如果存在且不在列表中，插入队首优先使用
- 多个 Key 支持轮换，分散 API 配额压力
- 无可用 Key 时抛出 BatchError

### 12.5.9 输出存储

处理结果写入两个数据库表：

| 存储位置 | 键 | 内容 |
|---|---|---|
| article_ai_outputs | article_id | 中文摘要、排版后 Markdown、英文翻译全文、source_hash、模型名 |
| article_translations | (article_id, target_lang, source_hash) | 翻译缓存（title/excerpt/summary/content/model） |

source_hash 用于检测源文章是否发生变化，确保翻译结果与源文章一致。由 build_article_source_hash() 计算。

翻译结果同时通过 upsert_article_translation() 写入 article_translations 表，作为翻译服务的缓存层。

## 12.6 其他批处理脚本概述

### 12.6.1 backend/services/taxonomy_service.py

标签分类服务，提供 build_tag_entries() 函数。核心逻辑：

- 从标题、正文、AI 分析结果中提取关键词
- 通过 normalize_keyword() 标准化关键词（合并同义词、统一格式）
- 过滤停用词 (KEYWORD_STOPWORDS) 和噪声系列 (SERIES_NOISE)
- 根据关键词的全局频次判断是否纳入
- 输出 (name, category, confidence) 三元组列表

### 12.6.2 backend/services/clustering_rules.py

栏目分配规则服务，提供 derive_column_slugs() 函数。根据文章特征（字数、类型、系列、标签、复旦关联）将文章映射到一个或多个栏目 slug。

### 12.6.3 backend/services/topic_engine.py

专题引擎，提供两个核心函数：

- auto_generate_topics(limit) --- 通过 API 端点触发，自动生成专题
- rebuild_topics(connection, limit_auto) --- 在数据库构建过程中调用，基于 TOPIC_AUTO_CLUSTERS 配置自动聚合文章生成专题页面

### 12.6.4 backend/services/tag_engine.py

标签引擎，提供 generate_tags_for_articles(limit, regenerate) 函数。支持批量为文章生成标签，可通过 API 端点 POST /api/tags/batch-generate 触发。

## 12.7 管道维护指南

### 增量更新场景

| 场景 | 操作步骤 |
|---|---|
| 新增文章来源 | 在链接文件中追加新行 -> 运行爬虫 -> 重建数据库 -> 重建 FAISS -> (可选) AI 批处理 |
| 修正已有文章 | 删除对应文章目录 -> 重新爬取 -> 重建数据库 -> 重建 FAISS |
| 仅更新 AI 处理 | 运行 article_ai_batch run-all（不影响数据库和 FAISS） |
| 完整重建 | 按顺序执行阶段 1 -> 2 -> 3 -> 4 |
| 新增数据源 | 创建新的链接文件和 SAVE_ROOT -> 修改 BUSINESS_DATA_DIR 配置 -> 全量重建 |

### 环境变量依赖

| 变量 | 用途 | 必需 |
|---|---|---|

| | | |
|---|---|---|
| GOOGLE_API_KEY | Gemini 嵌入向量生成 | 是（FAISS 构建阶段） |
| GEMINI_API_KEYS | 批处理多 Key 轮换 | 推荐（AI 批处理阶段） |
| PRIMARY_GEMINI_KEY | 优先使用的 Gemini Key | 否 |

## 典型耗时参考

| 阶段 | 500 篇文章估算 |
|---|---|
| 爬虫（5 并发） | 10--30 分钟（取决于网络和目标站点响应速度） |
| 数据库构建 | 1--3 分钟 |
| FAISS 索引（50/批） | 5--15 分钟（取决于 Gemini Embedding API 速率限制） |
| AI 批处理（20 worker） | 30--90 分钟（取决于文章长度和 API 配额） |

## 常见问题排查

| 问题 | 排查方向 |
|---|---|
| FAISS 索引构建失败 | 检查 GOOGLE_API_KEY 是否有效，API 配额是否充足 |
| AI 批处理卡住 | 查看 state/ 目录下各 worker JSON 的状态，检查日志 |
| 数据库重建后文章数不对 | 确认 BUSINESS_DATA_DIR 路径正确，检查是否有 content.txt 缺失 |
| 翻译结果为空 | 检查 TRANSLATION_FIELD_ALIASES 是否覆盖了 AI 输出的字段名 |
| 标签过少 | 调整全局阈值（关键词 >= 6 次、系列 >= 8 次），或检查停用词列表 |

# 13 --- 国际化与翻译

本章完整描述复旦商业知识库的双语支持实现，涵盖前端 i18n 系统的语言检测与切换机制、消息文件的键树结构、后端翻译服务的多层解析策略，以及列表级翻译与锁定文章预览翻译的处理逻辑。

## 13.1 前端 i18n --- LanguageProvider

### 13.1.1 架构概述

前端国际化基于 React Context 实现，核心文件位于 frontend/src/i18n/ 目录：

| 文件 | 职责 |
|---|---|
| LanguageContext.js | 创建 React Context 对象 |
| LanguageProvider.jsx | Context Provider 组件，封装语言状态管理和检测逻辑 |
| messages.js | 中英文消息键值对树 |

### 13.1.2 支持的语言

```
const SUPPORTED_LANGUAGES = new Set(['zh', 'en'])
```

系统支持两种语言：zh（中文，默认）和 en（英文）。任何不在此集合中的语言标识都会被规范化为 zh。

### 13.1.3 语言检测优先级 --- detectInitialLanguage()

页面加载时，LanguageProvider 按以下顺序检测用户语言偏好：

| 优先级 | 检测源 | 说明 | 示例 |
|---|---|---|---|
| 1 (最高) | URL 查询参数 ?lang= | 适用于分享链接和外部跳转 | ?lang=en |
| 2 | localStorage 键 fdsm-language | 用户上次选择的语言偏好 | localStorage.getItem('fdsm-language') |
| 3 (最低) | 浏览器 navigator.language | 浏览器默认语言 | en-US -> en；zh-CN -> zh |
| 回退 | 固定值 'zh' | SSR 环境或所有检测均未命中 | --- |

**详细逻辑：**

```
function detectInitialLanguage() {
if (typeof window === 'undefined') return 'zh'    // SSR 回退

// 1. URL 参数
const searchParams = new URLSearchParams(window.location.search)
const queryLang = normalizeLanguage(searchParams.get('lang'))
if (SUPPORTED_LANGUAGES.has(searchParams.get('lang') || '')) return queryLang

// 2. localStorage
const storedLanguage = normalizeLanguage(window.localStorage.getItem(STORAGE_KEY))
if (window.localStorage.getItem(STORAGE_KEY)) return storedLanguage

// 3. 浏览器语言
const browserLanguage = String(window.navigator.language || '').toLowerCase()
return browserLanguage.startsWith('en') ? 'en' : 'zh'
}
```

注意第 1 步的检测逻辑：只有当 lang 参数的原始值本身是合法语言代码时才使用（而非只看规范化后的结果），这避免了 ?lang=xyz 被错误规范化为 zh 并覆盖 localStorage 的偏好。

### 13.1.4 语言状态副作用

当 language 状态变化时，useEffect 执行以下三个副作用：

```
  useEffect(() => {
  if (typeof window === 'undefined') return

  // 1. 持久化到 localStorage
  window.localStorage.setItem(STORAGE_KEY, language)

  // 2. 更新 HTML lang 属性
  document.documentElement.lang = language === 'en' ? 'en' : 'zh-CN'

  // 3. 同步 URL 参数
  const url = new URL(window.location.href)
  if (language === 'en') {
    url.searchParams.set('lang', 'en')        // 英文模式添加 ?lang=en
  } else {
    url.searchParams.delete('lang')           // 中文模式移除 lang 参数
  }
  window.history.replaceState({}, '', ${url.pathname}${url.search}${url.hash})
}, [language])
```

| 副作用 | 说明 |
|---|---|
| localStorage.setItem | 持久化偏好，下次访问自动恢复 |
| document.documentElement.lang | 设置 HTML 根元素 lang 属性（en 或 zh-CN），影响浏览器字体渲染和可访问性 |
| URL 参数同步 | 英文模式下 URL 带 ?lang=en，中文模式移除参数。使用 replaceState 不产生浏览历史 |

### 13.1.5 Context 提供的值

LanguageProvider 向子组件提供以下 Context 值：

| 属性/方法 | 类型 | 说明 |
|---|---|---|
| language | string | 当前语言代码（'zh' 或 'en'） |
| isEnglish | boolean | 是否为英文模式的快捷判断 |
| setLanguage(lang) | function | 设置指定语言（会经过 normalizeLanguage 校验） |
| toggleLanguage() | function | 在中英文之间切换（en -> zh，zh -> en） |
| t(key) | function | 翻译函数，根据当前语言返回对应文本 |

### 13.1.6 翻译函数 t(key) --- resolveMessage()

t(key) 接受点分隔的键路径（如 'navbar.home'），按以下逻辑解析：

```
  1. 在当前语言的 messages 中查找 key
  └── 找到 string 类型的值 → 返回
  └── 未找到或不是 string →

2. 在中文 messages (messages.zh) 中查找 key（回退到中文）
  └── 找到 string 类型的值 → 返回
  └── 未找到 →

3. 返回 key 本身作为兜底
```

这确保了即使英文翻译缺失，也能显示中文文案而非空白。如果中文也不存在，显示 key 路径（如 'navbar.unknownKey'），便于开发者发现缺失的翻译。

## 13.2 消息文件结构 --- messages.js

### 13.2.1 文件格式

```
  export const messages = {
 zh: {
  appName: '复旦商业知识库',
  appSubtitle: 'Business-Only Knowledge Base',
  pageLoading: '页面加载中...',
  navbar: { ... },
  footer: { ... },
  article: { ... },
  articleCard: { ... },
 },
 en: {
  appName: 'Fudan Business Knowledge Base',
  appSubtitle: 'Business-Only Knowledge Base',
  pageLoading: 'Loading page...',
  navbar: { ... },
  footer: { ... },
  article: { ... },
  articleCard: { ... },
 },
}
```

messages 导出一个对象，顶层键为语言代码（zh、en），每个语言下的键树结构完全对称。

## 13.2.2 键树结构

### 顶层键

| 键 | 说明 |
|---|---|
| appName | 应用名称 |
| appSubtitle | 应用副标题 |
| pageLoading | 页面加载提示 |
| navbar | 导航栏相关文案 |
| footer | 页脚相关文案 |
| article | 文章详情页文案 |
| articleCard | 文章卡片组件文案 |

### `navbar` 子树

| 键 | 中文 | 英文 |
|---|---|---|
| home | 首页 | Home |
| membership | 会员 | Membership |
| audio | 音频 | Audio |
| video | 视频 | Video |
| insights | 洞察 | Insights |
| industry | 行业 | Industry |
| research | 研究 | Research |
| deansView | 院长说 | Dean's View |
| topics | 专题 | Topics |
| organizations | 机构 | Organizations |
| following | 关注 | Following |
| analytics | 分析 | Analytics |
| timeMachine | 时光机 | Time Machine |
| editorial | 编辑台 | Editorial |
| mediaStudio | 媒体工作台 | Media Studio |
| commercial | 预约演示 | Book Demo |
| admin | 管理台 | Admin |
| library | 我的资料库 | My Library |
| searchPlaceholder | 搜索商业文章、主题、人物 | Search articles, topics, people |
| signIn | 登录 | Sign in |
| emailSignIn | 邮箱登录 | Email sign in |
| signOut | 退出 | Sign out |
| authPending | 登录暂不可用 | Sign-in unavailable |
| languageZh | 中 | 中 |
| languageEn | EN | EN |

### `footer` 子树

| 键 | 中文 | 英文 |
|---|---|---|
| description | 复旦商业知识库汇集文章... | Fudan Business Knowledge Base brings articles... |
| commercial | 预约演示 / 商务咨询 | Book Demo / Business Inquiry |

## article 子树

文章详情页包含大量键，按功能可分为以下几组：

### 基础信息：

| 键 | 中文 | 英文 |
|---|---|---|
| loading | 文章加载中... | Loading article... |
| source | 查看来源 | View source |
| organization | 机构 | Organization |

### 读者数据：

| 键 | 中文 | 英文 |
|---|---|---|
| readerPulse | 读者数据 | Reader Pulse |
| realViews | 次浏览 | views |
| likes | 点赞 | likes |
| bookmarks | 收藏 | bookmarks |

### 互动操作：

| 键 | 中文 | 英文 |
|---|---|---|
| interaction | 互动 | Actions |
| interactionLoggedIn | 已登录，你的点赞和收藏会同步到账号。 | You are signed in... |
| interactionGuest | 访客可阅读公开内容，登录后可点赞和收藏。 | Guests can read public content... |
| login | 登录 | Sign in |
| like | 点赞 | Like |
| liked | 已点赞 | Liked |
| bookmark | 收藏 | Bookmark |
| bookmarked | 已收藏 | Bookmarked |
| loginRequired | 登录后才可以点赞或收藏。 | Sign in to like or bookmark... |
| authUnavailable | 登录暂不可用。 | Sign-in is unavailable... |
| reactionFailed | 操作未完成，请稍后重试。 | Action failed... |

### 会员墙（Membership Wall）：

| 键 | 中文 | 英文 |
|---|---|---|
| membershipWall | 会员内容 | Member Access |
| previewTitle | 当前仅开放文章预览 | Only the article preview is open... |
| currentIdentity | 当前身份 | Current identity |
| requiredAccess | 所需权限 | Required access |
| currentState | 当前状态 | Current state |
| previewState | 预览中 | Preview only |
| previewStateHint | 你现在看到的是可公开展示的摘要和正文预览。 | You are seeing the visible summary... |

### 内容区域：

| 键 | 中文 | 英文 |
|---|---|---|
| aiSummary | AI 摘要 | AI Summary |
| fullText | 正文 | Article |
| previewMode | 预览模式 | Preview mode |
| previewNote | 后续正文需要更高访问权限... | The rest of this article requires... |

### 推荐与入口：

| 键 | 中文 | 英文 |
|---|---|---|
| relatedTopics | 相关专题 | Related Topics |
| relatedRecommendations | 相关推荐 | Related Reads |
| membershipEntry | 会员入口 | Membership |
| membershipEntryTitle | 继续阅读，或升级获取更多访问 | Continue reading or unlock deeper access |
| membershipEntryBody | 会员页会展示当前身份... | The membership page shows your current access... |
| membershipBenefits | 查看会员权益 | View membership benefits |

**翻译相关：**

| 键 | 中文 | 英文 |
|---|---|---|
| translate | 查看英文版 | View English version |
| translated | 英文版 | English version |
| original | 查看原文 | View original |
| translationReady | 英文版已就绪 | English version ready |
| translationBadge | 英文版 | English version |
| translationHint | 在英文模式下，页面会优先展示... | In English mode, the page automatically... |
| translating | 正在生成英文版... | Preparing the English version... |
| translatingSubtext | 将保留当前可见摘要和正文结构。 | The current summary and body structure... |
| translationError | 英文版暂不可用，请稍后重试。 | The English version is unavailable... |
| previewTranslationNote | 当前英文版仅覆盖你现在可见的预览内 | This English version currently covers only... |
| translatedScopePreview | 预览英文版 | Preview translation |
| translatedScopeFull | 全文英文版 | Full translation |

**articleCard 子树**

| 键 | 中文 | 英文 |
|---|---|---|
| public | 公开 | Public |
| member | 会员 | Member |
| paid | 付费 | Paid |
| views | 次浏览 | views |
| likes | 点赞 | likes |
| bookmarks | 收藏 | bookmarks |
| read | 阅读文章 | Read article |

### 13.2.3 添加新翻译键的规范

1. 在 messages.zh 和 messages.en 中同时添加对应键和值
2. 键名使用 camelCase
3. 嵌套层级不超过 2 级（如 article.translating）
4. 组件中通过 t('article.translating') 引用

## 13.3 后端翻译服务 --- translation_service.py

### 13.3.1 三层翻译策略

后端翻译服务 get_article_translation() 采用三层回退策略解析英文翻译，确保在各种场景下都能返回可用结果：

```
  请求 GET /api/article/{id}/translation?lang=en
  │
  ▼
第一层：article_ai_outputs（预计算翻译）
  │
  │ 检查条件：translation_ready == True
  │         AND source_hash_matches_current == True
  │
  ├── 命中 → 返回预计算翻译（cached: True, model: "precomputed"）
  │
  ▼
第二层：article_translations（翻译缓存表）
  │
  │ 查询：WHERE article_id = ? AND target_lang = ? AND source_hash = ?
  │
  ├── 命中 → 返回缓存翻译（cached: True, model: 缓存记录的模型名）
  │
  ▼
第三层-a：英文原文直通
  │
  │ 检查条件：_is_english_source_article() == True
  │         （源文章前4000字符不包含CJK字符）
  │
  ├── 命中 → 原文直通，model = "source-pass-through"
  │       同时写入 article_translations 缓存
... (共 39 行，已截断显示)
```

### 13.3.2 第一层 --- 预计算翻译（article_ai_outputs）

由 article_ai_batch.py 批处理脚本预先生成并存储。通过 get_article_ai_output_detail(article_id) 查询。

**命中条件（两者同时满足）：**

- translation_ready == True --- 翻译已完成
- source_hash_matches_current == True --- 源文章未发生变化（source_hash 一致）

**返回字段来源：**

| 翻译字段 | AI 输出字段 | 说明 |
|---|---|---|
| title | translation_title_en | 英文标题 |
| excerpt | translation_excerpt_en | 英文摘要句 |
| summary | translation_summary_en | 英文摘要（经 normalize_summary_display_markdown 处理） |
| content | formatted_markdown_en 或 translation_content_en | 英文正文（优先取排版后版本） |
| summary_html | summary_html_en | 摘要 HTML 预览 |
| html_web / html_wechat | html_wechat_en | Web/微信 HTML 版本 |

如果文章处于锁定状态（access.locked == True），内容会经过 preview_markdown() 截断处理。

### 13.3.3 第二层 --- 翻译缓存（article_translations）

数据库查询：

```
  SELECT title, excerpt, summary, content, model, created_at, updated_at
FROM article_translations
WHERE article_id = ? AND target_lang = ? AND source_hash = ?
```

**缓存键组成：**

- article_id --- 文章 ID
- target_lang --- 目标语言（当前仅 "en"）
- source_hash --- 源文章内容哈希

source_hash 由 build_article_source_hash(article) 计算。当源文章内容发生变化时，source_hash 会改变，旧缓存自动失效，触发重新翻译。

### 13.3.4 第三层-a --- 英文原文直通

_is_english_source_article(article) 检测源文章是否已经是英文：

```
    sample = " ".join(
    str(part or "").strip()
    for part in (article.get("title"), article.get("main_topic"),
                 article.get("excerpt"), article.get("content"))
    if str(part or "").strip()
)
return not contains_cjk(sample[:4000])
```

取文章标题、主话题、摘要、正文拼接后的前 4000 字符，调用 contains_cjk() 检测是否包含 CJK 字符。如果不包含，视为英文原文。

**直通翻译的处理（`_build_passthrough_translation`）：**

| 字段 | 来源 |
|---|---|
| title | 原文标题（直接使用） |
| excerpt | main_topic 或 excerpt |
| content | 原文内容经 normalize_article_display_markdown() 处理 |
| summary | 从 excerpt 或 preview_markdown() 生成 |
| model | "source-pass-through" |

结果写入 article_translations 缓存，后续请求命中第二层。

### 13.3.5 第三层-b --- Gemini Flash 实时翻译

当以上层均未命中时，调用 ai_service.translate_article_to_english() 进行实时翻译：

```
    translated = translate_article_to_english(
    article["title"],
    article.get("main_topic") or article.get("excerpt") or "",
    article.get("content") or "",
)
```

翻译完成后通过 upsert_article_translation() 写入 article_translations 缓存，下次请求直接命中第二层。

前提条件： is_ai_enabled() 返回 True（Gemini API 已配置）。否则抛出 RuntimeError。

### 13.3.6 翻译响应结构

所有层返回统一的响应结构：

```
    {
    "article_id": int,          # 文章 ID
    "language": "en",           # 目标语言
    "source_hash": str,         # 源文章哈希
    "title": str,               # 英文标题
    "excerpt": str,             # 英文摘要句
    "summary": str,             # 英文 Markdown 摘要
    "summary_html": str | None,    # 摘要 HTML 预览
    "content": str,             # 英文正文
    "html_web": str | None,       # Web HTML 版本
    "html_wechat": str | None,     # 微信 HTML 版本
    "model": str,               # 翻译模型名
    "cached": bool,             # 是否来自缓存
    "content_scope": str,         # "full" 或 "preview"
    "access_locked": bool,        # 文章是否被锁定
    "updated_at": str,          # 翻译更新时间
}
```

### 13.3.7 HTML 渲染解析

`_resolve_translation_html()`

检查已有 HTML 是否为有效的复旦微信预览 HTML（通过 is_fudan_wechat_preview_html() 检测）。如果是，直接复用；否则返回 None。

`_resolve_summary_html()`

1. 检查存储的 HTML 是否为有效摘要预览 HTML（通过 is_summary_preview_html() 检测）
2. 如果有效，直接使用
3. 否则调用 render_summary_preview_html(summary, language) 生成新的摘要预览 HTML

## 13.4 翻译内容格式

### 13.4.1 四个翻译字段

翻译结果包含四个核心内容字段：

| 字段 | 说明 | 典型长度 |
|---|---|---|
| title | 英文标题 | 10--100 字符 |
| excerpt | 英文短摘要/导语 | 50--200 字符 |
| summary | 英文 Markdown 摘要 | 200--800 字符 |
| content | 英文全文（Markdown） | 1000--10000+ 字符 |

### 13.4.2 Markdown 后处理

翻译内容在返回前经过两个后处理函数：

| 函数 | 适用字段 | 功能 |
|---|---|---|
| normalize_article_display_markdown(content, language) | content | 规范化文章展示 Markdown |
| normalize_summary_display_markdown(summary, language) | summary | 规范化摘要展示 Markdown |

两个函数均接受 language 参数以进行语言相关的格式调整。

### 13.4.3 Source Hash 机制

source_hash 是翻译缓存失效的关键机制：

```
  文章源数据 → build_article_source_hash() → SHA 哈希
  │
  ▼
翻译缓存查询时：
  WHERE source_hash = 当前哈希
  └── 匹配 → 缓存有效
  └── 不匹配 → 缓存失效，重新翻译
```

当文章内容被编辑后，source_hash 变化，旧翻译自动作废，下次请求时触发重新翻译。

## 13.5 锁定文章预览翻译

### 13.5.1 什么是锁定文章

当用户的会员等级不足以阅读完整文章时，文章处于"锁定"状态（access.locked == True）。此时用户只能看到文章的预览部分。

### 13.5.2 预览翻译处理流程

```
  锁定文章翻译请求
  │
  ▼
正常走三层翻译策略获取完整翻译
  │
  ▼
content = preview_markdown(full_content)  ← 截断正文
  │
  ▼
返回响应：
  content_scope: "preview"
  access_locked: true
```

**关键行为：**

| 字段 | 锁定文章 | 完整文章 |
|---|---|---|
| content | preview_markdown(full_content) 截断后的预览 | 完整正文 |
| content_scope | "preview" | "full" |
| access_locked | true | false |
| summary | 完整摘要（不截断） | 完整摘要 |
| summary_html | 完整摘要 HTML | 完整摘要 HTML |

即使文章被锁定，摘要（summary）始终完整返回，仅正文（content）被截断。

### 13.5.3 预计算翻译的锁定处理

第一层（预计算翻译）在锁定场景下的特殊处理：

```
  full_content = str(
  precomputed.get("formatted_markdown_en") or
  precomputed.get("translation_content_en") or ""
)
content = preview_markdown(full_content) if article.get("access", {}).get("locked") else full_content
```

优先使用排版后的英文 Markdown（formatted_markdown_en），其次使用翻译原文（translation_content_en）。

### 13.5.4 直通翻译的锁定处理

英文原文直通也尊重锁定状态：

```
    access_locked = bool(article.get("access", {}).get("locked"))
html = _resolve_translation_html(
    ...,
    stored_html=None if access_locked else article.get("html_wechat"),
)
```

锁定文章不返回微信 HTML 版本，避免通过 HTML 泄露完整内容。

### 13.5.5 前端翻译状态提示

前端根据 content_scope 和 access_locked 展示不同的翻译状态提示：

| 条件 | 显示文案（中文） | 显示文案（英文） |
|---|---|---|
| 锁定 + 预览翻译 | 当前英文版仅覆盖你现在可见的预览内容。 | This English version currently covers only the preview content visible to you. |
| 完整翻译 | 全文英文版 | Full translation |
| 预览翻译 | 预览英文版 | Preview translation |

## 13.6 列表翻译 --- ?language=en 参数

### 13.6.1 列表接口的语言参数

多个列表接口支持 language 查询参数以返回英文版内容：

| 接口 | 参数 | 默认值 |
|---|---|---|
| GET /api/home/feed | `language=zh\ | en` zh |
| GET /api/articles/latest | `language=zh\ | en` zh |
| GET /api/time_machine | `language=zh\ | en` zh |
| GET /api/billing/plans | `language=zh\ | en` zh |
| GET /api/billing/me | `language=zh\ | en` zh |
| POST /api/search | request.language | zh |
| POST /api/chat | request.language | 自动检测 |

### 13.6.2 列表翻译与详情翻译的区别

| 维度 | 列表翻译 | 详情翻译 |
|---|---|---|
| 触发方 | ?language=en 查询参数 | GET /api/article/{id}/translation?lang=en |
| 翻译范 | 卡片标题、摘要等轻量字段 | 完整正文、摘要、HTML |
| 实时翻 | 否（仅返回已有翻译或中文原文） | 是（可触发 Gemini 实时翻译） |
| 缓存机制 | 由 catalog_service 在列表查询时处理 | 三层翻译策略 |

### 13.6.3 对话系统的语言处理

对话接口 (POST /api/chat) 的语言处理由 _normalize_language() 决定：

```
def _normalize_language(requested_language, text=""):
normalized = "en" if str(requested_language or "").strip().lower() == "en" else "zh"
if contains_cjk(text):
    return "zh"
return normalized
```

优先级：

1. 如果用户消息包含 CJK 字符 -> 强制中文
2. 否则使用 request.language 指定的语言
3. 默认中文

这确保了即使界面语言设为英文，当用户用中文提问时，AI 也以中文回复。

## 13.7 翻译系统性能考量

### 13.7.1 缓存层次

| 层次 | 位置 | 写入时机 | 失效条件 |
|---|---|---|---|
| L1 | article_ai_outputs | 批处理脚本写入 | source_hash 变化 |
| L2 | article_translations | 实时翻译后写入 | source_hash 变化 |
| L3 | 无缓存 | --- | 每次触发 Gemini API |

### 13.7.2 翻译成本优化

1. 批处理预计算：通过 article_ai_batch.py 在离线状态下批量翻译所有文章，避免运行时 API 调用
2. Source Hash 精确失效：只有源文章内容变化时才重新翻译，避免无谓的重复翻译
3. 英文直通：自动检测英文原文，跳过翻译
4. 模型链回退：Flash 模型失败时回退到 Chat 模型，提高翻译成功率

### 13.7.3 翻译质量保障

1. Prompt 明确要求"独立于摘要和排版，直接从中文原文翻译"，避免二次加工导致的信息损失
2. TRANSLATION_FIELD_ALIASES 容错机制应对 AI 输出格式变化
3. Markdown 后处理（normalize_article_display_markdown / normalize_summary_display_markdown）确保输出格式一致
4. 预览截断只作用于 content 字段，summary 始终完整

# 14 --- API 接口文档

本文档列出复旦商业知识库后端全部 REST API 端点，基于 FastAPI 实现。接口按功能模块分组，每个端点包含 HTTP 方法、完整路径、功能描述、请求参数、响应格式和权限要求。

**基础信息：**

| 项目 | 值 |
|---|---|
| 框架 | FastAPI |
| API 前缀 | /api（发布接口除外） |
| 认证方式 | Authorization Header（Bearer Token）或调试头 |
| 响应格式 | JSON |

**通用认证头：**

| Header | 说明 |
|---|---|
| Authorization | Supabase JWT Token（Bearer <token>） |
| X-Debug-User-Id | 调试模式用户 ID |
| X-Debug-User-Email | 调试模式用户邮箱 |
| X-Visitor-Id | 访客标识（用于匿名浏览统计） |

**权限等级说明：**

| 标记 | 含义 |
|---|---|
| 公开 | 无需认证即可访问 |
| 登录 | 需要有效的认证 Token |
| 付费会员 | 需要登录且拥有付费会员身份 |
| 管理员 | 需要登录且拥有管理员权限（require_admin_profile） |

## 14.1 首页模块（home）

路由文件：backend/routers/home.py
路由前缀：/api

### GET /api/home/feed

| 项目 | 说明 |
|---|---|
| 功能 | 获取首页信息流数据 |
| 权限 | 公开 |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| language | query | string | "zh" | 语言代码（zh / en） |

响应格式：HomeFeedResponse

## 14.2 文章模块（articles）

路由文件：backend/routers/articles.py
路由前缀：/api

### GET /api/articles/latest

| 项目 | 说明 |
|---|---|
| 功能 | 获取最新文章列表 |
| 权限 | 公开 |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| limit | query | int | 12 | 返回数量上限 |
| offset | query | int | 0 | 偏移量 |

| | | | | |
|---|---|---|---|---|
| language | query | string | "zh" | 语言代码 |

响应格式： list[ArticleCard]

排序方式：publish_date DESC, id DESC

## GET /api/articles/trending

| 项目 | 说明 |
|---|---|
| 功能 | 获取热门文章列表 |
| 权限 | 公开 |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| limit | query | int | 12 | 返回数量上限 |
| offset | query | int | 0 | 偏移量 |

响应格式： list[ArticleCard]

排序方式：综合评分（view_count + 4 likes + 6 bookmarks）降序，再按 publish_date 降序。

## GET /api/article/{article_id}

| 项目 | 说明 |
|---|---|
| 功能 | 获取文章详情 |
| 权限 | 公开（内容可能受会员等级限制） |

**路径参数：**

| 参数 | 类型 | 说明 |
|---|---|---|
| article_id | int | 文章 ID |

**请求头：**

| Header | 说明 |
|---|---|
| X-Visitor-Id | 访客标识，用于匿名浏览计数 |
| Authorization | 可选，用于获取完整内容和会员权限判断 |

响应格式： ArticleDetail

副作用： 自动记录一次文章浏览（record_article_view），来源标记为 "article-detail"。

## GET /api/article/{article_id}/cover

| 项目 | 说明 |
|---|---|
| 功能 | 获取文章封面图片 |
| 权限 | 公开 |

**路径参数：**

| 参数 | 类型 | 说明 |
|---|---|---|
| article_id | int | 文章 ID |

响应格式： FileResponse（图片二进制流）

## GET /api/article/{article_id}/translation

| 项目 | 说明 |
|---|---|
| 功能 | 获取文章英文翻译 |
| 权限 | 公开（翻译内容受会员等级限制） |

**路径参数：**

| 参数 | 类型 | 说明 |
|---|---|---|
| article_id | int | 文章 ID |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| lang | query | string | "en" | 目标语言（当前仅支持 en） |

响应格式： ArticleTranslationResponse

**错误码：**

| 状态码 | 说明 |
| --- | --- |
| 400 | 不支持的目标语言 |
| 503 | 翻译服务暂不可用 |

## GET /api/summarize_article/{article_id}

| 项目 | 说明 |
| --- | --- |
| 功能 | 获取文章 AI 摘要 |
| 权限 | 公开 |

**路径参数：**

| 参数 | 类型 | 说明 |
| --- | --- | --- |
| article_id | int | 文章 ID |

**响应格式：**

```
{
 "id": 123,
 "title": "文章标题",
 "source": "business",
 "publish_date": "2025-12-19",
 "link": "https://...",
 "summary": "Markdown 摘要",
 "summary_html": "<div>摘要 HTML 预览</div>"
}
```

摘要来源优先级：预计算摘要（article_ai_outputs.summary_zh） > 提取式摘要（ai_service.build_extractive_summary）。

## GET /api/article/{article_id}/engagement

| 项目 | 说明 |
| --- | --- |
| 功能 | 获取文章互动数据（浏览量、点赞、收藏） |
| 权限 | 公开 |

**路径参数：**

| 参数 | 类型 | 说明 |
| --- | --- | --- |
| article_id | int | 文章 ID |

响应格式： ArticleEngagement

## POST /api/article/{article_id}/reaction

| 项目 | 说明 |
| --- | --- |
| 功能 | 点赞或收藏文章 |
| 权限 | 登录 |

**路径参数：**

| 参数 | 类型 | 说明 |
| --- | --- | --- |
| article_id | int | 文章 ID |

请求体： ReactionRequest

```
{
 "reaction_type": "like",
 "active": true
}
```

| 字段 | 类型 | 说明 |
| --- | --- | --- |
| reaction_type | string | 反应类型（like / bookmark） |
| active | bool | 是否激活（true = 添加，false = 取消） |

响应格式： ArticleEngagement

错误码： 401 --- 未登录

## 14.3 搜索模块（search）

路由文件：backend/routers/search.py
路由前缀：/api

### POST /api/search

| 项目 | 说明 |
|---|---|
| 功能 | 搜索文章 |
| 权限 | 精确搜索（exact）公开；智能搜索（smart）需付费会员 |

请求体： SearchRequest

```
 {
 "query": "搜索关键词",
 "mode": "smart",
 "language": "zh",
 "filters": {},
 "sort": null,
 "page": 1,
 "page_size": 12
}
```

| 字段 | 类型 | 默认值 | 说明 |
|---|---|---|---|
| query | string | --- | 搜索关键词（必需） |
| mode | string | "smart" | 搜索模式：smart（语义搜索）/ exact（精确搜索） |
| language | string | "zh" | 语言 |
| filters | object | {} | 过滤条件 |
| sort | string | null | 排序方式 |
| page | int | 1 | 页码 |
| page_size | int | 12 | 每页数量 |

响应格式： SearchResponse

**错误码：**

| 状态码 | 条件 | 说明 |
|---|---|---|
| 401 | mode != "exact" 且未登录 | 智能搜索需要登录 |
| 403 | mode != "exact" 且非付费会员 | 智能搜索需要付费会员 |

### GET /api/suggest

| 项目 | 说明 |
|---|---|
| 功能 | 搜索自动补全建议 |
| 权限 | 公开 |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| query | query | string | "" | 搜索关键词前缀 |
| language | query | string | "zh" | 语言 |

响应格式： 补全建议列表

## 14.4 对话模块（chat）

路由文件：backend/routers/chat.py
路由前缀：/api

### POST /api/chat

| 项目 | 说明 |
|---|---|
| 功能 | AI 对话（知识库问答） |
| 权限 | 公开 |

请求体： ChatRequest

```
{
 "messages": [
  {"role": "user", "content": "什么是ESG？"}
 ],
 "session_id": "optional-session-id",
 "language": "zh"
}
```

| 字段 | 类型 | 说明 |
|---|---|---|
| messages | array | 对话消息列表（role + content） |
| session_id | string | 会话 ID（可选，为空则自动生成） |
| language | string | 语言偏好（可选，自动检测） |

**支持的快捷指令：**

| 指令 | 中文别名 | 功能 | 参数 |
|---|---|---|---|
| /brief [主题] | /简报 [主题] | 生成战略简报 | 主题关键词（必需） |
| /summarize [主题] | /总结 [主题]、/摘要 [主题] | 主题摘要总结 | 主题关键词（必需） |
| /compare [A] vs [B] | /比较 [A] 与 [B]、/对比 [A] 与 [B] | 对比分析 | 两个实体（与/和/vs 分隔） |
| /timeline [主题] | /时间线 [主题]、/脉络 [主题] | 时间线分析 | 主题关键词（必需） |
| /today [日期] | /今日一读、/一读、/今日、/今天、/今日简报 | 今日推荐阅读 | 可选日期 YYYY-MM-DD |
| /recommend [主题] | /继续阅读、/延伸阅读、/推荐 | 延伸阅读推荐 | 可选主题关键词 |

响应格式：ChatResponse

```
{
 "session_id": "uuid-hex",
 "answer": "AI 回答内容（Markdown）",
 "sources": [{"id": 1, "title": "...", "publish_date": "..."}],
 "follow_up_questions": ["/简报 AI", "/今日一读", "/推荐"],
 "confidence": 0.85
}
```

### GET /api/chat/sessions

| 项目 | 说明 |
|---|---|
| 功能 | 获取对话会话列表 |
| 权限 | 公开 |

响应格式：list[ChatSessionSummary]

### GET /api/chat/session/{session_id}

| 项目 | 说明 |
|---|---|
| 功能 | 获取对话会话详情（含完整消息历史） |
| 权限 | 公开 |

**路径参数：**

| 参数 | 类型 | 说明 |
|---|---|---|
| session_id | string | 会话 ID |

响应格式：ChatSessionDetail

### DELETE /api/chat/session/{session_id}

| 项目 | 说明 |
|---|---|
| 功能 | 删除对话会话 |
| 权限 | 公开 |

**路径参数：**

| 参数 | 类型 | 说明 |
|---|---|---|
| session_id | string | 会话 ID |

响应格式：ChatSessionDeleteResponse

## 14.5 栏目模块（columns）

路由文件：backend/routers/columns.py
路由前缀：/api

### GET /api/columns

| 项目 | 说明 |
|---|---|
| 功能 | 获取所有栏目列表 |
| 权限 | 公开 |

响应格式： list[ColumnSummary]

### GET /api/columns/{slug}/articles

| 项目 | 说明 |
|---|---|
| 功能 | 获取指定栏目下的文章列表 |
| 权限 | 公开 |

**路径参数：**

| 参数 | 类型 | 说明 |
|---|---|---|
| slug | string | 栏目 slug 标识 |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| page | query | int | 1 | 页码 |
| page_size | query | int | 12 | 每页数量 |

响应格式： 分页文章列表

## 14.6 专题模块（topics）

路由文件：backend/routers/topics.py
路由前缀：/api

### GET /api/topics

| 项目 | 说明 |
|---|---|
| 功能 | 获取所有专题列表 |
| 权限 | 付费会员 |

响应格式： list[TopicSummary]

### GET /api/topics/{slug}

| 项目 | 说明 |
|---|---|
| 功能 | 获取专题详情及文章列表 |
| 权限 | 付费会员 |

**路径参数：**

| 参数 | 类型 | 说明 |
|---|---|---|
| slug | string | 专题 slug 标识 |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| page | query | int | 1 | 页码 |
| page_size | query | int | 12 | 每页数量 |

响应格式： TopicDetail

副作用： 第一页请求时自动增加专题浏览计数（increment_topic_view_count_by_slug）。

### GET /api/topics/{topic_id}/timeline

| 项目 | 说明 |
|---|---|
| 功能 | 获取专题时间线 |

| | |
|---|---|
| 权限 | 付费会员 |

**路径参数：**

| 参数 | 类型 | 说明 |
|---|---|---|
| topic_id | int | 专题 ID |

响应格式：时间线数据

### GET /api/topics/{topic_id}/insights

| 项目 | 说明 |
|---|---|
| 功能 | 获取专题洞察数据 |
| 权限 | 付费会员 |

**路径参数：**

| 参数 | 类型 | 说明 |
|---|---|---|
| topic_id | int | 专题 ID |

响应格式：洞察数据

### POST /api/topics/auto-generate

| 项目 | 说明 |
|---|---|
| 功能 | 自动生成专题 |
| 权限 | 公开 |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| limit | query | int | 6 | 生成专题数量 |

响应格式：生成结果

## 14.7 标签模块（tags）

路由文件：backend/routers/tags.py
路由前缀：/api

### GET /api/tags

| 项目 | 说明 |
|---|---|
| 功能 | 获取所有标签（含分组信息） |
| 权限 | 公开 |

响应格式：TagsResponse

### GET /api/tags/cloud

| 项目 | 说明 |
|---|---|
| 功能 | 获取标签云数据（热门标签列表） |
| 权限 | 公开 |

响应格式：热门标签数组（TagsResponse.hot 的子集）

### GET /api/tags/{tag_slug}/articles

| 项目 | 说明 |
|---|---|
| 功能 | 获取指定标签下的文章列表 |
| 权限 | 公开 |

**路径参数：**

| 参数 | 类型 | 说明 |
|---|---|---|
| tag_slug | string | 标签 slug 标识 |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| page | query | int | 1 | 页码 |
| page_size | query | int | 12 | 每页数量 |

响应格式：分页文章列表

### POST /api/tags/batch-generate

| 项目 | 说明 |
|---|---|
| 功能 | 批量为文章生成标签 |
| 权限 | 公开 |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| limit | query | int | 50 | 处理文章数量上限 |
| regenerate | query | bool | false | 是否重新生成已有标签 |

响应格式：生成结果

## 14.8 组织/机构模块（organizations）

路由文件：backend/routers/organizations.py
路由前缀：/api

### GET /api/organizations

| 项目 | 说明 |
|---|---|
| 功能 | 获取机构列表 |
| 权限 | 公开 |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| limit | query | int | 60 | 返回数量上限 |

响应格式：list[OrganizationSummary]

### GET /api/organizations/{slug}

| 项目 | 说明 |
|---|---|
| 功能 | 获取机构详情及相关文章 |
| 权限 | 公开 |

**路径参数：**

| 参数 | 类型 | 说明 |
|---|---|---|
| slug | string | 机构 slug 标识 |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| page | query | int | 1 | 页码 |
| page_size | query | int | 12 | 每页数量 |

响应格式：OrganizationDetail

## 14.9 认证模块（auth）

路由文件：backend/routers/auth.py
路由前缀：/api/auth

### GET /api/auth/status

| 项目 | 说明 |
|---|---|

| | |
|---|---|
| 功能 | 获取当前用户认证状态 |
| 权限 | 公开（根据 Token 返回不同结果） |

响应格式： AuthStatusResponse

返回内容包括：认证状态、用户信息、会员信息、业务档案等。

### POST /api/auth/login

| 项目 | 说明 |
|---|---|
| 功能 | 邮箱密码登录（预览账号） |
| 权限 | 公开 |

请求体： AuthPasswordLoginRequest

```
{
 "email": "user@example.com",
 "password": "password123"
}
```

响应格式： AuthLoginResponse

```
{
 "authenticated": true,
 "auth_mode": "password",
 "user": {"id": "user-id", "email": "user@example.com"},
 "membership": {...},
 "business_profile": {...},
 "role_home_path": "/editorial"
}
```

错误码： 401 --- 邮箱或密码错误

## 14.10 会员模块（membership）

路由文件：backend/routers/membership.py
路由前缀：/api/membership

### GET /api/membership/me

| 项目 | 说明 |
|---|---|
| 功能 | 获取当前用户的会员信息 |
| 权限 | 公开（未登录返回默认 profile） |

响应格式： MembershipProfile

## 14.11 计费模块（billing）

路由文件：backend/routers/billing.py
路由前缀：/api/billing

### GET /api/billing/plans

| 项目 | 说明 |
|---|---|
| 功能 | 获取可用的付费计划列表 |
| 权限 | 公开 |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| language | query | string | "zh" | 语言代码 |

响应格式： BillingPlansResponse

### GET /api/billing/me

| 项目 | 说明 |
|---|---|
| 功能 | 获取当前用户的计费信息 |
| 权限 | 公开（未登录返回默认） |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| language | query | string | "zh" | 语言代码 |

响应格式： BillingMeResponse

### POST /api/billing/checkout-intent

| 项目 | 说明 |
|---|---|
| 功能 | 创建结账意向（购买/升级会员） |
| 权限 | 公开（服务端会识别用户） |

请求体： BillingCheckoutIntentRequest

```
 {
 "plan_code": "premium-yearly",
 "success_url": "https://example.com/success",
 "cancel_url": "https://example.com/cancel"
}
```

| 字段 | 类型 | 说明 |
|---|---|---|
| plan_code | string | 计划代码 |
| success_url | string | 支付成功跳转 URL |
| cancel_url | string | 支付取消跳转 URL |

响应格式： BillingCheckoutIntentResponse

## 14.12 个人中心模块（me）

路由文件：backend/routers/me.py
路由前缀：/api/me

### GET /api/me/dashboard

| 项目 | 说明 |
|---|---|
| 功能 | 获取用户仪表盘数据 |
| 权限 | 公开（未登录返回默认） |

响应格式： UserDashboardResponse

### GET /api/me/library

| 项目 | 说明 |
|---|---|
| 功能 | 获取用户资料库（收藏、点赞的文章等） |
| 权限 | 登录 |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| limit | query | int | 12 | 返回数量上限 |

响应格式： UserLibraryResponse

错误码： 401 --- 未登录

## 14.13 关注模块（follows）

路由文件：backend/routers/follows.py
路由前缀：/api/follows

### GET /api/follows

| 项目 | 说明 |
|---|---|
| 功能 | 获取当前用户的关注列表 |
| 权限 | 公开（未登录返回空列表） |

响应格式： FollowListResponse

### POST /api/follows

| 项目 | 说明 |
|---|---|
| 功能 | 关注/取消关注实体 |
| 权限 | 公开（未登录时服务端处理） |

请求体： FollowRequest

```
 {
 "entity_type": "tag",
 "entity_slug": "topic-ai-人工智能",
 "active": true
}
```

| 字段 | 类型 | 说明 |
|---|---|---|
| entity_type | string | 实体类型（tag / topic / column / organization） |
| entity_slug | string | 实体 slug |
| active | bool | true = 关注，false = 取消关注 |

响应格式： FollowToggleResponse

### GET /api/follows/watchlist

| 项目 | 说明 |
|---|---|
| 功能 | 获取关注内容的最新动态（watchlist feed） |
| 权限 | 公开 |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| limit | query | int | 24 | 返回数量上限 |
| entity_type | query | string | null | 过滤实体类型 |
| entity_slug | query | string | null | 过滤实体 slug |

响应格式： WatchlistResponse

## 14.14 媒体模块（media）

路由文件：backend/routers/media.py
路由前缀：/api/media

### GET /api/media/{kind}

| 项目 | 说明 |
|---|---|
| 功能 | 获取指定类型的媒体内容列表（音频/视频等） |
| 权限 | 公开（内容受会员等级限制） |

**路径参数：**

| 参数 | 类型 | 说明 |
|---|---|---|
| kind | string | 媒体类型（如 audio、video） |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| limit | query | int | 24 | 返回数量上限 |

响应格式： MediaHubResponse

### GET /api/media/admin/items

| 项目 | 说明 |
|---|---|
| 功能 | 管理后台获取媒体项列表 |
| 权限 | 管理员 |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| kind | query | string | null | 媒体类型过滤 |
| status | query | string | null | 状态过滤 |
| limit | query | int | 60 | 返回数量上限 |

响应格式： MediaAdminListResponse

### GET /api/media/admin/items/{media_id}

| 项目 | 说明 |
|---|---|
| 功能 | 获取单个媒体项详情 |
| 权限 | 管理员 |

**路径参数：**

| 参数 | 类型 | 说明 |
|---|---|---|
| media_id | int | 媒体项 ID |

响应格式： MediaItemDetail

### POST /api/media/admin/items

| 项目 | 说明 |
|---|---|
| 功能 | 创建新的媒体项 |
| 权限 | 管理员 |

请求体： MediaItemCreate

响应格式： MediaItemDetail

### PUT /api/media/admin/items/{media_id}

| 项目 | 说明 |
|---|---|
| 功能 | 更新媒体项 |
| 权限 | 管理员 |

**路径参数：**

| 参数 | 类型 | 说明 |
|---|---|---|
| media_id | int | 媒体项 ID |

请求体： MediaItemUpdate（仅传需要更新的字段）

响应格式： MediaItemDetail

### POST /api/media/admin/upload

| 项目 | 说明 |
|---|---|
| 功能 | 上传媒体文件 |
| 权限 | 管理员 |

请求体： multipart/form-data

| 字段 | 类型 | 说明 |
|---|---|---|
| kind | string (Form) | 媒体类型（必需） |
| usage | string (Form) | 用途，默认 "media" |
| file | UploadFile | 上传的文件（必需） |

响应格式： MediaUploadResponse

## 14.15 编辑后台模块（editorial）

路由文件：backend/routers/editorial.py
路由前缀：/api/editorial

### GET /api/editorial/dashboard

| 项目 | 说明 |
|---|---|
| 功能 | 获取编辑台仪表盘数据 |
| 权限 | 管理员 |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| limit | query | int | 6 | 返回最近稿件数量 |

响应格式： EditorialDashboardResponse

## GET /api/editorial/articles

| 项目 | 说明 |
|---|---|
| 功能 | 获取编辑稿件列表 |
| 权限 | 管理员 |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| limit | query | int | 40 | 返回数量上限 |
| status | query | string | null | 稿件状态过滤 |
| workflow_status | query | string | null | 工作流状态过滤 |

响应格式： list[EditorialArticleSummary]

## GET /api/editorial/source-articles

| 项目 | 说明 |
|---|---|
| 功能 | 搜索可导入的源文章列表 |
| 权限 | 管理员 |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| query | query | string | "" | 搜索关键词 |
| limit | query | int | 12 | 返回数量上限 |

响应格式： list[EditorialSourceArticleSummary]

## GET /api/editorial/source-articles/{article_id}/ai-output

| 项目 | 说明 |
|---|---|
| 功能 | 获取源文章的 AI 处理输出（摘要、排版、翻译） |
| 权限 | 管理员 |

**路径参数：**

| 参数 | 类型 | 说明 |
|---|---|---|
| article_id | int | 源文章 ID |

响应格式： EditorialAiOutputDetail

## POST /api/editorial/source-articles/{article_id}/import-ai

| 项目 | 说明 |
|---|---|
| 功能 | 从 AI 输出导入编辑稿件 |
| 权限 | 管理员 |

**路径参数：**

| 参数 | 类型 | 说明 |
|---|---|---|
| article_id | int | 源文章 ID |

请求体： EditorialAiImportRequest

响应格式： EditorialArticleDetail

## POST /api/editorial/articles

| 项目 | 说明 |
| --- | --- |
| 功能 | 创建新的编辑稿件 |
| 权限 | 管理员 |

请求体： EditorialArticleCreate

响应格式： EditorialArticleDetail

## POST /api/editorial/upload

| 项目 | 说明 |
| --- | --- |
| 功能 | 上传 Markdown 文件创建编辑稿件 |
| 权限 | 管理员 |

请求体： multipart/form-data（文件上传）

| 字段 | 类型 | 说明 |
| --- | --- | --- |
| file | UploadFile | Markdown 文件 |

响应格式： EditorialUploadResponse

```
{
 "filename": "article.md",
 "article": { ... }
}
```

## GET /api/editorial/articles/{editorial_id}

| 项目 | 说明 |
| --- | --- |
| 功能 | 获取编辑稿件详情 |
| 权限 | 管理员 |

**路径参数：**

| 参数 | 类型 | 说明 |
| --- | --- | --- |
| editorial_id | int | 编辑稿件 ID |

响应格式： EditorialArticleDetail

## PUT /api/editorial/articles/{editorial_id}

| 项目 | 说明 |
| --- | --- |
| 功能 | 更新编辑稿件 |
| 权限 | 管理员 |

**路径参数：**

| 参数 | 类型 | 说明 |
| --- | --- | --- |
| editorial_id | int | 编辑稿件 ID |

请求体： EditorialArticleUpdate（仅传需要更新的字段）

响应格式： EditorialArticleDetail

## POST /api/editorial/articles/{editorial_id}/auto-format

| 项目 | 说明 |
| --- | --- |
| 功能 | AI 自动排版编辑稿件 |
| 权限 | 管理员 |

**路径参数：**

| 参数 | 类型 | 说明 |
| --- | --- | --- |
| editorial_id | int | 编辑稿件 ID |

请求体： EditorialAutoFormatRequest

响应格式： EditorialArticleDetail

## POST /api/editorial/articles/{editorial_id}/autotag

| 项目 | 说明 |
| --- | --- |
| 功能 | AI 自动为编辑稿件生成标签 |
| 权限 | 管理员 |

**路径参数：**

| 参数 | 类型 | 说明 |
| --- | --- | --- |
| editorial_id | int | 编辑稿件 ID |

响应格式： EditorialArticleDetail

## POST /api/editorial/articles/{editorial_id}/workflow

| 项目 | 说明 |
| --- | --- |
| 功能 | 更新编辑稿件工作流状态 |
| 权限 | 管理员 |

**路径参数：**

| 参数 | 类型 | 说明 |
| --- | --- | --- |
| editorial_id | int | 编辑稿件 ID |

请求体： EditorialWorkflowRequest

响应格式： EditorialArticleDetail

## POST /api/editorial/articles/{editorial_id}/render-html

| 项目 | 说明 |
| --- | --- |
| 功能 | 将编辑稿件渲染为 HTML |
| 权限 | 管理员 |

**路径参数：**

| 参数 | 类型 | 说明 |
| --- | --- | --- |
| editorial_id | int | 编辑稿件 ID |

响应格式： EditorialHtmlResponse

## GET /api/editorial/articles/{editorial_id}/export

| 项目 | 说明 |
| --- | --- |
| 功能 | 导出编辑稿件为 HTML 文件 |
| 权限 | 管理员 |

**路径参数：**

| 参数 | 类型 | 说明 |
| --- | --- | --- |
| editorial_id | int | 编辑稿件 ID |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
| --- | --- | --- | --- | --- |
| variant | query | string | "web" | 导出变体（web / 其他） |

响应格式： HTML 文件下载（Content-Disposition: attachment）

## POST /api/editorial/articles/{editorial_id}/publish

| 项目 | 说明 |
| --- | --- |
| 功能 | 发布编辑稿件 |
| 权限 | 管理员 |

**路径参数：**

| 参数 | 类型 | 说明 |
| --- | --- | --- |
| editorial_id | int | 编辑稿件 ID |

响应格式： EditorialPublishResponse

## 14.16 分析模块（analytics）

路由文件：backend/routers/analytics.py
路由前缀：/api/analytics

### GET /api/analytics/overview

| 项目 | 说明 |
|---|---|
| 功能 | 获取站点分析概览数据 |
| 权限 | 管理员 |

响应格式： AnalyticsOverviewResponse

## 14.17 管理模块（admin）

路由文件：backend/routers/admin.py
路由前缀：/api/admin

### GET /api/admin/overview

| 项目 | 说明 |
|---|---|
| 功能 | 获取管理后台概览数据 |
| 权限 | 管理员 |

响应格式： AdminOverviewResponse

### GET /api/admin/memberships

| 项目 | 说明 |
|---|---|
| 功能 | 获取会员列表 |
| 权限 | 管理员 |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| limit | query | int | 100 | 返回数量上限 |
| query | query | string | "" | 搜索关键词（邮箱等） |

响应格式： MembershipListResponse

### PUT /api/admin/memberships/{user_id}

| 项目 | 说明 |
|---|---|
| 功能 | 更新用户会员信息 |
| 权限 | 管理员 |

**路径参数：**

| 参数 | 类型 | 说明 |
|---|---|---|
| user_id | string | 用户 ID |

请求体： MembershipUpdateRequest

```
 {
 "email": "user@example.com",
 "tier": "premium",
 "status": "active",
 "note": "升级备注",
 "expires_at": "2027-01-01"
}
```

| 字段 | 类型 | 说明 |
|---|---|---|
| email | string | 用户邮箱 |
| tier | string | 会员等级 |
| status | string | 状态 |

| | | |
|---|---|---|
| note | string | 备注 |
| expires_at | string | 到期时间 |

响应格式： MembershipSummary

### GET /api/admin/billing/orders

| 项目 | 说明 |
|---|---|
| 功能 | 获取计费订单列表 |
| 权限 | 管理员 |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| limit | query | int | 100 | 返回数量上限 |
| query | query | string | "" | 搜索关键词 |

响应格式： BillingOrdersResponse

## 14.18 商务模块（commerce）

路由文件：backend/routers/commerce.py
路由前缀：/api/commerce

### GET /api/commerce/overview

| 项目 | 说明 |
|---|---|
| 功能 | 获取商务概览数据 |
| 权限 | 公开 |

响应格式： CommerceOverviewResponse

### POST /api/commerce/demo-request

| 项目 | 说明 |
|---|---|
| 功能 | 提交预约演示请求 |
| 权限 | 公开 |

请求体： DemoRequestIn

```
 {
 "email": "user@company.com",
 "name": "联系人姓名",
 "company": "公司名称",
 "message": "需求描述"
}
```

响应格式： DemoRequestResponse

错误码： 400 --- 邮箱格式无效（不包含 @）

### GET /api/commerce/demo-requests

| 项目 | 说明 |
|---|---|
| 功能 | 获取预约演示请求列表 |
| 权限 | 管理员 |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| limit | query | int | 50 | 返回数量上限 |

响应格式： list[DemoRequestSummary]

### GET /api/commerce/demo-requests/export

| 项目 | 说明 |
|---|---|
| 功能 | 导出预约演示请求为 CSV 文件 |

| | |
|---|---|
| 权限 | 管理员 |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| limit | query | int | 200 | 导出数量上限 |

响应格式：CSV 文件下载

```
Content-Type: text/csv; charset=utf-8
Content-Disposition: attachment; filename="demo_requests.csv"
```

## 14.19 时光机模块（time_machine）

路由文件：backend/routers/time_machine.py
路由前缀：/api

### GET /api/time_machine

| 项目 | 说明 |
|---|---|
| 功能 | 获取时光机数据（按日期浏览历史文章） |
| 权限 | 公开 |

**请求参数：**

| 参数 | 位置 | 类型 | 默认值 | 说明 |
|---|---|---|---|---|
| date | query | string | null | 目标日期（YYYY-MM-DD格式），为空则使用当前日期 |
| language | query | string | "zh" | 语言代码 |

响应格式：TimeMachineResponse

## 14.20 发布模块（publishing）

路由文件：backend/routers/publishing.py
路由前缀：无 API 前缀（根路径）

### GET /sitemap.xml

| 项目 | 说明 |
|---|---|
| 功能 | 生成 XML 站点地图 |
| 权限 | 公开 |

响应格式：application/xml

### GET /rss.xml

| 项目 | 说明 |
|---|---|
| 功能 | 生成 RSS 订阅源 |
| 权限 | 公开 |

响应格式：application/rss+xml

### GET /robots.txt

| 项目 | 说明 |
|---|---|
| 功能 | 生成 robots.txt 文件 |
| 权限 | 公开 |

响应格式：text/plain; charset=utf-8

## 14.21 端点汇总表

### 公开接口（无需认证）

| 方法 | 路径 | 功能 |
|---|---|---|
| GET | /api/home/feed | 首页信息流 |
| GET | /api/articles/latest | 最新文章 |
| GET | /api/articles/trending | 热门文章 |
| GET | /api/article/{id} | 文章详情 |
| GET | /api/article/{id}/cover | 文章封面 |
| GET | /api/article/{id}/translation | 文章翻译 |
| GET | /api/summarize_article/{id} | 文章摘要 |
| GET | /api/article/{id}/engagement | 文章互动数据 |
| POST | /api/search (exact 模式) | 精确搜索 |
| GET | /api/suggest | 搜索建议 |
| POST | /api/chat | AI 对话 |
| GET | /api/chat/sessions | 会话列表 |
| GET | /api/chat/session/{id} | 会话详情 |
| DELETE | /api/chat/session/{id} | 删除会话 |
| GET | /api/columns | 栏目列表 |
| GET | /api/columns/{slug}/articles | 栏目文章 |
| GET | /api/tags | 标签列表 |
| GET | /api/tags/cloud | 标签云 |
| GET | /api/tags/{slug}/articles | 标签文章 |
| POST | /api/tags/batch-generate | 批量生成标签 |
| GET | /api/organizations | 机构列表 |
| GET | /api/organizations/{slug} | 机构详情 |
| GET | /api/auth/status | 认证状态 |
| POST | /api/auth/login | 邮箱密码登录 |
| GET | /api/membership/me | 会员信息 |
| GET | /api/billing/plans | 付费计划 |
| GET | /api/billing/me | 计费信息 |
| POST | /api/billing/checkout-intent | 创建结账意向 |
| GET | /api/me/dashboard | 用户仪表盘 |
| GET | /api/follows | 关注列表 |
| POST | /api/follows | 关注/取关 |
| GET | /api/follows/watchlist | 关注动态 |
| GET | /api/media/{kind} | 媒体列表 |
| GET | /api/commerce/overview | 商务概览 |
| POST | /api/commerce/demo-request | 预约演示 |
| POST | /api/topics/auto-generate | 自动生成专题 |
| GET | /api/time_machine | 时光机 |
| GET | /sitemap.xml | 站点地图 |
| GET | /rss.xml | RSS 订阅 |
| GET | /robots.txt | Robots.txt |

## 需要登录的接口

| 方法 | 路径 | 功能 |
|---|---|---|
| POST | /api/article/{id}/reaction | 点赞/收藏 |
| GET | /api/me/library | 我的资料库 |

## 需要付费会员的接口

| 方法 | 路径 | 功能 |
|---|---|---|
| POST | /api/search (smart 模式) | 智能搜索 |
| GET | /api/topics | 专题列表 |
| GET | /api/topics/{slug} | 专题详情 |
| GET | /api/topics/{id}/timeline | 专题时间线 |
| GET | /api/topics/{id}/insights | 专题洞察 |

## 需要管理员权限的接口

| 方法 | 路径 | 功能 |
|---|---|---|
| GET | /api/analytics/overview | 分析概览 |
| GET | /api/admin/overview | 管理概览 |
| GET | /api/admin/memberships | 会员列表 |
| PUT | /api/admin/memberships/{user_id} | 更新会员 |
| GET | /api/admin/billing/orders | 订单列表 |
| GET | /api/media/admin/items | 媒体管理列表 |
| GET | /api/media/admin/items/{id} | 媒体管理详情 |
| POST | /api/media/admin/items | 创建媒体 |
| PUT | /api/media/admin/items/{id} | 更新媒体 |
| POST | /api/media/admin/upload | 上传媒体文件 |
| GET | /api/editorial/dashboard | 编辑台仪表盘 |
| GET | /api/editorial/articles | 编辑稿件列表 |
| GET | /api/editorial/source-articles | 源文章搜索 |
| GET | /api/editorial/source-articles/{id}/ai-output | AI 输出详情 |
| POST | /api/editorial/source-articles/{id}/import-ai | 导入 AI 稿件 |
| POST | /api/editorial/articles | 创建编辑稿件 |
| POST | /api/editorial/upload | 上传 Markdown 稿件 |
| GET | /api/editorial/articles/{id} | 编辑稿件详情 |
| PUT | /api/editorial/articles/{id} | 更新编辑稿件 |
| POST | /api/editorial/articles/{id}/auto-format | AI 自动排版 |
| POST | /api/editorial/articles/{id}/autotag | AI 自动标签 |
| POST | /api/editorial/articles/{id}/workflow | 更新工作流 |
| POST | /api/editorial/articles/{id}/render-html | 渲染 HTML |
| GET | /api/editorial/articles/{id}/export | 导出 HTML |
| POST | /api/editorial/articles/{id}/publish | 发布稿件 |
| GET | /api/commerce/demo-requests | 预约列表 |
| GET | /api/commerce/demo-requests/export | 导出预约 CSV |